

AI Engine Tools and Flows User Guide

UG1076 (v2024.2) November 18, 2024

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.



Table of Contents

- Chapter 1: Overview.....4**
 - Navigating Content by Design Process..... 8
- Chapter 2: AI Engine Architecture Overview..... 9**
 - AI Engine Components..... 10
 - Tools..... 10
 - Documentation..... 12
- Chapter 3: Compiling an AI Engine Graph Application.....13**
 - Setting Up the Vitis Tool Environment..... 14
 - Compiling using v++ (Unified Compiler)..... 15
 - Macro for Device Architectures..... 28
 - AI Engine Compiler Guidance..... 29
 - Compiling AI Engine Graph for Independent Partitions..... 30
- Chapter 4: Simulating an AI Engine Graph Application..... 33**
 - x86 Functional Simulator..... 36
 - AI Engine Simulator..... 55
 - Hardware Emulation..... 85
 - External Traffic Generator..... 85
- Chapter 5: Performance Analysis of AI Engine Graph Application during Simulation..... 86**
 - Analyzing AI Engine Status in Hardware Emulation..... 86
 - AI Engine Simulation-Based Performance Analysis..... 87
 - Viewing the Run Summary in the Vitis IDE.....98
 - Trace View Data Visualization..... 102
 - AI Engine Stall Analysis in the Vitis IDE.....116
 - FIFO Depth Visualization in the Vitis IDE..... 136
- Chapter 6: Performance Analysis of AI Engine Graph Application on Hardware.....141**

Analyzing AI Engine Status in Hardware.....	141
Profiling the AI Engine.....	150
Profiling the NoC.....	192
Event Tracing in Hardware.....	201
System Level Trace.....	218
Viewing Guidance in the Vitis IDE.....	223
Memory Tile Event Trace for AI Engine-ML-based Devices.....	225
DMA Event Trace in AI Engine-ML Tiles.....	227
Chapter 7: Programming the PS Host Application.....	230
Host Programming on Linux.....	231
Host Programming for Bare-Metal.....	255
Host Programming Support Comparison Between Linux and Bare-Metal.....	260
Resetting and Reloading Flow of the AI Engine Array.....	261
Chapter 8: Mapper/Router Methodology.....	266
Design Convergence.....	266
Improving Design Performance.....	274
Appendix A: Event Trace Reference.....	278
Simulation Event Trace	278
Hardware Event Trace.....	280
Event API.....	281
Appendix B: Encrypting AI Engine Kernels.....	284
Appendix C: Mapper/Router Design Rule Checks.....	285
Appendix D: Additional Resources and Legal Notices.....	289
Finding Additional Documentation.....	289
Support Resources.....	290
References.....	290
Revision History.....	291
Please Read: Important Legal Notices.....	302

Overview

AMD Versal™ adaptive system-on-chips (SoCs) combine Processing Systems, Programmable Logic, and AI Engines with leading-edge memory and interfacing technologies to deliver powerful heterogeneous acceleration for any application. Most importantly, Versal adaptive SoCs hardware and software are targeted for programming and optimization by data scientists and software and hardware developers. Versal adaptive SoCs are enabled by a host of tools, software, libraries, IP, middleware, and frameworks to enable all industry-standard design flows.

Built on the TSMC 7nm FinFET process technology, the Versal portfolio is the first platform to combine software programmability and domain-specific hardware acceleration with the adaptability necessary to meet today's rapid pace of innovation. The portfolio includes six families of devices uniquely architected to deliver scalability and AI inference capabilities for a host of applications across different markets—from cloud—to networking—to wireless communications—to edge computing and endpoints.

The Versal architecture combines different engine types with a wealth of connectivity and communication capability and a network on chip (NoC) to enable seamless memory-mapped access to the full height and width of the device. AI Engines are SIMD VLIW processors for adaptive inference and advanced signal processing, and DSP Engines for fixed point, floating point, and complex MAC operations. Programmable Logic includes a combination of programmable logic (PL) blocks and memory, architected for high-compute density. Processing System, includes Arm® Cortex®-A72 and Cortex-R5F processors, allow for intensive compute tasks.

AI Engines

The Versal AI Core Series delivers breakthrough AI inference acceleration with AI Engines. This series is designed for a breadth of applications, including cloud for dynamic workloads and network for massive bandwidth, all while delivering advanced safety and security features. AI and data scientists, as well as software and hardware developers, can all take advantage of the high compute density to accelerate the performance of any application. Given the AI Engine's advanced signal processing compute capability, it is well-suited for highly optimized wireless applications such as radio, 5G, backhaul, and other high-performance DSP applications.

AI Engines are an array of very-long instruction word (VLIW) processors with single instruction multiple data (SIMD) vector units that are highly optimized for compute-intensive applications, specifically digital signal processing (DSP), 5G wireless applications, and artificial intelligence (AI) technology such as machine learning (ML).

AI Engines are hardened blocks that provide multiple levels of parallelism including instruction-level and data-level parallelism. Instruction-level parallelism includes a scalar operation, up to two moves, two vector reads (loads), one vector write (store), and one vector instruction that can be executed—in total, a 7-way VLIW instruction per clock cycle. Data-level parallelism is achieved via vector-level operations where multiple sets of data can be operated on a per-clock-cycle basis. Each AI Engine contains both a vector and scalar processor, dedicated program memory, local data memory, and can access adjacent local memory in any of three neighboring directions. It also has access to DMA engines and AXI4 interconnect switches to communicate via streams to other AI Engines or to the programmable logic (PL) or the DMA. Refer to the *Versal Adaptive SoC AI Engine Architecture Manual* ([AM009](#)) for specific details on the AI Engine array and interfaces.

The AI Engine-ML (AIE-ML) block is capable of delivering 2x compute throughput compared to its predecessor AI Engine blocks. The AIE-ML block, primarily targeted for machine learning inference applications, delivers one of the industry's best performance per Watt for a wide range of inference applications. Refer to the *Versal Adaptive SoC AIE-ML Architecture Manual* ([AM020](#)) for specific details on the AIE-ML features and architecture.

As an application developer, it is possible to use one of the white box or black box flows for running a ML inference application on AIE-ML variants. With the white box flow you can integrate custom kernels and dataflow graphs in the AIE-ML variants programming environment. A black box flow uses performance optimized Neural Processing Unit (NPU) IP from AMD to accelerate ML workloads in the AIE-ML variants.

AMD Vitis™ AI is used as a front-end tool that parses the network graph, performs optimization, quantization of the graph, and generates compiled code that can be run on the AIE-ML variants hardware. The AIE-ML variants core tile architecture provides support for a variety of precision fixed and floating-point datatypes. The architecture allows for pipe-lined vector processing and incorporates high-density, high-speed on-chip memory that can effectively store on-chip tensors. Additionally, it features versatile datamovers that are adept at handling multi-dimensional tensors in memory. With the proper selection of overlay processor architecture and spatial and temporal distribution of the input/output tensor in the on/off-chip memory, it is possible to achieve high computational efficiency of the AIE-ML variants processing cores.

[Chapter 2: AI Engine Architecture Overview](#) provides a high-level overview of the AI Engine architecture, tools, and documents that can be referenced for kernel programming.

AI Engine Kernels

An AI Engine kernel is a C/C++ program which is written using specialized APIs that target the VLIW vector processor. The AI Engine kernel code is compiled using the AI Engine compiler that is included in the AMD Vitis™ core development kit. The AI Engine compiler compiles the kernels to produce an ELF file that is run on the AI Engine processors.

AI Engine Graphs

An AI Engine program requires a data flow graph specification which is written in C++. This specification can be compiled and executed using the AI Engine compiler. An adaptive data flow (ADF) graph application consists of nodes and edges where nodes represent compute kernel functions, and edges represent data connections. Kernels in the application are compiled to run on the AI Engines. Refer to *AI Engine Kernel and Graph Programming Guide* ([UG1079](#)) for more information about how to develop, debug and optimize AI Engine kernels and graphs. It also include information on specialized graph constructs and ways to control the AI Engine graph.

Compiling and Simulating the Program

[Chapter 3: Compiling an AI Engine Graph Application](#) describes in detail the different types of compilation available with the AI Engine compiler, the options and input files that can be passed in, and the expected output. You can compile the graph and kernels independently, or as part of a larger system, and set up the design to capture and profile event trace data at runtime.

[Chapter 4: Simulating an AI Engine Graph Application](#) describes the AI Engine simulator in detail, as well as the x86 simulator for functional simulation. The AI Engine simulator simulates the graph application as a standalone entity, or as part of the hardware emulation of a larger system design.

[Using the Vitis Unified IDE](#) in the *Vitis Reference Guide* ([UG1702](#)) describes migrating, building, running, and debugging the AI Engine component in the Vitis Unified IDE. The Vitis tools embrace a bottom-up design flow that lets you develop components of a system and then integrate the components into a top-level system application. Refer to *Vitis Reference Guide* ([UG1702](#)) for a description of building the full system project incorporating the different components described above.

Integrating the AI Engine application into a Versal Design using Vitis

The AI Engine kernels and graph developed in the previous steps can be integrated into a larger Versal adaptive SoC system design that can consist of AI Engine kernels, HLS PL kernels, RTL kernels, and the host application. The Vitis compiler builds this larger system. Refer to *Vitis Reference Guide* ([UG1702](#)) for a description of building the full system project incorporating the different components described above.

Profiling and Debugging Designs with AI Engine

[Chapter 5: Performance Analysis of AI Engine Graph Application during Simulation](#) describes how to extract performance data by performing event tracing when running the hardware emulation build or the hardware build. This data can be used to further optimize the AI Engine kernels and graphs.

[Chapter 6: Performance Analysis of AI Engine Graph Application on Hardware](#) describes how to profile and extract performance data by performing event tracing when running the design in hardware. [Simulating and Debugging the AI Engine Component](#) in the *Vitis Reference Guide* (UG1702) provides information related to debugging the AI Engine component from within the Vitis Unified IDE.

[Debugging System Projects](#) in the *Embedded Design Development Using Vitis* (UG1701) shows you how to run and use the debug environment from the command line, or from the Vitis Unified IDE. The evaluation of the system performance and debugging the application are key steps to achieving your objectives for the system.

Controlling the AI Engine Graph

[Chapter 7: Programming the PS Host Application](#) describes the process of creating a host application to control the graph and PL kernels of the system. When your design is deployed in hardware, you can install drivers that facilitate initializing and controlling the graph execution via a host application running on the PS, or load and run the AI Engine graph at device boot time.

Application-specific AI Engine control code is generated by the AI Engine compiler as part of compiling the AI Engine design graph and kernel code. The AI Engine control code can:

- Control the initial loading of the AI Engine kernels.
- Run the graph for several iterations, update the runtime parameters (RTP) associated with the graph, exit and reset the AI Engines.

Note: A graph can have multiple kernels, input and output ports. The graph connectivity, which is equivalent to the nets in a data flow graph is either between the kernels, between kernel and input ports, or between kernel and output ports, and can be configured as a connection. A graph runs for an iteration when it consumes data samples equal to the window or stream of data expected by the kernels in the graph, and produces data samples equal to the window or stream of data expected at the output of all the kernels in the graph.

The Vitis core development kit provides platforms for building, simulating, debugging, and deploying your AI Engine designs. These platforms target a specific hardware board e.g., VCK190 board or VEK280 board. It enables development of a design including AI Engine and PL kernels with a host application that targets the Linux OS running on the Arm processor in the PS. Designs developed on this platform can be verified using the hardware emulation flow and run on the target hardware board.

AI Engine Methodology

[Chapter 8: Mapper/Router Methodology](#) describes the mapper and router methodology to be used when handling a failure in the AI Engine compiler during the mapper and/or router phase.

[AI Engine Hardware Profile and Debug Methodology](#) in the *Embedded Design Development Using Vitis* (UG1701) describes the five stage profile and debug methodology to use when you run a Versal design with AI Engine graphs in hardware.

Vitis Unified Integrated Design Environment



IMPORTANT! This document has been updated to reflect the use of the Vitis Unified IDE and the use of the `v++` common command line syntax to create AI Engine components. The classic Vitis IDE has been obsoleted. For information on migrating AI Engine graph applications from the classic IDE to the Vitis Unified IDE, see [Migrating Vitis Classic IDE Graph Applications to Vitis Unified IDE](#) in Vitis Reference Guide (UG1702).

The Vitis Unified IDE provides system project design and debug for heterogeneous computing systems, embedded system design, and data center acceleration. Elements of the system project includes AI Engine and High-Level Synthesis (HLS) component creation, platform creation, and embedded software design.

The Vitis Unified IDE uses a common command-line to compile and run the elements of the design. See [Compiling using v++ \(Unified Compiler\)](#) for a review of the `v++` command-line flows for developing AI Engine components.

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Embedded Software Development:** Creating the software platform from the hardware platform and developing the application code using the embedded CPU. Also covers XRT and Graph APIs. Topics in this document that apply to this design process include:
 - [Chapter 7: Programming the PS Host Application](#)
- **AI Engine Development:** Creating the AI Engine graph and kernels, library use, simulation debugging and profiling, and algorithm development. Also includes the integration of the PL and AI Engine kernels. Topics in this document that apply to this design process include:
 - [Chapter 3: Compiling an AI Engine Graph Application](#)
 - [Chapter 4: Simulating an AI Engine Graph Application](#)
- **System Integration and Validation:** Integrating and validating the system functional performance, including timing, resource use, and power closure.

AI Engine Architecture Overview

Programming the AI Engine array requires a thorough understanding of the algorithm to be implemented, the capabilities of the AI Engines, and the overall data flow between individual functional units. The AI Engine array supports three levels of parallelism:

- **SIMD:** Through vector registers that allow multiple elements to be computed in parallel.
- **Instruction level:** Through the VLIW architecture that allows multiple instructions to be executed in a single clock cycle.
- **Multicore:** Through the AI Engine array, where many AI Engines (from less than ten to several hundreds) can execute in parallel.

While most standard C/C++ code can be compiled for the AI Engine, the code might require substantial restructuring to achieve optimal performance on the AI Engine array. The power of an AI Engine is its ability to execute a vector MAC operation, load two 256-bit vectors for the next operation, store a 256-bit vector from the previous operation, and increment a pointer or execute another scalar operation in each clock cycle. To make use of the vector processor, the code needs to use AIE APIs or intrinsic functions and be structured for pipelined vector operations. The AI Engine compiler does not perform any auto or pragma-based vectorization. The code must be rewritten to use SIMD intrinsic data types (for example, `v8int32`) and vector intrinsic functions (for example, `mac(...)`), and these must be executed within a pipelined loop to achieve optimal performance. The 32-bit scalar RISC processor has an ALU, some non-linear functions, and data type conversions. Each AI Engine has access to a limited amount of memory, this means that large data sets need to be partitioned.

AI Engine kernels are functions that run on an AI Engine, and form the fundamental building blocks of a data flow graph specification. The data flow graph is a modified Kahn process network with deterministic behavior that does not depend on the various computational or communication delays. AI Engine kernels are declared as void C/C++ functions that take buffer or stream arguments for graph connectivity. Kernels can also have static data and runtime parameter arguments that can be either asynchronous or triggering. Each kernel should be defined in its own source file.

To achieve overall system performance, additional reading and experience is required with respect to the architecture, partitioning, as well as with the AI Engine data flow graph generation and optimizing data flow connectivity. More detailed information is found in the *Versal Adaptive SoC AI Engine Architecture Manual* ([AM009](#)), and the *Versal Adaptive SoC AIE-ML Architecture Manual* ([AM020](#)).

AMD provides DSP and communications libraries with optimized code for the AI Engine that should be used whenever possible. The supplied source code is also a great resource for learning about AI Engine kernel coding.

AI Engine Components

- **AI Engine:** Each AI Engine is a very long instruction word (VLIW) processor containing a scalar unit, a vector unit, two load units, and a single store unit.
- **AI Engine Tile:** An AI Engine tile contains an AI Engine, a local memory module together with several communication paths to facilitate data exchange between tiles.
- **AI Engine Array:** The AI Engine array consists of a 2D array of AI Engine tiles, where each AI Engine tile contains an AI Engine, memory module, and tile interconnect module.
- **AI Engine Program:** The AI Engine program consists of a data flow graph specification which is written as a C++ class. A top-level program will instantiate one or more graph classes and will be compiled and executed using the AI Engine tool chain.
- **AI Engine Kernels:** Kernels are written in C/C++ using AI Engine vector data types and intrinsic functions. These are the computation functions running on an AI Engine. The kernels form the fundamental building blocks of a data flow graph specification.
- **ADF Graph:** An ADF graph is a network with a single AI Engine kernel or multiple AI Engine kernels connected by data streams. It interacts with the programmable logic, global memory, and processing system with specific constructs like PLIO (port attribute in the graph that is used to make stream connections to or from the programmable logic), GMIO (port attribute in the graph that is used to make external memory-mapped connections to or from the global memory), and RTP.

Graphs can also be embedded into other graphs as sub-graphs, which allows for structuring the design into logical subsystems.

Tools

Vitis Unified Integrated Design Environment

The AMD Vitis™ unified integrated design environment (IDE) can be used to target system programming of AMD devices including AMD Versal™ devices with AI Engine array. The following features are available in the tool.

- An optimizing C/C++ compiler that compiles the kernels and graph code making all of the necessary connections, placements, and checks to ensure proper functioning on the device.

- A cycle approximate simulator, accelerated functional simulator, and profiling tools.
- A debugging environment that works in both simulation and hardware environments.

This IDE is also used to view compilation and simulation reports, and analyze output files generated by the AI Engine compiler. In addition, it can also be used to view and analyze system application reports. More information about this design environment and how it can be used to create components is found in [Using the Vitis Unified IDE](#) in the *Vitis Reference Guide* (UG1702).

Vitis Unified Command Line Tools

Command line tools are available to build, simulate, and generate output files and reports. Command line outputs which are generated by the IDE are captured to facilitate subsequent integration into customer build environments.

For information about the Vitis unified command line interface, see the *Vitis Reference Guide* (UG1702).

Vitis Model Composer

AMD Vitis™ Model Composer offers a high-level graphical entry environment based on Simulink® from MathWorks for simulation and code generation of designs that includes AI Engine, HLS, and RTL components. For more information, see the *Vitis Model Composer User Guide* (UG1483).

- Import AI Engine kernels, graphs, HLS kernels, and RTL based code into one Simulink® design for fast co-simulation.
- From the Simulink library browser, drag and drop optimized AI Engine functions from Vitis Libraries such as FFTs and Finite Impulse Response (FIR) filters into the design.
- Verify the design using stimulus generated in MATLAB or Simulink, visualize the results, and compare the results with the golden reference. Generate graph code and test vectors.
- Assemble imported and block library code to feed into downstream tools.

Documentation

The following links are useful in developing and programming your AI Engine kernels.

- The *AI Engine Intrinsic User Guide* ([UG1078](#)) is a list of all the intrinsics and data types supported in the current release. It is a good reference guide for all the intrinsics and data types supported by the AI Engine.
- *AI Engine-ML Intrinsic User Guide* ([UG1583](#))
- *AI Engine Kernel and Graph Programming Guide* ([UG1079](#))
- *AI Engine-ML Kernel and Graph Programming Guide* ([UG1603](#))
- *Embedded Design Development Using Vitis* ([UG1701](#))
- *Vitis Reference Guide* ([UG1702](#))
- *Versal Adaptive SoC AI Engine Architecture Manual* ([AM009](#))
- *Versal Adaptive SoC AI Engine Register Reference* ([AM015](#))
- *Versal Adaptive SoC AIE-ML Architecture Manual* ([AM020](#))
- *Versal Adaptive SoC AIE-ML Register Reference* ([AM025](#))
- The *ASIP Programmer Chess Compiler User Manual*, found on the *Versal AI Engines Secure Site*, provides a list of all the pragmas and functions to help optimize your AI Engine kernel code.
- *AI Engine API User Guide* ([UG1529](#))
- *Vitis Model Composer User Guide* ([UG1483](#))

Compiling an AI Engine Graph Application

This chapter describes all the command line options passed to the AI Engine compiler. It takes the code for the data flow graph, the code for individual kernels, and produces an executable that can be run on various AI Engine target platforms such as simulators, emulators, and AI Engine devices. The AI Engine compiler statically compiles the graph, mapping and subsequently placing its kernels within the AI Engines. It is important to note that the AI Engine graph and kernels can be compiled individually or combined into a standalone application. These in turn can be executed either through emulation, or directly, on the AI Engine array hardware.



TIP: Unless otherwise specified, all the input file paths are with respect to the current directory, and all the output file paths are with respect to the *Work* directory.

The graph and kernels can also be used as part of a larger subsystem design, that incorporates the AI Engine graph as an ELF application running on the AI Engine and programmable logic (PL) kernels running in the programmable logic of the device. The AI Engine compiler is used to compile the graph and kernels, whether in a standalone configuration or as part of a larger system. For more information on building the system project, see [Building and Running the System](#) in the *Embedded Design Development Using Vitis* (UG1701).

The AMD Vitis™ Unified IDE can be used to create and manage project build settings, and run the AI Engine compiler. Alternatively, you can build the project from the command line, or in a script or Makefile. Either approach lets you perform simulation or emulation to verify the graph application or the integrated system design, debug the design in an interactive debug environment, and build your design to run and deploy on hardware. For more information on using the Vitis Unified IDE to create and manage AI Engine graph applications, see [Using the Vitis Unified IDE](#) in the *Vitis Reference Guide* (UG1702).

Setting Up the Vitis Tool Environment

The AI Engine tools are delivered and installed as part of the Vitis unified software platform. Therefore, when preparing to run the AI Engine tools, for example, the AI Engine compiler and AI Engine simulator, you must set up the Vitis unified software platform. The AMD Vitis™ unified software includes embedded base platforms targeting select boards like the VCK190 and the VEK280. You can also download the Common Images for Embedded AMD Vitis™ Platforms from <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-platforms.html>. The common image package contains a prebuilt Linux kernel and root file system that can be used with the AMD Versal™ board for embedded design development together with the appropriate Vitis base platform.

For more information, see [Vitis Software Platform Release Notes](#) in the *Vitis Software Platform Release Notes (UG1742)*.

After the Vitis unified software platform is installed, run the following script to set all the necessary environment variables, such as `XILINX_VITIS` and `XILINX_HLS`.

```
source <Vitis_install_path>/Vitis/<version>/settings64.csh
```



TIP: The `settings64.sh` for the Bash shell is also provided in the same directory.

Set up your `PLATFORM_REPO_PATHS` environment variable to `$XILINX_VITIS/base_platforms`

```
setenv PLATFORM_REPO_PATHS $XILINX_VITIS/base_platforms
```



TIP: The `PLATFORM_REPO_PATHS` environment variable points to directories containing platform files (XPFM). This lets you specify platforms using only the folder name for the platform.

The AI Engine tools require a license which can be generated for free from any eligible AMD licensing account. The path to the license must be set in one of the following environment variables: `XILINXD_LICENSE_FILE` or `LM_LICENSE_FILE`.

You can validate the installation and setup of the tools by using one of the following commands.

```
vitis --help
```

Compiling using v++ (Unified Compiler)

AMD provides a standalone command line utility `v++ -c --mode aie` to compile the AI Engine graph and generate the `libadf.a` file for use in simulation, emulation, and hardware flows. The AMD Vitis™ Unified IDE uses the `v++ -c --mode aie` command line to compile the AI Engine graph code for either `x86simulator` or `aiesimulator` based on the target of the compilation process.

The `v++ -c --mode aie` command takes the graph, platform or part, and compiler options as inputs which can be either specified directly as part of the command or in a config file by adding `--config <my_settings>.cfg` argument.



TIP: Multiple config file arguments are allowed, with the first occurrence overruling any proceeding duplicate settings. It can be helpful to include common settings in the first config file, and application specific settings in a second config file.

In addition, the `v++ -c --mode aie` command supports AI Engine partitions and independent compilation and linking of ADF graphs into disjoint partitions.

The configuration file can contain a variety of options ranging from general compilation options like `include` and `target`, which are generic to different compilation modes. AI Engine compilation specific options are listed in the next section.

Note: For the complete list of v++ General Compilation Options, see [v++ General Compilation Options](#) in the Vitis Reference Guide (UG1702).



TIP: To access the command help, enter `v++ -c --mode aie --help`.

The contents of a configuration file can vary, but the sample `aiecompiler.cfg` file shown below uses the following commands.

```
include=<VITIS_INSTALL_PATH>/Vitis/2024.2/aietools/include
include=./aie
include=./data
include=./aie/kernels
include=./
platform= xilinx_vck190_base_202420_1.xpfm
target=hw

[AIE]
Xelfgen=-j4
workdir=./Work_3
Xmapper=DisableFloorplanning
output-archive=libadf_1.a
```

The options under the [AIE] section are applicable only to the AI Engine compilation. The following sections describe the `v++ -c --mode aie` command usage with an example and an insight on different AI Engine compile options which can be used in the configuration file.

File and Directory Names

File and Directory Names

The set of acceptable characters in file and directory names include as follows:

- Letters (a-z A-Z)
- Digits (0-9)
- Underscore (_)
- Hyphen
- Space
- Dot (.)

In addition:

- The name must contain at least one letter or number.
- The name must start with a letter or number.
- The name cannot end with a dot or space.

v++ Mode AI Engine

The AI Engine compilation mode provides for the development, optimization, analysis, and export of AI Engine graph applications (`libadf.a`). The AI Engine compilation mode uses the following command:

```
v++ -c --mode aie --target hw --platform xilinx_vck190_base_202420_1 \
--work_dir ./myWork --config ./config.cfg <Input File>
```

- `v++ -c --mode aie --target hw`: Indicates compilation of an AI Engine component for the HW target (default) indicating the generation of the `libadf.a` file for use in AI Engine simulation and to run on the physical device. You can also specify `x86sim` for the target to build the component for x86 functional simulator. The contents of the output directory depend on the target.
- `--platform xilinx_vck190_base_202420_1`: Specifies the platform or physical device to build the component for.
- `--work_dir ./myWork`: Specifies the work directory to use for building the AI Engine component.



TIP: By default the compiler writes all outputs to a directory called *Work* in a sub-directory of the current directory where the tool was launched, and creates a file called *libadf.a* in the current directory.

- `--config ./config.cfg`: Specifies the AI Engine component config file containing a variety of options available to the AI Engine compiler, x86 Simulator, and AI Engine Simulator which are part of the analysis and profiling tools for the AI Engine component.
- `<Input File>` specifies the data flow graph code that defines the `main()` application for the AI Engine graph. The input flow graph is specified using a data flow graph language. For a description of the data flow graph, refer to [Introduction to Graph Programming](#) in the *AI Engine Kernel and Graph Programming Guide* (UG1079).

The `v++ -c --mode aie` command options are described in the following sections. The AI Engine options should be added to a configuration file under the `[AIE]` section.



TIP: Some `[AIE]` options can be specified on the command line instead of in a config file, although using a config file is the recommended approach. To use an `[AIE]` option on the command line add `--aie.` to the start of the command name from the following sections. For example, to specify the constraints from the command line use `--aie.constraints`

The AI Engine component (with `--target hw`) can then be run through the `aiesimulator` using the following command:

```
aiesimulator --pkg-dir=./Work --i=../..
```

Where `--pkg-dir` specifies the *Work* directory where compilation occurs, and the `--i` specifies the input directory path for the simulator.

After compilation the AI Engine component (with `--target x86sim`) can then be run through the `x86simulator` using the following command:

```
x86simulator --pkg-dir=./Work --i=../..
```

AI Engine Options

The following options apply to the AI Engine compilation process.

- **constraints:**

Constraints (location, bounding box, etc.) can be specified using a JSON file. This option lets you specify one or more constraint files.

```
constraints=constraints.json
```

- **heapsize:**

Heap size (in bytes) used by each AI Engine

In an AI Engine, the size of the heap determines the maximum amount of memory that can be dynamically allocated at any given time.

The heap size for each AI Engine is measured in bytes, and by default, the size is set at 1024 bytes.

The stack, heap, and sync buffer (sync buffer is a memory of size 32 bytes, used to store graph run iteration information) are allocated up to the tile data memory. Before changing the heap size to a different value, ensure that the sum of the stack, heap, and sync buffer sizes does not exceed the tile data memory, for example, 32768 bytes in AI Engine device.

This option is also used for allocating any remaining file-scoped data that is not explicitly connected in the user graph.

```
heapsize=512
```

- **log-level:**

Log level for verbose logging. Only applicable when used with `verbose`. The specified range can be from 0 to 5, with increasing details provided in the log file as the number increases.

- 0 is the same as the `quiet` option
- 1 is the default logging of the compiler
- 2 is the default logging specified with `verbose`
- 3 - 5 are increased logging details when used with `verbose`

```
log-level=4
```

- **pl-freq:**

Specifies the interface frequency (in MHz) for all PL kernels and PLIOs. The default frequency is a quarter of the AI Engine clock frequency, and the maximum frequency is half of the AI Engine clock frequency. The PL frequency specific to each interface can be explicitly provided in the graph.

```
pl-freq=500
```

- **pl-register-threshold:**

Specifies the frequency (in MHz) threshold for registered AI Engine-PL crossings. The default frequency is one-eighth of the AI Engine clock frequency.

```
pl-register-threshold=300
```

Note: For more details, regarding the use of this setting, see *AI Engine Kernel and Graph Programming Guide* ([UG1079](#)).



IMPORTANT! Values above a quarter of the AI Engine array frequency are ignored, and a quarter is used instead.

- **stacksize:**

Stack size (in bytes) used by each AI Engine tile. The default stack size is set to 1024 bytes. Used as a standard compiler calling convention including stack-allocated local variables and register spilling.

The stack is a region of memory used by programs to store temporary data during execution. In the case of an AI Engine, every tile relies on a specific amount of stack memory to run the kernel mapped to it. The amount of memory allocated as stack space is measured by the number of bytes assigned to each tile. This size of the stack is an essential consideration when designing and optimizing the performance of the AI Engine.

By default, the stack size for each tile is set at 1024 bytes. This value, takes into account factors like stack-allocated local variables and register spilling, which occur when a register no longer has room for data, so it is "spilled" onto the stack temporarily to free up register space. It is essential to optimize the stack size for each tile to prevent potential issues that could arise, such as memory overflows, which can lead to errors and other performance problems.



TIP: The stack, heap, and sync buffer (32 bytes) are allocated up to 32768 bytes of data memory. Before changing the stack size to a different value, ensure that the sum of the stack, heap, and sync buffer sizes does not exceed 32768 bytes.

```
stacksize=512
```

CDO Options

The CDO options apply to generator code for graph configuration and initialization in configuration data object (CDO) format. This is used during SystemC-RTL simulation and during actual hardware execution.

- **broadcast-enable-core:**

Enables all AI Engine cores associated with a graph using broadcast. This option reserves one broadcast channel in the array for core enabling purpose. This is enabled by default, and can be disabled by setting the argument to false.

```
broadcast-enable-core=false
```

Compiler Debugging Options

The Debugging options specify features of the compiler to enable troubleshooting errors during the build process, such as increased log details and consistency checking in the source code.

- **adf-api-log-level:**

ADF API log-level. Available values are as follows:

- 0: errors
- 1: level-0 + warnings
- 2: level-1 + info messages
- 3: level-2 + debug messages

The default is 2.

```
adf-api-log-level=3
```

- **kernel-linting:**

Perform consistency checking between graphs and kernels. Accepted values are true and false. The default is false.

```
kernel-linting=true
```

- **quiet:**

Suppress output of the compiler. Accepted values are true and false. The default is false.

```
quiet=true
```

- **verbose:**

Verbose output of the compiler. Accepted values are true and false. The default is false.

```
verbose=true
```



TIP: Can be used with `log-level` to increase the verbosity of the logs. For example:

```
log-level=4
```

Design Rule Check Options

The AI Engine mapper and router support configuring a list of Design Rule Checks (DRCs). The DRC commands shown below should be entered into the config file outside of any heading ([AIE] for instance).

- **drc.disable:**

Disable the DRC rule check for the specified ID. A disabled check is not executed.

```
drc.disable=AIE-ROUTER-3
```

- **drc.enable:**

Enable a previously disabled DRC rule check for the specified ID.

```
drc.enable=AIE-ROUTER-3
```

- **drc.severity:**

Change the severity of a DRC rule check. The format of the argument is

`<ID>:<severity>[:context]`

Where:

- `<ID>` is the DRC rule ID which can be found on DRC messages reported by the tool.
- `<severity>` represents the new severity to assign to the specified ID.
- `[context]`

```
drc.severity=AIE-ROUTER-3:warning
```

- **drc.waive:**

Waive the Design Rule Check for the specified ID. A check is still performed, but marked as waived, meaning that you don't care about this specific check for some reason.

```
drc.waive=AIE-ROUTER-3
```

Related Information

[Mapper/Router Design Rule Checks](#)

File Options

The file options are used for file management activities through the config file, such as specifying include files and output file names.

- **include:**

This option can be used to include additional directories in the include path for the compiler front-end processing. Specify one or more include directories.

- **output:**

Specifies an output.json file that is produced by the front end for an input data flow graph file. The output file is passed to the back-end for mapping and code generation of the AI Engine device. This is ignored for other types of input.

- **output-archive:**

Specify output archive name which will contain compiled AI Engine artifacts (default: `libadf.a`). The output archive is written to the component directory above the Work directory.

```
output-archive=myGraph.a
```

Module Specific Options

Options that apply to kernel implementation on AI Engine tiles.



IMPORTANT! Only AI Engine kernels that were modified are recompiled in subsequent compilations of the AI Engine graph. Any un-modified kernels will not be recompiled; therefore, the application of these options might require touching kernel source to be recompiled.

- **Xchess arg:**

Used to pass kernel specific options to the AI Engine compiler, which in turn uses the CHES compiler to compile code for each AI Engine. The option string is specified as `<kernel>:<optionid>=<value>`. The option string is included during compilation of generated source files on the AI Engine where the specified kernel is mapped.

```
Xchess=main:darts.xargs=-nb
```

- **Xelfgen arg:**

Can be used to pass additional command-line options to the ELF generation phase of the compiler, which is currently run as a `make` command to build all AI Engine ELF files. The values for this option can be provided in quotes which is especially useful if you want to pass special characters. For example, `Xelfgen="CLANG_OPTS=' -g1 -mllvm --issue-limit=6' "`.

For example, to limit the number of parallel compilations to four, use `Xelfgen=" -j4 "`.

Note: If errors with `bad_alloc` appear in the log, or if the Vitis IDE crashes, it could be due to insufficient memory on your workstation. A possible workaround (other than increasing the available memory on your machine) is to limit the parallelism used by the compiler during code generation by passing this option to the Elf Generator Makefile (`-j1` or `-j2`).

```
Xelfgen=-j2
```

- **Xmapper arg:**

Can be used to pass additional command-line options to the mapper phase of the compiler. These are options to try when the design is either failing to converge in the mapping or routing phase, or when trying to achieve better performance via reduction in memory bank conflict.

```
Xmapper=DisableFloorplanning
```

- **Xpreproc arg:**

Pass general option to the PREPROCESSOR phase for all source code compilations AIE/PS/PL/x86sim of the AI Engine component (e.g., -D...).

```
Xpreproc=-D<var>=<value>
```

- **Xpslinker arg:** Pass general option to the PS LINKER phase of the AI Engine component (e.g., -L... -l...).

```
Xpslinker=-L<libpath> -l<libname>
```

- **Xrouter arg:**

Pass general option to the ROUTER phase.

```
Xrouter=dmaFIFOsInFreeBankOnly
```

- **Xx86sim arg:**

Pass x86sim-specific option to the compiler.

```
Xx86sim=clangStaticAnalyzer
```

- **fast-floats:**

Enable fast implementation for linear floating point scalar operations like `add`, `sub`, `mul` and `compare`. Accepted values are `true` and `false`. The default is `false`.

```
fast-floats=true
```

- **fast-nonlinearfloats:**

Enable fast implementation for non-linear floating point scalar operations like `sine/cosine`, `sqrt` and `inv`. Accepted values are `true` and `false`. The default is `false`.

```
fast-nonlinearfloats=true
```

- **fastmath:**

Enable fast implementations of `float2fix`, `fplt` and `fpge`. Accepted values are `true` and `false`. The default is `false`.

```
fastmath=true
```

- **float-accuracy:**

Option available only for AI Engine-ML. It selects the required floating-point compute accuracy emulated with multiple `bfloat16` numerical type operations:

```
float-accuracy=<arg>
```

Available argument values are:

- `safe`: Accuracy is slightly better than single precision floating-point (FP32).
- `fast`: Improved performance with similar accuracy to FP32 (default).
- `low`: Best performance with better accuracy than FP16 and `bfloat16`.

Event Tracing Options

Options to define the event-tracing characteristics of the compiled kernels and graph. These options prepare the design for event tracing during runtime. See [Event Trace Build Flow](#) for more information.

- **event-trace:**

Event trace configuration value.

```
event-trace=runtime
```

Note: The following event trace configuration values are now deprecated: `functions`, `functions_partial_stalls`, `functions_all_stalls`. For ease of use purpose, these configuration values must now be specified in the `xrt.ini` file for hardware emulation or hardware run.

- **event-trace-port:**

Set the AI Engine event tracing port. Accepted values are `plio` and `gmio`. The `gmio` option uses the AI Engine-to-NoC pathway to capture trace data. The alternative is to use `plio` which uses the AI Engine-PL pathway to capture trace data. This uses programming logic resources to capture data from AI Engine to DDR. The default value is `gmio` which is the recommended event-trace-port configuration.

```
event-trace-port=plio
```

- **graph-iterator-event:**

Generates `user_event()` whenever the graph iterator is incremented. This provides the ability to delay the start of the hardware event trace based on the graph iteration.

- **num-trace-streams:**

Specifies the number of trace streams. The default value is 16.

```
num-trace-streams=32
```

- **trace-plio-width:**

PLIO width for trace streams (default: 64)

Optimization Options

- **xlopt:**

Enable kernel optimizations based on the specified value:

- 0: No kernel optimizations.
- 1: Computation of heap size, generation of guidance based on LLVM IR analysis, and insertion of loop pragmas.
- 2: The same optimizations as 1 with the addition of loop peeling for unrolled loops, and automatic inlining.

```
xlopt=2
```

- **Xxloptstr:**

Option string to enable or disable optimizations in XLOpt level 1,2.

- -xinline-threshold=T: set inlining threshold to T (default T = 5000, only for optlevel=2).
- -annotate-pragma: insertion of loop unrolling, pipelining, and flattening pragmas (default = true)
- -align-global-array=0,1 if set to 1, global arrays are aligned on 2 times the width of the physical memory bank. This automatic alignment optimizes load/store performance. Load/Store ports bitwidth is twice the width of a physical bank and aligned on even index physical bank. This option automatically aligns the data to the even index bank address.

```
Xxloptstr=-annotate-pragma
```

Miscellaneous Options

- **--swfifo-threshold:** Above this specified threshold, a FIFO is implemented as a DMA FIFO in an AI Engine application, and generates an error in an AI Engine-ML application. To avoid an error message in AI Engine-ML, you can increase the threshold but this requires additional resources to route the design.
- **--evaluate-fifo-depth:**

This option analyzes re-convergent data paths. Data might be sent on multiple paths and sometimes they can re-converge which can result in a deadlock. Such deadlocks can be resolved by adding FIFOs to the appropriate data paths.

The steps for evaluating and resolving deadlocks as a result of re-convergent data paths is as follows:

1. Compile the design with this option.
2. Run `aiesimulator` on the design.

3. Open Vitis Unified IDE with the simulation `run_summary`
4. Note the FIFO depth displayed in the Estimated FIFO column in the Nets table. This FIFO depth value is the recommended value (in 32-bit words) to be used with the `fifo_depth(net)` constraint on a specific net connection in the graph to resolve the deadlock situation.
5. Apply the recommended FIFO depth value using the `fifo_depth` constraint on specified nets on the graph, and recompile the design. In AI Engine-ML if the recommendation is above the current fifo depth threshold, you might have to increase it using the `--swfifi-threshold` option.

- **disable-multirate:**

Disable multirate in ADF graphs. Accepted values are true and false. The default is false.

```
disable-multirate=true
```

- **no-init:**

This option disables initialization of window buffers in AI Engine data memory. This option enables faster loading of the binary images into the SystemC-RTL co-simulation framework. Accepted values are true and false. The default is false.

```
no-init=true
```

- **nodot-graph:**

By default, the AI Engine compiler produces DOT and PNG files to visualize the user-specified graph and its partitioning onto the AI Engines. This option can be used to eliminate the dot graph output. Accepted values are true and false. The default is false.

```
nodot-graph=true
```

Aiecompiler to Vitis Command Line Interface Migration

Using the v++ CLI

The AI Engine applications developed to compile using the `aiecompiler` command must be migrated to use the Vitis CLI (`v++ -c --mode aie`) to support AI Engine partitions and independent compilation and linking of ADF graphs. and this section describes the procedure to migrate using an example.

Consider the following `aiecompiler` command.

```
aiecompiler --include=<Vitis Installation Path>/20XX.X/aietools/include" --
stacksize=512 --include="./aie" --include="./data" --include="./aie/
kernels" --include="." --platform <PLATFORM_REPO_PATHS>/
xilinx_vck190_base_20XXX0_1/xilinx_vck190_base_20XXX0_1.xpfm --workdir=./
Work --target=hw aie/graph.cpp
```

You can convert the above command to use unified CLI as shown below.

```
v++ -c --mode aie --include=<Vitis Installation Path>/20XX.X/aietools/
include" --aie.stacksize=512 --include="./aie" --include="./data" --
include="./aie/kernels" --include="." --platform <PLATFORM_REPO_PATHS>/
xilinx_vck190_base_20XXX0_1/xilinx_vck190_base_20XXX0_1.xpfm --
aie.work_dir=./Work --target=hw aie/graph.cpp
```

Here:

- `v++ -c --mode aie` is the unified CLI compile option.
- Some generic options, such as `--include`, and `--platform`, are common across different `v++` compile modes (for example, `v++ -c --mode hls`).
- The option `-stacksize=512` is specific to the `aie` mode. So, in the unified CLI command you need to prefix the option with `-aie.` as in:

```
-aie.stacksize=512
```

Note: The `--aie.work_dir` option in the unified CLI command is the equivalent option to the `-workdir` in the `aiecompiler` command.

Using the `v++` Configuration File

Using the same example as the previous section, `aiecompiler --export-config=aiecompiler.cfg <Arguments to export>` can be used to generate a `aiecompiler.cfg` which can be used by the `v++` command. That file is passed to the `v++` command using `--config` option as shown below.

```
v++ -c --mode aie --config aiecompiler.cfg --target=hw aie/graph.cpp
```

A sample `aiecompiler.cfg` file is shown below. The first two commented lines show the corresponding `v++` and `aiecompiler` commands as reference.

```
# v++ command: v++ -c --mode aie --config aiecompiler.cfg aie/graph.cpp

# aiecompiler command: aiecompiler --include=<Vitis Installation Path>/
20XX.X/aietools/include --include=./aie --include=./data --include=./aie/
kernels --include=./ --target=hw --platform=<PLATFORM_REPO_PATHS>/
xilinx_vck190_base_20XXX0_1/xilinx_vck190_base_20XXX0_1.xpfm --
stacksize=512
--workdir=./Work aie/graph.cpp

# current directory: "<Current directory>"
include=<Vitis Installation Path>/20XX.X/aietools/include
include=./aie
```

```
include=./data
include=./aie/kernels
include=./
target=hw
platform=<PLATFORM_REPO_PATHS>/xilinx_vck190_base_20XXX0_1/
xilinx_vck190_base_20XXX0_1.xpfm

[aie]
stacksize=512
workdir=./Work
```

Note: The AI Engine specific options are added under the section [aie].

Macro for Device Architectures

Macro `__AIE_ARCH__`

AI Engine compiler supports predefined macro `__AIE_ARCH__` to differentiate different architectures:

- `__AIE_ARCH__=10`: AI Engine device has a predefined value of 10.
- `__AIE_ARCH__=20`: AI Engine-ML device has a predefined value of 20.

The macro can be used inside AI Engine kernel or AI Engine graph code. When designing for different architectures, the macro can be used to guard for specific architectures.

For example, in the kernel code, the macro is used to choose architecture specific API:

```
void fir24(input_buffer<__int16>,...> &iwin, output_buffer<__int16> &owin) {
    ...
    #if __AIE_ARCH__==10
        acc=aie::sliding_mul_sym<4,4>(...);
    #elif __AIE_ARCH__==20
        acc=aie::sliding_mul<4,8>(...);
    #endif
    ...
}
```

Following example shows how to use the macro in the graph code to choose buffer sizes based on a specific architecture:

```
//Using __AIE_ARCH__ macro to define buffer size for AI Engine device
#if __AIE_ARCH__==10
    static const int NUM=4096;
//Using __AIE_ARCH__ macro to define buffer size for AI Engine-ML device
#elif __AIE_ARCH__==20
    static const int NUM=8192;
//Covering rest of the cases - recommended for PS compilation
#else
    static const int NUM=4096;
#endif
```



```
using namespace adf;
class topgraph: public adf::graph
{
public:
    kernel k;

    topgraph(){
        .....
        k=kernel::create(mykernel);
        dimensions(k.in[0])={NUM};
        dimensions(k.out[0])={NUM};
    }
};
```

Note: If the graph code is included in the PS code compilation, you must add `-D__AIE_ARCH__=<NUM>` preprocessor definition to the Arm cross compiler to differentiate the devices.

AI Engine Compiler Guidance

After the AI Engine compiler completes the compilation of an AI Engine design it analyzes the design and provides guidance on how to improve the design based on AI Engine rules or best software practices. Some guidance might be corrected by AI Engine compiler automatically. The guidance file lists all the findings with severity, category with tile number, details, correction if done by the AI Engine compiler and suggested resolutions.

The guidance file, `guidance.html`, is located in the `Work/reports` directory. Use a web browser to review this guidance file. It is recommended that you update your design per the design guidance report before running the design in the simulator or on hardware.

The following are examples of the AI Engine compiler-generated guidance available in `Work/reports/guidance.html`.

- Variables are referenced before being initialized.

Id	Name	Severity	Impact	Full Name	Categories	Details	Resolution
1	AIE-MEM-04	advisory		Memory Management	Memory_25_0	Global variable 'state' reads its default/external initialization in kernel 'producer', and is not explicitly written before this read	Ensure that for all global variables, an initialization/write (default = zero) precedes the read. For more information, refer to UG1076.

- Global variables are initialized locally from a kernel implementation.

Id	Name	Severity	Impact	Full Name	Categories	Details	Resolution
1	AIE-MEM-04	advisory		Memory Management	Memory_24_0	Global variable 'state' reads its default/external initialization in kernel 'producer', and is not explicitly written before this read	Ensure that for all global variables, an initialization/write (default = zero) precedes the read. For more information, refer to UG1076.

- Array of data is not 128 bits (16 bytes) aligned.

Id	Name	Severity	Impact	Full Name	Categories	Details	Resolution
1	AIE-MEM-01	advisory		Memory Management	Memory_0_0	Alignment of global array lut1 is not 128 bits. Automatically correcting it.	Align global arrays to 16 byte boundary. Refer to the chess user manual for details on specifying the alignment constraints.

- Use of `__restrict` qualifier causes undefined behavior. This undefined behavior is exhibited running on hardware only.

Id	Name	Severity	Impact	Full Name	Categories	Details	Resolution
3	AIE-MEM-05	advisory		Memory Management	Memory_24_0	Store to restrict child pointer *_Z23equalizer_24tap_complexP12input_window6cint16EP13output_windowIS0_E::inputcb.addr must be used in a different block-level scope than the parent pointer	Ensure proper usage of restrict keyword. Refer to the formal definition of restrict in Section 6.7.3.1 of C99 Standard.
4	AIE-MEM-05	advisory		Memory Management	Memory_24_0	Store to restrict child pointer *_Z23equalizer_24tap_complexP12input_window6cint16EP13output_windowIS0_E::outputcb.addr must be used in a different block-level scope than the parent pointer	Ensure proper usage of restrict keyword. Refer to the formal definition of restrict in Section 6.7.3.1 of C99 Standard.

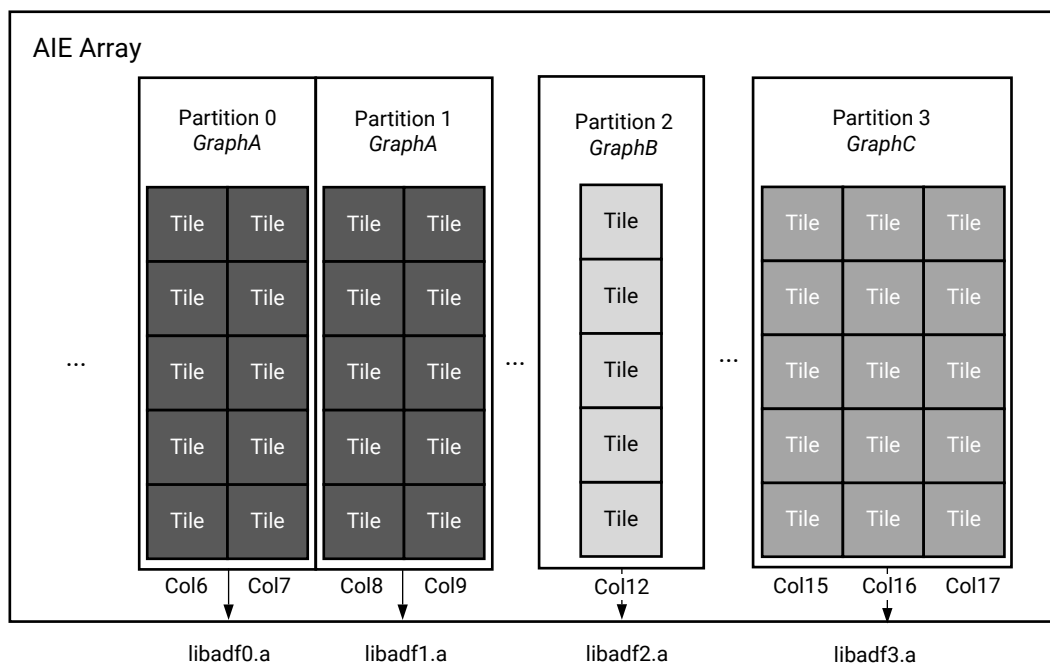
Note: The `--xlopt ≥ 1` option is required to compile a design that allows the AI Engine compiler to generate the appropriate guidance file.

Compiling AI Engine Graph for Independent Partitions

The AI Engine supports column-based partitions. This is enabled by creating independent graphs that can be compiled and simulated separately. Each graph is mapped to a specific column or a contiguous set of columns on the AI Engine array. During the `v++ link` stage, multiple partitions can be integrated together, provided there is no overlap in the resource usage by the graphs in those partitions.

The following picture shows an example of AI Engine with multiple partitions.

Figure 1: AI Engine Partitions



X30071-110624

A partition in the AI Engine array can be defined as

`<START_COLUMN>:<NUM_OF_COLUMN>:<PARTITION_NAME>`, and passed to the Vitis command line using `v++ -c -mode aie --config <CONFIG_FILE.cfg>`.

Example configuration (`<CONFIG_FILE.cfg>`):

```
[aie]
enable-partition=6:2:pr0
```

This configuration restricts the compiled graph to columns 6 and 7, and names the partition `pr0`.

Each partition is compiled independently. The same AI Engine graph can be used in different partitions. Each partition is compiled independently but with different locations inside the device. Multiple partition compilation outputs (for example, `libadf0.a`, `libadf1.a`...) are used at `v++ link` time.

Note: When multiple partitions overlap inside the device, `v++ link` will report error about the conflict.

The graphs are differentiated by their partition names and graph names in the host code. Absence of an explicit `enable-partition` option is interpreted as a single partition comprising the entire AI Engine array, which will overlap (i.e., conflict) with any other partitions at `v++ link` time.

The `v++ link` command can link multiple independent compilation results together; for example:

```
v++ -l --platform ${PLATFORM} -t hw --config system.cfg -o ${XSA} libadf0.a
libadf1.a libadf2.a libadf3.a
```

The graph ports in the graphs are prefixed with the partition name, for example (in the configuration file for `v++ linker`):

```
[connectivity]
stream_connect=ai_engine_0.pr0_Dataout0:s2mm_1.s
stream_connect=datagen.out:ai_engine_0.pr0_Datain0
```

Note: The port name `Dataout0` is renamed as `pr0_Dataout0`. Thus, there will be no conflicts with port names from other partitions.

The `v++ package` command is called to merge multiple compilation results together, such as:

```
v++ -p -t hw -f ${PLATFORM} \
--package.rootfs ${ROOTFS} \
--package.kernel_image ${IMAGE} \
--package.boot_mode=sd \
--package.image_format=ext4 \
--package.defer_aie_run \
--package.sd_file ${HOST_EXE} \
${XSA} \
libadf0.a libadf1.a libadf2.a libadf3.a
```

When specifying multiple `libadf.a` files for `v++ link` and `v++ package`, make sure that they are specified with the same order, because the order will affect the metadata generation and might impact the application execution.

For more information about how `v++` works, see the *Vitis Reference Guide* ([UG1702](#)).

The graph name used in the PS host code for XRT API is prefixed with the partition name, for example:

```
auto ghdl=xrt::graph(device,uuid,"pr0_gr");//partition name = "pr0", graph  
name = "gr"
```

For more information about how to use XRT API to control partitions in the host code, see [Controlling AI Engine Partitions](#).

Note: The Analysis view of the Vitis Unified IDE can open only one partition compilation result at a time. Event trace result can be analyzed in Vitis Unified IDE with only one partition at a time. The Bare-metal application can be built with one partition result at a time.

Simulating an AI Engine Graph Application

This chapter describes the various execution targets available to simulate AI Engine applications at different levels of abstraction, accuracy, and speed. AI Engine graphs can be simulated in four different simulation environments.

The x86 simulator is a fast-functional simulator as described in [x86 Functional Simulator](#). It should be used to functionally simulate your AI Engine graph and is useful for functional development and verification of kernels and graphs. It, however, does not provide timing, resource, or performance information.

The AI Engine simulator (`aiesimulator`) models the timing and resources of the AI Engine array, while using transaction-level SystemC models for the NoC, and DDR memory. This allows for quick performance analysis of your AI Engine applications and accurate estimation of the AI Engine resource use, with cycle-approximate timing information.

In both x86 functional simulator and AI Engine simulator, the PL interface to AI Engine can be exercised through untimed external traffic. Similarly, both use the user graph's `main()` function as C-test bench to configure and control the AI Engine. This test bench data input and output is not timed, and the user graph's `main()` function acts as a virtual simulation platform.



TIP: In addition to using the x86 functional simulator and AI Engine simulator, it is possible to test the AI Engine graph on a VCK190 board using the precompiled AI Engine test harness. This test harness is designed to provide a very simple and natural transition from the AI Engine simulation environment to testing on hardware. With the test harness, AI Engine graphs running in simulation can be taken to hardware in only a few minutes and with only a few minor modifications. More details about the AI Engine test harness can be found here: <https://xilinx.github.io/AI-Engine-Test-Harness/>

To simulate the entire system including AI Engine graph and PL logic along with XRT-based host application to control the AI Engine and PL, for a specific board and platform, use the Vitis hardware emulation flow. This flow includes the SystemC model of the AI Engine, transaction-level SystemC models for the NoC, DDR memory, PL Kernels (RTL), and PS (running on QEMU). Hardware emulation targets to perform C-RTL co-simulation of the full system.

The following table lists the four simulation flows and whether they support functional or performance level debug, and the level of support for source code debug. It also recommends the usage of these simulation flows in appropriate stages of your AI Engine design development.

Table 1: Simulation Flows

Design Development Stage	Simulation Tool Flow	Source Level Debug	Simulation Debug Type
AI Engine kernel and graph debug	x86 Simulator	Yes	AI Engine Functional Debug
AI Engine graph performance debug	AI Engine Simulator	Allows stepping through the AI Engine compiler generated assembly code which aids in code optimization, however, source level visibility could be limited due to compiler optimization	AI Engine Performance Analysis and Debug
System level simulation and performance debug	Vitis Hardware Emulation	Possible—however, provides limited source level visibility due to compiler optimization	System Performance Analysis and Debug

Simulation Models

The following table lists the simulation flows and recommends the usage of these simulation flows in appropriate stages of your AI Engine design development and the type of simulation model used for the various AMD Versal™ architecture domains. The type of simulation model and the simulation tool flow used determine the accuracy of the simulation results.

Table 2: Simulation Models

Simulation Tool Flow	AI Engine Kernels	PL Kernels	PL I/O Support	PL Platform	NoC/DDR Model	PS Model
x86 Simulator	x86 threads	N/A	File IOs/ External Traffic generators	N/A	N/A	N/A
AI Engine Simulator	SystemC	N/A	File IOs/ External Traffic generators	N/A	SystemC	N/A
Vitis Hardware Emulation	SystemC	SystemC/RTL/HLS-C	PL Kernels/ External Traffic generators	RTL/SystemC	SystemC	QEMU

Simulation Features

You can obtain profiling data when you run your design in `aiesimulator` or hardware emulation. Analyzing this data helps you gauge the efficiency of the kernels, the stall and active times associated with each AI Engine, and pinpoint the AI Engine kernel whose performance might not be optimal. This also allows you to collect data on design latency, throughput, and bandwidth. Details on running and analyzing profile data can be found in [Chapter 5: Performance Analysis of AI Engine Graph Application during Simulation](#).

The event trace feature allows you to capture and analyze a system-level view of program execution. It can be helpful in identifying problems during program execution including correctness and performance issues. The AI Engine architecture has direct support for generation, collection, and streaming of events as trace data during simulation and hardware emulation. Details on running and analyzing event trace data can be found in [Chapter 5: Performance Analysis of AI Engine Graph Application during Simulation](#).

The `x86simulator` and `aiesimulator` simulate the design with the `main()` function of `graph.cpp`. QEMU emulation support is available for host applications in hardware emulation. In hardware emulation, you can create a host application targeting Bare metal or Linux-XRT, and emulate the host application as well. Test bench data can be provided to the x86 simulator and AI Engine simulator via the graph's `main()` function which acts like a virtual test bench platform. There are various levels of test bench support available for the simulation flows. Test bench data can either be file based or via an external traffic generator. Additional details on this feature can be found in [Using Traffic Generators for AI Engine Designs](#) in *Embedded Design Development Using Vitis* (UG1701).

The data flowing between the AI Engine kernels is available and can be viewed either as data snapshot files or via the Vitis IDE. Additional details on the x86 simulator snapshot feature can be found in [Data Snapshots](#). Additional information on data visualization of trace data on the results of running the AI Engine simulator can be found in [Trace View Data Visualization](#).

The following table lists the type of simulation features supported with the four simulation flows.

Table 3: Simulation Features

Simulation Tool Flow	Trace	Profile	Application main()	Test Bench Support	AI Engine Data Flow Visibility
x86 Simulator	No	No	The main() function in <code>graph.cpp</code>	File-based/ External traffic generators	Yes (via the snapshot feature)
AI Engine Simulator	Yes	Yes	The main() function in <code>graph.cpp</code>	File-based/ External traffic generators	Yes (via Trace view in the Vitis IDE)
Vitis Hardware Emulation	Yes	Yes	The main() function in host application targeting BareMetal or Linux-XRT	HLS-C/RTL datamovers/ external traffic generators	Yes (via Trace view in the Vitis IDE)

Note: The `main` function for simulation must have a return value. Otherwise, `v++ -c --mode aie` will error out.

x86 Functional Simulator

When starting development on AI Engine graphs and kernels, it is critical to verify the design behavior. This is called functional simulation and it is useful in identifying errors in the design. For example, window sizes in the graph are related to the number of iterations in a kernel. Troubleshooting to see if every kernel in a large graph is absorbing and generating the right number of samples can be time consuming and iterative in nature. The x86 simulator is an ideal choice for testing, debugging, and verifying this kind of behavior because of the speed of iteration and the high level of data visibility it provides the developer. The x86 simulation does not provide timing, resource, or performance information. The x86 simulator is running exclusively on the tool development machine. This means its performance and memory use are defined by the development machine.

While the AI Engine simulator fully models the memory of the graphs and kernels, this also means that the AI Engine simulator is limited by the memory space of the AI Engine. The x86 simulator is not limited by this and provides a nearly unlimited amount of debug `printf()`s, large arrays, and variables. When combined with the ability to single step through the kernel, very complex design issues can quickly be isolated and addressed.

Several macros are provided to improve the kernel debug. These macros are intended for use with the x86 simulator and can be left in the code for maintainability purposes. With the benefits of the x86 simulator come some trade offs. There are several types of graph constructs that are not supported and cycle accurate behavior cannot be guaranteed to match between the AI Engine simulator and the x86 simulator. The x86 simulator is not a replacement for the AI Engine simulator. The AI Engine simulator must still be run on all designs to verify behavior and obtain performance information. For different stages of project development one tool or the other might better suit your needs.

Note: To learn more about x86 simulation limitations, refer to [Limitations](#).

To run the x86 simulator, change the AI Engine compiler target to `x86sim`.

```
v++ -c --mode aie --target=x86sim graph.cpp
```

After the application is compiled for x86 simulation, the x86 simulator can be invoked as follows.

```
x86simulator
```

To see the available x86simulator options, there is a help dialog displayed with `x86simulator --help`.

The compiled binary for x86 native simulation is produced by the AI Engine compiler under the `Work` directory (see [Chapter 3: Compiling an AI Engine Graph Application](#)) and is started automatically by the x86 simulator.

The input and the output files are specified in the following snippet of graph code.

```
adf::input_plio in1=adf::input_plio::create( "In", adf::plio_32_bits,
    "In1.txt");
adf::output_plio out1=adf::output_plio::create( "Out", adf::plio_32_bits,
    "Out1.txt");
```

When running, the x86 simulator looks in the current working directory for `In1.txt` which is one of the inputs used by the ADF graph. To distinguish the output files for the x86 simulator from the output files for the AI Engine graphs and kernels, it is critical to verify the design simulator the `Out1.txt` are located in `current_working_dir/x86simulator_output/data/`.

If a comma separated value (CSV) input file is used, the input filename is replaced with `In1.csv`.

Optionally, the output files produced by the simulator can be compared with a golden output ignoring white space differences.

```
diff -w <data>/golden.txt <data>/output.txt
```

Simulator Logs and Output can be visualized in the Vitis IDE:

Figure 2: Log View

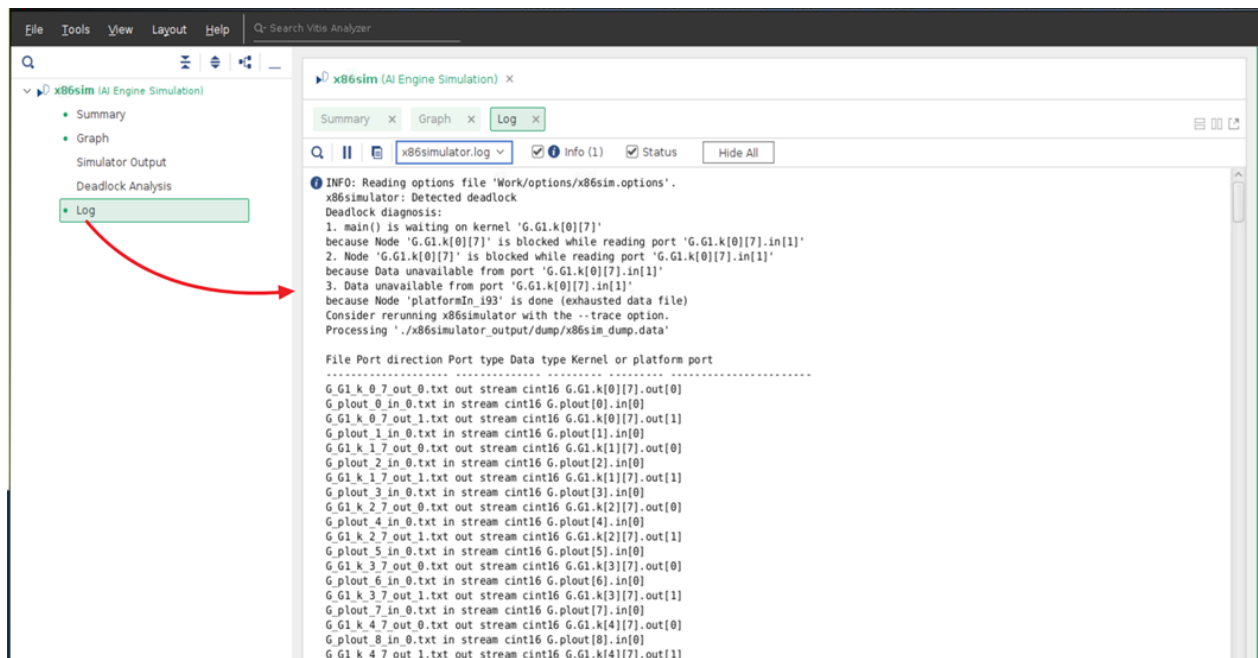
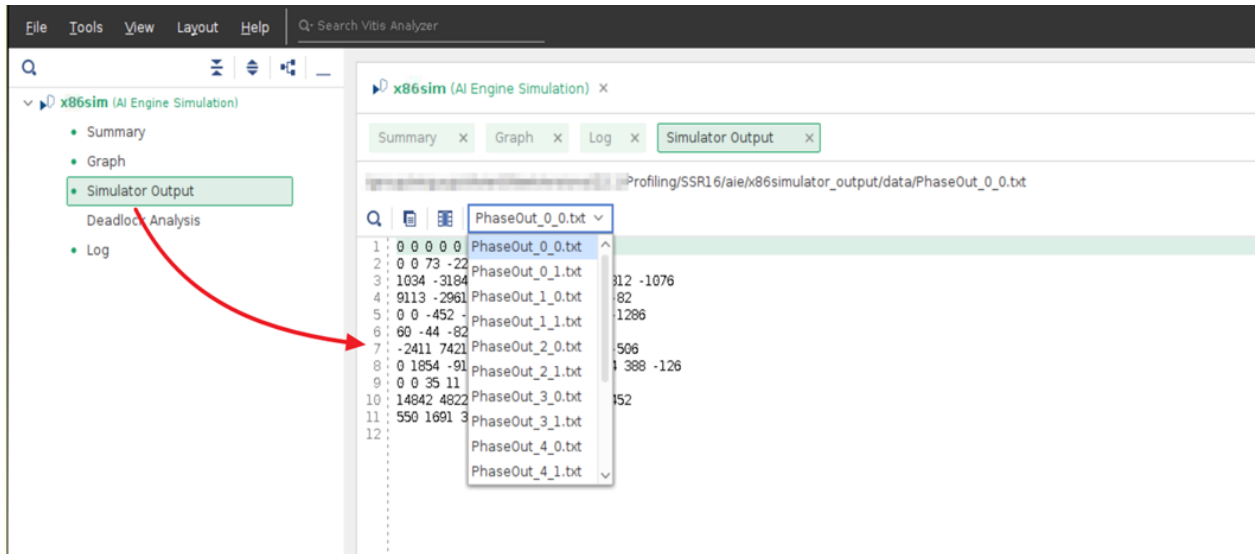


Figure 3: Simulator Output View



The Simulator Output View allows you to select which output should be displayed.

Compiling the Design

The GNU debugger allows for C/C++ debugging similar to an IDE based debugger. It allows setting of breakpoints, single stepping, stepping over functions, and multiple hit counts on breakpoints. For AI Engine kernel development the x86 simulator enables single step debugging of kernel code using GDB.

The `target` argument for the AI Engine compiler must be set to `x86sim` to use GDB.

```
v++ -c --mode aie --target=x86sim graph.cpp
```

Simulator Options

The complete set of the x86 simulator (`x86simulator`) options are described in this section.

Table 4: x86 Simulator Options

Option	Description
<code>-h, --help --h</code>	Shows the help message.
<code>--pkg-dir=PKG_DIR</code>	Sets the package directory, for example: <code>./Work</code>
<code>--i, -i, --input-dir=PATH</code>	Sets the input directory.
<code>--o, -o, --output-dir=PATH</code>	Sets the output directory.
<code>--timeout=secs</code>	Terminates simulation after a specified number of seconds immediately after the program begins. Seconds refers to the wall clock time.

Table 4: x86 Simulator Options (cont'd)

Option	Description
<code>--dump</code>	Generates snapshots of the data traffic on kernel ports. A snapshot file is generated for each port on a kernel. For more information, see Data Snapshots .
<code>--dump-size=SIZE</code>	Sets the maximum size of the snapshot file in MB. The snapshot file is generated by the <code>--dump</code> option. Note: When the snapshot file exceeds the specified maximum size, any additional writes to this file are ignored and the following warning is shown: WARNING: Snapshot File will exceed its maximum size, further writes to this file will be ignored.
<code>--gdb</code>	Invokes from GDB.
<code>--valgrind</code>	Runs the simulator under Valgrind to detect memory access violations.
<code>--valgrind-gdb</code>	Runs the simulator under Valgrind and debug via GDB server.
<code>--valgrind-args=ARGS</code>	Overrides the default options for Valgrind. Used with <code>--valgrind</code> .
<code>--disable-stop-on-deadlock</code>	By default the simulator is monitoring the simulation and stops the simulation when deadlock is detected. This option disables this deadlock monitoring process.
<code>--trace</code>	Enables trace of kernel stall events.
<code>--trace-print</code>	Prints kernel stall events during simulation.

Data Snapshots

The `x86simulator --dump` data snapshot feature allows users to dump and inspect data traffic at kernel ports without using the debugger. This feature does not require any instrumentation of kernel code. It also helps diagnose simulation result mismatches between two versions of the same design with `x86simulator`. Data snapshots are supported for kernel ports including streams, windows, packet streams, cascade streams, and RTP ports. This feature is also supported for all platform level ports like PLIO, GMIO, RTP ports, as well as for PS accesses into a specific RTP port.

For input window ports, a data snapshot is taken each time the kernel acquires the window and the snapshot includes the margin. For output window ports, a snapshot is taken each time the kernel releases the window and it excludes the margin. In either case the snapshot consists of the kernel iteration count and sample data. One text file per kernel port is generated. In addition, the iteration count of the kernel is also recorded.

For example, a data snapshot for a graph with a chain of two kernels with window ports where the first window has a non-zero margin is as follows.

```
$x86simulator --pkg-dir=./Work --i=.. --dump
INFO: Reading options file './Work/options/x86sim.options'.
Processing './x86simulator_output/dump/x86sim_dump.data'
File Port direction Port type Data type Kernel or platform port
-----
platform_src_0.txt out window cint16 platform.src[0]
```

```
mygraph_first_in_0.txt in window cint16 mygraph.first.in[0]
mygraph_second_out_0.txt out window cint16 mygraph.second.out[0]
platform_sink_0.txt in window cint16 platform.sink[0]
mygraph_first_out_0.txt out window cint16 mygraph.first.out[0]
mygraph_second_in_0.txt in window cint16 mygraph.second.in[0]
Wrote './x86simulator_output/dump/platform_src_0.txt'
Wrote './x86simulator_output/dump/mygraph_first_in_0.txt'
Wrote './x86simulator_output/dump/mygraph_second_out_0.txt'
Wrote './x86simulator_output/dump/platform_sink_0.txt'
Wrote './x86simulator_output/dump/mygraph_first_out_0.txt'
Wrote './x86simulator_output/dump/mygraph_second_in_0.txt'
Simulation completed successfully returning zero
$> more ./x86simulator_output/dump/mygraph_first_in_0.txt
# port: mygraph.first.in[0]
# port_dir: in
# port_type: window
# data_type: cint16
# Iteration 1; snapshot 1
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 1
2 3
4 5
6 7
8 9
10 11
12 13
14 15
16 17
18 19
20 21
22 23
24 25
26 27
28 29
30 31
32 33
34 35
36 37
38 39
40 41
42 43
44 45
46 47
48 49
50 51
52 53
54 55
# Iteration 2; snapshot 2
48 49
50 51
52 53
54 55
```

```

56 57
58 59
60 61
62 63
1 0
3 2
5 4
7 6
9 8
11 10
13 12
15 14
17 16
19 18
21 20
23 22
25 24
27 26
29 28
31 30
33 32
35 34
37 36
39 38
41 40
43 42
45 44
47 46
49 48
51 50
53 52
55 54
57 56
59 58
61 60
63 62

```

For stream ports, a data snapshot is taken for each chunk of four bytes of data. Each snapshot includes the value of TLAST and the kernel iteration count. For cascade stream ports, the granularity is eight bytes.

For example, a data snapshot for a graph with three kernels connected via stream ports is as follows.

```

$ x86simulator --pkg-dir=./Work --i=.. --dump
INFO: Reading options file './Work/options/x86sim.options'.
Processing './x86simulator_output/dump/x86sim_dump.data'

```

```

File Port direction Port type Data type Kernel or platform port
-----
platform_src_0.txt out stream int32 platform.src[0]
fifo_graph_dist_in_0.txt in stream int32 fifo_graph.dist.in[0]
fifo_graph_aggr_out_0.txt out stream int32 fifo_graph.aggr.out[0]
platform_sink_0.txt in stream int32 platform.sink[0]
fifo_graph_comp0_in_0.txt in stream int32 fifo_graph.comp0.in[0]
fifo_graph_comp1_in_0.txt in stream int32 fifo_graph.comp1.in[0]
fifo_graph_dist_out_0.txt out stream int32 fifo_graph.dist.out[0]
fifo_graph_comp0_out_0.txt out stream int32 fifo_graph.comp0.out[0]
fifo_graph_aggr_in_0.txt in stream int32 fifo_graph.aggr.in[0]

```

```
fifo_graph_comp1_out_0.txt out stream int32 fifo_graph.comp1.out[0]
fifo_graph_aggr_in_1.txt in stream int32 fifo_graph.aggr.in[1]
```

```
Wrote './x86simulator_output/dump/platform_src_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_dist_in_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_aggr_out_0.txt'
Wrote './x86simulator_output/dump/platform_sink_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_comp0_in_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_comp1_in_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_dist_out_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_comp0_out_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_aggr_in_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_comp1_out_0.txt'
Wrote './x86simulator_output/dump/fifo_graph_aggr_in_1.txt'
Simulation completed successfully returning zero
```

```
$ more ./x86simulator_output/dump/fifo_graph_dist_in_0.txt
# port: fifo_graph.dist.in[0]
# port_dir: in
# port_type: stream
# data_type: int32
# Iteration 1
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
...
```

The compiler generates the configuration file `./Work/options/x86sim_dump.config` to specify what data to dump when running `x86simulator --dump`. Following is an example of compiler generated `./Work/options/x86sim_dump.config`:

```
kernel=yes
gr.k[0].in[0]=yes
gr.k[0].out[0]=yes
gr.k[0].out[1]=yes
gr.k[1].in[0]=yes
gr.k[1].in[1]=yes
gr.k[1].out[0]=yes

platform_io=no
gr.in.out[0]=no
gr.dataout.in[0]=no
```

Following categories based on the design are supported in the configuration file:

Table 5: X86simulator Dump Configurations

No.	Category	Default Dump Configuration	Nodes / Ports under the Category
1	kernel	yes	Input/Output ports of all the kernels
2	shared_buffer	yes	Shared buffer as a whole
3	external_buffer	yes	External buffer as a whole
4	shared_buffer_port	no	Ports of all the shared buffers
5	external_buffer_port	no	Ports of all the external buffers
6	platform_io	no	PLIO, GMIO, FileIO, PS:RTP
7	pktmerge	no	Pkt merge ports
8	pktsplit	no	Pkt split ports
9	bypass	no	Bypass port

If the dump configuration is set to `yes`, all entries under the category are dumped, irrespective of their control value.

If the dump configuration is set to `no`, partial entries under the category can be selected to be dumped to save disk space and running time. For example, following configurations will only to dump `gr.k[0].in[0]`:

```
kernel=no
gr.k[0].in[0]=yes
gr.k[0].out[0]=no
gr.k[0].out[1]=no
gr.k[1].in[0]=no
gr.k[1].in[1]=no
gr.k[1].out[0]=no
```

If no dump configuration is set to a category, the value of the previous category will be inherited. For example, the following entries `g.ifm1` and `g.wts1` are under the category `shared_buffer`. However, the dump configuration for `shared_buffer` is not specified. So, the value of the previous category `kernel` which is `yes` is inherited. Then, the entries `g.ifm1` and `g.wts1` are all dumped.

```
kernels=yes
g.k1.in[0]=yes
g.k1.in[1]=yes
g.k1.out[0]=yes

g.ifm1=yes
g.wts1=no
```

Deadlock Detection

AI Engine users can run into simulator hangs. A common cause is insufficient input data for the requested number of graph iterations, mismatch between production and consumption of stream data, cyclic dependency with stream, cascade stream or asynchronous windows, or wrong order of blocking protocol calls (acquisition of async window, read/write from streams).

The x86 simulator will automatically detect deadlock, unless the `--disable-stop-on-deadlock` option is specified. If a deadlock is detected, simulation is stopped, and a deadlock diagnosis report is printed.

Additionally, a graph is generated in `x86simulator_output/simulator_state_post_analysis.dot`. This is a `.dot` file that encodes a description of the graph in terms of a block diagram where the agents involved in the deadlock are highlighted in red. To get this file transformed into a `.png` file, you must use the `dot` program as follows.

```
dot -Tpng x86simulator_output/simulator_state_post_analysis.dot >
simulator_state_post_analysis.png
```

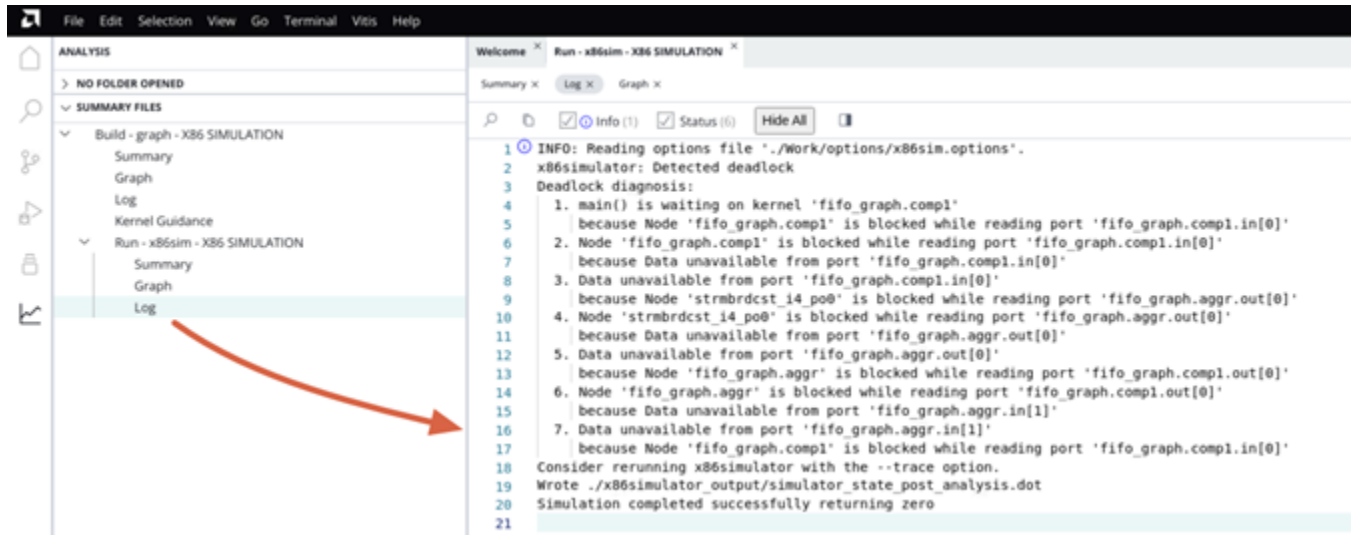


IMPORTANT! Absence of deadlock for x86 simulation does not mean absence of deadlock in SystemC simulation. x86 simulation does not model timing and resource constraints and thus there are fewer causes of deadlock. On the other hand, if x86 simulation deadlocks, SystemC simulation deadlocks as well, so it is beneficial to fix the deadlock in x86 simulation before proceeding with SystemC simulation.

Note: The `--disable-stop-on-deadlock` option is not supported for use cases with an external test bench.

The textual description of the deadlock can be visualized in the Vitis IDE:

Figure 4: Deadlock Diagnosis in the Analysis View



Trace Report

Trace capability is used for debugging simulation hangs without the need for instrumenting kernel code or using `gdb`. Trace capability is not available with external test bench. There are two use models.

The first use model applies to standalone x86 simulation without external test bench. At the time that the x86 simulator detects deadlock, it prints a message indicating that the design is deadlocked and the simulation will be terminated and suggests to rerun the simulation with the trace option enabled. When you re-run the simulation again and the trace option is enabled, the simulator produces a textual report on the events that occurred during the simulation. Studying this report can help you identify the root cause of the deadlock. The root cause can be as simple as insufficient input data in a data file of the simulation platform, or it can be more involved.

A second use model applies to simulations where `--trace` and `--timeout=secs` options are enabled. After the timeout expires, the simulator terminates and produces the trace report. As in the first use model, analysis of the trace report can help identify the root cause of the deadlock.

The following options are available:

- `--trace` to get a full trace report at the end of the simulation.
- `--trace-print` to get output displayed on the console while the simulation is running.

Trace Report Content

The trace report shows the sequence of events that occurred during the simulation of your design in the x86 simulator. The name of the generated file is `x86simulator_output/trace/x86sim_event_trace.data.txt`. The kind of events that are recorded include:

- The start of a kernel iteration
- The end of a kernel iteration
- The start of a stream stall, i.e., the start of a read from a stream port of a kernel that blocks because of lack of data
- The end of a stream stall, i.e., the point where a read from a stream port that initially blocked, finally returns
- The start of a lock stall, i.e., the start of an attempt to acquire a window port where the lock is initially not available
- The end of a lock stall, i.e., the point in time where an attempt to acquire a window port that initially blocked, finally returns

Output of `--trace-print` versus `--trace`

The output of `--trace-print` is not as polished as the file generated by `--trace`. If you want to quickly see what is going on in the simulation, or if you are planning to terminate the simulation via `CTRL-C` instead of `--timeout=secs`, use `--trace-print`.

The columns of the output of `--trace-print` contain the following information:

- Timestamp: It is the same as in `x86sim_event_trace.data.txt`.
- Internal name of the kernel (`x86sim_event_trace.data.txt` uses the user name).
- Event type.
- Numeric value whose meaning depends on the event type: It encodes the port that you are waiting on for a lock or stream stall. It encodes the iteration number of a start of an iteration event.

Memory Access Violations and Valgrind

Memory access violations occur when a kernel is reading or writing out of bounds of an object or reading uninitialized memory. This can manifest itself in multiple ways like a simulator crash or hang. It can also cause simulator results to be non-repeatable. The `x86simulator --valgrind` option will find memory access violations in kernel source code.

Note: Valgrind needs to be installed for this feature to work. AMD recommends using Valgrind version 3.16.1 and above.

This option allows detection of memory access violations in kernel source code during x86 simulation with Valgrind. The following kinds of access violations can be detected.

- Out-of-bounds write
- Out-of-bounds read
- Read of uninitialized memory

There are two ways to use this option:

- `x86simulator --valgrind`: This runs the simulation with access violation detection turned on. At the end of the simulation, Valgrind prints a report on access violations. If there are none, the report ends with *ERROR SUMMARY: 0 errors from 0 contexts*. Otherwise, the report lists each access violation found. This includes a stack trace, which highlights the line number in the kernel source code where the access violations occurred.
- `x86simulator --valgrind-gdb`: This runs the simulation with access violation detection turned on and debug with GDB. The simulation comes up in GDB and is halted at `main()`. At this point you can set additional breakpoints. After continuing, the simulation will stop if an access violation is detected. At this point you can inspect local variables and the stack to diagnose the problem.

In either case, some arguments can be added to the Valgrind command using `flag --valgrind-args='list of arguments for valgrind'`.

In the following example, arguments are used to specify not to track memory leakage, and to display the overall stack when discovering an access violation. The following command is used:

```
--valgrind-args='-v --leak-check=no --track-origins=yes'
```

The snippet below shows the simulation running this command.

Figure 5: Simulation with Valgrind Arguments

```
x86simulator --pkg-dir=Work --valgrind-gdb --valgrind-args='-v --leak-check=no --track-origins=yes'
==1021329== Memcheck, a memory error detector
==1021329== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1021329== Using Valgrind-3.16.1 and LibVEX; rerun with -h for copyright info
==1021329== Command: Work/pthread/sim.out
==1021329==
==1021329== (action at startup) vgdb me ...
==1021329==
==1021329== TO DEBUG THIS PROCESS USING GDB: start GDB like this
==1021329==   /path/to/gdb Work/pthread/sim.out
==1021329== and then give GDB the following command
==1021329==   target remote | /tools/baton/valgrind/3.16.1/lib/valgrind/../../bin/vgdb --pid=1021329
==1021329== --pid is optional if only one valgrind process is running
==1021329==
```

Note: Running the `x86simulator` with the Valgrind option will increase the simulation run time.

Using GDB

After successful compilation with the appropriate target you can launch the simulation and automatically attach a GDB instance to it. To launch an interactive GDB session run the command with the switch `--gdb` as follows.

```
x86simulator --gdb
```

By default, when running the x86 simulator with the `--gdb` command line switch it breaks immediately before entering `main()` in `graph.cpp`. This pauses execution before any AI Engine kernels have started because the graph has not been run. To exit GDB type `quit` or `help` for more commands.

Setting a breakpoint can be done in multiple ways. One method uses the following syntax:

```
break <kernel_function_name>
```

By typing `continue` (shorthand `c`) the debugger runs until the breakpoint in `<kernel function name>` is reached. When the breakpoint is reached it is possible to examine local stack variables and function arguments. The following table shows some commonly used GDB instructions to allow examination of these variables.

Table 6: Common GDB Instructions

Command	Description
<code>info stack</code>	Reports the function call stack at the current breakpoint.
<code>info locals</code>	Shows the current status of local variables within the scope of the function call shown in the call stack.
<code>print <local_variable_name></code>	Prints the current value of a single variable.
<code>finish</code>	Exits the current function call but keeps the simulation paused.
<code>continue</code>	Causes the debugger to run to completion.

GDB is a very powerful debugger with many features. Complete documentation of GDB is beyond the scope of this document. For GDB documentation, see <https://sourceware.org/gdb/current/onlinedocs/gdb.pdf> and <https://sourceware.org/gdb/current/onlinedocs/refcard.pdf>.

Macros

AMD provides several predefined compiler macros to help using the x86 simulator. In your top level graph test bench (usually called `graph.cpp`) it can be useful to use the following pre-processor macros with a conditional `#if` to help include or exclude appropriate code.

Table 7: Macros

Macro	Description
<code>__X86SIM__</code>	Use this predefined macro to designate code that is applicable only for the <code>x86sim</code> flow.
<code>__AIESIM__</code>	Use this predefined macro to designate code that is applicable only for the <code>aiesimulator</code> flow.
<code>X86SIM_KERNEL_NAME</code>	Use this macro with <code>printf()</code> to tag instrumentation text with the kernel instance name.

Note: For macros surrounded with underscores `_` there are two underscore characters at the front and behind.

The following is an example of the macro code.

```
myAIEgraph g;
#ifdef __AIESIM__ || defined(__X86SIM__)
int main()
{
    g.init();
    g.run(4);
    g.end();
    return 0;
}
#endif
```



TIP: The `__AIESIM__` macro is used in the AI Engine simulator only and `__X86SIM__` is applicable for the x86 simulator.

The previous example shows the `__X86SIM__` macro surrounding the `main()` which is used by a `graph.cpp` file. This `main()` must be excluded from emulation flows and these macros provide that flexibility. Additionally, consider using the `__X86SIM__` macro to selectively enable debug instrumentation only during x86 simulation.

Printf() Macros

The x86 simulator executes multiple kernels in parallel on separate threads. This means that `printf()` debug messages can often be interleaved and non-deterministic when viewed in standard output. To help identify which kernel is printing which line the `X86SIM_KERNEL_NAME` macro can be useful. `__FILE__` is a preprocessor macro that expands to full path to the current file. The following is an example showing how to combine it with `printf()`.

Note: To use `X86SIM_KERNEL_NAME` you must include `adf/x86sim/x86simDebug.h` as shown in the following code.

```
#include <adf/x86sim/x86simDebug.h>

void simple(input_window_float * in, output_window_float * out) {
    for (unsigned i=0; i<NUM_SAMPLES;i++) {
        float val = window_readincr(in);
        window_writeincr(out, val+0.15);
    }
    static int count = 0;
    printf( "%s: %s %d\n", __FILE__, X86SIM_KERNEL_NAME, ++count );
}
```

Using printf() in Kernels

Vector data types are the most commonly used types in AI Engine kernel code. To debug vector operations within a kernel it is helpful to use `printf`.

Using printf() with Vector Data Types

To `printf()` the native vector types you can use a technique as follows.

```
v4cint16 input_vector;
...
int16_t* print_ptr = (int16_t*)&input_vector;
for (int qq=0; qq<4;qq++) //4 here so we print two int16s, real + imag
per loop.
    printf( "vector re: %d, im: %d\r\n", print_ptr[2*qq], print_ptr[2*qq+1]);
}
```

With the AI Engine simulator the `--profile` option is required in order to observe `printf()` outputs. With the x86 simulator no additional options are needed to enable `printf` calls. This is one of the benefits of the x86 simulator.



IMPORTANT! AMD recommends avoiding `std::cout` in kernel and host code. If `std::cout` is used its outputs can appear interleaved, given the multi-threaded nature of the x86 simulator. Using `printf()` is recommended instead.

Limitations

The x86 simulator uses a simplified model of the AI Engine architecture. The placement and routing of the various kernels of the AI Engine array is a simplified model in this x86 simulator. The routing delays and FIFO lengths are not modeled in the x86 simulation.

x86 simulations are composed of multiple threads sharing the x86 processor. As a result, memory access conflicts, and stream access conflicts are not modeled.

When targeting the x86 simulator, the AI Engine compiler translates all vector processor specific instructions into a series of x86 instructions. Therefore, the x86 simulator will not estimate any design throughput. Furthermore, software pipelining is not modeled in the x86 simulator.

For information about how to handle the same table definition in multiple kernels, refer to the following.

- For AI Engine, see [Shared Graphed-Scoped Tables](#) in the *AI Engine Kernel and Graph Programming Guide* (UG1079).
- For AI Engine-ML, see [Shared Graphed-Scoped Tables](#) in the *AI Engine-ML Kernel and Graph Programming Guide* (UG1603).

Memory Model

Kernels that require retaining state from one invocation (iteration) to the next can use global or static variables to store this state. Variables with static storage class, such as global variables and static variables are a cause of discrepancies between x86 simulation and AI Engine simulation. The root cause is that for x86 simulation, the source files of all kernels are compiled into a single executable, whereas for AI Engine simulation each kernel targeting an AI Engine is compiled

independently. Thus, if a variable with static storage class is referred to by two kernels and those kernels are mapped to the same AI Engine, the variable is shared for both x86 simulation and AI Engine simulation. However, if those kernels are mapped to different AI Engines, then the variable is still shared for x86 simulation, but for AI Engine simulation each AI Engine has its own copy and there is no sharing. This leads to mismatches between x86 simulation and AI Engine simulation if the variable is both read and written to by the kernels.

The preferred way of modeling state to be carried across kernel iterations is to use a C++ kernel class (see [C++ Kernel Class Support](#) in *AI Engine Kernel and Graph Programming Guide (UG1079)*). This avoids the pitfall of variables with static storage class. Alternatively the storage class of the global or static variable can be changed to *thread_local*, but just for x86 simulation. In this case, each instance of the kernel has its own copy of the variable in x86 simulation. This matches the behavior of AI Engine simulation if using the variable are mapped to different AI Engines. In the following example, the kernel carries the state across kernel iteration via the global variable *delayLine* and the static variable *pos*. This causes mismatches between x86 simulation and AI Engine simulation if there are multiple kernel instances using this source file. This can be avoided by changing the storage class of these variables to *thread_local*.

Original kernel source code:

```
// fir.cpp
#include <adf.h>
using namespace adf;
cint16 delayLine[16] = {};
void fir(input_buffer<cint16> *in1,
        output_buffer<cint16> *out1)
{
    static int pos = 0;
    ..
}
```

Reworked kernel source code:

```
// fir.cpp
#include <adf.h>

#ifdef __X86SIM__
cint16 delayLine[16] = {};
#else
thread_local cint16 delayLine[16] = {};
#endif
void fir(input_buffer<cint16> *in1,
        output_buffer<cint16> *out1)
{
    using namespace adf;
#ifdef __X86SIM__
    static int pos = 0;
#else
    static thread_local int pos = 0;
#endif
    ..
}
```

Another possibility is to use the macro `X86SIM_THREAD_LOCAL` to make the global read/write thread safe. The macro is defined in `adf.h` as follows:

```
#ifndef _X86SIM_
#define X86SIM_THREAD_LOCAL thread_local
#else
#define X86SIM_THREAD_LOCAL
#endif
```

which makes it defined only for X86 simulations.

The previous code is simplified as follows:

```
// fir.cpp
#include <adf.h>

X86SIM_THREAD_LOCAL cint16 delayLine[16] = {};

void fir(input_buffer<cint16> *in1,
        output_buffer<cint16> *out1)
{
    using namespace adf;
    static X86SIM_THREAD_LOCAL int pos = 0;
    ..
}
```

Graph API Calls

Graph constructs for timed execution (`graph.wait(N)`, `graph.resume(N)` and `graph.end(N)`) behave differently in the AI Engine simulator and the x86 simulator. For the AI Engine simulator `N` specifies the processor cycle timeout, whereas for the x86 simulator it specifies a wall clock timeout in milliseconds. Thus, if your test bench uses timed execution the AI Engine simulator and x86 simulator might produce different results, in particular, the amount of output data produced might differ.

Preprocessor Considerations

When running the AI Engine compiler with a target of `x86sim` the compiler ignores the `--Xchess` option. This means that the `x86sim` flow does not support kernel-specific compile options.

To understand this better consider the following example. A common method of making compile-time modifications to C code is using preprocessor directives such as `#ifndef`. To control these preprocessor directives it is helpful to pass `#defines` through the command line compiler options. The following example code block takes two different actions based on a preprocessor directive.

```
void example_kernel()
{
    #ifndef SIM
        printf( "Simulation Mode\n" );
    #else
        printf( "Default Mode\n" );
    #endif
}
```

To define the SIM macro at compile time with the AI Engine compiler targeting hardware (hw) you can do the following.

```
v++ -c --mode aie -target=hw -Xchess="example_kernel:-DSIM"
```

Because the `-Xchess` argument is ignored when the compilation target is set to `x86sim`, SIM is not defined for the x86 simulator case and the output of the kernel is Default Mode.

If you need to specify preprocessor options with the x86 simulator you can do so using `v++ -c --mode aie -target=x86sim --Xpreproc` instead of `-Xchess`. Any options passed in this manner applies to all source code and all target flows.

Table 8: AI Engine Compiler Command Line Options

Option	Description
<code>--Xchess=<string></code>	Can be used to pass kernel specific options to the CHES compiler that is used to compile code for each AI Engine. The option string is specified as <code><kernel-function>:<optionid>=<value></code> . This option string is included during compilation of generated source files on the AI Engine where the specified kernel function is mapped.
<code>--Xpreproc=<string></code>	Pass general option to the PREPROCESSOR phase for all source code compilations (AIE/PS/PL/x86sim). For example: <code>--Xpreproc=-D<var>=<value></code>

Packet Switching and RTP

The x86 simulator and AI Engine simulator have some important differences when it comes to packet switching and runtime parameters (RTP). See [Run-Time Parameter Specification](#) and [Explicit Packet Switching](#) in *AI Engine Kernel and Graph Programming Guide (UG1079)*, if you are not familiar with these constructs.

Both packet switching and RTPs exhibit behavior that can manifest itself as a difference between the AI Engine simulator and the x86 simulator. It is important to understand that this dissimilarity is still correct in both cases. As discussed in [Explicit Packet Switching](#) in *AI Engine Kernel and Graph Programming Guide* (UG1079), packet switched streams are non-deterministic in all design flows (x86 simulator, AI Engine simulator, and hardware emulation), but proper use of packet headers in the kernels associated with those streams can make the compute order deterministic.

Packet Switching

Packet Stream connections have a field known as the packet ID. If the source of the packet ID field comes from within the ADF graph, the x86 simulator uses the canonical, zero-based indexing scheme for packet IDs. The first branch on the output of a split node has a packet ID equal to 0 followed by 1, 2, 3 and so on up to 31 (5-bit Packet ID). If the source of the packet ID field comes from outside the ADF graph there will be discrepancies between the AI Engine simulator and the x86 simulator. To resolve these discrepancies and for additional information on providing custom packet IDs when the source is outside the ADF graph, see [Packet Switching and the AI Engine Simulator](#) in *AI Engine Kernel and Graph Programming Guide* (UG1079).

The nature of packet merging means that the x86 simulator and the AI Engine simulator produce non-deterministic results. If an AI Engine on a packet split branch finishes processing data before any of the other branches its data appears on the output of the packet merge first. Exactly which core finishes first is highly dependent on both the kernel code and the incoming data. So any downstream processing blocks must be prepared to handle this behavior.

Synchronous and Asynchronous RTP

Both synchronous and asynchronous run-time parameters are fully supported in the x86 simulator. However the precise timing and cycle accuracy of when an RTP update occurs differs between the x86 simulator and the AI Engine simulator. Asynchronous RTPs in particular do not affect a kernel on a specific known cycle by their very nature of being asynchronous. This is true of asynchronous RTPs in both the x86 simulator and the AI Engine simulator.

Note: The x86 simulator by its nature is a functional level simulator whereas the AI Engine simulator models cycles (approximately). It is expected there will be differences. AMD recommends you consider partitioning your design into pieces that benefit from the x86 simulator.

Best Practices

The x86 simulator works best with the following recommendations.

- Keep variables in kernels function scoped.
- Keep the x86 simulator test bench simple - use `init()`, `run(X)`, and `end()`.
- Keep variables in the ADF graph class scoped (see [Single Kernel Graph](#) and [C++ Kernel Class Support](#) in *AI Engine Kernel and Graph Programming Guide* (UG1079)).

AI Engine Simulator

The AMD Versal™ adaptive SoC AI Engine multi-threaded simulator (`aiesimulator`) includes the modeling of the global memory (DDR memory) and the network on chip (NoC) in addition to the AI Engine array. When the application is compiled using the simulation target, the AI Engine multi-threaded simulator can be invoked as follows.

```
aiesimulator --pkg-dir=./Work
```

This will run the AI Engine simulator in the multi-threaded mode consuming some CPU threads.

Using Threads in the AI Engine Simulation

The calculation for the default number of threads used during AI Engine Simulation is based on the number of CPU cores available on the user machine and the number of active AI Engine core tiles used in the AI Engine Design.

In any multi-threaded process, there is overhead associated with the launching and synchronization of threads. The default threads consumed during AI Engine simulation is the optimal number of threads calculated by the multi-threaded model so that the overall computation on each thread should be substantial when compared to the overhead.

By default, the `graph.run()` function with no argument specifies a graph that runs forever. The AI Engine compiler generates code to execute the data flow graph in a perpetual `while` loop, thus simulation also runs perpetually. To create terminating programs for debugging, specify `graph.run(<number_of_iterations>)` in your graph code to limit the execution for the specified number of iterations. The specified number of iterations can be any positive integer value. `graph::run(-1)` also specifies a graph that runs forever.

The AI Engine simulator command first configures the simulator as specified in the compiler generated `Work/config/scsim_config.json` file. This includes loading PL IP blocks and their connections, configuring I/O data file drivers, and configuring the NoC and global memory (DDR memory) connections. It then executes the specified PS application and finally exits the simulator.

The AI Engine simulator has an optional `--profile` option, which enables `printfs` in kernel code to appear on the console, and also generates profile information. Also, the `--dump-vcd <filename>` option generates a value change dump (VCD) for the duration of the simulation. The `--simulation-cycle-timeout <number-of-cycles>` can be used to exit the simulation after a given number of clock cycles.



IMPORTANT! If you do not provide either the clock cycles or the number of runs to `graph.run()`, the simulation runs forever. You need to press **Ctrl+C** twice to exit the simulator.



TIP: You might observe cycle count differences between simulation runs on the same design. This is because the simulator waits for a few seconds for all pending transactions (such as DMA) to finish. During this wait time, the simulator process is still ticking but can be context-switched by the OS and total cycles can be different for each run. To ensure that the total cycles are the same for each run, you should use the AI Engine simulator `--simulation-cycle-timeout` option to stop the simulator on the exact cycle. The total cycles that appear on the profiling report are same on each run.



IMPORTANT! Do not include `<iostream>` in the kernel code to enable `printfs`. The use of `#include <iostream>` in the kernel code results in a compilation error for both the x86 simulator and the `aiesimulator` simulators.

The AI Engine simulator reports Average Throughput of each PLIO at the end of the simulation.

The report is printed on the console and in `AIESimulator.log`

```
Enabling core(s) of graph G
Waiting for core(s) of graph G to finish execution ...
core(s) are done executing
Exiting!
Cores are done executing but the simulation will run for some more cycles
to allow PLIO to be flushed
Stopping Simulator.
```

```
Info: /OSCI/SystemC: Simulation stopped by user.
```

```
-----
Port Name          | Type          | Average Throughput
-----
Input_0            | IN            | 1499.787536 MBps
Input_1            | IN            | 1480.061002 MBps
Input_2            | IN            | 1500.666709 MBps
Input_3            | IN            | 1499.567904 MBps
Output_0           | OUT           | 1380.798274 MBps
Output_1           | OUT           | 1361.883229 MBps
Output_2           | OUT           | 1380.798274 MBps
Output_3           | OUT           | 1380.798274 MBps
-----
```

```
JSON file generated successfully!
IP-INFO: deleting ip PSIP_ps_i24
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
IP-INFO: deleting packet ip
[INFO] : Simulation Finished, Sim result: 0 Total Simulation time 19696001
ps
AIEMLSim feature license is released.
```

The same report is generated in the summary table of the Analysis View of the AMD Vitis™ Unified IDE.

Figure 6: Summary Table in Analysis View

Kernels I/O Buffers Ports Nets Tiles Interface Channels										
NAME	TYPE	DATA WIDTH	FREQUENCY (MHZ)	THROUGHPUT (MBYTES/S)	BUFFERS	CONNECTED PORTS	COLUMN	CHANNEL ID	LOCATION CONSTRAINT	
G (8)										
➤ Input: Input_0 (data/input.txt)	PLIO	128	325.0	1499.787536		1	20	0		
➤ Input: Input_1 (data/input.txt)	PLIO	128	325.0	1480.061002		1	17	0		
➤ Input: Input_2 (data/input.txt)	PLIO	128	325.0	1500.666709		1	16	0		
➤ Input: Input_3 (data/input.txt)	PLIO	128	325.0	1499.567904		1	21	0		
⬅ Output: Output_0 (data/output0.txt)	PLIO	128	325.0	1380.798274		1	19	0		
⬅ Output: Output_1 (data/output1.txt)	PLIO	128	325.0	1361.883229		1	18	0		
⬅ Output: Output_2 (data/output2.txt)	PLIO	128	325.0	1380.798274		1	15	0		
⬅ Output: Output_3 (data/output3.txt)	PLIO	128	325.0	1380.798274		1	22	0		

For designs using External Traffic Generators in AI Engine Simulation, throughput values might not be accurate because external traffic generators are not cycle accurate.

```
|INFO | in_interpolator | Total number of bytes sent : 2048
|INFO | out_interpolator | Total number of bytes received : 4096
|INFO | in_classifier | Total number of bytes sent : 4096
WARNING::[ XTLM_IPC::006 ] out_interpolator Closing Socket
|INFO | out_classifier | Total number of bytes received : 4096
WARNING::[ XTLM_IPC::006 ] in_interpolator Closing Socket
WARNING::[ XTLM_IPC::006 ] out_classifier Closing Socket
WARNING::[ XTLM_IPC::006 ] in_classifier Closing Socket
Stopping Simulator.
```

```
Info: /OSCI/SystemC: Simulation stopped by user.
INFO: The PLIO throughput may not be accurate when an External Traffic
Generator is used.
```

```
-----
Port Name          | Type          | Average Throughput
-----
in_interpolator    | IN            | 315.640219 MBps
in_classifier      | IN            | 516.259138 MBps
out_classifier     | OUT           | 618.282816 MBps
out_interpolator   | OUT           | 617.686090 MBps
-----
```

```
JSON file generated successfully!
IP-INFO: deleting ip PSIP_ps_i6
[INFO] : Simulation Finished, Sim result: 0 Total Simulation time 27676 ns
AIEsim feature license is released.
```

The Average Throughput will not be reported if the graph is run indefinitely (`graph.run(-1)`). Only if AI Engine simulator is stopped using `--simulation-cycle-timeout=<ns>` can you see the Average Throughput at the end of the simulation.

For designs with stalls or deadlock, the Average Throughput reported is 0.

The Average Throughput report in HW Emulation is found in the `simulate.log` file.

Note: The AI Engine simulator reports Average Throughput for PLIOs only.

Graphs and Sub-Graphs Simulation Runtimes

On an AI Engine design consisting of a larger graph composed of smaller sub-graphs, it is recommended to simulate the individual sub graphs first. Once the sub-graphs are verified and meet requirements in terms of functionality and performance, simulation of the larger graph can be attempted. Typically, depending on the size of the sub graphs, simulation time for the larger graph might take longer.



RECOMMENDED: *Simulate larger graphs only after you have simulated the individual sub-graphs.*

Simulator Options

The complete set of the AI Engine simulator (`aiesimulator`) options are described in this section. In most cases, pointing to `pkg-dir` is sufficient.

Table 9: AI Engine Simulator Options

Options	Description
<code>-h, --help</code>	Shows the help message and exits.
<code>--display-run-interval=<time in ns></code>	Displays time stamp periodically after each time interval (in ns). For example, <code>aiesimulator --display-run-interval=10 --pkg-dir=./Work --i=..</code> will display the time stamp every 10 ns.

Table 9: AI Engine Simulator Options (cont'd)

Options	Description
<code>--dump-vcd=<file></code>	Dump VCD waveform information into <file>. Because the tool appends .vcd to the specified file name, it is not necessary to include the file suffix. Note: This option allows you to access a subset of the signals. If you want all the signals or a different subset of the signals you should use the <code>--options-file</code> option to specify the category of signals to be generated in the VCD file. Note: To generate the XPE file from the aiesimulator generated vcd file, use the <code>vcdanalyze</code> tool as follows: <pre>vcdanalyze --vcd <vcdfile> -s=<STARTTIME> -e=<ENDTIME> --xpe [OPTIONS]</pre> The available options are: • <code>-s=<STARTTIME></code>: Specifies the start time in ps. • <code>-e=<ENDTIME></code>: Specifies the end time in ps. • <code>--out=<RPT></code>: Generates the workload estimation report in .rpt format. • <code>-v</code>: Generates verbose output (Optional). • <code>--xpe</code>: Generates the power estimate report. • <code>--xpe-dir=<XPEOutputDir></code>: Specifies the directory for generating XPE reports. The default is <code>./aiesim_xpe</code>. For information on using the XPE file, see <i>Power Design Manager User Guide (UG1556)</i>. Note: The <code>--online</code> option and <code>--dump-vcd</code> option cannot be used together. If both options are specified, only the <code>--online</code> option takes effect.
<code>--options-file=<file></code>	Text file used to select and generate specific VCD data. For example, <code>aiesimulator --options-file=<options>.txt --dump-vcd</code>. The <options.txt> file will specify modules whose data needs to be generated as VCD data. This switch should be used along with <code>--dump-vcd</code> switch for generating VCD.
<code>--enable-memory-check</code>	Enable runtime program and data memory boundary access check. Any violation access will be reported as an [ERROR] message. This option is disabled by default. When you run a simulation with this option enabled, and open the run summary in the Vitis IDE, the graph view displays a Memory Violations tab. This tab reports all the memory violations detected in the simulation run. You can click on a specific violation and navigate to the line in the source code causing the violation. For more information, see Memory Violations.
<code>--hang-detect-time=<time in ns></code>	The <code>aiesimulator</code> will exit if all active cores are in a stalled state after the time period (in ns), provided by this option. For example, if the option <code>--hang-detect-time=10</code> is provided to the <code>aiesimulator</code>, if after 10 ns all the active cores are in a stalled state, the <code>aiesimulator</code> will exit.
<code>-i</code>	Alias of <code>--input-dir=<dir></code> option.
<code>-o</code>	Alias of <code>--output-dir=<dir></code> option.
<code>--pkg-dir=<dir></code>	Specify the package directory, for example, <code>./Work</code> .

Table 9: AI Engine Simulator Options (cont'd)

Options	Description
<code>--profile</code>	Generates profiling data for all used cores. Allows generation of <code>printf</code> trace messages on <code>stdout</code> and collects profiling statistics during simulation. This can slightly slow down the simulator. Optionally, specify profiling of specific cores by using <code>--profile=(col,row) (col,row)...</code> Note: This option is not supported in <code>hw_emu</code>.
<code>--simulation-cycle-timeout=<cycles></code>	Run the application for a given number of cycles after it is loaded. The cycles are AI Engine-based cycles, which in turn depend on the frequency that the AI Engine clocked at. TIP: Specify the <code>--simulation-cycle-timeout</code> option to end the simulation session after the specified number of cycles. However, when specifying simulation timeout during the debug process, be sure to specify a larger number of cycles because the debug will terminate when the timeout cycle is reached.
<code>--online -text -wdb</code>	Use this option with <code>-text</code> and <code>-wdb</code> to parse VCD data on-the-fly, to produce an <code>event.txt</code> file, and waveform database (WDB) output file. For example, <code>aiesimulator --pkg-dir=./Work --i=.. --online -text -wdb</code> will produce a WDB file and an event text file that can be opened in the Vitis IDE. Note: The <code>--online</code> option and <code>--dump-vcd</code> option cannot be used together. If both options are specified, only the <code>--online</code> option takes effect.
<code>--output-time-stamp</code>	Use <code>--output-time-stamp=no</code> to get output file without time values and <code>--output-time-stamp</code> to get uniform time value (in ns) in output file.

An AI Engine simulator display run example follows:

```
aiesimulator --pkg-dir=Work/ --display-run-interval=10

AIEMLsim feature license is found.
INFO: Executing config: Work//config/scsim_config.json
set use_simple_noc to: 1
Loading device config from: VC2802.json
  AIE_WORK_DIR = Work/
Initializing AIE driver...
Initializing ADF API...
XAIEFAL: INFO: Resource group Avail is created.
XAIEFAL: INFO: Resource group Static is created.
XAIEFAL: INFO: Resource group Generic is created.
IP-INFO: [ps_i24_ps_main] IP loaded.
Warning: ps_lib_path is empty, And supported simulators are XSIM, RIVIERA,
XCELIUM, VCS and QUESTA

Info: DEVICE FILE: VC2802.json

Info: AIE SOLUTION FILE: Work//arch/aieshim_solution.aiesol
Disable unused tiles
xpe directory: BaseDesign/aie/Work//reports/graph.xpe
[AIESIM_OPTIONS]: aiesim_options file path aiesimulator-output/
aiesim_options.txt
[INFO]: AIE Cycle-approximate Model
Running Dispatch Server on port: 45443
```



```

INFO: Running AIE2 MTMODEL Simulation with 3 threads
[INFO]: Disable Unused Memory Tiles
AIE2 ISS rlp4
ISS disables unused tiles
[INFO]: Array constructed
[INFO]: Shim constructed
[INFO]: mem_row constructed
-----
SLAVE STREAM INFO: | Port Name          | Port Width      | Stream
Type              | Channel Index   |
-----
STREAM            | 160, 161        | S00_AXIS        | 128          | PL
STREAM            | 136, 137        | S01_AXIS        | 128          | PL
STREAM            | 128, 129        | S02_AXIS        | 128          | PL
STREAM            | 168, 169        | S03_AXIS        | 128          | PL
-----
MASTER STREAM INFO: | Port Name          | Port Width      | Stream
Type              | Channel Index   |
-----
STREAM            | 114, 115        | M00_AXIS        | 128          | PL
STREAM            | 108, 109        | M01_AXIS        | 128          | PL
STREAM            | 90, 91          | M02_AXIS        | 128          | PL
STREAM            | 132, 133        | M03_AXIS        | 128          | PL
-----
Enabled fast PM writes.
Enabled fast DM writes.
0 s IP-INFO: Starting to send data from file: default case ./data/input.txt
Starting to receive data into file: aiesimulator_output/data/output0.txt
IP-INFO: Starting to send data from file: default case ./data/input.txt
Starting to receive data into file: aiesimulator_output/data/output1.txt
IP-INFO: Starting to send data from file: default case ./data/input.txt
Starting to receive data into file: aiesimulator_output/data/output2.txt
IP-INFO: Starting to send data from file: default case ./data/input.txt
Starting to receive data into file: aiesimulator_output/data/output3.txt
IP-INFO: [ps_i24_ps_main] IP started.
60 ns Initializing graph traceGraph...
530 ns Configuring PL-Interface for graph traceGraph...
1130 ns Loading elfs of graph G...
Initializing graph G...
1180 ns Resetting cores of graph G...
1210 ns Configuring DMAs of graph G...
1220 ns Configuring PL-Interface for graph G...
1280 ns Set 16 iterations for the core(s) of graph G
Enabling core(s) of graph G
1460 ns Waiting for core(s) of graph G to finish execution ...
14470 ns core(s) are done executing

```

Simulation Input and Output Data Streams

The AI Engine simulator supports both a CSV and TXT file format as inputs and outputs to the simulator.

To generate the CSV based file format from an existing TXT file, AMD provides a utility script.

The command example below shows the options available with the script:

```
$XILINX_VITIS/data/emulation/scripts/aiesim_convert_plio_txt2csv.py -txt


```

- `-txt` is used to specify the input txt file
- `--datawidth` to specify AIE kernel data type width 32, 64, etc.
- `--buswidth` is the PLIO width 32, 64, 128, etc.
- `--complex` to specify that the AI Engine data type is complex.

For more details on the options available in the script, use the `--help` option.

The generated CSV file format has columns and rows as shown below.

Where,

- `DATA:<n>` is the command to represent the data values. The argument `n` is the value to indicate the number of times data should be repeated. The default value of `n` is 1.
 - The number of `D` columns is PLIO width / Data width of the AIE kernel.
- `TLAST` and `TKEEP` are optional columns to indicate the values to be driven with the data beat. Default value of `TKEEP` is -1 and `TLAST` is 0.
- `STALL:<n>` represents empty cycle(s). The argument `n` indicates the number of empty cycles from the PL domain. Default value of `n` is 1.
- `COMMENT` is used to add notes for better readability of the input file.

The following section describes the CSV and TXT formats.

TXT File Format

Input File

The default bit width for input/output streams is 32 bits. The bit width specifies the number of samples per line on the simulation input file. The interpretation of the samples on each line of the input file is dependent on the data type expected and the PLIO data width. The following table shows how the samples in the input data file are interpreted, depending on the data type and its corresponding PLIO interface specification.

Table 10: Data Type and PLIO Width

Data Type	PLIO 32 bit	PLIO 64 bit	PLIO 128 bit
	<code>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_32_bits, "input.txt");</code>	<code>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_64_bits, "input.txt");</code>	<code>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_128_bits, "input.txt");</code>
int8	4 values per line. For example: 6 8 3 2	8 values per line. For example: 6 8 3 2 6 8 3 2	16 values per line. For example: 6 8 3 2 6 8 3 2 6 8 3 2 6 8 3 2
int16	2 values per line. For example: 24 18	4 values per line. For example: 24 18 24 18	8 values per line. For example: 24 18 24 18 24 18 24 18
int32	Single value per line 2386	2 values per line. For example: 2386 2386	4 values per line. For example: 2386 2386 2386 2386
int64	N/A	45678	2 values per line. For example: 45678 95578
cint16	1 cint value per line – real, imaginary. For example: 1980 485	2 cint values per line. For example: 1980 45 180 85	4 cint values per line. For example: 1980 485 180 85 980 48 190 45
cint32	N/A	1 cint value per line – real, imaginary. For example: 1980 485	2 cint values per line. For example: 1980 45 180 85
float	1 floating point value per line. For example: 893.5689	2 floating point values per line. For example: 893.5689 3459.3452	4 floating point values per line. For example: 893.5689 39.32 459.352 349.345
cfloat	N/A	1 floating point cfloat value per line, real, imaginary. For example: 893.5689 24156.456	2 floating point cfloat values per line, real, imaginary. For example: 893.5689 24156.456 93.689 256.46

PLIO and Packet Stream Interface Requirements

When the TXT file is used to provide data that represents a PLIO port and packet stream interface, the following TXT file requirements should be followed.

1. `tkeep` is always valid. The `tkeep` signal set to False is not supported.
2. Multiple numbers of data samples in a line can be combined and sent, depending on the width of the interface. The first data sample is sent in the lowest bits of the interface. For example, if the data type is `int16` and the AI Engine to PL interface is 64 bits wide, the line 0 1 2 3 is sent as `0x0003000200010000` to the AI Engine.

```
0 1 2 3
```

3. `tlast` in the TXT file denotes that the following line has `tlast` equals 1. When `tlast` is 1, the number of data samples sent to the AI Engine can be equal to or less than the AI Engine to PL interface width. For example, if the data type is `int16` and the AI Engine to PL interface is 64 bits wide, the last line 4 5 is sent as `0x00050004` with `tlast` to the AI Engine:

```
0 1 2 3
tlast
4 5
```

- For the packet stream interface, `tlast` equals 1 denotes the end of the packet. The packet header should be specified in unsigned decimal format. For example, if the AI Engine to PL interface is 64 bits wide, the following lines send the packet header `0x8fff0000` (which is 2415853568 in unsigned decimal):

```
tlast
2415853568
```

For the packet stream interface, whose data type is `int16` and the AI Engine to PL interface is 64 bits wide, the following lines send the packet header `0x8fff0000`, packet data `0x0`, and then packet data `0xffffeffff` to the AI Engine:

```
2415853568 0
tlast
-1 -2
```

Output File

On each output PLIO port a file containing the stream content can be created automatically by the simulator using the same type of declaration as the input PLIO data files.

```
adf::output_plio out1 =
adf::output_plio::create("DataOut1",adf::plio_32_bits,"output1.txt");
adf::output_plio out2 =
adf::output_plio::create("DataOut2",adf::plio_64_bits,"output2.txt");
adf::output_plio out3 =
adf::output_plio::create("DataOut3",adf::plio_128_bits,"output3.txt");
```

The same format as the input applies for the output created file. Depending on the data type and the PLIO bitwidth, a number of data will be displayed on each line. Each output line is timestamped by the simulator so that you can estimate the data throughput during the simulation. Timestamp units can be in:

- picosecond (ps)
- nanosecond (ns)
- microsecond (us)
- millisecond (ms)
- second (s)

If the stream comes from a source that generates a TLAST flag at the end of a frame, this TLAST is also written in the output file. Below is an example of such an output file.

```
...
T 15984 ns
4552 4555
T 15988 ns
4558 4561
T 15992 ns
4564 4567
T 15996 ns
4570 4573
T 16 us
```

```

4576 4579
T 16004 ns
4582 4585
T 16008 ns
4588 4591
T 16012 ns
4594 4597
T 16016 ns
4600 4603
T 16020 ns
4606 4609
T 16024 ns
TLAST
4612 4615
T 17940 ns
4618 4621
T 17944 ns
4624 4627
T 17948 ns
4630 4633
T 17952 ns
...

```

As a summary each output of the PLIO port has the following format:

- Timestamp
- TLAST, TKEEP
- Sample DATA values

From the output you can estimate the throughput of the designs for this PLIO port. Timestamps are related only to valid output. When the PLIO port is quiet, there is no indication on the output file.

Compute the throughput as the number of output samples divided by the timestamp difference:

$$\text{Throughput} = \text{NumberOfSamples} / (\text{LastTimestamp} - \text{FirstTimestamp})$$

This simple equation will overestimate the throughput if you do not take into account all the clock cycles occurring after the last output sample.

This might be highly overestimated if this is a frame-based output. In that case, the output has the following format:

- Frame 0 output (tstart_0 up to tend_0)
- Quiet interframe
- Frame 1 output (tstart_1 up to tend_1)
- Quiet interframe
- ...
- Frame N-1 output (tstart_N-1 up to tend_N-1)
- Quiet interframe

- Frame N output (tstart_N up to tend_N)

In that case you have to take into account the interframe timelapse for each frame output. This can be done if you use only the N first frames (from 0 to N-1). In that case, replace the time stamps of the throughput equation with:

- FirstTimestamp = tstart_0
- LastTimestamp = tstart_N (the first output timestamp of the last frame)

Below is an example Python script that calculates the PLIO throughput. It analyzes the simulation output file and calculates the throughput.

```
import numpy as np
from math import *
import sys
import argparse

def GetTime_ns(Stamp):
    Time_ns = float(Stamp[1])
    if(Stamp[2] == 'ps'):
        Time_ns = Time_ns/1000.0
    elif(Stamp[2] == 'us'):
        Time_ns = Time_ns*1000.0
    elif(Stamp[2] == 'ms'):
        Time_ns = Time_ns*1000000.0
    elif(Stamp[2] == 's'):
        Time_ns = Time_ns*1000000000.0
    return(Time_ns)

def ReadFile(filename):
    # Detect the number of data per PLIO output
    fdr = open(filename, 'r')
    ts = fdr.readline()
    d = fdr.readline()
    dw = d.split()
    fdr.close()

    coltime = 0
    coldata = 1
    numdata = len(dw)
    coltlast = numdata + 1

    # Initializes the output array
    # Format: timestamp (in ns) val1 val2 ... valN TLAST (0 or 1)
    a = np.zeros((0,numdata+2))
    fdr = open(filename, 'r')
    line = ''
    lnum = 0;

    while line != "":
        line = fdr.readline()
        if line== '':
            continue
        res = line.split()

        if(res[0] != 'T'): # It should be a timestamp
```

```

        continue

    l = np.zeros((1,numdata+2))
    # Extract the time stamp
    l[0][0] = GetTime_ns(res)

    line = fdr.readline()
    res = line.split()
    # extract the TLAST
    if(res[0]== 'TLAST'):
        tlast = 1
        line = fdr.readline()
        res = line.split()
    else:
        tlast = 0

    l[0,coltlast] = tlast
    # Extract all values
    for i in range(numdata):
        l[0,i+1] = float(res[i])

    # Appends to the whole array
    a = np.append( a , l,axis=0)

fdr.close()
return(a)

def Throughput(Filename,IsComplex):
    V = ReadFile(Filename)
    print( "\n===== ")
    print(Filename)
    print( "\n")

    NRows = V.shape[0]
    NCols = V.shape[1]
    NFullFrames = int(np.sum(V[:,NCols-1]))
    print( "Number of Full Frames: " + str(NFullFrames))

    # Basic Throughput computation
    if IsComplex:
        Ratio = 0.5
    else:
        Ratio = 1
    RawThroughputMsps = float(NRows*(NCols-2))/(V[NRows-1,0]-
V[0,0])*Ratio*1000.0
    print( "Raw Throughput: %.2f" % RawThroughputMsps)

    # If the output is frame based, compute a more precise throughput
    tlast = np.where(V[:,NCols-1] == 1.0)
    if(len(tlast[0])<=1):
        TotalThroughput = RawThroughput
    else:
        tlast = tlast[0]
        EndRow = tlast[len(tlast)-2]+1
        # EndRow is the number of Rows I take into account for the number of
datasource
        # The timestamp I am interested in is the timestamp of the next
transaction
        TotalThroughputMsps = float(EndRow*(NCols-2))/(V[EndRow,0]-
V[0,0])*Ratio*1000.0
    print( " Throughput: %.2f" % TotalThroughputMsps)

```

```

print( "\n")

# Entry point of this file
if __name__ == "__main__":
    parser = argparse.ArgumentParser(prog=sys.argv[0], description='Compute
the throughput corresponding to some output of AIE Simulations')
    parser.add_argument('--iscomplex', action='store_true', help='Indicates
Complex data in the file')
    parser.add_argument('filename', nargs='+')
    Args = sys.argv
    Args.pop(0)
    args = parser.parse_args(Args)

for f in args.filename:
    Throughput(f,args.iscomplex)

```

CSV File Format

The AI Engine simulator supports the CSV file format for PLIO-based simulation. This new format provides more control over transactions and allows for cycle-by-cycle injection of traffic to AI Engine. This file format also offers the flexibility to add empty clock cycles to the traffic injection, use the `STALL` command to inject multiple empty cycles, and specify `TLAST` and `TKEEP` values explicitly. With these features, you can simulate complex scenarios and analyze the behavior of the design.

To associate the CSV file with a PLIO-based input or output, you need to specify the CSV file during the graph PLIO creation.

```

adf::input_plio in = adf::input_plio::create( "DataIn", adf::plio_32_bits,
"data/input.csv");
adf::input_plio out = adf::output_plio::create( "DataOut",
adf::plio_32_bits, "data/output.csv");

```

Input File

The input CSV file to the AI Engine simulator can be defined as a number of comma separated columns of data. The first row in the file is the header line. The header line is defined as a `CMD` column, followed by data `D` columns, followed by the `TLAST` and `TKEEP` columns. The number of data `D` columns should be equal to the PLIO buswidth/datawidth of kernel data being input. For example the header below corresponds to data being provided for a PLIO buswidth of 32-bit, and datawidth of `int16`. Hence, there are two data sample `D` columns in the header line.

```
CMD, D, D, TLAST, TKEEP
```

The `DATA` command must be used to provide data input in the lines that follow the header line.

```

CMD, D, D, TLAST, TKEEP
DATA, 16, 20, 0, -1
DATA, 15, 11, 0, -1

```


The `STALL` command must be used to specify the number of empty cycles to be injected in the data traffic. This command will mimic back pressure in the input traffic that can be useful in simulating real-life data input.

```
CMD, D, D, TLAST, TKEEP
DATA, 16, 20, 0, -1
DATA, 15, 11, 0, -1
STALL:100
```

Table 11: Input Columns

Column	Description
CMD	A pre-defined set of keywords for the parser to act on. Possible values are <code>DATA</code>, <code>STALL:<n></code>, and <code>COMMENT</code>. • <code>DATA</code>: This command is followed by an optional argument <code><n></code> separated by colon, as in <code>DATA:<n></code>. ◦ <code><n></code> indicates how many times the data should be repeated as input into AI Engine. Default value of <code><n></code> is 1. • <code>STALL</code>: This command represents an empty PL cycle and is followed by an optional argument <code><n></code> separated by colon as in <code>STALL:<n></code>. ◦ If you specify <code>STALL:100</code>, there will be no data driven for 100 cycles from the PL domain. If the PL frequency is 100 MHz, a stall duration of 1000 ns is present from the PL side. ◦ If you do not specify the argument value <code><n></code>, the default value of <code>STALL</code> is 1. • <code>COMMENT</code>: This command represents a comment.
D	• Represents the data values sent to AI Engine kernel, as per the kernel data width. For example, INT8, or INT32. • Data column can be repeated based on the data width and PLIO bus width combination. The number of <code>D</code> columns equals $\text{PLIO buswidth} / \text{datawidth}$. • Data specified in the <code>D</code> columns are sent to the AI Engine in the order specified in the CSV file.

Table 11: Input Columns (cont'd)

Column	Description
TKEEP	The TKEEP column can be used to specify how the D (data) columns are processed. Based on the PLIO bus width, the allowable value of the TKEEP column and its impact will differ. The TKEEP value can be empty. If it is empty, there is no effect on the data values. 32 bit PLIO buswidth: - Possible values : 0x0 to 0xF - Effect: If AI Engine PLIO buswidth is 32 bit, the TKEEP value is ignored. 64 bit PLIO buswidth: - Possible values : 0x0 to 0xFF - Effect: 0x0 <= TKEEP <= 0x0F → Lower 32 bit is valid 0x0F < TKEEP <= 0xFF → Full 64 bit is valid 128 bit PLIO buswidth: - Possible values : 0x0 to 0xFFFF - Effect: 0x0000 <= TKEEP <= 0x000F → Lower 32 bit is valid 0x000F < TKEEP <= 0x00FF → Lower 64 bit is valid, that is, two 32 bit words are valid. 0x00FF < TKEEP <= 0x0FFF → Lower 96 bit is valid, that is, three 32 bit words are valid. 0x0FFF < TKEEP <= 0xFFFF → All 128 bit is valid, that is, four 32 bit words are valid. Below are examples of appropriate TKEEP combinations and their impact (in comment) during the simulation for a CSV file that represents 32 bit data type, 64 bit PLIO bus width. <pre> CMD, D, D, TKEEP, TLAST COMMENT, All 64 bit is valid DATA, 1234, 5543, -1, 0 DATA, 1234, 5543, 0x0F, 0 DATA, 1234, 5543, 0xFF, 1 DATA, 1234, 5543, 0xFF, 0 COMMENT, Only lower 32 bit is valid DATA, 1234, 5543, 0x0F, 1 COMMENT, Empty D column with 0x0<=TKEEP<=0x0F and TLAST=1 COMMENT, Only 1234 is valid DATA, 1234, , 0x0F, 1 </pre>
TLAST	- The TLAST column is used to indicate whether the end of the data stream is reached. If the TLAST value is not specified, the value is 0. - The TLAST value can be either 0 or 1. The default TLAST value is 0. - TLAST = 1 indicates the end of the data stream.



IMPORTANT! In the CSV file, the column headers and their corresponding values must be comma separated even when an empty value is specified. For example, below is the CSV file with empty TLAST and TKEEP.

```
CMD, D, D, TLAST, TKEEP
DATA, 3, 2, , ,
```

Table 12: Data Type and PLIO Width

Data Type	PLIO 32 bit	PLIO 64 bit	PLIO 128 bit
	<pre>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_32_bits, "input.csv");</pre>	<pre>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_64_bits, "input.csv");</pre>	<pre>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_128_bits, "input.csv");</pre>
int8	Four data columns expected. For example: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 6, 8, 3, 2, -1, 0</pre>	Eight data columns expected. For example: <pre>CMD, D, D, D, D, D, D, D, D, TKEEP, TLAST DATA, 6, 8, 3, 2, 6, 8, 3, 2, -1, 0</pre>	16 data columns expected. For example: <pre>CMD, D, D, D, D, D, D, D, D, D, D, D, D, D, D, D, D, TKEEP, TLAST DATA, 6, 8, 3, 2, 6, 8, 3, 2, 6, 8, 3, 2, 6, 8, 3, 2, -1, 0</pre>
int16	Two data columns expected. For example: <pre>CMD, D, D, TKEEP, TLAST DATA, 24, 18, -1, 0</pre>	Four data columns expected. For example: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 24, 18, 24, 18, -1, 0</pre>	Eight data columns expected. For example: <pre>CMD, D, D, D, D, D, D, D, D, TKEEP, TLAST DATA, 24, 18, 24, 18, 24, 18, 24, 18, -1, 0</pre>
int32	One data column expected. <pre>CMD, D, TKEEP, TLAST DATA, 2386, -1, 0</pre>	Two data columns expected. For example: <pre>CMD, D, D, TKEEP, TLAST DATA, 2386, 2386, -1, 0</pre>	Four data columns expected. For example: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 2386, 2386, 2386, 2386, -1, 0</pre>
int64	N/A	One data column expected. <pre>CMD, D, TKEEP, TLAST DATA, 45678, -1, 0</pre>	Two data columns expected. For example: <pre>CMD, D, D, TKEEP, TLAST DATA, 45678, 95578, -1, 0</pre>
cint16	Two data columns expected to represent one <code>cint</code> value (real, imag). For example: <pre>CMD, D, D, TKEEP, TLAST DATA, 1980, 485, -1, 0</pre>	Two <code>cint</code> values (real, imag) represented by four data columns expected. For example: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 1980, 45, 180, 85, -1, 0</pre>	Four <code>cint</code> values (real, imag) represented by 8 data columns expected. For example: <pre>CMD, D, D, D, D, D, D, D, D, TKEEP, TLAST DATA, 1980, 485, 180, 85, 980, 48, 190, 45, -1, 0</pre>

Table 12: Data Type and PLIO Width (cont'd)

Data Type	PLIO 32 bit	PLIO 64 bit	PLIO 128 bit
	<code>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_32_bits, "input.csv");</code>	<code>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_64_bits, "input.csv");</code>	<code>adf::input_plio in = adf::input_plio::create("DataIn1", adf::plio_128_bits, "input.csv");</code>
cint32	N/A	One <code>cint</code> value (real, imag) represented by two data columns expected. For example: CMD, D, D, TKEEP, TLAST DATA, 1980, 485, -1, 0	Two <code>cint</code> values represented by four data columns expected. For example: CMD, D, D, D, D, TKEEP, TLAST DATA, 1980, 45, 180, 85, -1, 0
float	One data column expected. For example: CMD, D, TKEEP, TLAST DATA, 893.5689, -1, 0	Two data columns expected. For example: CMD, D, D, TKEEP, TLAST DATA, 893.5689, 3459.3452, -1, 0	Four data columns expected. For example: CMD, D, D, D, D, TKEEP, TLAST DATA, 893.5689, 39.32, 459.352, 349.345, -1, 0
cfloat	N/A	One floating point <code>cfloat</code> value represented using two data columns expected. For example: CMD, D, D, TKEEP, TLAST DATA, 893.5689, 24156.456, -1, 0	Two floating point <code>cfloat</code> values represented using four data columns expected. For example: CMD, D, D, D, D, TKEEP, TLAST DATA, 893.5689, 24156.456, 93.689, 256.46, -1, 0
bfloat16	Two data columns expected. For example: CMD, D, D, TKEEP, TLAST DATA, 3.14062, 3.14062, -1, 0	Four data columns expected. For example: CMD, D, D, D, D, TKEEP, TLAST DATA, 3.14062, 3.14062, 3.14062, 3.14062, -1, 0	Eight data columns expected. For example: CMD, D, D, D, D, D, D, D, TKEEP, TLAST DATA, 3.14062, 3.14062, 3.14062, 3.14062, 3.14062, 3.14062, 3.14062, 3.14062, -1, 0

Note: For integer data types, both decimal and hexadecimal representation of the data format are supported in the input/output CSV format. To use the hexadecimal format, set `bool hex` parameter to true in the `adf::input_plio::create()` API. For details, see [input_plio/output_plio](#) in *AI Engine Kernel and Graph Programming Guide* (UG1079).

Note: For floating point data types, both floating point representation and exponential representation of the data format is supported in the input CSV file.

CSV File Format and Error Checks During Simulation

For a CSV with incorrect data, an ERROR message is generated during AI Engine simulation execution. Below are some of the error checking conditions during simulation when a CSV file is provided as input.



TIP: The first line in the CSV should not be a comment. It must have only headers.

Table 13: CSV File Error

Error Scenario	CSV Input	Error Message
The number of DATA columns are not equal to PLIO buswidth / datawidth	For example, if buswidth = 64, and datawidth = 16, you need four D columns. Two Few D Columns: <pre>CMD, D, D, D, TKEEP, TLAST DATA, 10, 100, 64, -1, 0</pre>	[ERROR] : In <filename>, the number of data (D) columns are not equal to PLIO buswidth/sample datawidth. Expected <expected_number>
	Two Many D Columns: <pre>CMD, D, D, D, D, D, TKEEP, TLAST DATA, 10, 50, 150, 100, 90, -1, 0</pre>	
Partial data provided in the DATA columns of the CSV	For example, if buswidth = 64, and datawidth = 16, you need four D columns. You must provide data samples in all four D columns. If partial data is provided, appropriate TKEEP with TLAST=1 needs to be set. The TKEEP values will determine whether the lower 32 bit is valid or the full 64 bit is valid. Missing One Data Sample: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 10, 30, 60, , -1, 0</pre> Missing Two Data Samples: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, , , 50, 64, -1, 0</pre>	[ERROR] : In <filename>, in line <linenumber>, partial data is being fed to plio . To feed partial data, provide appropriate TKEEP value with TLAST = 1
Empty line in the CSV file	Empty Line: <pre>CMD, D, D, D, D, TKEEP, TLAST</pre>	This line is ignored by the tool.
Invalid value specified in the CMD or D column	Invalid Command in the CMD Column: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA:*(#\$, 10, 20, 30, 40, -1, 0</pre>	[ERROR] : Invalid command specified at <Filename:line number>
	Invalid Data Sample in D Columns: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, D, 20, 30, 40, -1, 0 CMD, DATA, DATA, DATA, DATA, TKEEP</pre>	[ERROR] : Invalid data value detected <filename:line number>.
DATA columns ordered incorrectly	D Columns are Not Side-by-Side (Incorrect Order): <pre>CMD, D, TKEEP, TLAST, D, D, D DATA, 10, -1, 0, 20, 30, 40</pre>	[ERROR] : Ensure all the data columns (D) are specified in sequence right after the CMD column in <Filename>

Table 13: CSV File Error (cont'd)

Error Scenario	CSV Input	Error Message
DATA value is out of range for a specific datatypes	For example, if data type is <code>int8</code> , buswidth is 32, and data sample 2323 is out of range. DATA Value 2323 is Out of Range: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 34, 2323, 23, 21, -1, 0</pre>	[ERROR] : Out of range data value detected in <filename>. Datatype is <datatype_provided_from_graph>. Range is <corresponding_range>
DATA value provided is invalid	Invalid Data Sample in D Column: <pre>CMD, D, D, D, D, TKEEP, TLAST DATA, 1.23###, 10, 10, 10, -1, 0</pre>	[ERROR] : In <filename>, invalid data value is provided at <line number>.

Output File

Similar to the input CSV file, you can specify a CSV file be generated for a PLIO output port as well. The simulator write the simulation output corresponding to the appropriate PLIO port into the specified output CSV file.

```
adf::output_plio out1 =
adf::output_plio::create("DataOut1",adf::plio_32_bits,"output1.csv");
adf::output_plio out2 =
adf::output_plio::create("DataOut2",adf::plio_64_bits,"output2.csv");
adf::output_plio out3 =
adf::output_plio::create("DataOut3",adf::plio_128_bits,"output3.csv");
```

The CSV file output includes both `TLAST` and `TKEEP` data. Each output line also includes the timestamp so that you can estimate the data throughput during the simulation. The timestamp is output in `ns` as the last column in the output CSV file.

```
...
CMD, D, D, TLAST, TKEEP, TIME_NS
DATA:1, 0, 1, 0, -1, 720
DATA:1, 2, 3, 0, -1, 918.4
DATA:1, 4, 5, 0, -1, 1116.8
DATA:1, 6, 7, 0, -1, 1315.2
DATA:1, 8, 9, 0, -1, 1513.6
DATA:1, 10, 11, 0, -1, 1712
DATA:1, 12, 13, 0, -1, 1910.4
DATA:1, 14, 15, 0, -1, 2108.8
DATA:1, 16, 17, 0, -1, 2307.2
DATA:1, 18, 19, 0, -1, 2505.6
DATA:1, 20, 21, 0, -1, 2704
DATA:1, 22, 23, 0, -1, 2902.4
DATA:1, 24, 25, 0, -1, 3100.8
DATA:1, 26, 27, 0, -1, 3299.2
DATA:1, 28, 29, 0, -1, 3497.6
DATA:1, 30, 31, 0, -1, 3696
DATA:1, 32, 33, 0, -1, 3894.4
DATA:1, 34, 35, 0, -1, 4092.8
DATA:1, 36, 37, 0, -1, 4291.2
DATA:1, 38, 39, 0, -1, 4489.6
DATA:1, 40, 41, 0, -1, 4688
DATA:1, 42, 43, 0, -1, 4886.4
```

```

DATA:1, 44, 45, 0, -1, 5084.8
DATA:1, 46, 47, 0, -1, 5283.2
DATA:1, 48, 49, 0, -1, 5481.6
DATA:1, 50, 51, 0, -1, 5680
DATA:1, 52, 53, 0, -1, 5878.4
DATA:1, 54, 55, 0, -1, 6076.8
DATA:1, 56, 57, 0, -1, 6275.2
DATA:1, 58, 59, 0, -1, 6473.6
DATA:1, 60, 61, 0, -1, 6672
DATA:1, 62, 63, 0, -1, 6870.4
DATA:1, 0, 1, 0, -1, 7068.8
DATA:1, 2, 3, 0, -1, 7267.2
DATA:1, 4, 5, 0, -1, 7465.6
DATA:1, 6, 7, 0, -1, 7664
DATA:1, 8, 9, 0, -1, 7862.4
DATA:1, 10, 11, 0, -1, 8060.8
DATA:1, 12, 13, 0, -1, 8259.2
DATA:1, 14, 15, 0, -1, 8457.6
...

```

Note: For floating point data types, only exponential representation of the data format is supported in the output CSV file. For example, below is the output CSV file format of the float (32-bit) data type.

```

CMD, D, TLAST, TKEEP, TIME_NS
DATA:1, 2.002000093e+00, 0, 0, 3264
DATA:1, 4.004000187e+00, 0, 0, 3267.2
DATA:1, 6.006000042e+00, 0, 0, 3270.4
DATA:1, 8.008000374e+00, 0, 0, 3273.6
DATA:1, 1.001000023e+01, 0, 0, 3276.8
DATA:1, 1.201200008e+01, 0, 0, 3280
DATA:1, 1.401399994e+01, 0, 0, 3283.2
DATA:1, 1.6016000075e+01, 0, 0, 3286.4
DATA:1, 1.801799965e+01, 0, 0, 3328
DATA:1, 2.002000046e+01, 0, 0, 3331.2
DATA:1, 2.202199936e+01, 0, 0, 3334.4
DATA:1, 2.402400017e+01, 0, 0, 3337.6
DATA:1, 2.602599907e+01, 0, 0, 3340.8
DATA:1, 2.802799988e+01, 0, 0, 3344
DATA:1, 3.003000069e+01, 0, 0, 3347.2
DATA:1, 3.203200150e+01, 0, 0, 3350.4
DATA:1, 3.403400040e+01, 0, 0, 3395.2

```

Viewing the Stalls in Analysis View of the Vitis Unified IDE

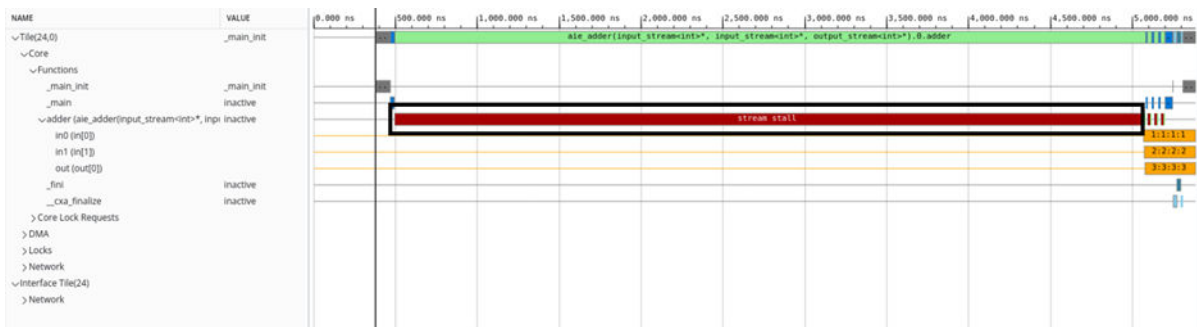
Using `STALL` command in the CSV file, after you run the AI Engine and opt to generate simulation trace data, you can open the simulation run summary and view the stalls in the Vitis unified IDE. The `STALL:<n>` inserts stalls for `n` empty cycles from the PL domain. You can see the stall duration based on the PL frequency. For example, `STALL:100` inserts 100 empty cycles from the PL for the duration of 1000 ns when the PL frequency is 100 MHz.

Example 1: Design with direct stream connections (no buffer) results in a stream stall for the 500 empty cycles (from the PL domain) specified.

Figure 7: CSV Entry with Stall for 500 Empty Cycles

```
1 CMD.D.TLAST,KEEP
2 STALL:500,,,
3 DATA,1,0,-1
4 DATA,1,0,-1
5 DATA,1,0,-1
6 DATA,1,0,-1
7 DATA,1,0,-1
8 DATA,1,0,-1
9 DATA,1,0,-1
```

Figure 8: Stream Stall for 500 Empty Cycles in Analysis View of the Vitis Unified IDE

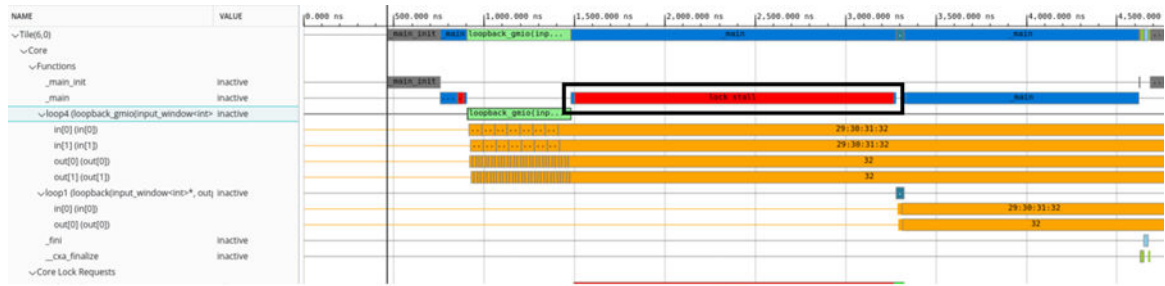


Example 2: Design where streams are connected via buffer results in a lock stall for the 1000 empty cycles (from the PL domain) specified.

Figure 9: CSV Entry with Stall for 1000 Empty Cycles

```
1 CMD.D.D.TLAST,KEEP
2 STALL:1000,,,
3 DATA,1,2,0,-1
4 DATA,3,4,0,-1
5 DATA,5,6,0,-1
6 DATA,7,8,0,-1
7 DATA,9,10,0,-1
8 DATA,11,12,0,-1
```


Figure 10: Lock Stall for 1000 Empty Cycles in Analysis View of the Vitis Unified IDE



Hang Detection

AI Engine designs might experience simulator hangs. Some common causes include:

- Insufficient input data for the specified number of iterations.
- Mismatched production and consumption rates in stream data.
- Cyclic dependencies with streams.
- Cascade streams or asynchronous buffers.
- Incorrect order of blocking protocol calls, such as, acquisition of asynchronous buffers or read/write from streams.

Using the `aiesimulator` option `--hang-detect-time=<time_in_ns>` will allow the tool to check for a hang situation on the kernels and trigger an exit of the simulation after a specified delay. It can detect lock stalls and stream stalls, whether it is circuit switched or packet switched communication. By default, `aiesimulator` runs with `--hang-detect-time=100000` (i.e., 100000 ns). To turn off the hang detection, set `--hang-detect-time=-1`.

Below are some examples of hang detection output.

Figure 11: Stream Stall Detection

```
WARN : T=2588000.000000 ps All Cores in Hang State for 2000ns
|----- Core Stall Status -----|
(24,1) -> Stream stall -> MS0 at T=668400.000000 ps
(24,2) -> Stream stall -> MS0 at T=574000.000000 ps
(25,1) -> Stream stall -> SS0 at T=587600.000000 ps
(25,2) -> Stream stall -> SS1 at T=561200.000000 ps
|-----|
[CRITICAL WARNING]: Closing Simulator
[INFO]: Use aiecompiler option --evaluate-fifo-depth to get fifo guidance on hang for designs with reconvergent path.
Exiting!
Cores are done executing but the simulation will run for some more cycles to allow PLIO to be flushed
Stopping Simulator.
```

Figure 12: Lock Stall Detection

```

Enabling core(s) of graph rx0
Waiting for core(s) of graph rx0 to finish execution ...
852600 ps Array::all cores with kernels are in hang condition, stopping simulation
|----- Core Stall Status -----|
(24,1) -> Lock stall -> Lock_East
(24,2) -> Lock stall -> Lock_East
(25,1) -> Lock stall -> Lock_North
|-----|
Exiting!
core(s) are done executing

```

Note: For larger designs, run `aiesimulator` with a hang detection value suitable for the design.

Memory Violations

The AI Engine simulator can detect memory violations at runtime. By specifying the `--enable-memory-check` option, `aiesimulator` can detect memory violations and show messages, such as:

```

4040800 ps [ERROR]
tl.aie_logical.aie_xtlm.math_engine.array.tile_32_1.cm.proc: dme_lda_e_out
access out of boundary! address = 0x3b000 prog_cntr = 0x00001101000000
4040800 ps [ERROR]
tl.aie_logical.aie_xtlm.math_engine.array.tile_32_1.cm.proc: dmo_lda_e_out
access out of boundary! address = 0x3b010 prog_cntr = 0x00001101000000

```

The memory violations can be shown in the Vitis IDE by opening the run summary. The Memory Violations tab shows the tile in which the violation occurred. Click the PC values of the violations to cross probe with the source code to determine where the issue originates. Below is one example of memory violations:

Figure 13: Memory Violations

ID	TILE	TIME (PS)	PC	KERNEL	BAD ADDRESS	MODULE	ERROR MESSAGE
1	Tile 132.0	4040800	832	N01	0x3b000	dme_lda_e_out	[ERROR] tl.aie_logical.aie;
2	Tile 132.0	4040800	832	N01	0x3b010	dmo_lda_e_out	[ERROR] tl.aie_logical.aie;

Evaluating FIFO Depth To Break Deadlocks

When your system is deadlocked, adding FIFOs within the design might help. In a simple system, you can infer the FIFO sizes from the design and do a manual static analysis. However, you do not know exactly how many clock cycles are spent in the `main_init()`, the `main()`, and the beginning of your `kernel`.

The tool computes these FIFO lengths from an AI Engine simulation for you. You can compile your graph with the AI Engine compiler using the flag `--evaluate-fifo-depth`. This option generates infinite FIFOs on all relevant paths.

Once your code is compiled, run a standard AI Engine simulation with `aiesimulator`. Near the beginning of the simulation you will get this warning:

```
[CRITICAL WARNING]: Design was compiled with --evaluate-fifo-depth option,
before deploying the design on hardware compile without --evaluate-fifo-
depth option.
```

1. Open Vitis Unified IDE and open the simulation `run_summary`.
2. Note the Estimated FIFO column, and apply the recommended number of FIFOs using the `fifo_depth` constraint on specific nets.

Figure 14: Vitis Unified IDE

NAME	VARIABLE	SOURCE GRAPH NODE	SOURCE PORT	DESTINATION GRAPH NODE	DESTINATION PORT	LATENCY (CYCLES)	FIFO DEPTH (WORDS)	FIFO DEPTH CONSTRAINT	BUFFERS	SWITCH COUNT	SWITCH FIFOs	ESTIMATED FIFO (WORDS)	PEAK FIFO (WORDS)
net0 (11)	<unnamed>.net0	Input: input64_1 (data/input_64.csv)	out[0]	k[0]	sin	52	104	0		13	0		97
net0_bcast1	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[1]	sin	48	96	0		12	0	20	117
net0_bcast2	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[2]	sin	44	88	0		11	0	40	137
net0_bcast3	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[3]	sin	40	80	0		10	0	60	157
net0_bcast4	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[4]	sin	36	72	0		9	0	80	177
net0_bcast5	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[5]	sin	32	64	0		8	0	100	197
net0_bcast6	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[6]	sin	28	56	0		7	0	120	217
net0_bcast7	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[7]	sin	24	48	0		6	0	140	237
net0_bcast8	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[8]	sin	20	40	0		5	0	154	251
net0_bcast9	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[9]	sin	16	32	0		4	0	154	251
net0_bcast10	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[10]	sin	12	24	0		3	0	170	267
net0_bcast11	<unnamed>.net	Input: input64_1 (data/input_64.csv)	out[0]	k[11]	sin	8	16	0		2	0	193	290

The estimated FIFO depth are given in the `Estimated FIFO (Words)` column. These values can be used in the graph to insert FIFOs with the specified length using the following syntax:

```
fifo_depth(net) = value;
```

Where `net` is a connection on which you want to insert a FIFO.

Latency and Throughput Estimates

During an AI Engine simulation, average throughput is automatically calculated and displayed at the end of the simulation. If you generate a VCD file during the simulation, it can also provide estimates for continuous latency and throughput.

Continuous latency refers to how the latency between specified input and output ports changes over time, whereas continuous throughput refers to how the throughput at a specified input or output port changes.

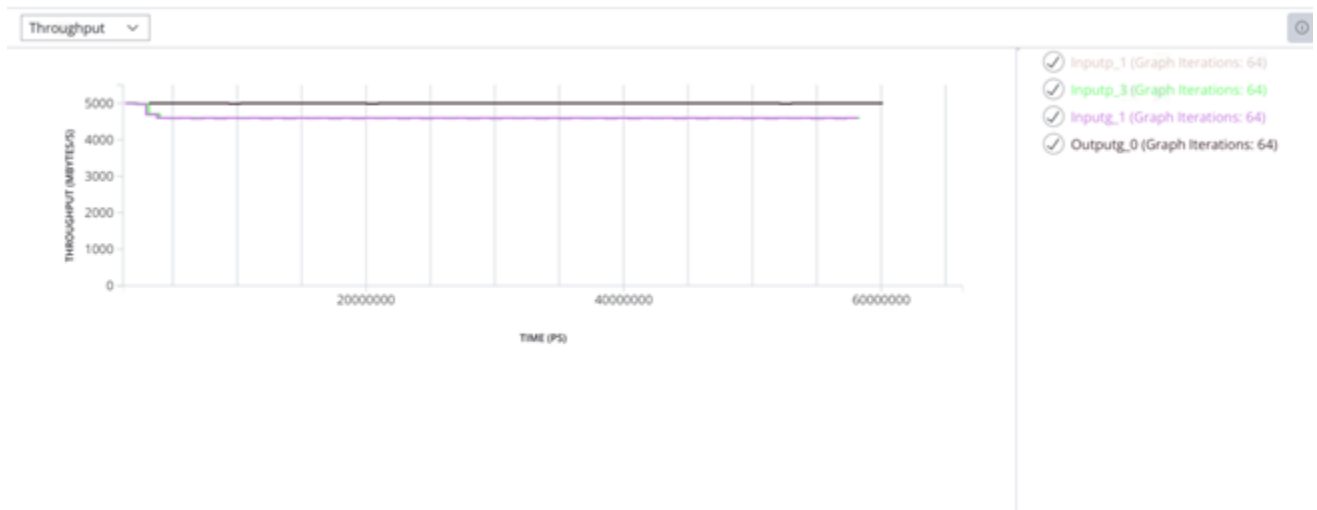
Input and output ports can be either PLIO or GMIO. You can generate a VCD file with the command:

```
aiesimulator --pkg-dir=./Work --dump-vcd foo --options-file=aiesim-
options.txt
```

To minimize the size of the VCD file, only generate SHIM and STREAM_SWITCH trace data. In the `aiesim-options.txt` file, set `AIE_DUMP_VCD_SHIM` and `AIE_DUMP_VCD_STREAM_SWITCH` to `true` while setting other options to false as follows:

```
AIE_DUMP_VCD_IO=false
AIE_DUMP_VCD_CORE=false
AIE_DUMP_VCD_SHIM=true
AIE_DUMP_VCD_MEM=false
AIE_DUMP_VCD_STREAM_SWITCH=true
AIE_DUMP_VCD_CLK=false
```

Figure 15: Continuous Throughput



Latency and Continuous Latency

The Vitis IDE has a feature that allows you to analyze latency and throughput of the graph. If you open the `default.run_summary` file using Vitis Analyzer, you can view the latency measurements for the AI Engine Array in the `Trace` section of the `Latency` tab. You can access this feature by running:

```
vitis_analyzer aie/aiesimulator_output/default.aierun_summary
```

The Latency tab displays three columns with latency data:

- **First Latency:**

The time it takes for the first input data to reach the first output data.

- **Last Latency:**

The time it takes for the last input data to reach the last output data.

- **Average Latency:** The difference between the average output sample time and average input sample time.

In addition, all corresponding inputs are shown for each output.

Figure 16: Latency Table

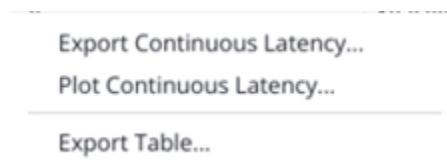
	Latency	Lock Stalls	DMA Stalls	Memory Stalls	Events	Kernels	I/O
NAME	FIRST LATENCY (PS)						
▼ ● I/O Latency (4)							
▼ ◀ Outputg_0 (2)	1876000						
▶▶ Inputg_0	1876000						
▶▶ Inputg_1	1868800						
▼ ◀ Output: Outputp_0 (data/output0.txt) (2)	1876000						
▶▶ Input: Inputp_0 (data/input.txt)	1876000						
▶▶ Input: Inputp_1 (data/input.txt)	1876000						
▼ ◀ Outputg_1 (2)	2078400						
▶▶ Inputg_3	2078400						
▶▶ Input: Inputp_3 (data/input.txt)	1876000						
▼ ◀ Output: Outputp_1 (data/output1.txt) (2)	2084000						
▶▶ Input: Inputp_2 (data/input.txt)	1876000						
▶▶ Inputg_2	2084000						

For more information about the latency of a simulation:

1. Right-click on an input port line, and select one of three options **Export Continuous Latency**, **Plot Continuous Latency**, or **Export Table**.

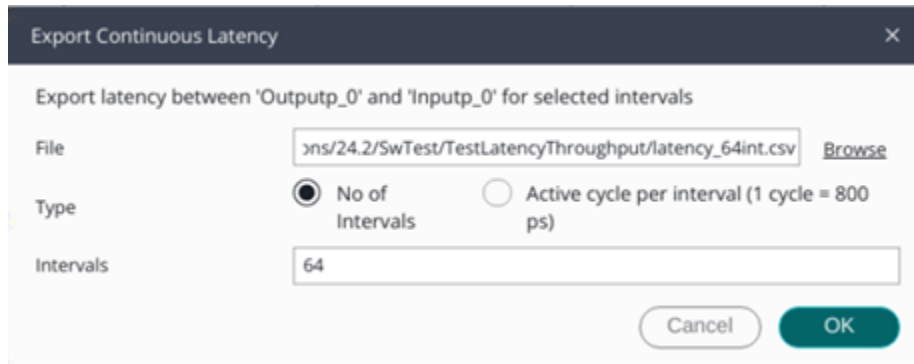
Note: **Export Table** option is only available for an output row.

Figure 17: Options in the Context Menu for Input Ports



2. When you choose to export or plot continuous latency, provide the filename and window width in the dialog box. The continuous latency and throughput are calculated over consecutive windows, and you can select either the number of intervals or active cycles.
 - Number of intervals: The simulation duration is divided into N intervals on which the average latency (or throughput) is computed.
 - Active Cycles: The interval length specified in terms of number of cycles.

Figure 18: Export Continuous Latency Dialog Box

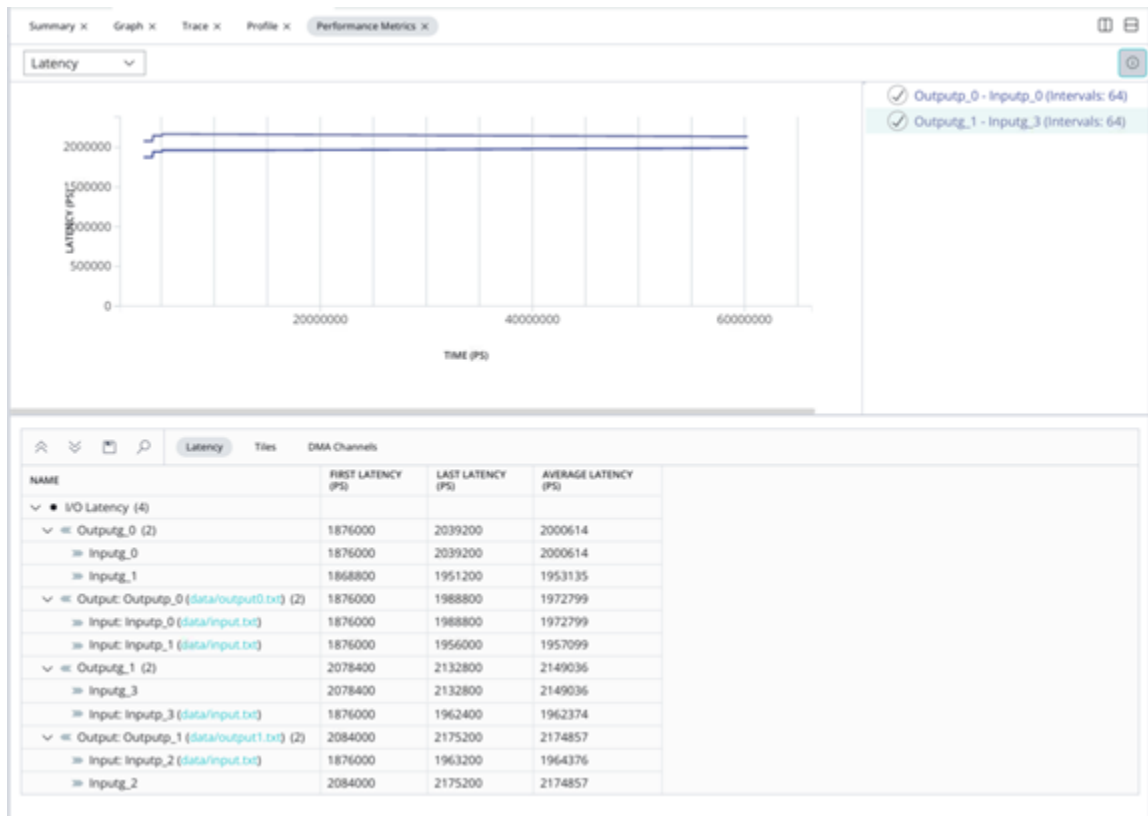


A CSV file is then generated.

```
Start Time (ps), End Time (ps), Latency (ps) between 'Inputp_0' and
'Outputp_0' for '64' intervals
3362400, 4181599, 1876000
4252800, 5071999, 1944800
5143200, 5962399, 1964000
6033600, 6852799, 1964800
...
58593600, 59412799, 1991200
59484000, 60303199, 1991200
```

After you make this selection, a CSV file is generated containing the start and end times of each interval, as well as the computed latency between input and output ports. The CSV file can be used by post processing scripts for further analysis. To view the latency graph, select the `Plot Continuous Latency` option. The same dialog box appears, but the latency plot is displayed in Vitis Analyzer. Several plots can be displayed on the same graph, as shown in the following image.

Figure 19: Continuous Latency



Throughput and Continuous Throughput

Throughput estimates are available also in the Trace section but in the I/O tab. The THROUGHPUT (MBYTES/S) column gives you the average throughput on each port, computed on the total duration of the simulation.

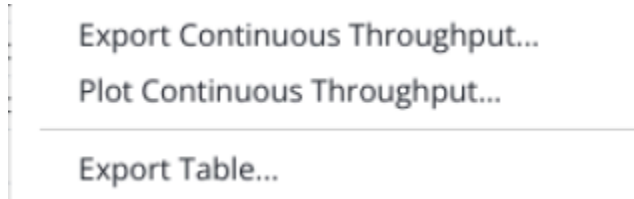
Figure 20: Throughput Table

NAME	TYPE	DATA WIDTH	FREQUENCY (MHz)	THROUGHPUT (MBYTES/S)	BUFFERS	CONNECTED PORTS	COLUMN	INTERFACE CHANNEL	DMA CHANNEL	LOCATION CONSTRAINT	PACKET IDS
▼ G (12)											
Input: Inputp_0 (data/input.txt)	PLIO	128	312.5	4616.706821	2	1	22	0			
Input: Inputp_1 (data/input.txt)	PLIO	128	312.5	4614.041511	2	1	24	0			
Input: Inputp_2 (data/input.txt)	PLIO	128	312.5	4613.781645	2	1	30	0			
Input: Inputp_3 (data/input.txt)	PLIO	128	312.5	4616.706821	2	1	25	0			
Inputg_0	GMIO	32		1157.585349	2	1	18	3	0		
Inputg_1	GMIO	32		1155.935599	2	1	18	7	1		
Inputg_2	GMIO	32		1156.931420	2	1	34	3	0		
Inputg_3	GMIO	32		1156.931420	2	1	26	3	0		
Output: Outputp_0 (data/output0.txt)	PLIO	128	312.5	4604.057776	2	1	23	0			
Output: Outputp_1 (data/output1.txt)	PLIO	128	312.5	4603.216970	2	1	31	0			
Outputg_0	GMIO	32		1150.174099	2	1	18	3	1		

For more information about the throughput of a simulation:

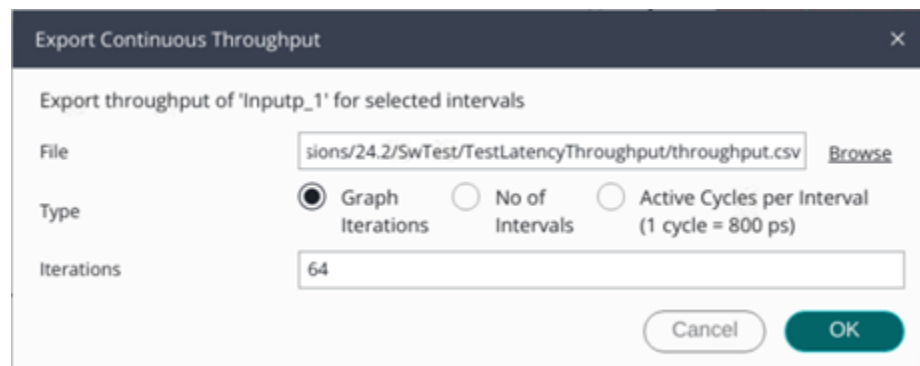
1. Right-click on an input port line, and choose to export continuous throughput, plot continuous throughput, or export table.

Figure 21: Options in the Context Menu for all Ports



2. When you choose to export or plot continuous throughput, provide the filename and window width in the dialog box. Window width is specified as follows:
 - Graph Iterations: In the case of Graph Iterations, the throughput is calculated based on equally sized sections of input or output data. By default, the iteration number used during the simulation is used for this calculation, which provides the most comprehensible results regarding the evolution of throughput over time.
 - Number of intervals: The simulation duration is divided into N number of intervals to compute the average throughput for each interval.
 - Active cycles per interval: Specify the length of the interval in terms of the number of active cycles per interval.

Figure 22: Export of Plot Continuous Throughput Dialog Box



If Export Continuous Throughput has been selected, a CSV file is generated.

```
Start Time (ps), End Time (ps), Throughput (Mbytes/s) for 'Inputp_1 for
'64' graph iterations
1486400, 2305600, 5000.0
2305600, 3128000, 4980.545
3128000, 4000000, 4697.2476
4000000, 4890400, 4600.1797
4890400, 5781600, 4596.0503
...
55673600, 56564800, 4596.0503
56564800, 57456000, 4596.0503
57456000, 58347200, 4596.0503
```


The CSV file contains the start and end times of each interval, as well as the computed throughput for the ports. To view the throughput graph, select the `Plot Continuous Throughput` option.

Hardware Emulation

To simulate the entire system, including AI Engine graph and PL logic along with XRT-based host application to control the AI Engine and PL, for a specific board and platform, you must use the Vitis hardware emulation flow. This flow includes the SystemC model of the AI Engine, transaction-level SystemC models for the NoC, DDR memory, PL Kernels (RTL), and PS (running on QEMU). You can also include RTL logic and test bench PL logic for your platform or design. Details on the flow can be found in [Application Verification Using Vitis Emulation Flow](#) in the *Embedded Design Development Using Vitis (UG1701)*.

For details on speed and accuracy of the system including AI Engine, PL models, NoC, DDR memory, PS and other I/O models, see [Speed and Accuracy of Hardware Emulation](#) in the *Embedded Design Development Using Vitis (UG1701)*.

Scope of HW Emulation is limited to modeling only functional performance of PS-to-AIE communication. Tests that rely on PS-to-AIE latency, that include features such as asynchronous RTP, where PS updates are propagated asynchronously, or tests that involve PS-initiated DDR activities might be impacted by the reduced accuracy.

External Traffic Generator

In AI Engine simulation and x86 simulation, the PL connections are replaced by files. These files must be provided with already created data for the input and they are created during the simulation runtime for the output. This can be cumbersome when a PL kernel is right in the middle of the graph. In that case, there is no direct relationship between the output values and the re-entered values on the PLIOs.

In software and hardware emulation, you must provide HLS and/or RTL code for PL located kernels. This can be a problem if these PL kernels are not designed yet.

In order to overcome these difficulties, you can write external traffic generators in Python, C++ or System Verilog. The same traffic generator can be used in simulation and emulation. For additional information, see [Using Traffic Generators for AI Engine Design](#) in *Embedded Design Development Using Vitis (UG1701)*.

Performance Analysis of AI Engine Graph Application during Simulation

A system-level view of program execution can be helpful in identifying problems during program execution including correctness and performance issues. Problems such as missing or mismatching locks, buffer overruns, and incorrect programming of DMA buffers are examples that are difficult to debug by using explicit print statements or by using traditional interactive debuggers. A systematic way of collecting system level traces for the program execution is needed. The AI Engine architecture has direct support for generation, collection, and streaming of events as trace data during simulation, hardware emulation, or hardware execution.

Analyzing AI Engine Status in Hardware Emulation

Generating and Analyzing AI Engine Status

During hardware emulation, status output can be generated periodically. The status and warnings can be analyzed in the Analysis View of the AMD Vitis™ Unified IDE for debug purposes.

1. Add the following options in your `xrt.ini` to output the AI Engine status. This is a one-time set up that results in periodic output of status data, including deadlock detection.
 - `aie_status=true`: Turns on the output generation feature.
 - `aie_status_interval_us`: Specifies the interval of the AI Engine status output.

Example:

```
[Debug]
aie_status=true
aie_status_interval_us=1000
```

The following files are generated by hardware emulation.

- `xrt.run_summary`: A summary file generated by hardware emulation that can be opened in the Analysis View of the Vitis Unified IDE.

- `aie_status_edge.json`: A status of AI Engine and AI Engine memory.
 - `aieshim_status_edge.json`: A status of AI Engine interface tiles.
 - `summary.csv`: A summary CSV file.
2. The summary and JSON files must be copied from the QEMU Linux system to your local system using the `scp` command. The status files must be in a local directory prior to the analysis in Analysis View of the Vitis Unified IDE. For more details on how to copy these files to your local system, see [Running the System on Embedded Processor Platform](#) in the *Embedded Design Development Using Vitis (UG1701)*.
 3. Analyze the summary and JSON files in Analysis View of the Vitis Unified IDE. For details, see [Analyzing AI Engine Status](#).

AI Engine Simulation-Based Performance Analysis

In simulation, to view time stamped events, different event types, and data associated with each event, value change dump (VCD) files can be used. VCD files provide a detailed dump of the simulated hardware signals. Additionally, a profile summary provides annotated details for the overall application performance.

AI Engine Simulation-Based Profiling

Profiling Data Generation

In the simulation framework, the AI Engine simulator can generate a profiling report for the complete application. This report is generated using the flag `--profile`.

```
aiesimulator -pkg-dir=Work --profile
```

Text files and XML files are generated in the directory `aiesimulator_output`. Two types of files are generated for the tile located in column C and row R. The `*_funct` reports the number of calls and number of cycles for each function. The `*_instr` is a report that goes down to the assembly code. To visualize the report, use the Analysis View of the Vitis Unified IDE.

```
vitis -a aiesimulator_output/default.aierun_summary
```

The Profile tab opens the Profile report, which shows a menu of sections that show information.

- **Summary:** Reports the total cycle count, total instruction count, and program size in memory.
- **Function Reports:** Shows several key indicators of the functions in the graphs.
 - **Number of calls:** Reports the number of times the function is executed

- **Total function time (cycles and %):** Reports the function execution time (in cycles and as a percent). This is the time required to execute the code within a function, exclusive of any calls to its descendants.
- **Total function + descendant time (cycles and %):** Reports the function execution time, as well as the execution time of the descendant functions (descendant functions are functions called by the function whose profile information is being reported). The "Total Function + descendant time" represents the total time required to execute the code within a function and in any function it calls, including the time spent in its descendant functions. It is important to note that the time includes the time spent in the function itself as well as the time spent in all the functions it calls, directly or indirectly.
- **Min/Avg/Max function time (cycles):** Reports the minimum/average/maximum function execution time (in cycles and as a percent).
- **Min/Avg/Max function + descendant time (cycles):** Reports the minimum/average/maximum function execution time, as well as the execution time of the descendant functions (descendant functions are functions called by the function whose profile information is being reported).
- **Program counter Low/High:** Reports the lowest and highest program counter value for a specific function.
- **Profile Details:** Shows the assembly code, function by function, with useful precisions. The columns are as shown in the following table.

Table 14: Profile Details Column Description

Column Name	Content
PC	Program counter
Instruction	Up to 16 bytes for each line
Assembly	Assembly code mnemonic with the full 7-way instruction word
Exe-count	Number of times this line has been executed by the processor
Cycles	Number of cycles required
Wait States	For some instructions you can have memory conflicts which end up into a number of wait-states
Relative cycle use within function	Shown as '*' lines where the relative length visually shows the relative cycles use of this instruction within the function
Relative cycle use within simulation	Shown as '*' lines where the relative length visually shows the relative cycles use of this instruction within the simulation (including <code>main()</code> and all functions)
Relative wait-state use within function	Shown as 'W' lines where the relative length visually shows the relative cycles used by wait-states during this instruction within the function
Relative wait state use within simulation	Shown as '*' lines where the relative length visually shows the relative cycles used by wait states during this instruction within the simulation (including <code>main()</code> and all functions)

Performance Debug with Profiling Data

Performance improvement of a design should start by optimizing the function that takes most of the cycles. After it is done, you can optimize functions that take less and less proportion of the cycles. For this purpose, the total function time graph will help you in selecting these functions.

For the optimization itself, you can use pragmas (`chess_prepare_for_pipelining`, `chess_loop_range`) for the usual pipelining, unrolling of loops. The Profile Details tab provides insight about wait states. Even if your inner-loop is perfectly optimized, you might lose some cycles due to wait-states. These wait-states occur when there are conflicts in resource access.

- Two reads or one read and one write within the same memory bank, either from the local AI Engine, or two contiguous AI Engines.
- The local AI Engine tries to access a bank (either read or write) while a memory DMA is accessing it for some data transfer.

Here is an example of profile details.

Figure 23: Profile Details

PC	Instruction	Assembly	Exe-count	Cycles	User Count	Wait states
1296 30 08 19 ff d4 02 90 04 aa df 60 3f		MOV.s10 m0, #32; MOV.s9 r9, #2; PADDS [p0], #8; MOV.u20 p3, #171872; NOP	4	4	0	0
1308 08 ac 49 68 67 d2 08 06 22 01 01 3f		PADDA [p1], #24; VLDB wc0, [p3]; LSHL r12, r6, r9; MOV.u20 c10, #557313; NOP	4	4	0	0
1320 6a a8 18 c0 04 ff 98 04 02 80 00 3f		LDA p2, [p0], #16; MOV.s9 r8, #0; ADD r12, r12, #1; MOV.u20 c12, #0; NOP	4	4	0	0
1332 41 a6 16 c0 e7 d3 92 04 02 f2 10 3f		LDA cb1, [p0], #12; MOV.s9 r6, #7; LSHL r9, r12, r9; MOV.u20 c13, #12816; NOP	4	4	0	0
1344 49 d0 17 c1 65 00 f0 00 12 00 01 3f		LDA cb1, [p0]; MOV.s9 r7, #11; ADD.NRM s0, r7, #1; MOV.T.s12 c10, #1; NOP	4	4	0	0
1356 40 a6 59 c0 83 9f 3e 04 00 00 00 3f		LDA cb0, [p1], #12; MOV.s9 r9, #4; ADD lc, r9, #1; MOV.u20 ch0, #0; NOP	4	4	0	0
1368 48 b2 51 c0 68 06 7a 04 22 41 01 3f		LDA cs0, [p1], #28; MOV.s9 r1, #3; MOV.s12 r13, #12; MOV.u20 c11, #33025; NOP	4	4	0	0
1380 69 d0 50 c1 e0 00 01 00 6f 06 9a df		LDA p1, [p1]; MOV.s9 r0, #15; NOP; MOV.s12 r12, #8; MOV p3, p1	4	4	0	0
1392 31 04 12 c2 70 00 00 00 12 40 01 3f		MOV.s10 m1, #16; MOV.s9 r2, #19; NOP; MOV.T.s12 c11, #1; NOP	4	4	0	0
1404 80 00 00 00 04 00 07 f7		NOP; MOV.u20 ch1, #0; NOP	4	4	0	0
1412 80 00 00 00 40 60 62 07 f7		NOP; MOV.u20 ls, #1568; NOP	4	4	0	0
1420 80 00 00 00 40 6c 68 07 f7		NOP; MOV.u20 le, #1664; NOP	4	4	0	0
1428 00 01		NOP	4	4	0	0
1430 1a 10 80 00 00 00 00 37		VLDA wd0, [p2], #0, cycl; NOP; NOP	4	4	0	0
1438 1e 10 85 68 40 00 00 17		VLDA wd1, [p2], #0, cycl; VLDB wr2, [p2]; NOP	4	4	0	0
1446 00 01		NOP	4	4	0	0
1448 aa c5		MOV p4, p2	4	4	0	0
1450 0e 10 b8 03		NOP; VLDB wr3, [p2], #0, cycl	4	8	0	0
1454 00 01		NOP	4	4	0	0
1456 aa d5		MOV p5, p2	4	4	0	0
1458 00 01		NOP	4	4	0	0
1460 0a 10 b8 03		NOP; VLDB wr2, [p2], #0, cycl	4	4	0	0
1464 1a d1 00 00 00 00 00 37		VLDA wd0, [p4]; NOP; NOP	4	4	0	0
1472 00 01		NOP	4	4	0	0
1474 aa c5		MOV p4, p2	4	4	0	0
1476 00 00 09 15 01 00 3c 80 09 3d a0 8f		NOP; NOP; VMOV xa, xb; NOP; VMUL.48 am0, yd.c16, r9, c3, r7, wc0.s16, #0, c2, c0	4	4	0	0
1488 1e d1 47 08 50 00 00 40 00 00 94 11 23		VLDA wd1, [p5]; VLDB wr3, [p2], #0, cycl; NOP; NOP; NOP; VMAC.48 am0, ya.c16, r8, c3, r6, wc0.s	4	4	0	0
1504 80 00 01 00 12 33 41 47		NOP; NOP; VMUL.48 am1, yd.c16, r9, c3, r7, wc0.s16, #0, c2, c0	4	4	0	0
1512 80 00 01 41 12 76 45 47		NOP; NOP; VMAC.48 am1, yd.c16, r9, c3, r7, wc0.s16, #4, c2, c1	4	4	0	0
1520 00 00 00 00 10 00 00 46 ab 40 00 92 01 34 b4 10		NOP; NOP; NOP; MOV p5, p2; NOP; VMUL.48 am0, yd.c16, r13, c3, r1, wc0.s16, #0, c2, c0	4	8	0	4
1536 1a d1 05 08 50 00 00 40 00 00 92 01 30 34 14		VLDA wd0, [p4]; VLDB wr2, [p2], #0, cycl; NOP; NOP; NOP; VMUL.48 am1, yd.c16, r12, c3, r0, wc0.	4	4	0	0
1552 00 00 00 00 10 00 00 64 54 00 00 96 11 35 34 54		NOP; NOP; NOP; VMOV xa, xb; NOP; VMAC.48 am1, yd.c16, r13, c3, r2, wc0.s16, #4, c2, c1	4	4	0	0
1568 00 00 01 aa c1 00 3c b0 89 81 a2 0f		NOP; NOP; MOV p4, p2; NOP; VMAC.48 am0, yd.c16, r12, c3, r0, wc0.s16, #4, c2, c1	56	56	0	0
1580 00 00 09 15 01 00 3c 80 09 3d a0 8f		NOP; NOP; VMOV xa, xb; NOP; VMUL.48 am0, yd.c16, r9, c3, r7, wc0.s16, #0, c2, c0	56	112	0	56
1592 1e d1 47 08 50 00 00 40 00 00 94 11 23		VLDA wd1, [p5]; VLDB wr3, [p2], #0, cycl; NOP; NOP; NOP; VMAC.48 am0, ya.c16, r8, c3, r6, wc0.s	56	56	0	0
1608 e0 20 49 00 12 33 41 47		NOP; VST.48.SRSS am1, s0, [p4], #1, cycl; VMUL.48 am1, yd.c16, r8, c3, r6, wc0.s16, #0, c2, c0	56	56	0	0
1616 00 00 00 00 10 00 00 46 ab 40 00 92 01 30 34 14		NOP; NOP; VST.48.SRSS am0, s0, [p1], #1, cycl; NOP; NOP; VMAC.48 am1, yd.c16, r9, c3, r7, wc0.	56	56	0	0
1632 00 00 00 1c 14 09 46 ab 40 00 92 01 30 34 14		NOP; NOP; VST.48.SRSS am1, s0, [p1], #1, cycl; NOP; MOV p5, p2; NOP; VMUL.48 am0, yd.c16, r13, c3,	56	112	0	56
1648 1a d1 05 08 5c 10 09 40 00 00 92 01 30 34 14		VLDA wd0, [p4]; VLDB wr2, [p2], #0, cycl; VST.48.SRSS am0, s0, [p1], #1, cycl; NOP; NOP; VMUL.48	56	56	0	0
1664 00 00 00 00 10 00 00 64 54 00 00 96 11 35 34 54		NOP; NOP; NOP; VMOV xa, xb; NOP; VMAC.48 am0, yd.c16, r13, c3, r2, wc0.s16, #4, c2, c1	56	56	0	0
1680 00 00 0f 91 a1 00 3c b0 89 81 a2 0f		NOP; NOP; MOV.s9 cs2, #28; NOP; VMAC.48 am0, ya.c16, r12, c3, r0, wc0.s16, #4, c2, c1	4	4	0	0

In this screenshot, the inner-loop is first localized with *ls* and *le* which are the loop-start and loop-end PC. This inner loop is shown within the blue rectangle. This seems correct as the Exe-Count is much higher on these lines than on the others.

In the second step, an instruction that is constantly taking two clock cycles instead of one, due to wait-states is localized:

- Exe-count = 56
- Cycles = 112

The VMUL instruction takes data in register yd and coefficients in wc0. A load taking seven clock cycles before the data is effectively in the register. The third step is to analyze the code seven clock cycles before the first iteration (3) or within the loop on the previous iteration (3'). There are two loads in each case which are on the same bank (all the reads are from the same window in the source code).

AI Engine Simulation-Based Value Change Dump

In the simulation framework, the AI Engine simulator can generate a detailed dump of the hardware signals in the form of value change dump (VCD) files. A defined set of abstract events describes the execution of a multi-kernel AI Engine program in terms of these events. The output of a VCD file is enabled using the `aiesimulator --dump-vcd` command.

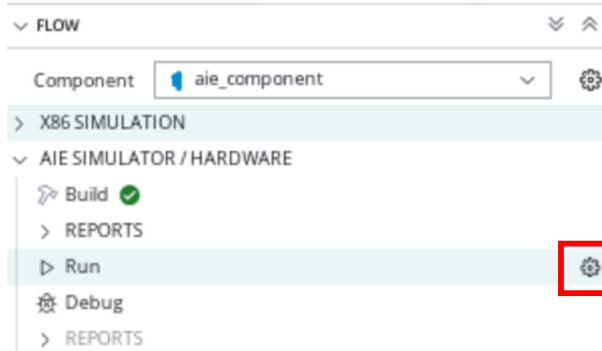
After simulation, or emulation, the VCD file can be processed into events and viewed on a timeline in the Vitis IDE. The events contain information such as time stamps, different event types, and data associated with each event. This information can be correlated to the compiler generated debug information. This includes program counter values mapped to function names and instruction offsets, and source level symbolic data offsets for memory accesses.

The abstract AI Engine events are independent of the VCD format and will be directly extracted from the hardware. The events traces can be produced as plain text, comma-separated values (CSV), or in waveform database (WDB), and the generated event trace data can be viewed in the Analysis View of the Vitis unified IDE.

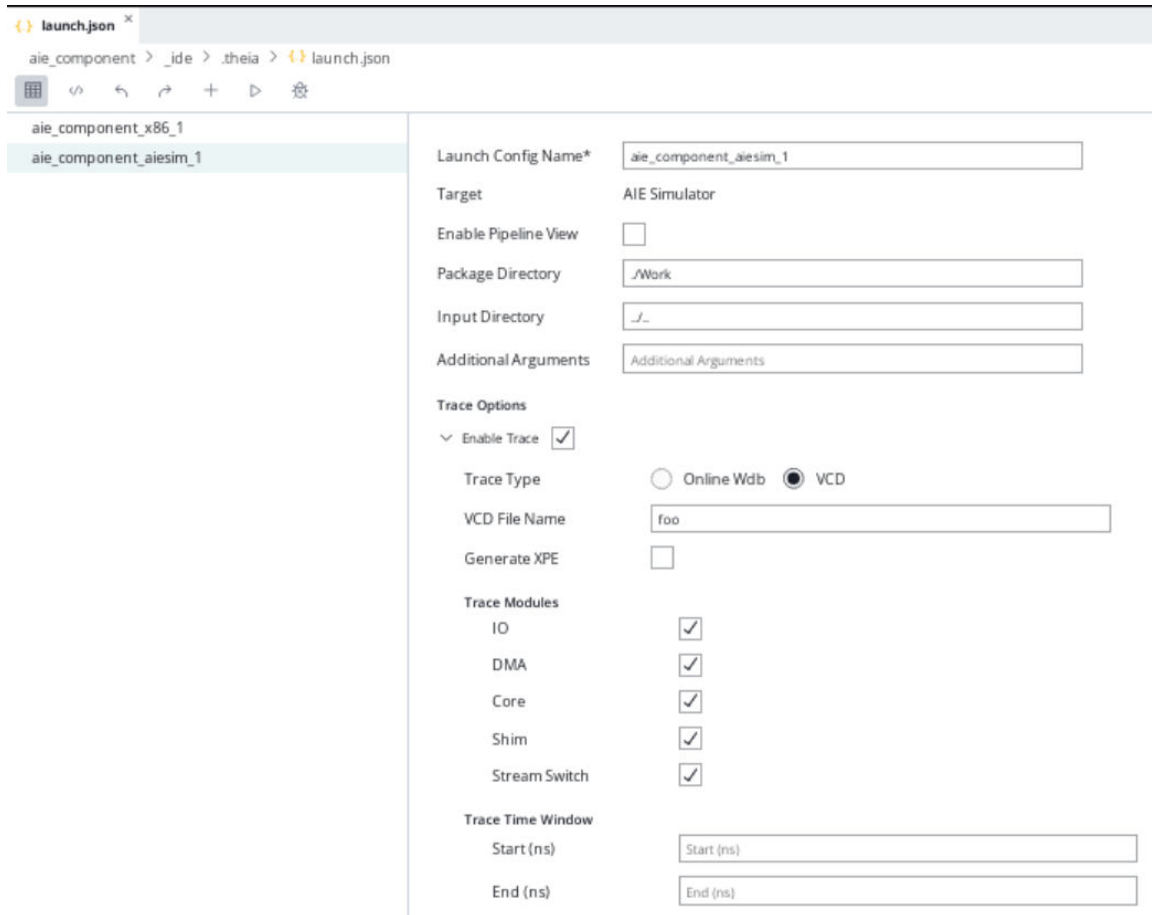
Generating a VCD File in the IDE

To generate a VCD file from the Vitis IDE

1. From the Flow Navigator, click the **AI Engine Run Settings** button, as shown in below image.



2. Click the `launch.json` file of the aiesimulator, and set the fields as follows:
 - Select the **Enable Trace** check box.
 - Set Trace Type to **VCD**.
 - Enter a filename in the VCD File Name field.



3. Right-click on your AI Engine graph project from the **Explorer** view and select **Run As → Run Configurations**. This opens up the Run Configurations dialog box for the current project.
4. Select the **AI Engine Emulator** option and double-click to open a new configuration. Select the **Generate Trace** check box to enable trace capture, and select the **VCD Trace** button. By default, this produces a VCD dump in a file called `foo.vcd` in the current directory. You can rename the file if you like. By default, CLK and STREAM_SWITCH signals are disabled from full VCD trace.

The VCD file can also be generated by invoking the AI Engine simulator with the `--dump-vcd <filename>` option on the command line. The VCD file is generated in the same directory as the simulation is run. Assuming that the program is compiled using the AI Engine compiler, the simulator can be invoked in a shell with the VCD option.

```
$ aiesimulator --pkg-dir=./Work --dump-vcd=foo
```

This command produces the VCD file (`foo.vcd`) which is written to the current directory.

AI Engine Trace from VCD

The `vcdanalyze` utility is provided to generate an AI Engine event trace from the VCD file. This process is integrated into the Vitis tool flow automatically. From the Vitis IDE, after a simulation run has finished capturing AI Engine events, you can right-click on the project from the **Explorer** view and select **Analyze AIE Events**. The trace data is produced under the current project at `trdata.aiesim/` and various views are automatically loaded into the current project.

The raw event trace under the directory `trdata.aiesimevents.txt` should look like the following:

```
time=1741000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=65536,data1=0,tlast=0
time=1742000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=196610,data1=0,tlast=0
time=1743000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=327684,data1=0,tlast=0
time=1743000,event=CORE_RESET,col=1,row=0
time=1744000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=458758,data1=0,tlast=0
time=1745000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=589832,data1=0,tlast=0
time=1746000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=720906,data1=0,tlast=0
time=1747000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=851980,data1=0,tlast=0
time=1748000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=983054,data1=0,tlast=0
time=1749000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=1,data1=0,tlast=0
time=1750000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=131075,data1=0,tlast=0
time=1751000,event=FROM_PL,name=tl.me.shim.tile_0_0.pl_interface.pl_to_shim0.data0,col=0,streamid=0,data0=262149,data1=0,tlast=0
time=2186000,event=DM_WRITE_REQ,col=0,row=0,port=tl.me.array.tile_0_1.mm.dm.port_AXI_write_b6
time=2190000,event=DM_WRITE_REQ,col=0,row=0,port=tl.me.array.tile_0_1.mm.dm.port_AXI_write_b7
time=2194000,event=DM_WRITE_REQ,col=0,row=0,port=tl.me.array.tile_0_1.mm.dm.port_AXI_write_b6
time=2198000,event=DM_WRITE_REQ,col=0,row=0,port=tl.me.array.tile_0_1.mm.dm.port_AXI_write_b7
time=2202000,event=DM_WRITE_REQ,col=0,row=0,port=tl.me.array.tile_0_1.mm.dm.port_AXI_write_b2
time=2206000,event=DM_WRITE_REQ,col=0,row=0,port=tl.me.array.tile_0_1.mm.dm.port_AXI_write_b3
```

The following command produces the AI Engine trace data for `foo.vcd` in text form in the `./trdata.aiesim/events.txt` file.

```
vcdanalyze -vcd foo.vcd
```



TIP: Use `vcdanalyze -h` to get help for the command.

The following command produces the waveform data files from the AI Engine trace data from the `foo.vcd` file.

```
vcdanalyze -vcd foo.vcd -wdb
```


Note: The simulator models the hardware performance of AI Engine when possible. The AI Engine simulator can be more optimistic than the actual hardware, where two DMA FIFOs in the same AI Engine tile are accessed via the same (North, East, West or South) interface. For this corner case, there is arbitration in the memory controller shared between the DMA FIFOs; this effect is not fully reflected in the simulation.

Generating VCD with Select Signals

When `--dump-vcd` option is provided to the `aiesimulator` all (except `CLK,STREAM_SWITCH`) signals are generated in the VCD file. This could slow down simulation. Alternatively, using options to generate select signals associated with specific modules can improve simulation speed. To debug a design, it is recommended that you run `aiesimulator` with `--dump-vcd` to generate all the signals. After the issue is narrowed down, run the `aiesimulator` with options to generate a subset of VCD signals. This can be achieved by passing `aiesimulator` with specific options through an options text file. Signals are selected based on the options provided in the text file.

To enable VCD generation for specific modules, select the module type in a text file, and pass the file as input to the `aiesimulator` command line. The contents of the file is merged into the generated `aie_simulator/aiesim_options.txt`. The usage of this file in the HW emulation flow remains the same. Options passed to the `aiesimulator` command line will have higher precedence over the same options provided in the options file.

```
aiesimulator --options-file=<user_aiesim_options.txt> --dump-  
vcd=<file_name>
```

The list of selections can be found below. By default, all modules except `CLK` and `STREAM_SWITCH` are set to `true`. Set the option to `false` to turn off modules.

Row/Column Selection

```
AIE_DUMP_VCD_ROW=(<row start index>,<row end index>),( <row start  
index>,<row end_index>)...  
AIE_DUMP_VCD_COL=(<column start index>,<column end index>),( <column start  
index>,<column end_index>)...
```

- **Row:** Use `AIE_DUMP_VCD_ROW` to generate VCD data for all the active cores in the selected row. You must specify absolute coordinates that consider the AI Engine interface row as `row 0`.

Example: To generate VCD data for active cores present between rows 1 to 4, specify it as `AIE_DUMP_VCD_ROW=(2,5)` in the `options.txt` file.

- **Column:** Use `AIE_DUMP_VCD_COL` to generate VCD data for all the active cores present in the selected column.

Example: To generate VCD data for active cores present between columns 23 to 26, specify it as `AIE_DUMP_VCD_COL=(23,26)` in the `options.txt` file.

- **Time interval:** Use `AIE_DUMP_VCD_INTERVAL` to generate VCD data for a given time period. The VCD file generated will contain data related to all active cores for the selected time period. Only one time period specification is supported.

Note: `End time -1` will mean end of simulation.

Example: To generate VCD data from 600 ns to 700 ns, use
`AIE_DUMP_VCD_INTERVAL=(600,701)`.

Tile Type Selection

```
AIE_DUMP_VCD_CORE=true/false
AIE_DUMP_VCD_SHIM=true/false
AIE_DUMP_VCD_MEM=true/false (valid for AIEngine ML designs only)
```

- **Core:** Use `AIE_DUMP_VCD_CORE` to generate VCD data with signals associated with processor modules, programs and data memory.
- **Shim:** Use `AIE_DUMP_VCD_SHIM` to generate VCD data with signals associated with all AI Engine interface tiles.
- **Mem:** Use `AIE_DUMP_VCD_MEM` to generate VCD data with signals associated with all mem tiles. This option valid for AI Engine-ML designs only.

Tile Submodule Selection

To output VCD data around tile submodules, you must select the tile type category described above.

```
AIE_DUMP_VCD_IO=true/false
AIE_DUMP_VCD_DMA=true/false
AIE_DUMP_VCD_STREAM_SWITCH=true/false
AIE_DUMP_VCD_CLK=true/false
```

This categorization of module signals is directly related to the category of the information shown on the Analysis View of the Vitis Unified IDE:

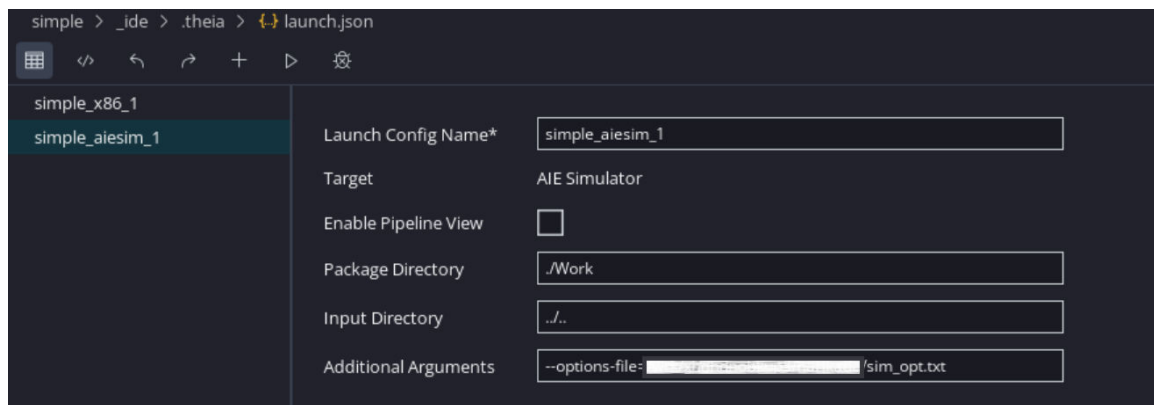
- **I/O:** Use `AIE_DUMP_VCD_IO` to generate VCD data with signals associated with AI Engine kernel input/output, locks, stalls. To generate this VCD data you must specify this option with any one of the tile type options: `AIE_DUMP_VCD_CORE`, `AIE_DUMP_VCD_SHIM`, or `AIE_DUMP_VCD_MEM`.
- **DMA:** Use `AIE_DUMP_VCD_DMA` to generate VCD data with signals associated with DMA transfers, and locks. To generate this VCD data you must specify this option with any one of the tile type options: `AIE_DUMP_VCD_CORE`, or `AIE_DUMP_VCD_MEM`.

- **Stream Switch:** Use `AIE_DUMP_VCD_STREAM_SWITCH` to generate VCD data with signals associated with all the streams on the stream switch boundary. To generate this VCD data you must specify this option with any one of the tile type options: `AIE_DUMP_VCD_CORE`, `AIE_DUMP_VCD_SHIM`, or `AIE_DUMP_VCD_MEM`.
- **Clock:** Use `AIE_DUMP_VCD_CLK` to generate VCD data for the clock signal.

Using Vitis IDE to Generate a VCD with Select Signals

To generate a VCD file with select signals, follow the instructions for generate a VCD, as outlined in [Generating a VCD File in the IDE](#). In the Run Settings dialog box in the Vitis IDE, specify a user simulation options file in the Additional Arguments field, such as, `--option-file=<user_sim_options.txt>`. The following figure shows how to specify the user simulation options file in order to generate a selective VCD.

Figure 24: Vitis IDE to Enable Selective VCD File Generation

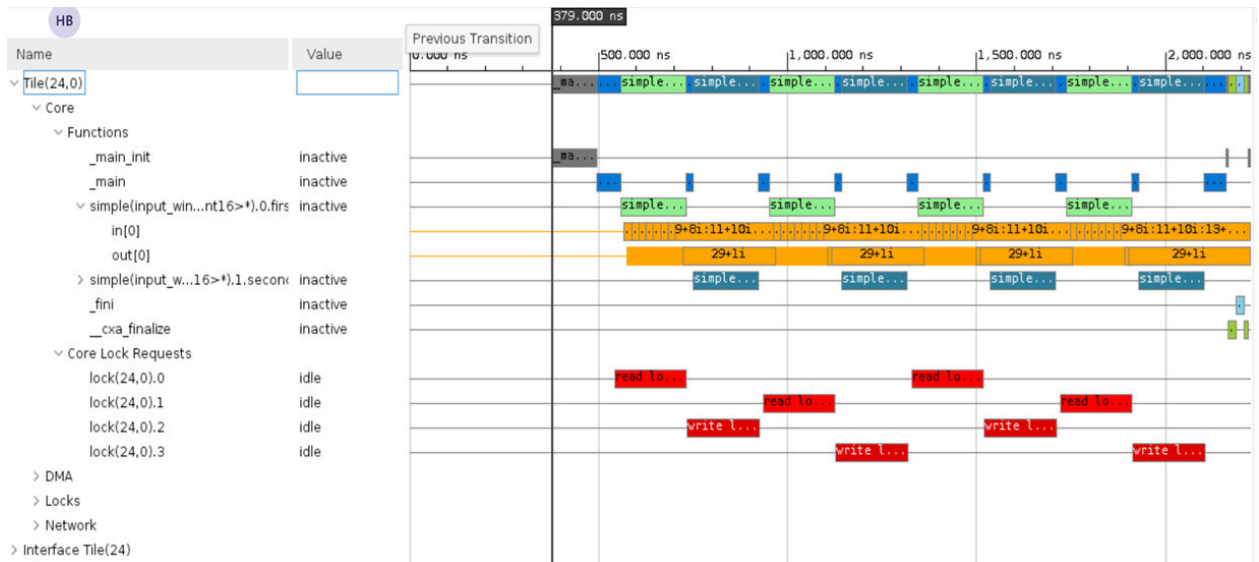


Viewing the Generated VCD in the Vitis Unified IDE

The Vitis Unified IDE displays the signals generated in the VCD. By default, stream switch FIFO signals, and signals connected between the PLIO and the AI Engine are generated in the VCD and are displayed in the IDE. The following section explains the details of the VCD signal selection.

- **AI Engine I/O:** When the `AIE_DUMP_VCD_IO` option is enabled in the `options.txt` file, the I/Os for each of the kernels are displayed in the IDE. All the stall signals for lock requests can also be seen.

Figure 25: AI Engine VCD I/O Data Display



- **DMA:** When the `AIE_DUMP_VCD_DMA` is enabled in the `options.txt` file, the DMA transaction on the buffers are displayed in the IDE. Also displayed:
 - DMA transactions.
 - MM2S, and S2MM transactions with their channel IDs.
 - Buffer descriptors (`BD0`, `BD1`).
 - The associated locks (`lock0`, `lock1`) with the values on the locks which indicate if the locks were acquired for read or write.

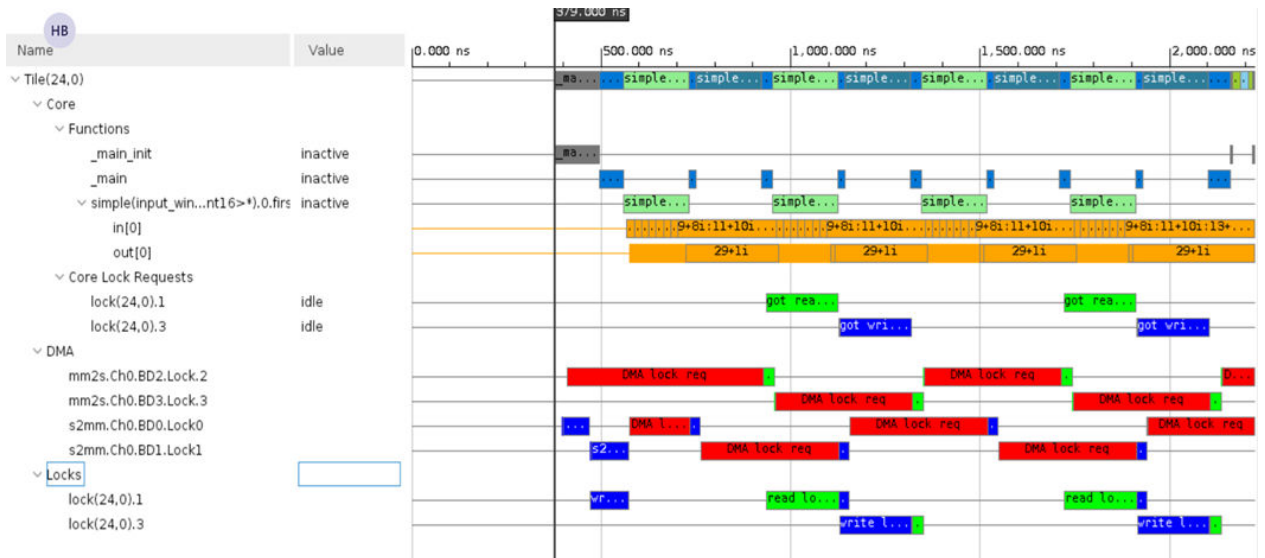
Figure 26: DMA VCD Data Display



- **Core:** When the `AIE_DUMP_VCD_CORE` option is enabled in the `options.txt` file, the core signals, which is a combination of kernels, I/O and the DMA signals, are displayed. Also displayed are:
 - All functions and their transitions.
 - All the stalls on the functions.
 - The kernel I/O with the core requesting read and write locks.
 - DMA transactions.
 - MM2S, and S2MM transactions with their channel IDs.

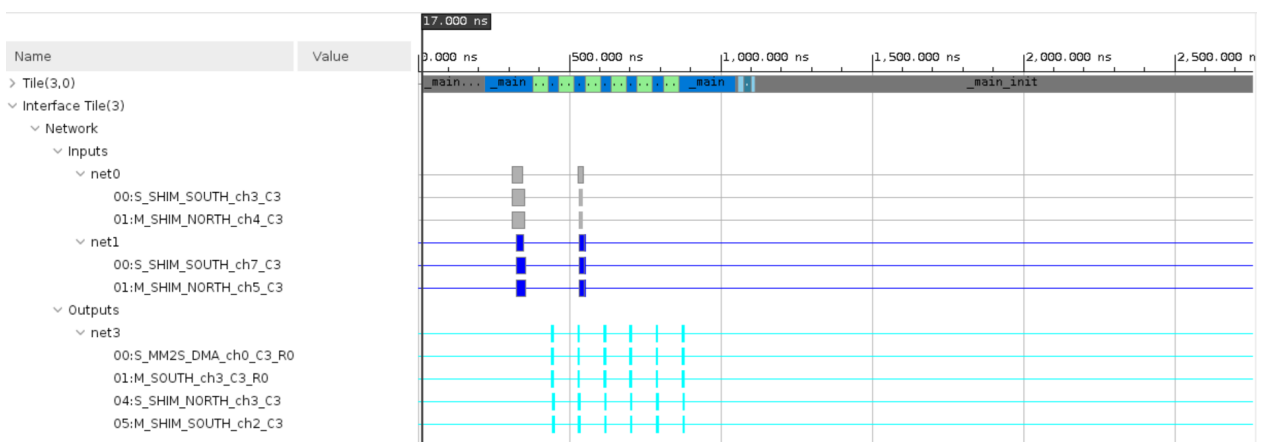
- Buffer descriptors.
- The associated locks with the values on the locks which indicate if the locks were acquired for read or write.

Figure 27: Core Data Display



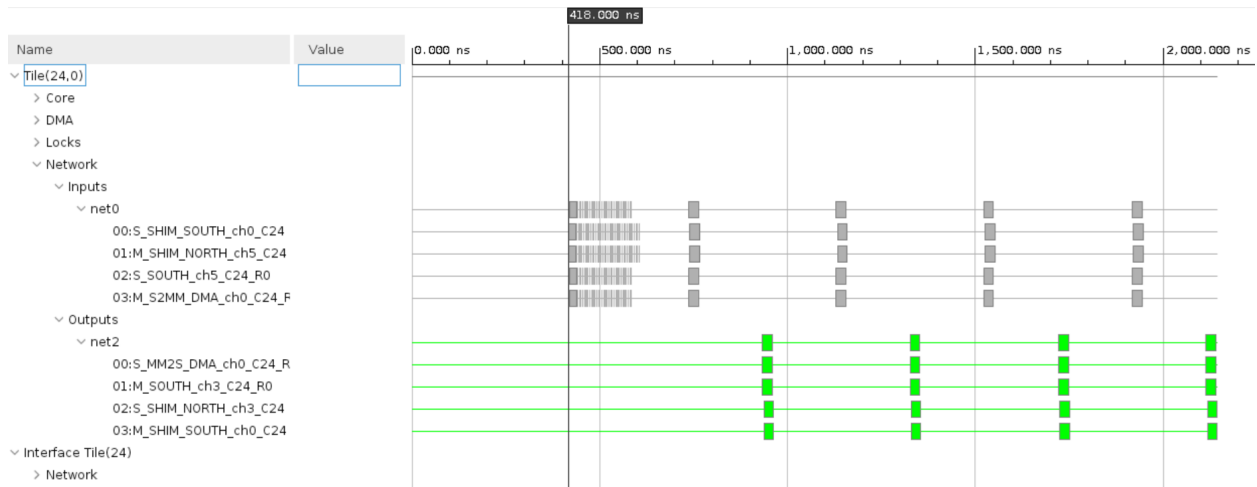
- **AI Engine Interface Tile (Shim):** When the AIE_DUMP_VCD_SHIM option is enabled in the options.txt file, the AI Engine interface tile signals are displayed. Also displayed are:
 - The streams to and from the tiles.
 - Net mapped to the tile.
 - Nets that pass through the tiles if the stream switch option is enabled along with the tile option.

Figure 28: AI Engine Interface Tile Data Display



- **Stream Switch:** When the `AIE_DUMP_VCD_STREAM_SWITCH` option is enabled in the `options.txt` file, the stream switch signals are displayed. The Trace contains all the nets related to the network for all the tiles under the stream switch. Only the network events can be seen in the trace.

Figure 29: Stream Switch Data Display



Viewing the Run Summary in the Vitis IDE

After running the system, whether in simulation, hardware emulation, or in hardware, a `run_summary` report is generated when the application has been properly configured.

During simulation of the AI Engine graph, the AI Engine simulator or hardware emulation, captures performance and activity metrics and writes the report to the output directory `./aiesimulator_output` and `./sim/behav_waveform/xsim`. The generated summary is called `default.aierun_summary`.

The `run_summary` can be viewed in the Vitis IDE. The summary contains a collection of reports, capturing the performance profile of the AI Engine application captured as it runs. For example, to open the AI Engine simulator run summary use the following command:

```
vitis -a ./aiesimulator_output/default.aierun_summary
```

The Vitis IDE opens displaying the Summary page of the report. The tool lists the different reports that are available in the summary. For a complete understanding of the Analysis view, see [Working with the Analysis View \(Vitis Analyzer\)](#) in the *Vitis Reference Guide* (UG1702).

Note: The `default.aierun_summary` also contains the some of the same reports as `<GRAPH_TB_FILE_NAME>.aiecompile_summary`. These reports are Graph and Array. To see those reports go to the [Opening AI Engine Compilation Summary Reports](#) in the *Vitis Reference Guide* (UG1702).

Report Summary

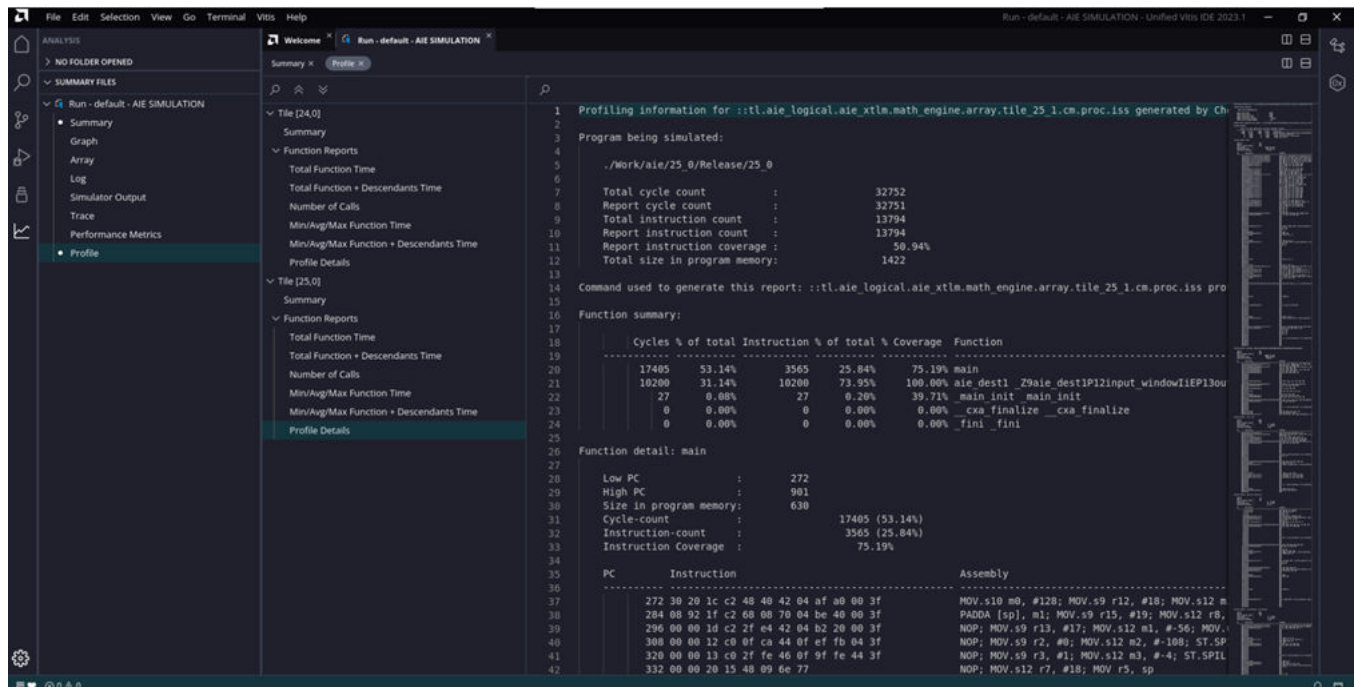
This is the top-level of the report, and reports the details of the run, such as date, tool version, and the command-line used to launch the simulator.

Profile Summary

When the `aiesimulator --profile` option is specified, the simulator collects profiling data on the AI Engine graph and kernels presenting a high-level view of the AI Engine graphs, kernels-mapped to processors, with tables and graphic presentation of metric data.

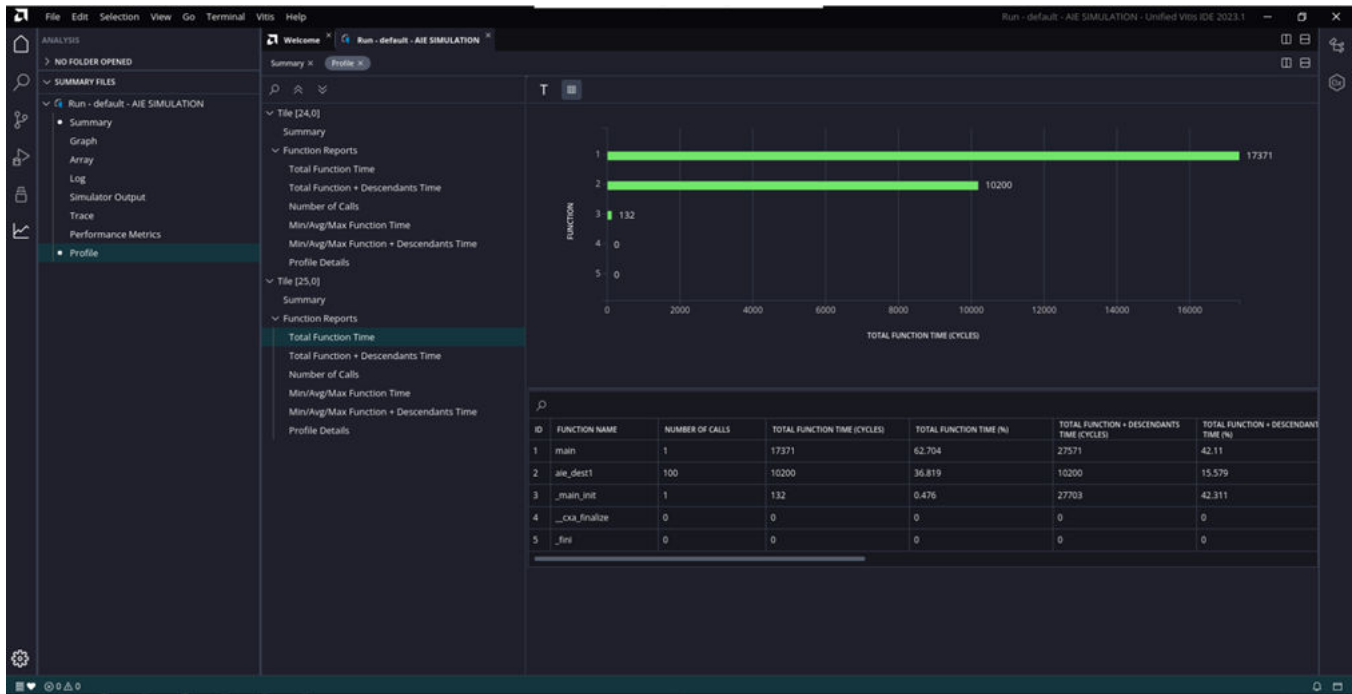
The Profile Summary provides annotated details regarding the overall application performance. All data generated during the execution of the application is grouped into categories. The Profile Summary lets you examine processor/DMA memory stalls, deadlock, interference, critical paths, and maximum contention. This is useful for system-level performance tuning and debug. System performance is presented in terms of latency (number of cycles taken to execute the system) and throughput (data/time taken). Sub-optimal system performance forces you to examine and control (thru constraints) mapping and buffer packing, stream and packet switch allocation, interaction with neighboring processors, and external interfaces. An example of the raw Profile Summary report is shown.

Figure 30: Profile Summary



Specific tables can be used to see profile information specific to the kernels. This is shown as a chart with a table showing what is running on the tiles. The following is an example chart.

Figure 31: Example Chart



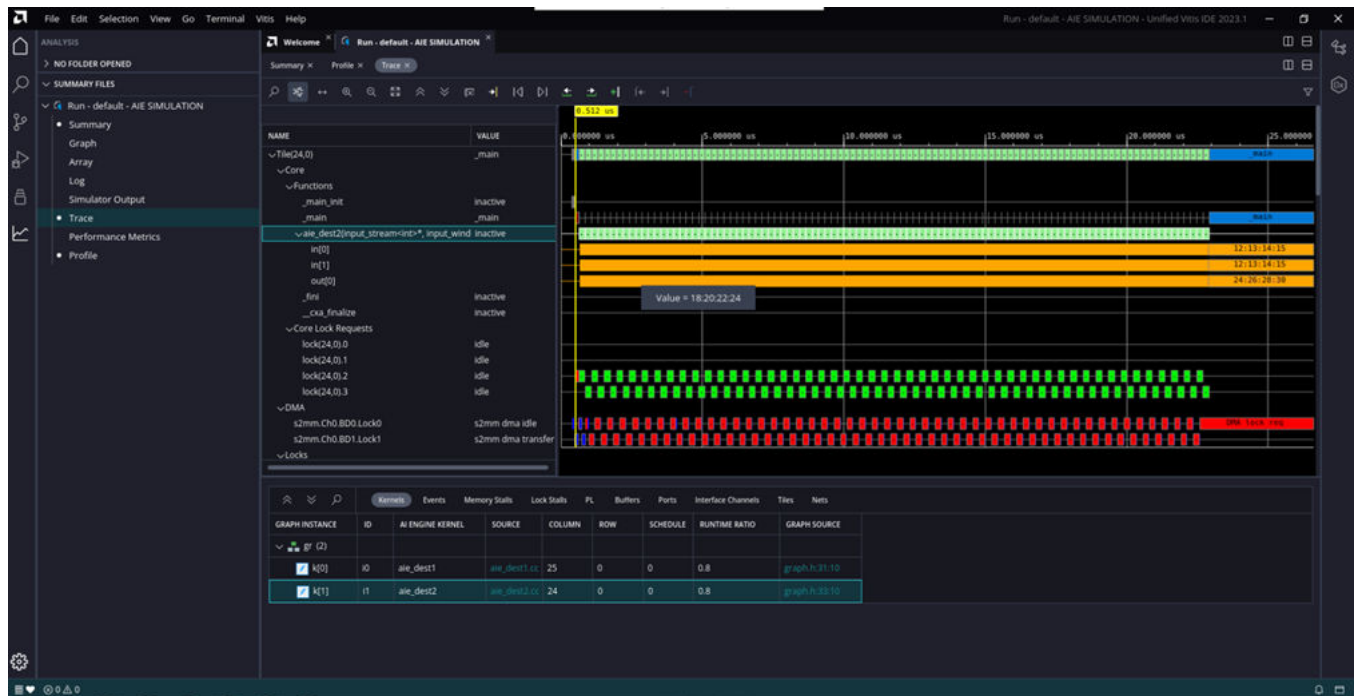
This view shows a chart with a Total Function Time that is the total cycles the function used in running the graph. The y-axis shows the id of the function that can be referenced in the ID column of the table below. This information can be useful in determining where time is being spent in a function and helps with potential optimization or debug. The table lists the:

- ID of the function profiled
- The function name
- The number of times the function was executed
- The total time taken in cycles to execute the function
- The total function execution time as a percent of the total execution time of the graph
- The total time taken in cycles to execute the function and the functions(descendants) called from within it
- The total time as a percent to execute the function and the functions(descendants) called from within it

Trace Report

Issues such as missing or mismatching locks, buffer overruns, and incorrect programming of DMA buffers are difficult to debug using traditional interactive debug techniques. Event trace provides a systematic way of collecting system level traces for the program events, providing direct support for generation, collection, and streaming of hardware events as a trace. The following image shows the Trace report open in the Vitis IDE.

Figure 32: Trace Report



Note: This example illustrates kernel function and functions that are added by the compiler:

- `_main`: Core `main` function. This is different from the function used in the top-level file.
- `_main_init`: Kernel `init` function that runs once per graph execution.
- `_cxa_finalize`: Calls destructors of global C++ objects.
- `_fini`: This section holds executable instructions that terminate the process. When a program exits normally, the system runs the code in this section.

Note: If the VCD file is too large and it takes too much time for the Vitis IDE to analyze the VCD and open the Trace view, you can do an online analysis of the VCD when running the AI Engine simulator. The IDE then opens the existing WDB and event files instead of analyzing the VCD file. The command for AI Engine simulator is as follows.

```
aiesimulator --pkg-dir=./Work --online -wdb -text
```

Features of the trace report include the following.

- Each tile is reported. Within each tile the report includes core, DMA, locks, and I/O if there are PL blocks in the graph.
- There is a separate timeline for each kernel mapped to a core. It shows when the kernel is executing (blue) or stalled (red) due to memory conflicts or waiting for stream data.
- Use lock IDs in the core, DMA, and locks sections to identify how cores and DMAs interact with one another by acquiring and releasing locks.
- The lock section shows the activities of the locks in the tile, both the allocation and release for read and write lock requests. A particular lock can be allocated by nearby tiles. Thus, this section does not necessarily match the core lock requests of the core shown in the left pane of the image.
- If a lock is not released, a red bar extends through the end of simulation time.
- Clicking the left or right arrows takes you to the start and end of a state, respectively.
- The data view shows the data flowing through stream switch network with slave entry points and master exit points at each hop. This is most useful in finding the routing delays, as well as network congestion effects with packet switching, where one packet might get delayed behind another packet when sharing the same stream channel.

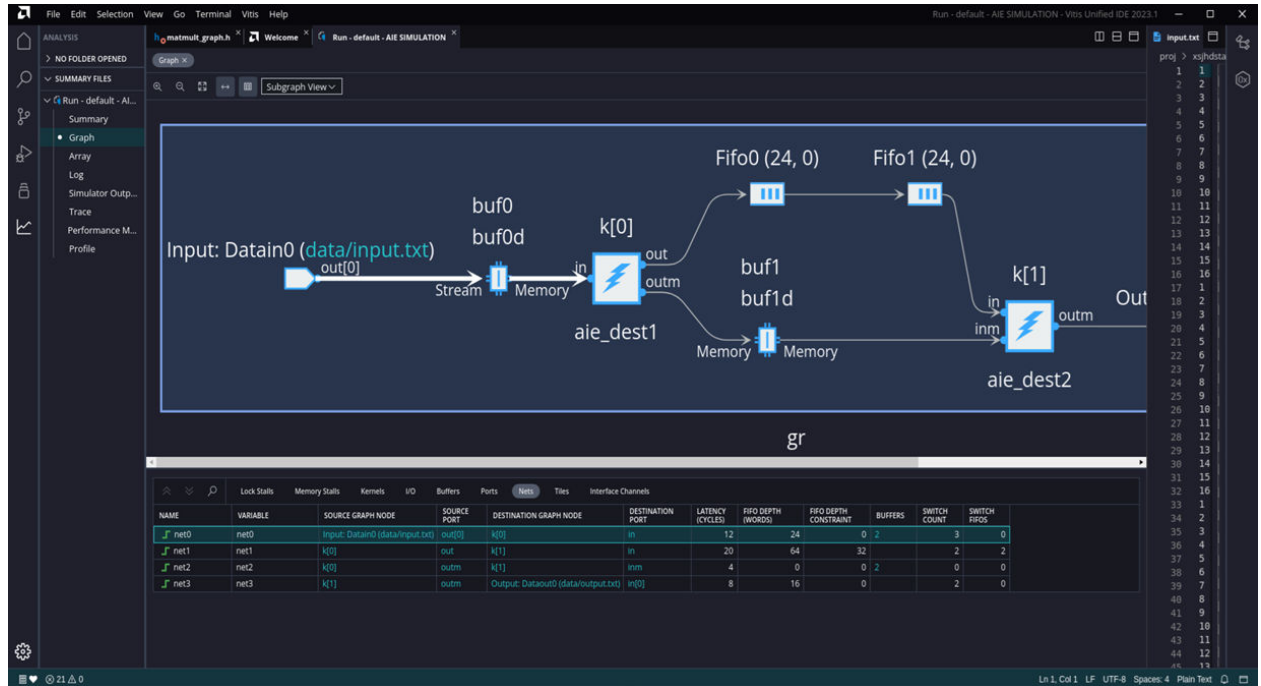
Trace View Data Visualization

Trace view data visualization allows you a side-by-side view of events at the I/O ports, allowing you to look into the design and examine the relative event timing. This provides insight into how the design works in a multi-core environment.

Window Data Analysis

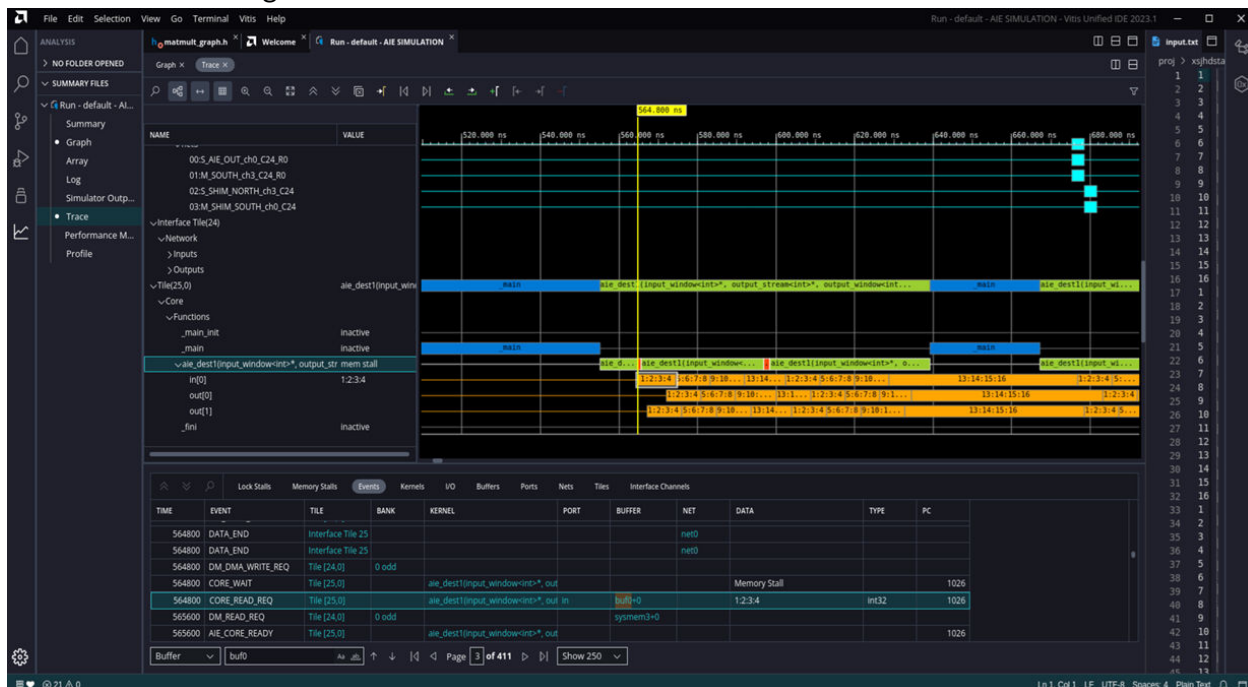
Window data analysis in the Trace view allows you to cross probe from a specific input/output window net connections in the graph view to the appropriate position in the Event Trace view. You can trace window input data as it is being buffered prior to the kernel starting execution. To perform data analysis on a window interface, use the following steps.

1. The following image shows a net connection that has been selected in the Graph view. The selected net (`net0`) is highlighted both in graph view and in the nets table at the bottom of the window. In addition the Graph view also associates the simulation input and output files with the appropriate nets. The simulation input file associated with `net0` is displayed on the right.



2. In the graph view, select the file source that connects to `net0`, as shown in the previous image. The input data is displayed at the right hand side of the window.

3. Select the kernel in Graph view, and switch to the Trace view to view the events and detailed event timing.



4. In the events table at the bottom of the screen, select **Buffer** from the column selector.
5. Type `buf0` in the adjacent (filter) box.
6. Core events that match `buf0` are highlighted in the events table. Browse through the events if there are multiple events using the **Previous/Next** toolbar buttons to show the matched events.
7. The highlighted data matches the data from the input file, shown in the right-hand pane.

Note: The AI Engine event kernel does not start until all the data is available at the window interface.

Using these steps, you can inspect I/O data for the required window connections to ensure the correctness of the I/O data to and from a kernel.

Buffer Data Types

Table 15: Buffer Data Types

Data Types	Applicable to		Display Format Example
	AIE (Y/N)	AIE-ML (Y/N)	
int8 ¹	Y	Y	±123
int16	Y	Y	±12345
int32	Y	Y	±1234567890
int64	Y	Y	±1234567890123456789

Table 15: Buffer Data Types (cont'd)

Data Types	Applicable to		Display Format Example
	AIE (Y/N)	AIE-ML (Y/N)	
uint8	Y	Y	123
uint16	Y	Y	12345
uint32	Y	Y	1234567890
uint64	Y	Y	12345678901234567890
cint16	Y	Y	±12345±12345i
cint32	Y	Y	±1234567890±1234567890i
float	Y	Y	±1.234567
cfloat	Y	Y	±1.234567±1.234567i
v4cint16	Y	Y	±1+2i:3+4i:5+6i:7+8i
v4int32	Y	Y	±1:2:3:4
v4cint32	Y	Y	±1+2i:3+4i
v4int64 ⁴	N	N	±1:2/3:4
v4float	Y	Y	±1.000000:2.000000:3.000000:4.000000
v4cfloat	Y	Y	±1.000000+2.000000i:3.000000+4.000000i
v8int16	Y	Y	±1:2:3:4:5:6:7:8
v8cint16	Y	Y	±1+2i:3+4i:5+6i:7+8i
v8int32	Y	Y	±1:2:3:4:5:6:7:8
v8float			±1.000000:2.000000:3.000000:4.000000:5.000000:6.000000:7.000000:8.000000
v16int8	Y	Y	±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v16uint8	Y	Y	1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v16int16 ²	Y	Y	±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v16uint16 ²	N	Y	1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v16int32	Y	Y	±1:2:3:4:5:6:7:8
v16float ²	Y	Y	±1.000000:2.000000:3.000000:4.000000:5.000000:6.000000:7.000000:8.000000:9.000000:10.000000:11.000000:12.000000:13.000000:14.000000:15.000000:16.000000
v16cfloat ²	Y	Y	±1.000000+2.000000i:3.000000+4.000000i:5.000000+6.000000i:7.000000+8.000000i:9.000000+10.000000i:11.000000+12.000000i:13.000000+14.000000i:15.000000+16.000000i
v32int8 ³	Y	Y	±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16:17:18:19:20:21:22:23:24:25:26:27:28:29:30:31:32
v32uint8 ³	Y	Y	1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16:17:18:19:20:21:22:23:24:25:26:27:28:29:30:31:32
v32int16			±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v32cint16	Y	Y	±1+2i:3+4i:5+6i:7+8i:9+10i:11+12i:13+14i:15+16i
v32int32			±1:2:3:4:5:6:7:8
v32float	Y	Y	±1.000000:2.000000:3.000000:4.000000:5.000000:6.000000:7.000000:8.000000

Table 15: Buffer Data Types (cont'd)

Data Types	Applicable to		Display Format Example
	AIE (Y/N)	AIE-ML (Y/N)	
v64int8	Y	Y	±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16:17:18:19:20:21:22:23:24:25:26:27:28:29:30:31:32 ±33:34:35:36:37:38:39:40:41:42:43:44:45:46:47:48:49:50:51:52:53:54:55:56:57:58:59:60:61:62:63:64
v64uint8	Y	Y	1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16:17:18:19:20:21:22:23:24:25:26:27:28:29:30:31:32 33:34:35:36:37:38:39:40:41:42:43:44:45:46:47:48:49:50:51:52:53:54:55:56:57:58:59:60:61:62:63:64
v64int16	Y	Y	±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16 ±17:18:19:20:21:22:23:24:25:26:27:28:29:30:31:32

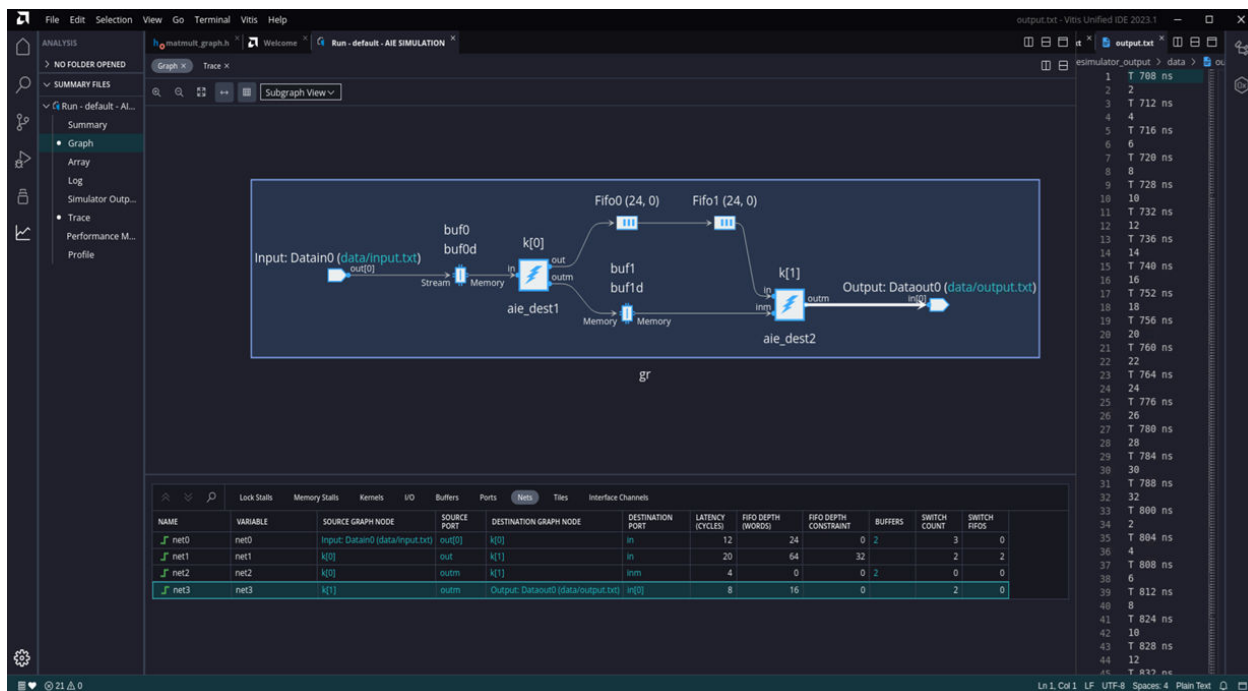
Notes:

1. Preceding 0 and '+' not displayed.
2. Currently data misaligned or out of order in timeline view and event table.
3. In the case of 256-bit data, each 128-bit data is separated by ' / ' instead of ' : '. For example, 1:2:3:4 / 5:6:7:8.
4. Two samples at timestamp x, next two samples at timestamp x+1.

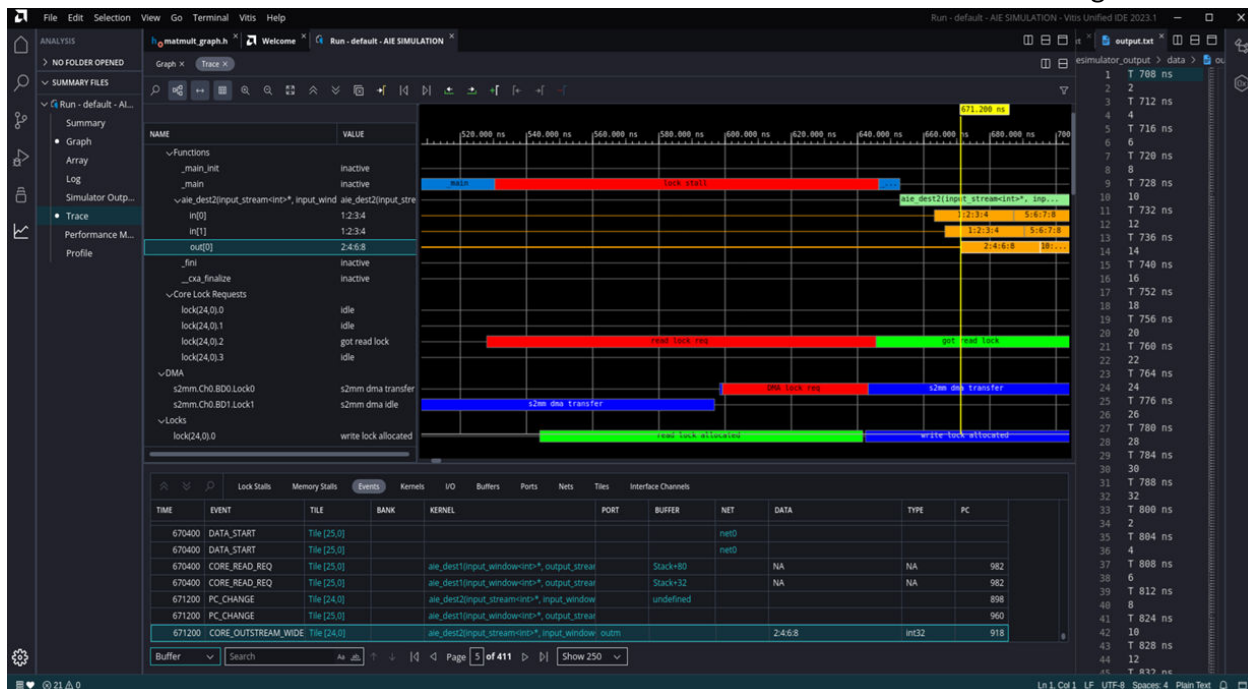
Stream Data Analysis

Stream data analysis in the Trace view allows you to cross probe from a specific input/output stream net connection in the graph view to the appropriate position in the Event Trace view. You can trace stream input data as it is being received during the kernel execution. To perform data analysis on a stream interface, use the following steps.

1. The following image shows a kernel port that has been selected in the Graph view. The selected net with ID, `net3`, is highlighted both in the Graph view and in the net table at bottom of the window.



2. Select the kernel and switch to the Trace to view the events and detailed event timing.



3. Move the time marker to the beginning of the stream data by dragging the vertical line (time marker) in the upper part of the window.

4. In the upper part of the window you can examine the data associated with the selected input port.
5. In the events table at the bottom of the screen, the expected time and event are highlighted in the events table.

Using these steps, you can inspect I/O data for the required stream connections to ensure the correctness of the I/O data to and from a kernel.

Supported Stream Data Types

Table 16: Supported Stream Data Types

Data Types	Display Format
int8 ¹	±123
int16	±12345
int32	±1234567890
int64 ³	±1234567890123456789
uint8	123
uint16	12345
uint32	1234567890
uint64 ³	12345678901234567890
cint16	±12345±12345i
cint32 ²	±1234567890±1234567890
float	±1.234567
cfloat ²	±1.234567±1.234567
acc48	±1:2:3:4:5:6:7:8
cacc48	±1+1i_2+2i_3+3i_4+4i
acc80 ⁴	N/A
cacc80 ⁴	N/A
accfloat	±1.000000:2.000000:3.000000:4.000000:5.000000:6.000000:7.000000:8.000000
caccfloat	1.000000+2.000000i:3.000000+4.000000i:5.000000+6.000000i:7.000000+8.000000i
v2cint32	±1+2i:3+4i
v4cint16	±1+2i:3+4i:5+6i:7+8i
v4int32	±1:2:3:4
v4float	±1.000000:2.000000:3.000000:4.000000
v8int16	±1:2:3:4:5:6:7:8
v16int8	±1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v16uint8	1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16
v8acc48	±1:2:3:4:5:6:7:8

Table 16: Supported Stream Data Types (cont'd)

Data Types	Display Format
v4cacc48	$\pm 1+2i_3+4i_5+6i_7+8i$

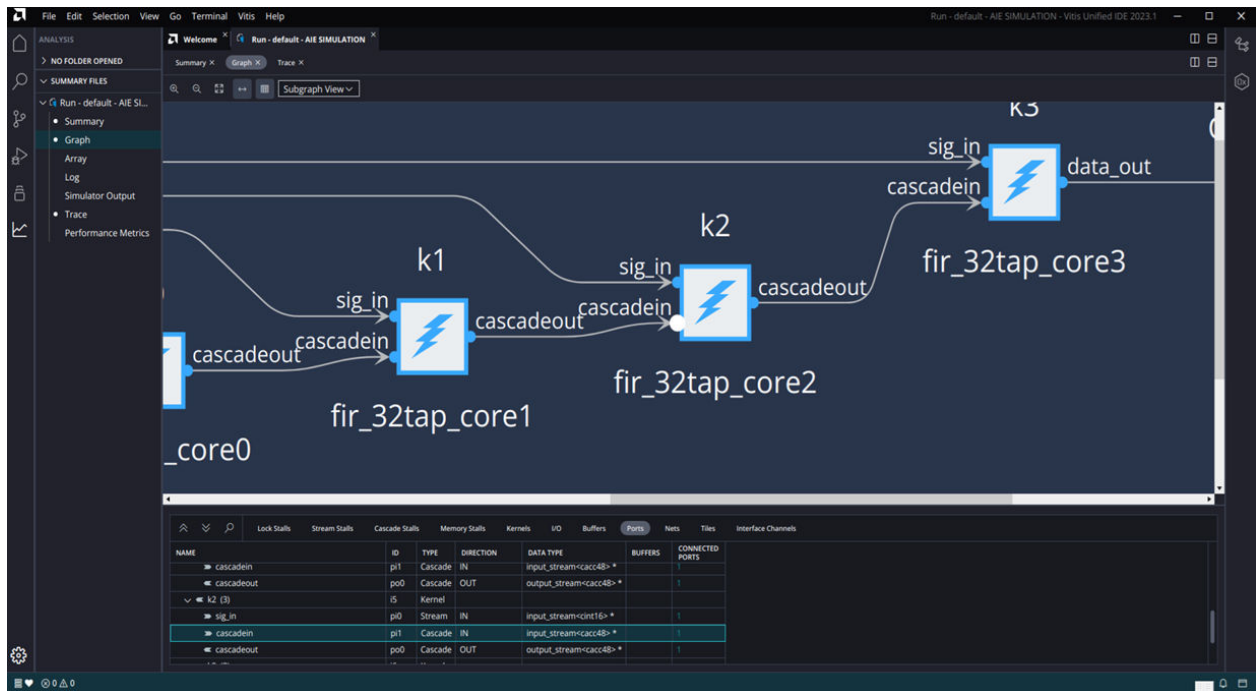
Notes:

1. Preceding 0 and '+' not displayed.
2. Real part followed by imaginary part at x and x+1 timestamp.
3. Least significant 32 bits value and Most significant 32 bits value are displayed at x and x+1 timestamp.
4. Currently under development.

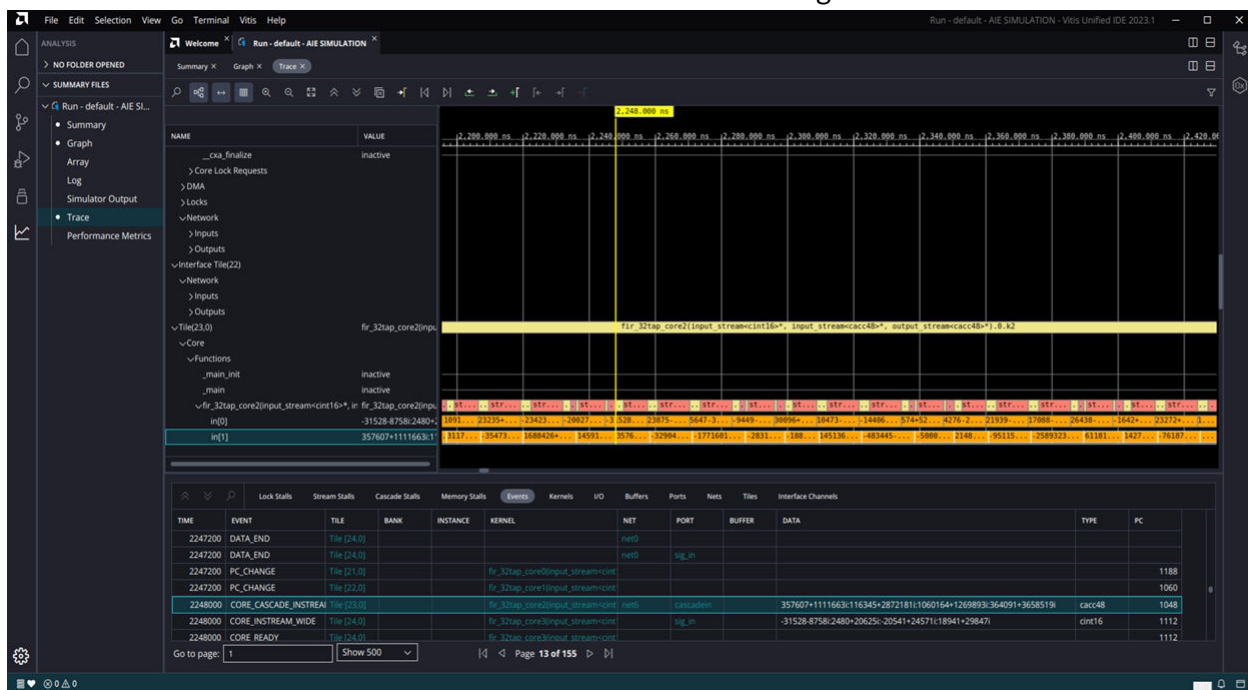
Cascade Data Analysis

Tracing cascaded data is similar to trace stream data. Cascade data analysis in the Trace view allows you to cross probe from a specific input/output cascade connection in the Graph view to the appropriate position in the Event Trace view. You can trace cascade input data as it is being received during the kernel execution. To perform data analysis on a cascade interface, use the following steps.

1. The following image shows a kernel port that has been selected in the Graph view. The selected port is highlighted both in the Graph view and in the ports table at bottom of the window.



2. Switch to the Trace view to the events and detailed event timing.



3. View the data associated with the selected input port.
4. Put the cursor in the timeline of the Trace view. In the events table to view data in DATA column.

Using these steps, you can inspect I/O data for the required cascade connections to ensure the correctness of the I/O data to and from a kernel.

Supported Cascade Data Types

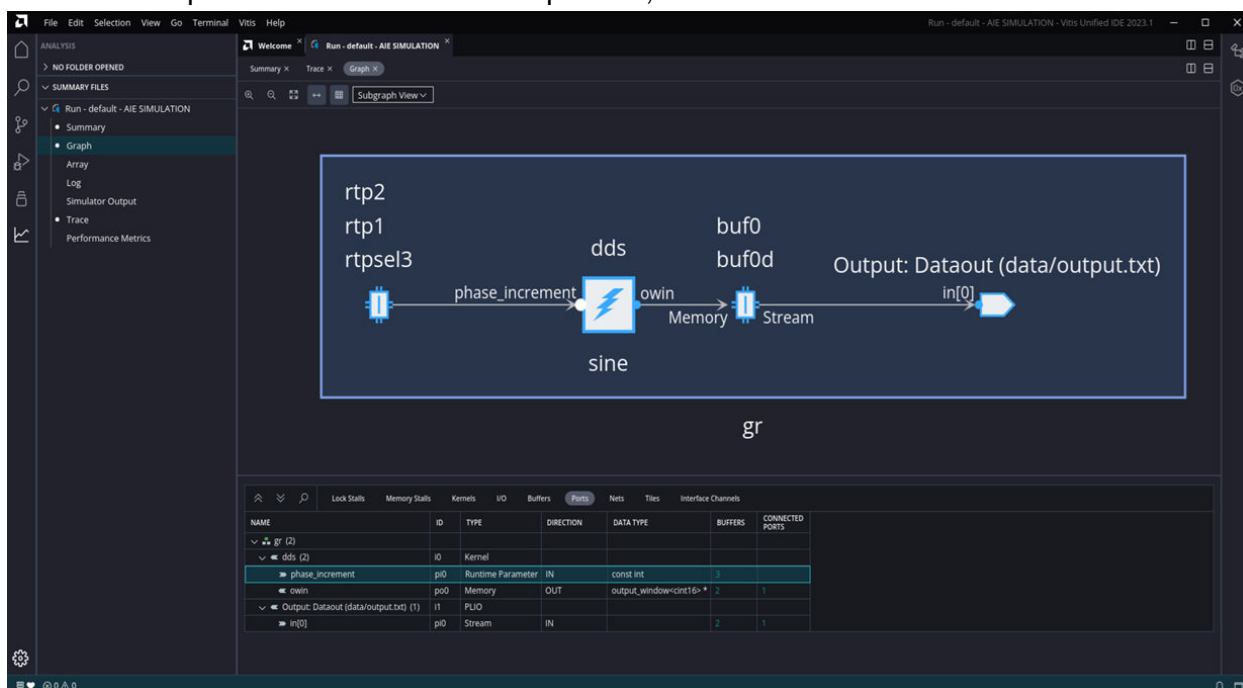
Table 17: Supported Cascade Data Types

Data Types	Display Format
acc48	$\pm 1:2:3:4:5:6:7:8$
cacc48	$\pm 1+1i_2+2i_3+3i_4+4i$
acc32	$\pm 1:2:3:4:5:6:7:8$
cacc32	$\pm 1+1i_2+2i_3+3i_4+4i$
int32	$\pm 1:2:3:4:5:6:7:8$

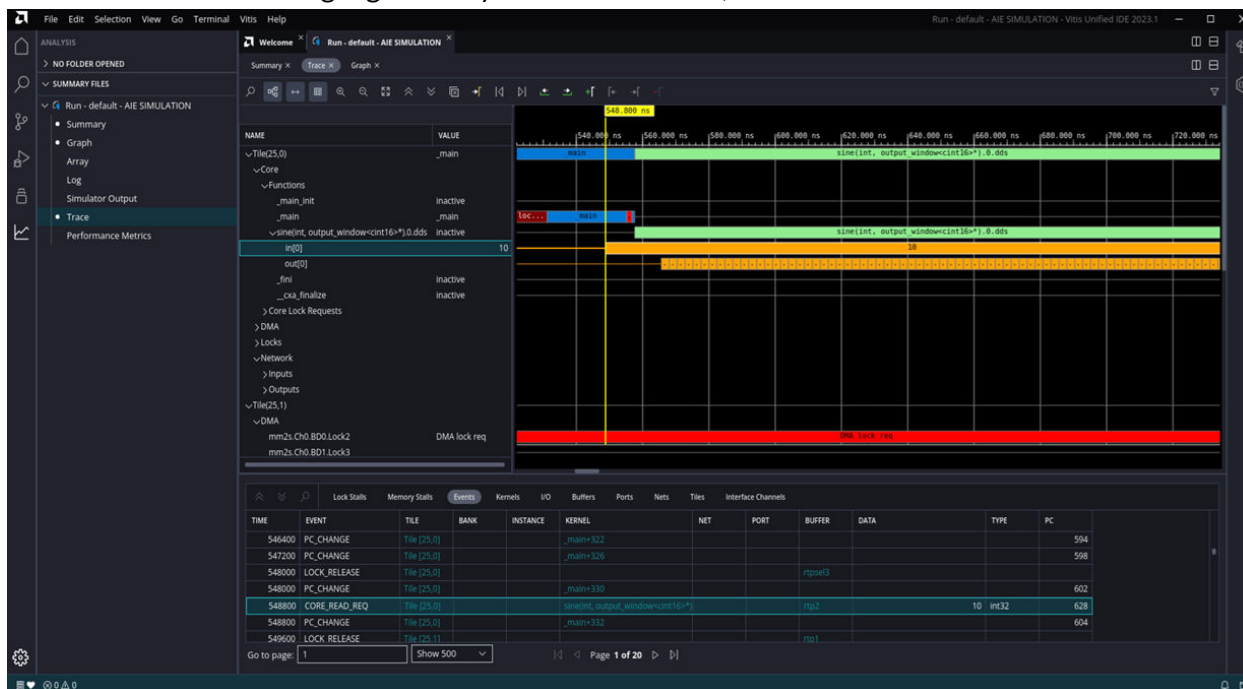
Scalar RTP Data Analysis

Scalar RTP data analysis in the Trace view allows you to cross probe from a specific scalar input/output RTP port in the Graph view to the appropriate position in the trace Events view. You can trace RTP data as it is being read or written during the kernel execution. To perform data analysis on a RTP port, use the following steps.

1. Click the RTP port of the kernel in the Graph view, as shown below.



2. Switch to Trace view, and the selected port is highlighted. The Events view also shows the RTP data read and write operations by the kernel. Put the cursor at the beginning of the RTP data in Trace view will highlight the cycle in Events view, as shown below.



Note: Only scalar input, inout synchronous, and asynchronous RTP data can be analyzed in the Trace view. The vector RTP data cannot be visualized.

Data Display Limitations

The limitations of the trace view data visualization feature are as follows.

- In certain cases, data will repeat twice in the timeline because very-long instruction word (VLIW) CPU fetches data from two different load ports at same cycle, so memory arbitrates the two load data in two consecutive cycles, so that same data is repeated in the timeline.
- 64-bit non-vector window data types are displayed as two 32-bit high and low values.

For example, an unsigned, 64-bit integer `0x0000000100000002` is displayed as 2L and 1H separately. L and H represent low and high 32 bits.

Another example is a complex 32-bit value where the real 32 bits are displayed first, followed by the imaginary 32 bits.

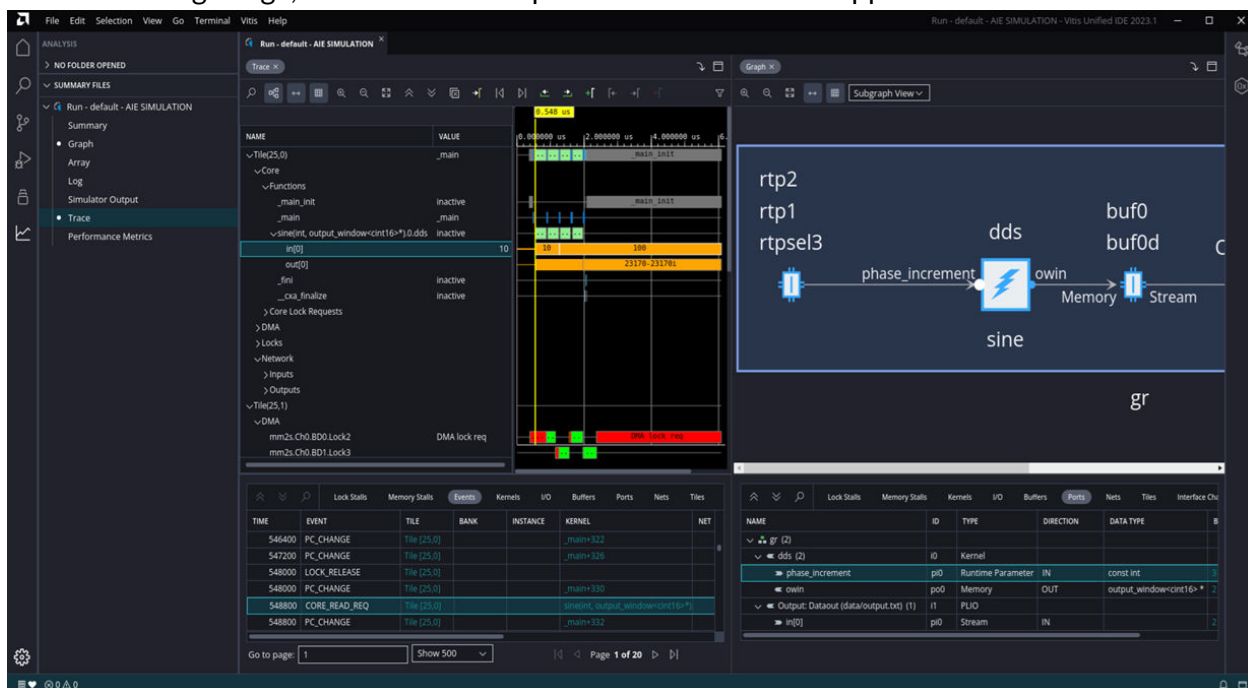
- The `input_pktstream/output_pktstream` data type is displayed in integer format.
- The display of the `int64` data type used in a templated class is not supported.
- Kernel functions are not visible in the trace if they are inlined by the AI Engine compiler.
- Inlined kernel function I/O ports are not visible in the trace.

Cross Probing

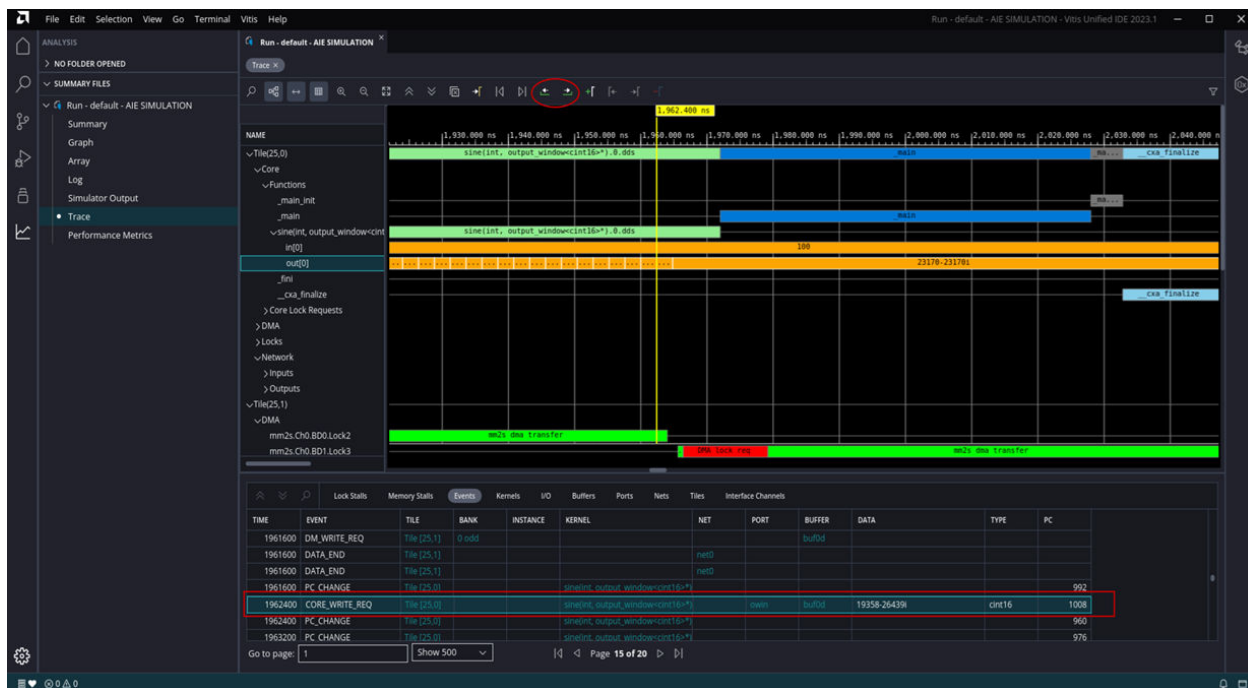
Debugging a design is a complicated process and often needs to switch between views/reports. The Vitis IDE supports this functionality and allows detailed inspection of events and their associated timing. Cross probing allows you to probe data in different views allowing you to view I/O data from different perspectives, including data on a particular tile or port, and at what time an event occurred, all within the same window. The Trace and Graph views can be shown in the same window and moving the time marker in the Trace view or selecting object(s) in the Graph view applies to both views simultaneously.

1. Use the New Horizontal Group and New Vertical Group toolbar buttons ( ) to manage views/reports.

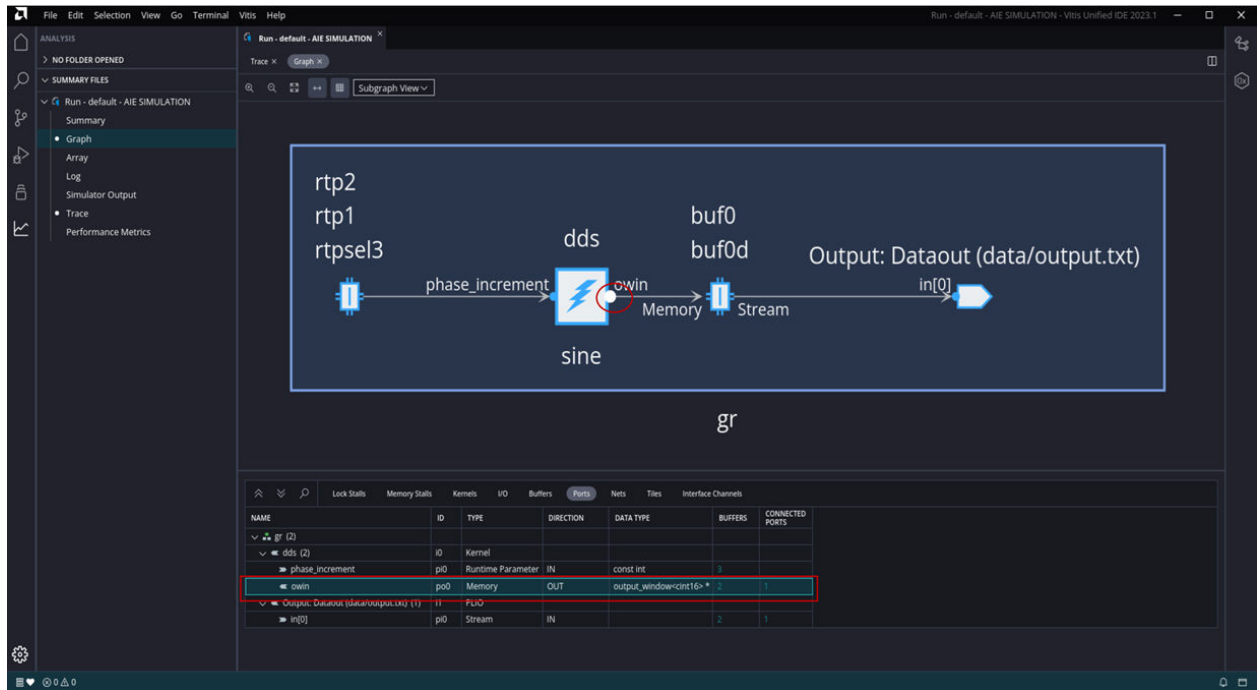
2. In the following image, the Trace and Graph view are selected to appear in the same window.



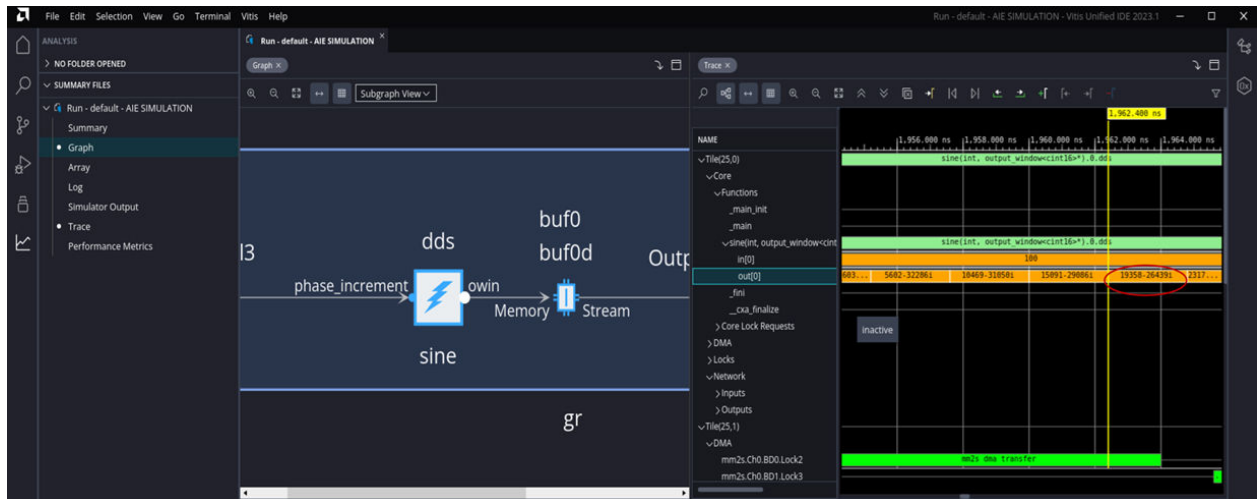
3. Use the filter button () to select tiles, functions, input/output ports, DMA, and locks to be included or excluded from the Trace view to focus on areas of interest.
4. Drag the time marker to move the time backwards and forwards to evaluate events at a given time. Events occurring after the selected time are highlighted in the events table of the Trace view at the bottom of the screen.



- Select objects in the Graph view to map graph, tiles, I/O ports, and net connections to see the object ID, type, direction, data type, buffers, and connected ports.



- Values of I/O ports are available in the Trace view, shown circled in the following image. This example design uses complex 16 bits value type (`cnt16`).

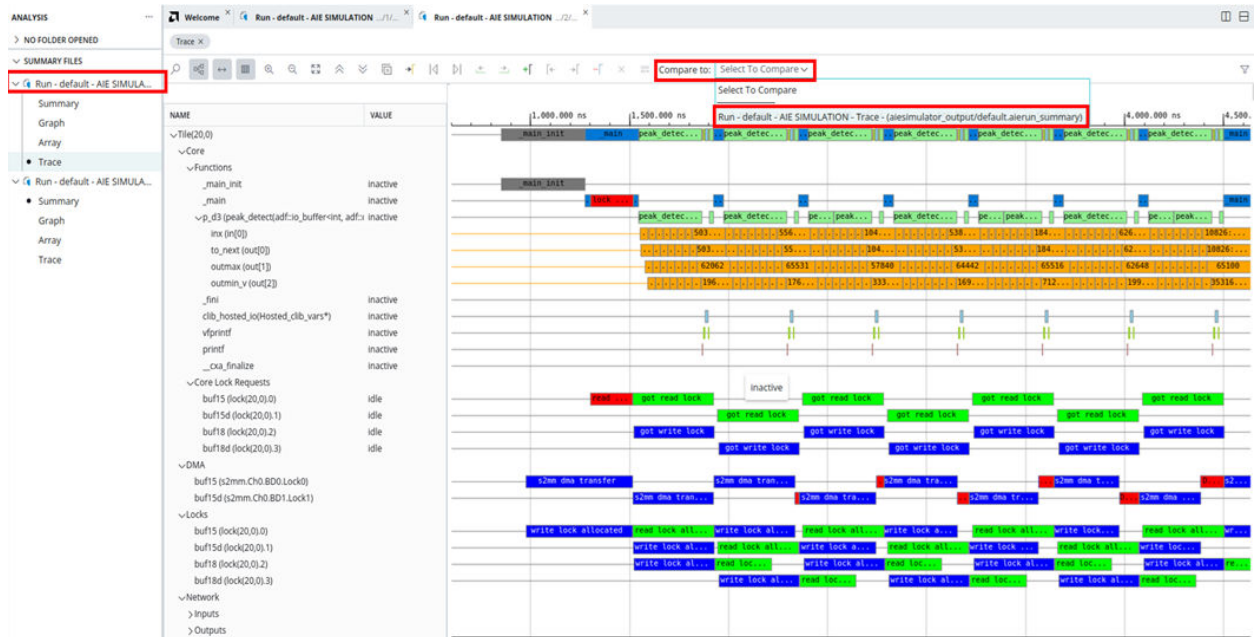


These steps show an output port object in the Graph view (on the right side) at 1,962,400 ns into the design execution and in the Trace view (on the left side) where the output data is available. Moving the time marker to a later time in the Trace view, the events in the events table (lower portion of the Trace view) are highlighted to indicate which events happened at that time. This information is useful to relate events in a multi-processor environment. Other examples can leverage this cross probing feature in the Vitis IDE.

Trace Compare

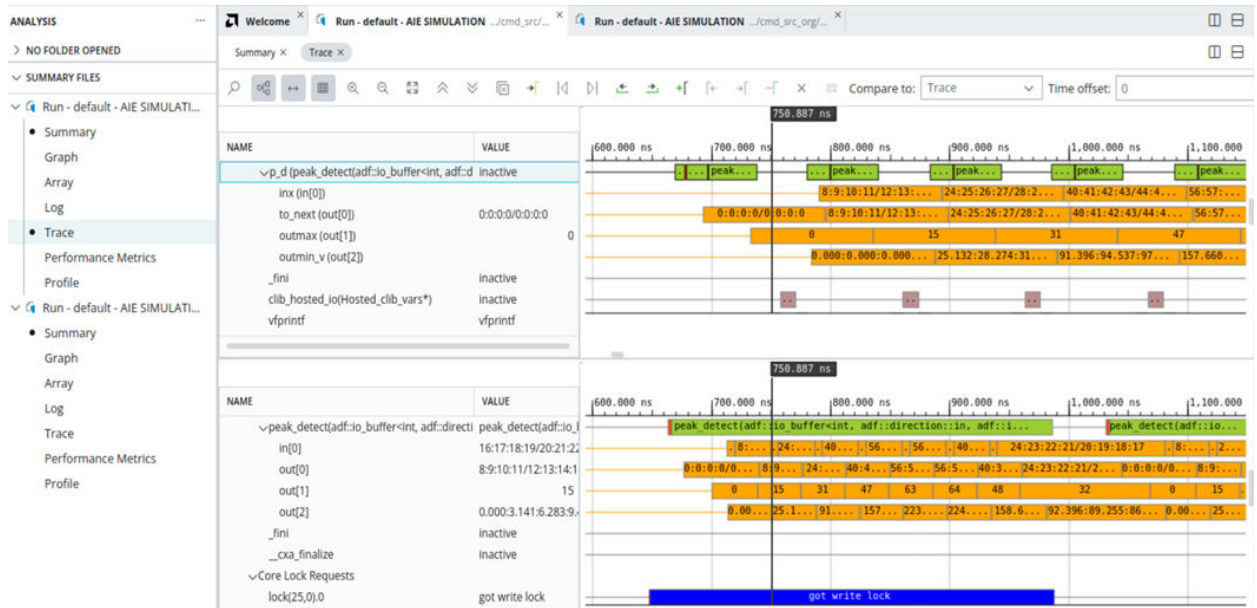
Trace Compare in the Vitis IDE allows the comparison of two different design executions. The comparison between two design runs helps to see the impact of one or more variances introduced in a design run. This allows you to inspect performance difference regardless of what variables are adjusted.

1. Open the two summary files to be compared.
2. Select the Trace view for either design and select **Compare** → **Select To Compare** and from the drop down choose the run summary of the other design to start comparing the trace. The compare can start from any trace.



Note: When you change any of the IDE Trace options, it applies to all the Trace views in the Trace Compare mode.

This example uses the same design but one has added `printf()` statements in the kernel. In this Trace Compare example, the upper design expended time on the `printf()` slows down the overall execution of the design.



AI Engine Stall Analysis in the Vitis IDE

AI Engine execution can be stalled by stall logic from multiple sources include the following.

- **External memory-mapped AXI4 master:** Any external AXI4 master (for example, PS) can issue a stall signal to a specific AI Engine.
- **Lock modules:** AI Engine has access to locks for hardware synchronization. When acquiring a lock, if the core does not get a lock, then the AI Engine will be stalled until the lock becomes unlocked.
- **Empty or full AXI4-Stream interfaces:** The AI Engine can be stalled when reading from an empty input FIFO or writing to a full output FIFO.
- **Data memory collisions:** Memory stall can occur between two different AI Engines or when one AI Engine tries to access a single memory bank (for example, trying to load and store in the same cycle to the same memory bank).
- **Event actions from the event unit:** The AI Engine can be stalled by event actions from the event unit.

When an AI Engine stalls, all memory interfaces to AI Engine including program memory interface are stalled. The stall is resolved when the cause of the stall has been fixed.

The IDE can use the VCD trace from AI Engine simulation to do stall analysis that shows an overview of the stall status in metrics. It also helps you detect where the stall happens and the possible causes.

To do stall analysis in the IDE, it is advised that you run the AI Engine simulator with `--online -wdb -text` options to generate event trace information in the background.

```
aiesimulator --pkg-dir=./Work --online -wdb -text
```

Note: If the VCD file is needed for analysis, `--dump-vcd` option of AI Engine simulator can be used instead. However, the IDE takes time to generate event trace from the VCD file. Especially when the design is large, the time becomes non-negligible. Therefore, it is recommended to use `vcdanalyze` together with AI Engine simulator to prepare event trace for the IDE to perform AI Engine stall analysis.

```
aiesimulator --pkg-dir=./Work --dump-vcd foo
vcdanalyze --vcd=foo.vcd --wdb --text
```

Note: Do not use `--outdir` option of `vcdanalyze` to place output data in directories other than default.

For the steps to launch the IDE to view the AI Engine simulation result, see [Viewing the Run Summary in the Vitis IDE](#).

```
vitis -a ./aiesimulator_output/default.aierun_summary
```

The following additional settings are required for AI Engine stall analysis in the Vitis IDE working in the HW emulation flow.

1. Write a simulator option file (for example, `sim_options.txt`) with content:

```
AIE_PKG_DIR=<ABSOLUTE_PRJ_PATH>/Work
AIE_DUMP_VCD=foo
```

2. Launch HW emulation with options:

```
./launch_hw_emu.sh -aie-sim-options <ABSOLUTE_PATH_TO>/sim_options.txt
```

3. Optionally run `vcdanalyze`:

```
cd ./sim/behav_waveform/xsim/; vcdanalyze --pkg-dir=../../Work --
vcd=foo.vcd --wdb -text
```

4. Launch the Vitis IDE:

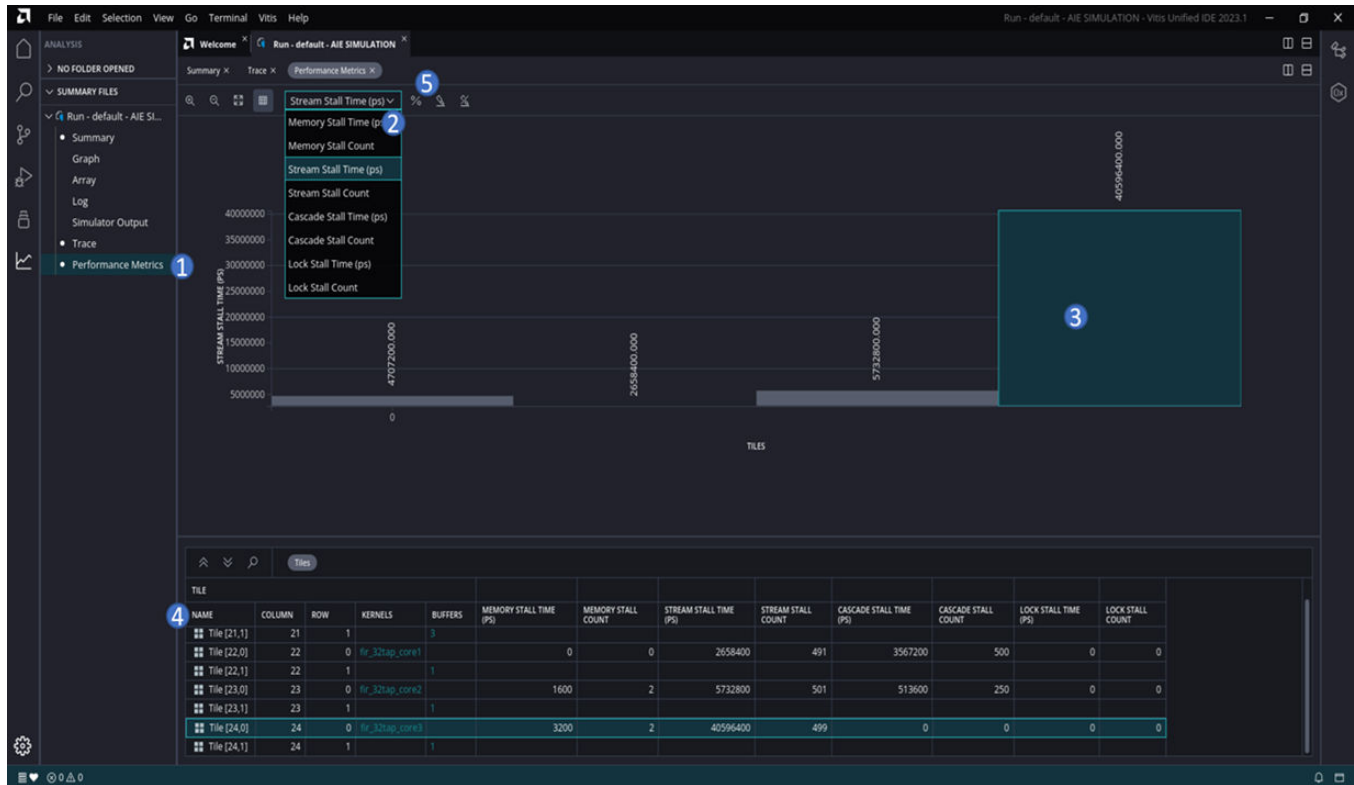
```
vitis -a ./sim/behav_waveform/xsim/default.aierun_summary
```

Note: `default.run_summary` is generated for third-party simulators. For AI Engine, a run summary file is generated for Questa, Vcs, Xcelium, and Riviera. For AI Engine-ML, a run summary is generated only for Questa, and VCS.

Performance Metrics

The Performance Metrics view shows the percentage of each type of stall against total simulation time or total stall count.

Figure 33: Performance Metrics View



1. Click on the Performance Metrics view.
2. Choose the stall type from the drop-down list. The metrics separate different types of stalls. It will show the existing stalls in the simulation result.
 - **Lock Stall Time (ps):** Percentage of lock stall time during which buffers in the AI Engine tile are being acquired.
 - **Lock Stall Count:** The number of lock stalls in a specific tile.
 - **Memory Stall Time (ps):** Percentage of memory stall time that memory accesses in the AI Engine tile have conflicts.
 - **Memory Stall Count:** The number of memory stalls in a specific tile.
 - **Stream Stall Time (ps):** Percentage of stream stall time due to full or empty stream.
 - **Stream Stall Count:** The number of stream stalls in a specific tile.
 - **Cascade Stall Time (ps):** Percentage of cascade stream stall time that either due cascade stream is full or empty.
 - **Cascade Stall Count:** The number of cascade stream stalls in a specific tile.
3. Each AI Engine tile is shown as a bar in the Performance Metrics view. The higher the stall percentage, the more towards the right the bar will be. More attention should be paid to the highest bars. Click one of the bars to select the tile to focus.

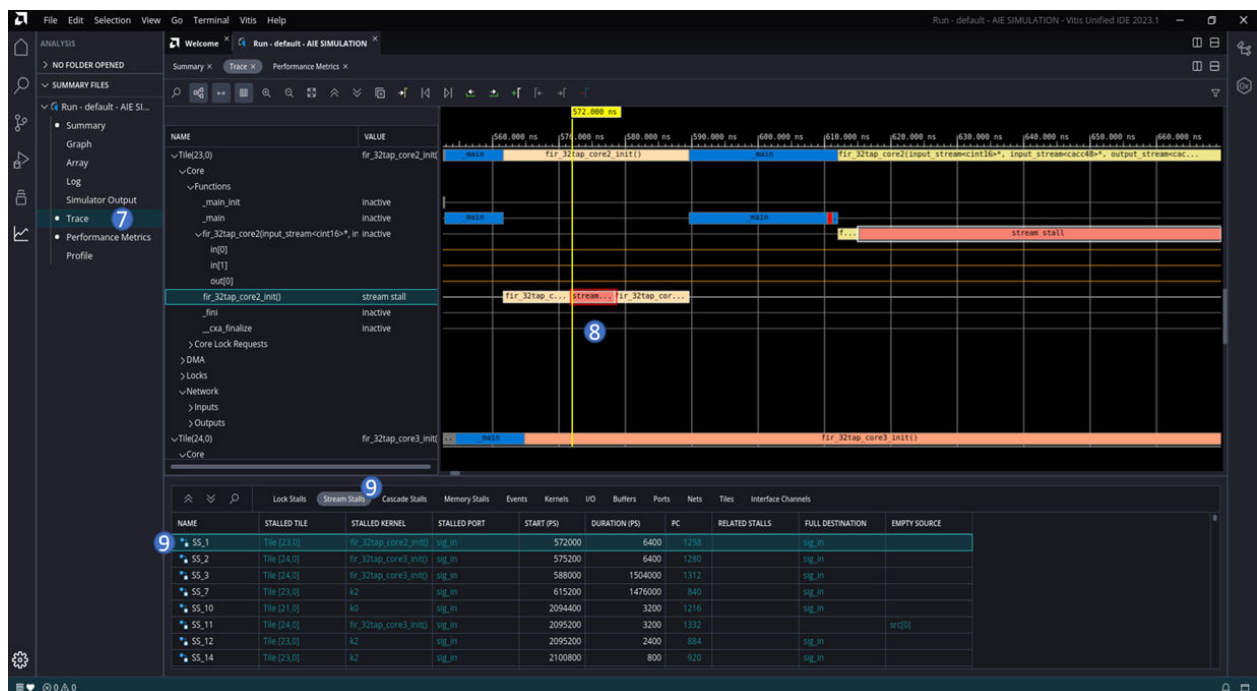
- The Tiles view at the bottom lists all the AI Engine tiles with the information about column, row, kernels, buffers, and all the stall times, percentage, as well as count. You can click on the title row to sort on a specific column.



TIP: When the number or information in the view is in green, it can be cross-probed with other views.

- There is a % button beside the drop-down list. By clicking on this button, you can switch between displaying the stall in percentage or absolute value.
- While exploring the information in the Performance Metrics view, it is usually helpful to cross-probe with other views. For example:

Figure 34: Trace View



- Choose the tile that has the largest stall time. Then go to the Trace view to see the position and frequency of the stalls.
- Zoom in and out of the Trace view for a better view of the stall in the timeline.
- Click on the Stalls view and from the drop-down list, select the type of stalls to examine. Select a stall in the Stalls view to highlight it in the Trace view.

The Performance Metrics view, Trace view, Graph view, and Array view can be cross-probed between each other. Graph view helps in understanding where the stall happens in the graph, and Array view helps in viewing the positions of the objects in hardware. Additional details about analyzing each type of stall are explained in the following sections.

Lock Stall Analysis

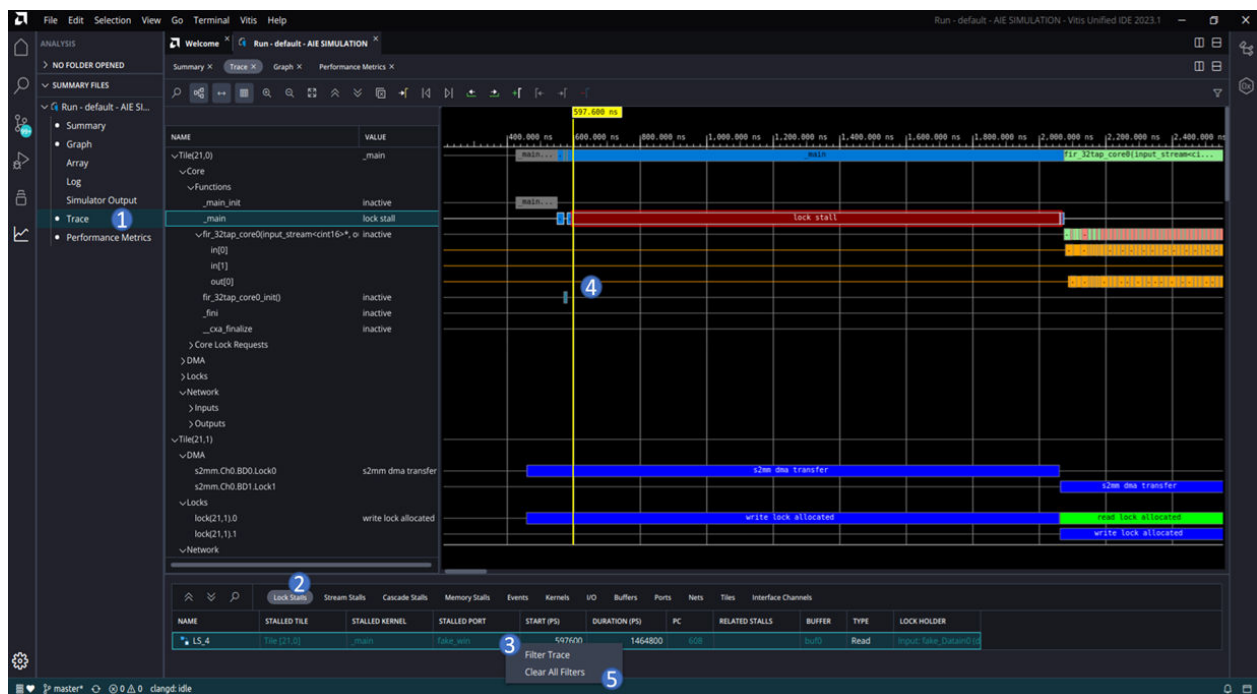
The Performance Metrics analysis tab can help identify if the lock stall needs to be analyzed. The following steps illustrate how a lock stall can be analyzed starting in the Vitis IDE.

1. Select the Trace view.
2. Select the Lock Stalls view.



TIP: The Stalls view is available with the Trace view, Graph view, and Array view.

Figure 35: Lock Stall in Trace View



Each stall has the following information associated with it.

- **NAME:** The lock stall is named LS_<NUM>. The number is unique across all types of stalls. The earlier the stall happens, the smaller the number.
- **Stalled Tile:** The AI Engine tile where the stalled kernel is located.
- **Stalled Kernel:** The kernel that is stalled. It is named <Kernel_function_name>.<Schedule_ID>.<Graph_instance_name>. Sometimes it is shown as _main and then cross-probe is required to find the real kernel function.
- **Start (ps):** The start time of the stall.
- **Duration (ps):** The duration of the stall.
- **PC:** Program counter when the stall happens.

- **Type:** The stalled kernel tries to *read* or *write* the buffer.
- **Buffer:** The buffer that the stalled kernel tries to read or write.
- **Stalled Port:** The port of the stalled kernel that tries to read or write the buffer.
- **Lock Holder:** The source that is holding the lock of the buffer.
- **Related Stall:** Other stalls that can cause a stall.



TIP: *The items in green can be cross-probed with other views.*

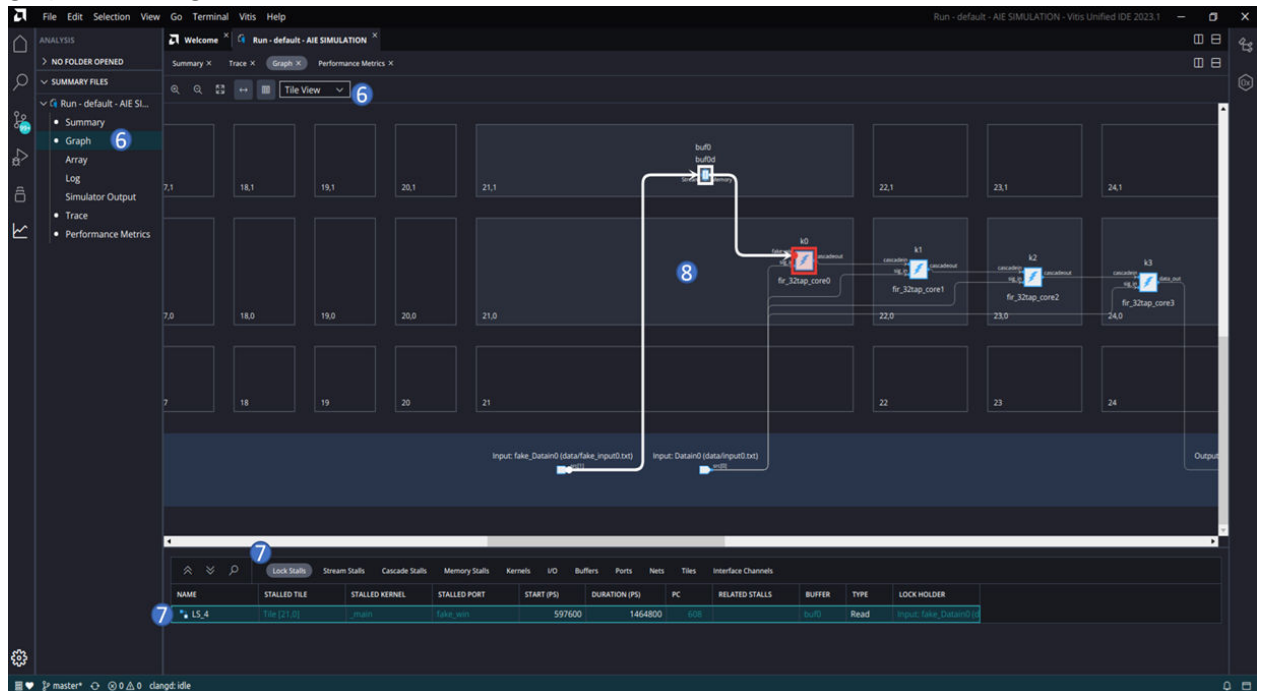
3. Select one row of the stall. It will go to the start of the stall in Trace view. It is optional to filter all the signals that are related to the stall by right-clicking the stall and choosing **Filter Trace**. In Trace view, the signals related to the stall are shown. Non-related signals are hidden. Exploring the trace is easier when the design is large.



TIP: *Filter Trace may not show signals related to the related stalls.*

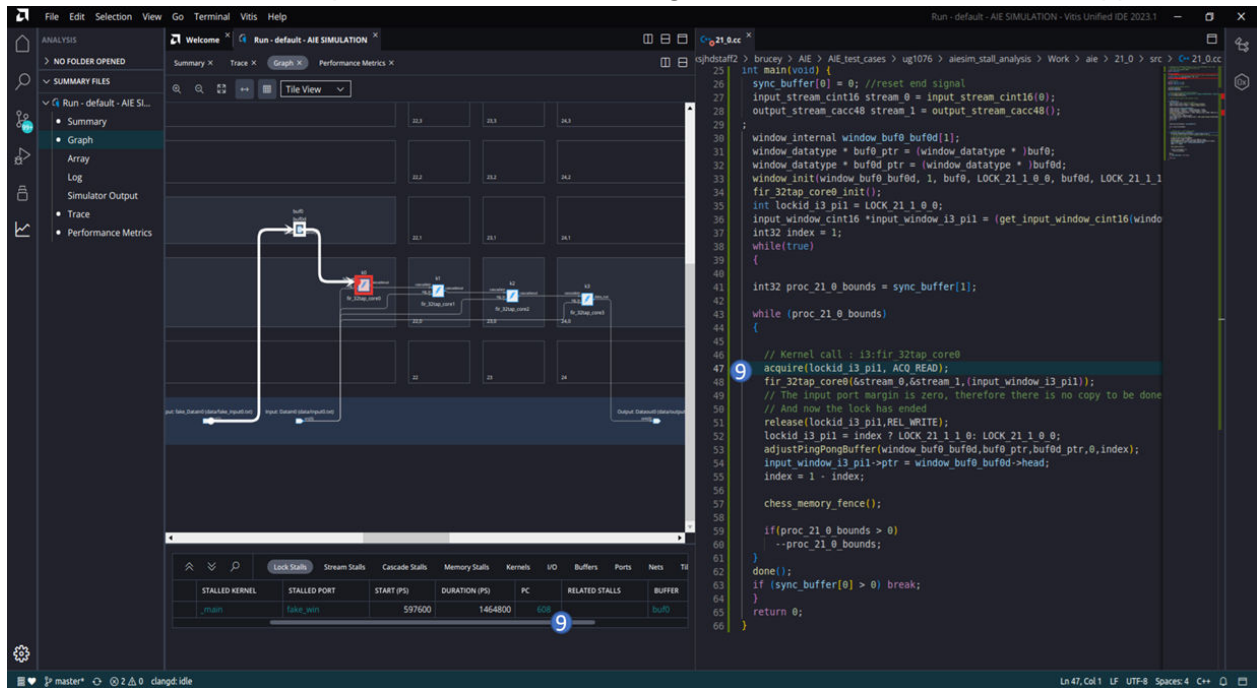
4. Trace view can be used to view the lock stall in timeline. For a specific lock stall, it can be seen when the write lock and read lock are allocated. From the position of the stall and events before and after the stall, the reason for the stall can be analyzed. For example, if write lock has already been allocated and the lock type is *Read*, then it indicates that the buffer has not been released by the producer. The consumer is waiting for it to be readable. The producer can be found in the Lock Holder.
5. To clear the previously filtered trace, right-click and choose **Clear All Filters**.

6. It is usually helpful to have an overview of the stall path in Graph view. Select Graph view and then select **Tile View** from the drop-down list in the view. Tile view of Graph view shows graph in AI Engine tiles.



7. If the Stalls view is not shown, select **Lock Stalls** from the drop-down list and choose the stall to be analyzed. It will highlight the related paths in the Tile view of the Graph view.
8. The red path shows where the stall occurs. The white path shows the source to where the stall occurs.

9. Click the PC value. This opens the source code and goes to the line where the stall happens.



The following table lists scenarios that can cause a lock stall and possible solutions.

Table 18: Lock Stall Scenarios and Solutions

Source	Target	Destination	Stall Type	Possible Solution
AI Engine kernel	Lock of sync window	AI Engine kernel	Lock stall	- If single buffer is used, use PING-PONG buffer (default) or place kernels into same AI Engine tile. - If the kernels are unbalanced in execution time, balance throughput between kernels.
AI Engine kernel	Lock of async window (window_acquire and window_release API)	AI Engine kernel	Lock stall	- If a single buffer is used, use a PING-PONG buffer (default). - Acquire and release buffer in-time. Use a local buffer as needed.
PL interface	Lock of window	AI Engine kernel	Lock stall	- Ensure that the PL interface throughput matches the AI Engine throughput. - Check that the PL interface frequency and width are set properly. See AI Engine/Programmable Logic Integration in <i>AI Engine Kernel and Graph Programming Guide (UG1079)</i>.

Table 18: Lock Stall Scenarios and Solutions (cont'd)

Source	Target	Destination	Stall Type	Possible Solution
AI Engine	Lock of window	PL interface	Lock stall	- Ensure that the PL interface throughput matches AI Engine throughput. - Check that the PL interface frequency and width are set properly. See AI Engine/Programmable Logic Integration in <i>AI Engine Kernel and Graph Programming Guide</i> (UG1079).

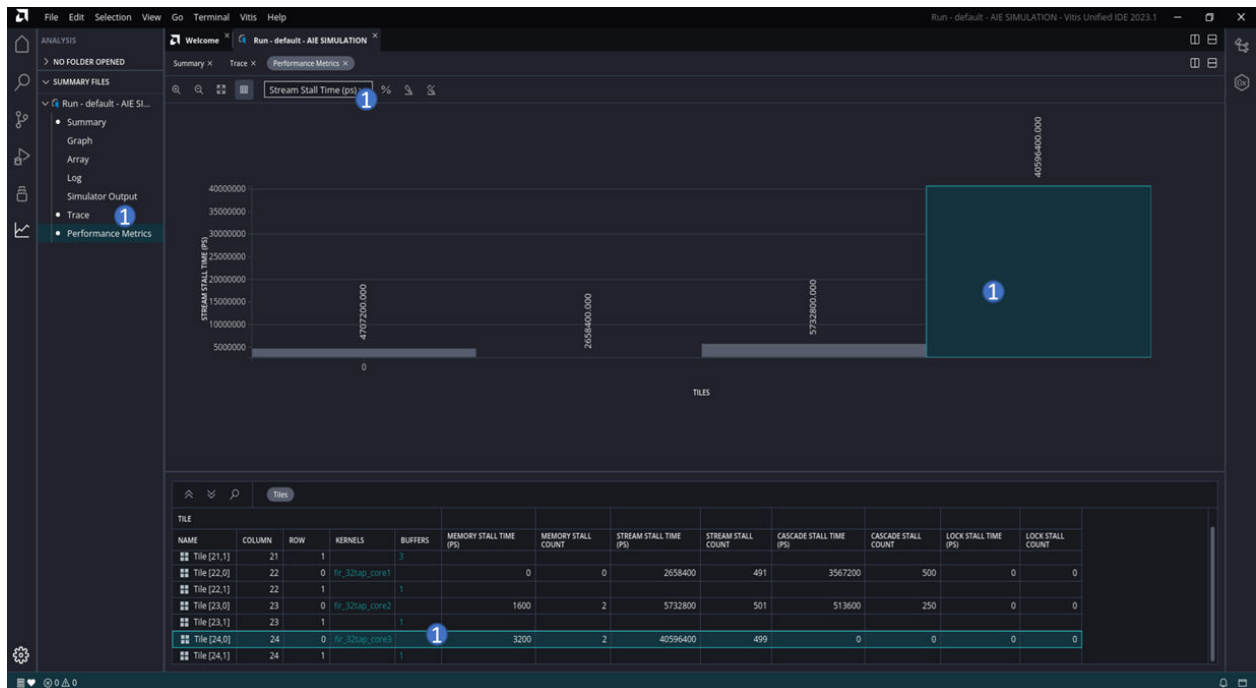
Note: DMA lock stall is not included in the Vitis IDE lock stall analysis.

Stream Stall Analysis

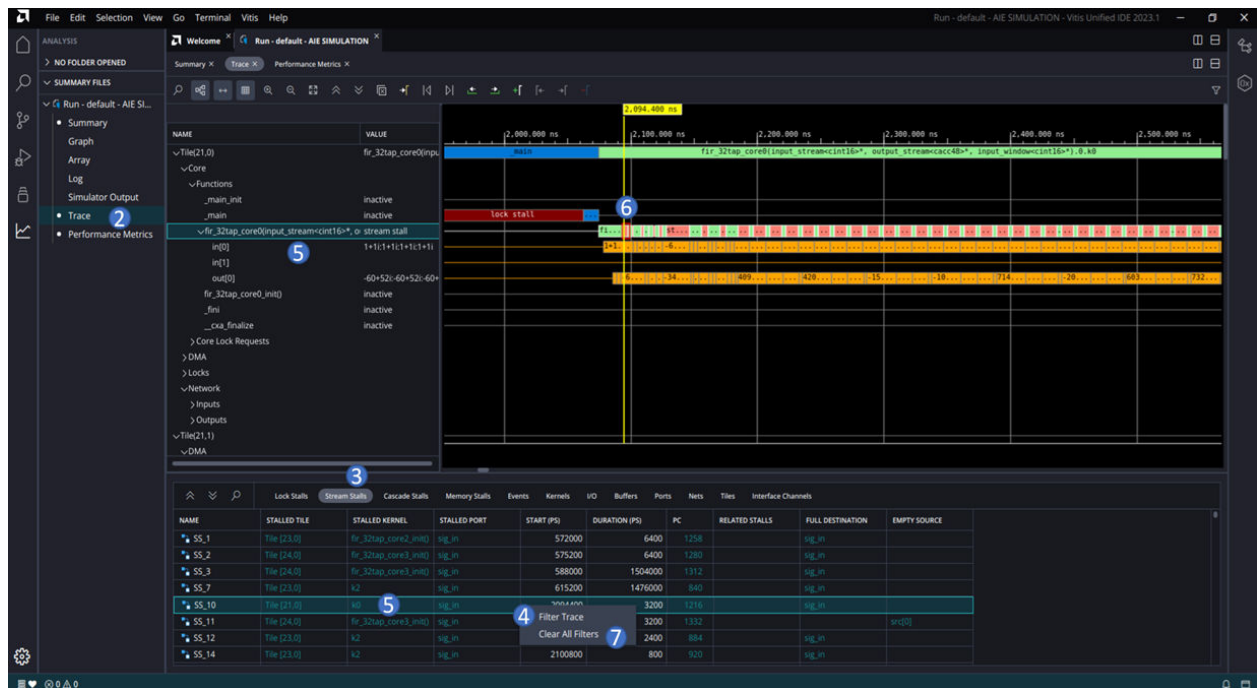
From the Performance Metrics view, you can identify if a stream stall needs to be analyzed, and also the tile/tiles that are causing the stall.

The following steps illustrate how a stream stall can be analyzed starting in the Performance Metrics tab in the Vitis IDE.

1. In the Performance Metrics view, select **Stream Stall Time (%)** to view stream stalls across all tiles. Identify the tile(s) to be analyzed. Note that the objects in Performance Metrics view can be cross-probed with Trace view, Graph view, and Array view. For example, selecting the tile in the Performance Metrics view to highlight the tile in Trace view can help quickly locate the tile.

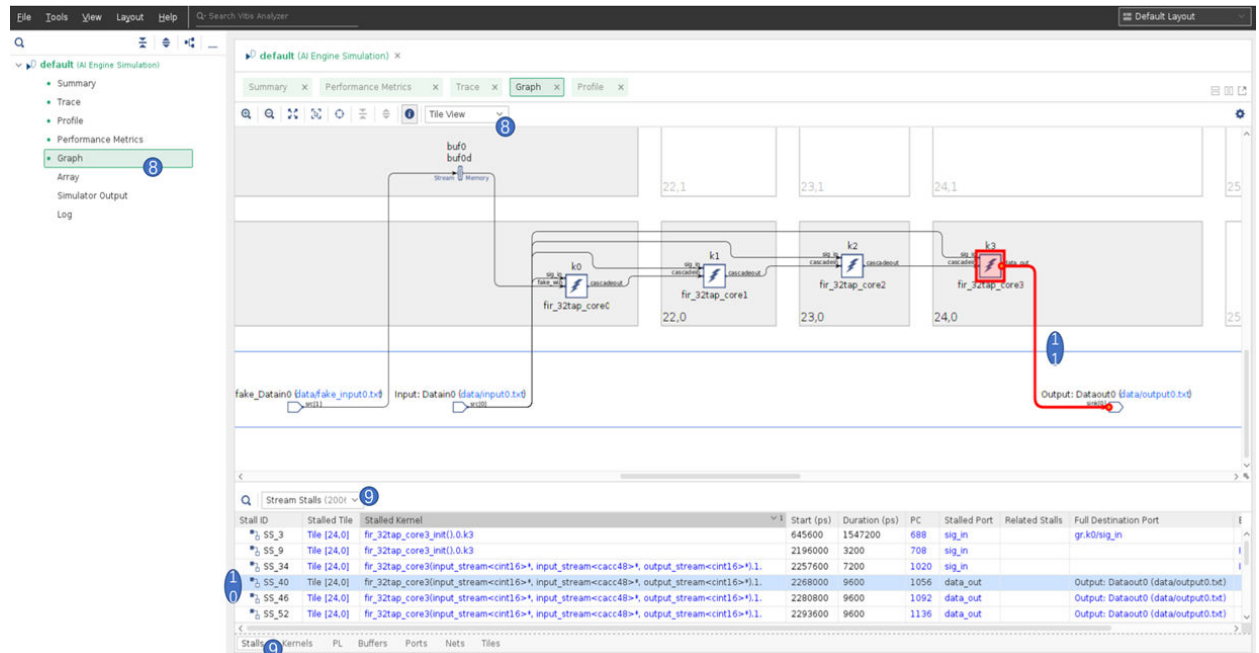


2. Select the Trace view.



- Select the **Stream Stalls** view. In the Stalls view, stream stalls have the following information and the objects in blue can be cross-probed with other views by clicking on it.
 - NAME:** The stream stall is named `SS_<NUM>`. The earlier the stall happens, the smaller the number. The number is unique across all types of stalls.
 - Stalled Tile:** The AI Engine tile where the stalled kernel is located.
 - Stalled Kernel:** The kernel that is stalled. It is named `<Kernel_function_name>.<Schedule_ID>.<Graph_instance_name>`. Sometimes it is shown as `_main` and then cross-probe is required to find the real kernel function.
 - Start (ps):** The start time of the stall.
 - Duration (ps):** The duration of the stall.
 - PC:** Program counter when the stall happens.
 - Stalled Port:** The port of the stalled kernel.
 - Related Stalls:** Other stalls that can cause the stall.
 - Full Destination:** The port that the stalled kernel cannot write into because it is full.
 - Empty Source:** The port that the stalled kernel cannot read from because it is empty.
- Clicking on a stream stall in Stalls view will go to the start of the stall in the Trace view. Right-click the stall and select **Filter Trace** as needed. After filtering trace, the signals related to the stall are shown in the Trace view. Non-related signals are hidden. Exploring the trace using filter trace is clearer when the design is large.

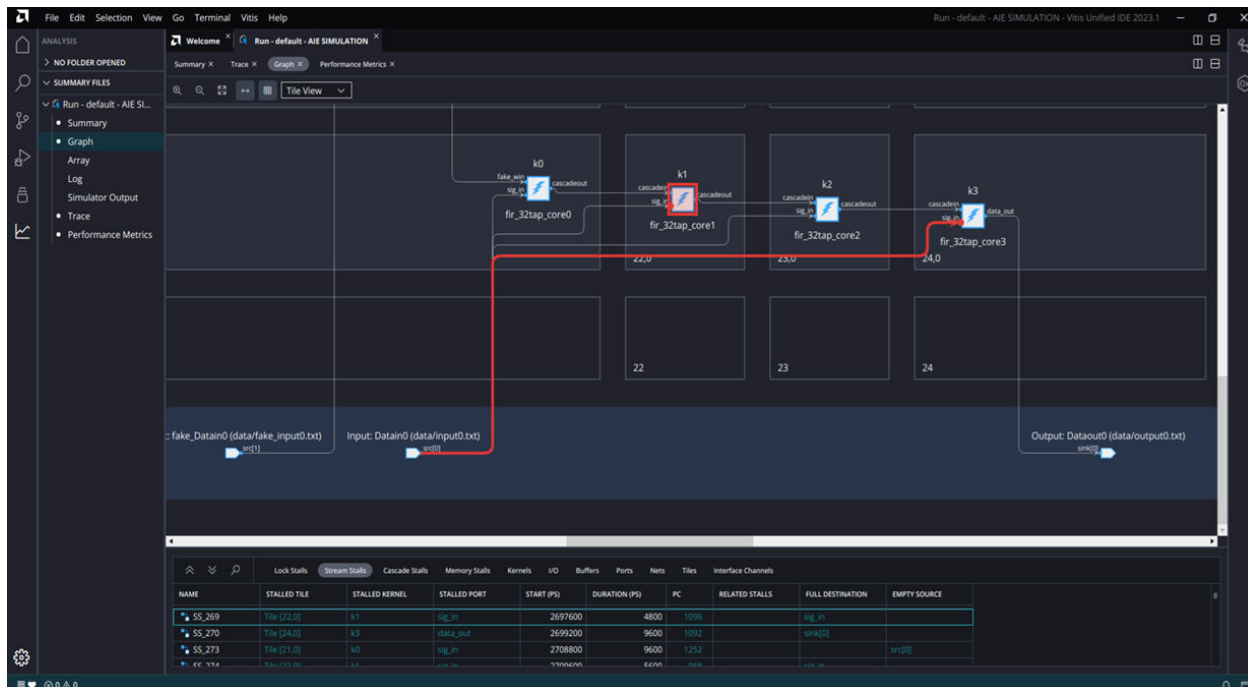
5. Objects in blue in the Stalls view can be clicked and cross-probed. For example, clicking the kernel in Stalls view will highlight the kernel in Trace view.
6. Zoom in and out of the Trace view to explore the stalls. From the position of the stall, how frequently do similar stalls occur, events before the stall and related stalls (if any), can give you a hint on why the stall occurs.
7. To clear the previously filtered trace, right-click and select **Clear All Filters**.
8. It is helpful to have an overview of the stall path in Graph view. Select **Graph** view and then select **Tile View** from the drop-down list.



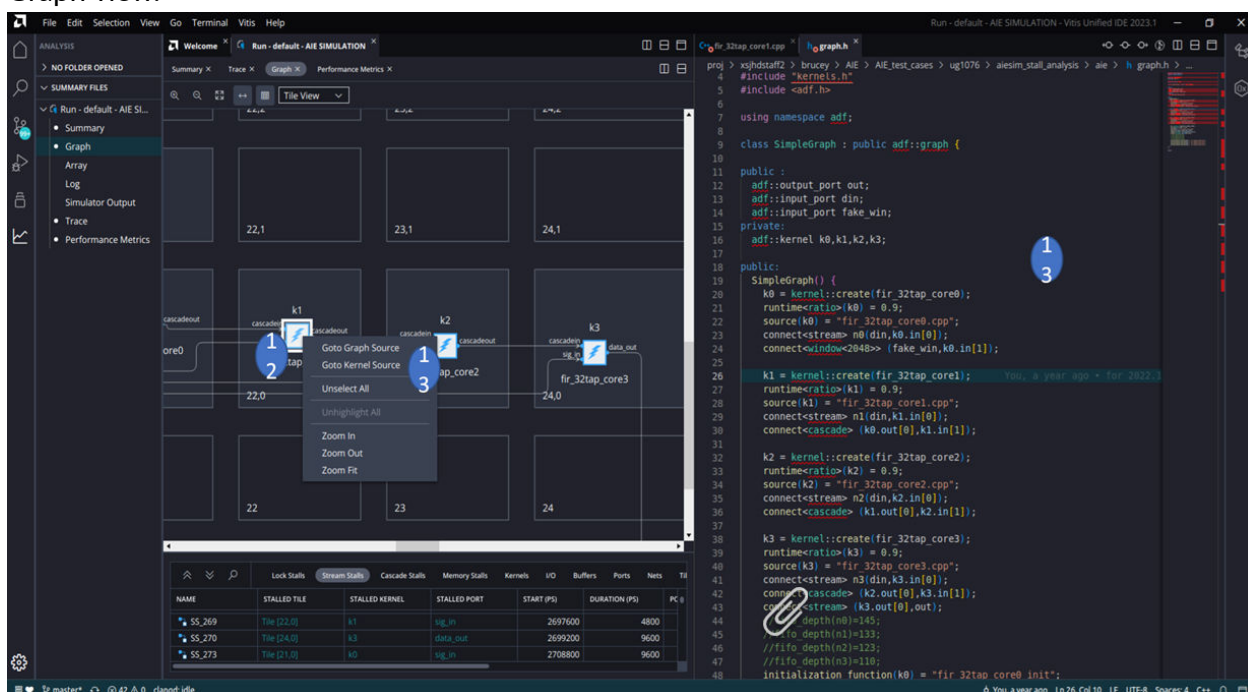
9. Select **Stalls** view and then select **Stream Stalls** from the drop-down list.
10. Explore the stream stalls in the Stalls view. Click on a stream stall in the Stalls view to have an overview of the stall in the graph. The red path shows where the stall occurs. It can be from a stalled kernel to full destination port, or from an empty source port to the stalled kernel.



TIP: If a stream multicasts to multiple destinations and a stream stall occurs when the stream does not have enough FIFO for all destinations, the highlighted stalled kernel and stalled net cannot be connected (separately in red). Meaning that all destinations of the multicast stream as a whole must be analyzed for the stream stall. An example of multicast stream stall in the Graph view is shown in the following figure.



- It can open graph source code or kernel source code from Graph view or Array view. Select the kernel instance by clicking the kernel object in the Stalls view or clicking the kernel in the Graph view.



- Right-click the kernel instance in the Graph view, and select either **Goto Graph Source** or **Goto Kernel Source**. It will open the graph source code or kernel source code.
- Correlate the graph source code and kernel source code with the stalls analyzed, editing the source code as needed.

The following table lists some possible scenarios that can cause a stream stall and possible solutions.

Table 19: Stream Stall Scenarios and Solutions

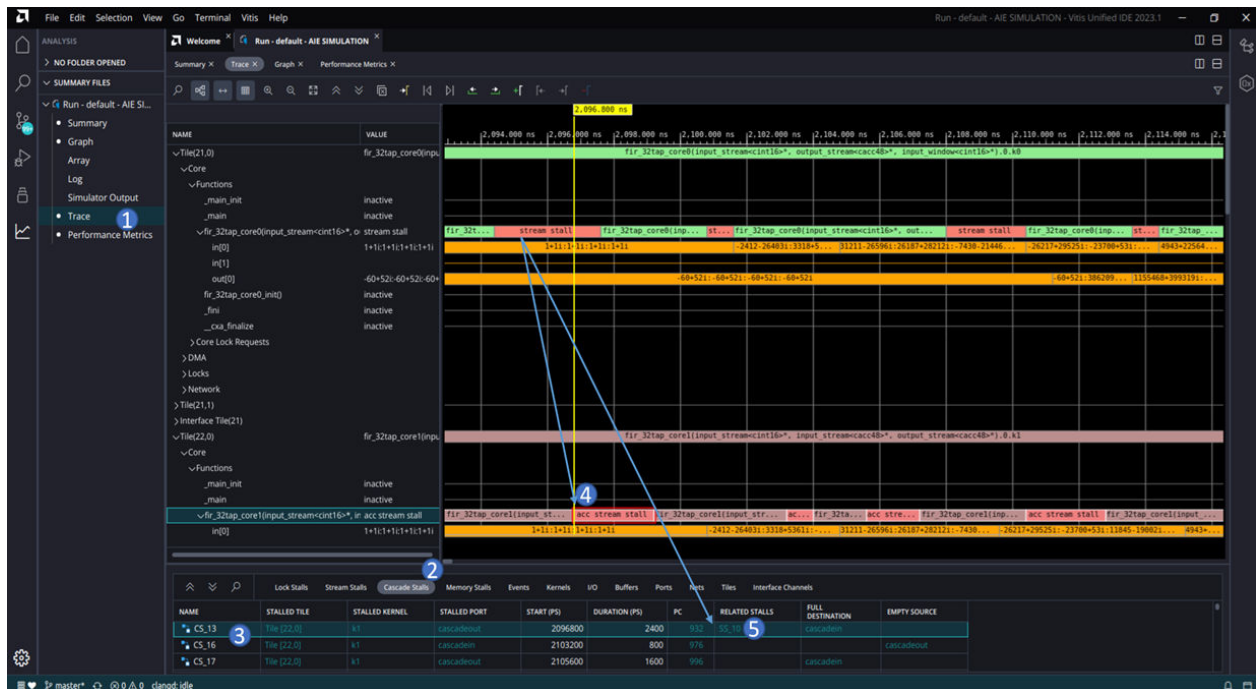
Source	Destination	Stall Type	Possible Solution	Notes
Stream	Stream	Stream stall	- Increase FIFO depth. See FIFO Depth Constraints. - Adjust stream read and write instructions in source or destination kernels.	
Stream	Multiple streams	Stream stall	- Increase FIFO depth. See FIFO Depth Constraints. - Insert DMA FIFO or set different FIFO depth for different destination nets. See FIFO Depth Constraints.	Multicast
Stream	Multiple streams of multiple kernels in same AI Engine	Stream stall	- Put multiple kernels into different AI Engines - Add enough FIFO to streams to the kernels.	Multicast
Multiple streams	Multiple streams	Stream stall	- Adjust instructions to match between different streams. - Increase FIFO depth (ssFIFO or DMA FIFO).	
PLIO	Stream	Stream stall	Maximize AI Engine-PL interface bandwidth. For example, 64-bit interface, highest frequency (1/2 AI Engine frequency) for PL. Or 128-bit interface (note this uses two 64-bit channels for a 128-bit interface). See AI Engine-PL Interface Performance in <i>AI Engine Kernel and Graph Programming Guide</i> (UG1079).	
Stream	PLIO	Stream stall	Same as above.	
Stream (32 bits per iteration)	PLIO	Stream stall	Send TLAST for each 32 bits. See AI Engine-PL Interface Performance in <i>AI Engine Kernel and Graph Programming Guide</i> (UG1079).	

Cascade Stall Analysis

From Performance Metrics analysis, you can identify the cascade stall that needs to be analyzed.

1. Choose the **Trace** view.
2. Choose **Cascade Stalls** view.
3. Select a **Cascade stall**.
4. The **Cascade stall** is highlighted in Trace view.

Figure 36: Cascade Stall in Trace View



Each cascade stall has the following information.

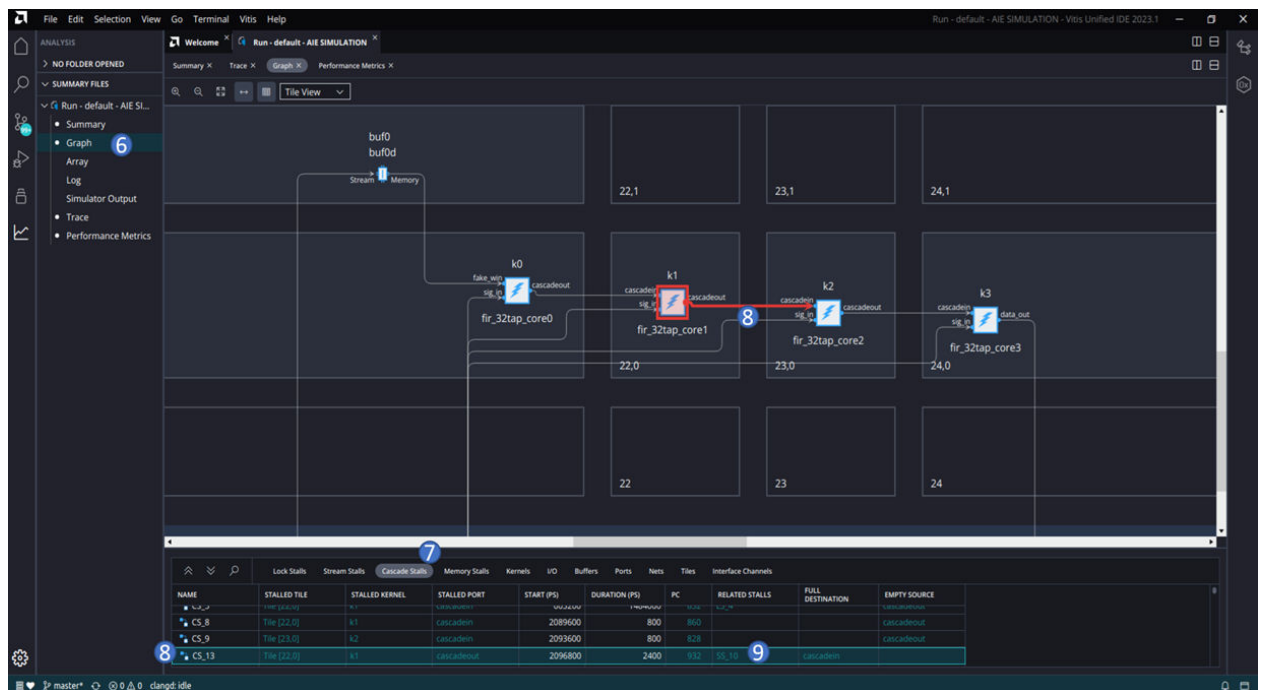
- **NAME:** The stream stall is named CS_<NUM>. The earlier the stall happens, the smaller the number. The number is unique across all types of stall.
- **Stalled Tile:** The AI Engine tile where the stalled kernel is located.
- **Stalled Kernel:** The kernel that is stalled. It is named <Kernel_function_name>.<Schedule_ID>.<Graph_instance_name>. Sometimes it is shown as _main and then cross-probe is required to find the real kernel function.
- **Start (ps):** The start time that the stall happens
- **Duration (ps):** The duration of the stall.
- **PC:** Program counter when the stall happens.
- **Stalled Port:** Which port of the stalled kernel is stalled.
- **Related Stalls:** Other stalls that might cause the stall.
- **Full Destination:** The port that the stalled kernel cannot write into because it is full.
- **Empty Source:** The port that the stalled kernel cannot read from because it is empty.

Explore the cascade stalls in the Trace view and see how they are related.

5. It is possible that other types of stalls cause the cascade stall. Explore the other types of stalls to analyze why that happened. Related stalls in the Stalls view show which stall causes the cascade stall.

6. Viewing the stalls happen in Graph view can help identify the cause of the stall. Select **Graph view**.
7. Select **Cascade Stalls** view.
8. Click a path in the Stalls view and the stall path is shown in red in Graph view.
9. Exploring the cascade stall and its related stalls, can help in finding a hint about the cause of the stall. Clicking **Related Stalls** can also show the related stall in red. For example, click SS_16 in Related Stalls to see the path in red, which gives a hint about where the stall starts.

Figure 37: Cascade Stall in Graph View



The following table lists some possible scenarios that cause cascade stall and possible solutions.

Table 20: Cascade Stall Scenarios and Solutions

Source	Destination	Stall type	Possible solution
Cascade Stream	Cascade Stream	Cascade Stall	- Adjust cascade stream read/write instructions in kernel execution cycles to match between source and destination kernels - Analyze other sources that cause the cascade stall
Stream + Cascade Streams	Stream + Cascade Streams	Cascade Stall	- Adjust instructions to match between different streams - Analyze other sources that cause the cascade stall

Memory Stall Analysis

AI Engine can perform several vector load or store operations per cycle. However, for the load or store operations to be executed in parallel, they must target different memory banks. Memory stall happens when multiple access on the same bank of memory on the same cycle.

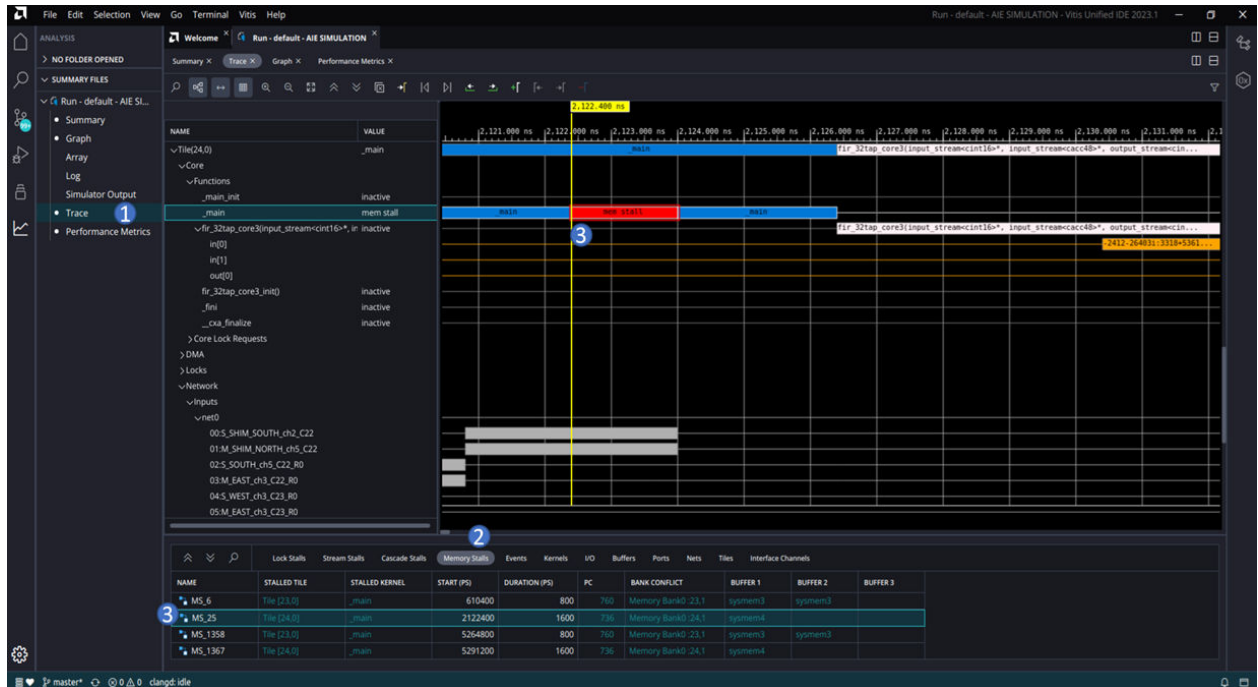
The kinds of memory include window buffer and DMA FIFO between kernels, RTP buffer, and system memory. System memory includes kernel synchronization information in the first 32 bytes, stack, and heap. Static variables are in the heap and function control logics are in the stack. System memory occupies continuous memory banks. Tool can automatically or manually place window buffer, RTP buffer, DMA FIFO, and system memory on specific banks. To alleviate memory stalls between these memories, try to place them into separate banks if possible. But memory stall can still happen between these kinds of memories if separate banks cannot be found for all the memories, or multiple accesses are happening on the same memory.

In general, the compiler tries to schedule many memory accesses in the same cycle when possible, but there are some exceptions. Memory accesses coming from the same pointer are scheduled on different cycles. If the compiler schedules the operations on multiple variables or pointers in the same cycle, memory bank conflicts can occur. Each memory bank has its arbitrator to arbitrate between all requests, and the arbitration is round-robin. After every request has been addressed, the memory stall is released.

From Performance Metrics analysis, you can identify if the memory stall needs to be analyzed.

1. Select the **Trace** view.
2. Select the **Memory Stalls** table.

Figure 38: Memory Stall in the Trace View



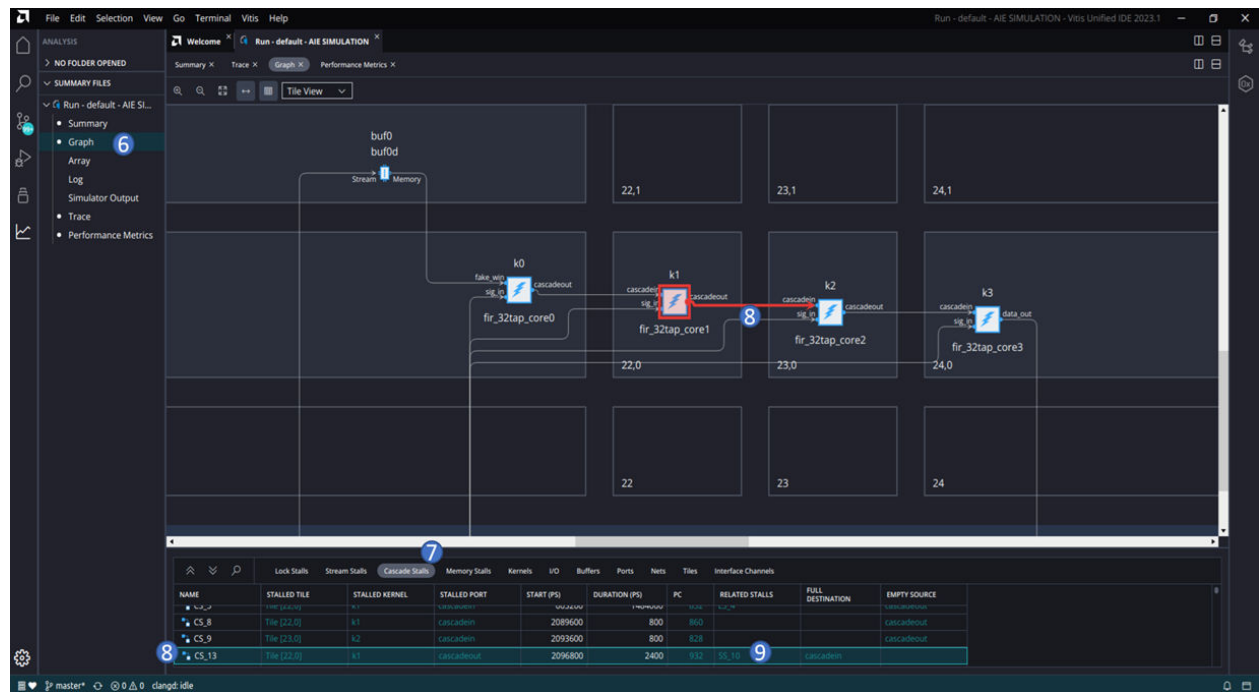
The stall is named as MS_<NUM>. The number is increased by time. Each stall has the following associated information.

- **NAME:** The memory stall id. The earlier the stall happens, the smaller the number. The number is unique across all types of stall.
 - **Stalled Tile:** The AI Engine tile where the stalled kernel is located.
 - **Stalled Kernel:** The kernel that is stalled. It is named `<Kernel_function_name>.<Schedule_ID>.<Graph_instance_name>`. Sometimes it is shown as `_main` and then cross-probe is required to find the real kernel function.
 - **Start (ps):** The start time that the stall happens
 - **Duration (ps):** The duration of the stall.
 - **PC:** Program counter when the stall happens.
 - **Bank Conflict:** The memory bank where the stall happens on.
 - **Buffer 1, Buffer 2, Buffer 3:** The buffers that cause the memory stall. It can be one buffer or multiple buffers.
3. When you click each line of the stalls in the Stalls view, it goes to the start of the memory stall in the Trace view. Zoom in and out of the Trace view to observe how frequently the memory stalls occur and the position of the stall in kernel running.

Note: If large number of memory stalls occur repeatedly in the running kernel, it indicates that the stalls can happen inside loop. It is best to investigate and resolve. If memory stalls only happen once at the start of the kernel running, or a very small number of stalls happen in kernel running, it can usually be neglected. From the name of the buffers that cause the stall, it can be identified whether it is window buffer or system buffer or something else. If it is a window buffer or RTP buffer that can be controlled in the graph, one way is to place it manually using constraints if better placement can be identified. If it is system memory (named `system<NUM>+<NUM>`), it is required to identify the variables that are involved in the stall.

- Click the row of the specific stall and switch to the Events view.

Figure 39: Events View of Memory Stall



- The Events view shows the events that happen in the device. The cycle where the memory stall happens is highlighted. You can see the tile on which the `DM_BANK_CONFLICT` event has occurred and the data that is being read or written.
- Try to explore some cycles before or after the stall cycle to find more hints.

In some cases, the tool schedules to read or write variables at the same bank in a cycle. One way to resolve it is documented in the Load and Store with Virtual Resource Annotations section in *AI Engine Kernel and Graph Programming Guide* ([UG1079](#)). For example, redefine points to the variables and annotate them with `__aie_dm_resource_a`. Following is an example code :

```
const v8cint16 __aie_dm_resource_a* __restrict coeff = (v8cint16
__aie_dm_resource_a*) eq_coef0; const v8cint16 coe = *coeff;
v16cint16 __aie_dm_resource_a* __restrict p_buff = (v16cint16
__aie_dm_resource_a*) &delay_line; v16cint16 buff=*p_buff;
```

The following table lists some possible scenarios that cause memory stalls and possible solutions.

Table 21: Memory Stall Scenarios and Solutions

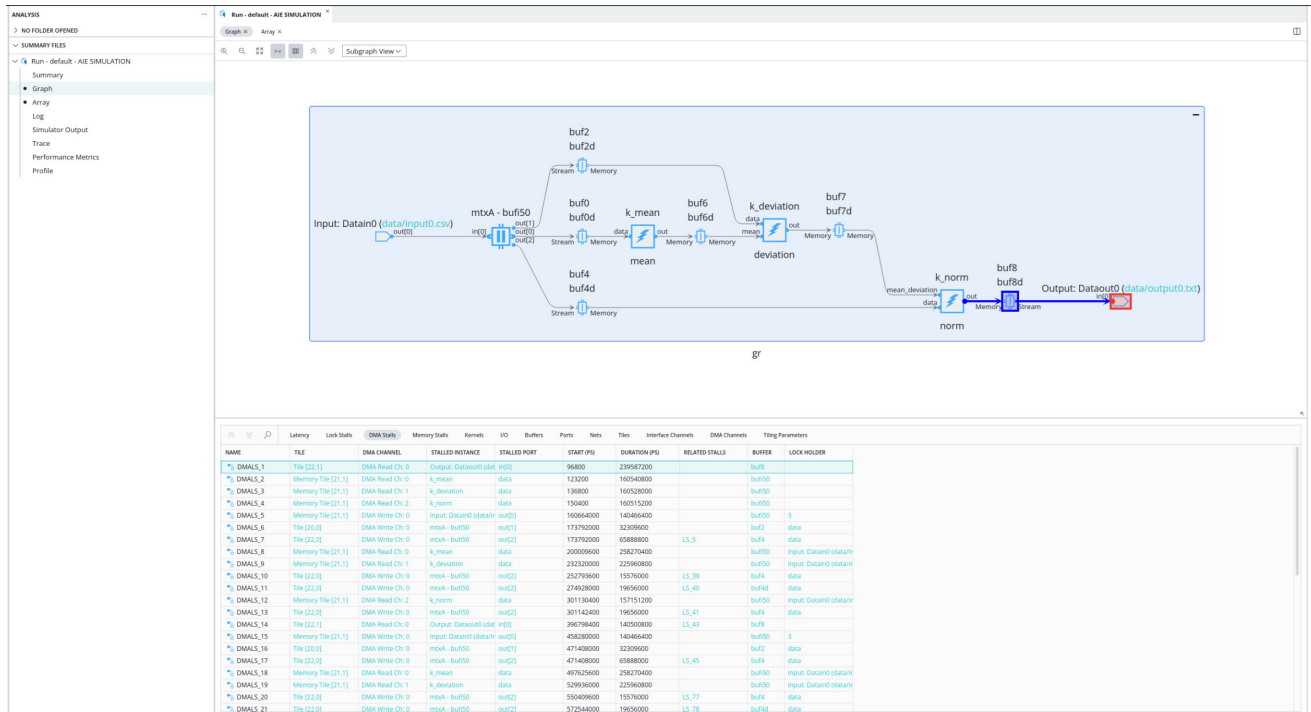
Source	Target	Stall Type	Possible Solution	Note
Single kernel	Buffers on a single memory bank	Memory stall	- Dispatch buffers to different banks (buffers include system memory, RTP, window buffer, DMA, and FIFO). See Memory Stalls. - Guide compiler scheduling with virtual memory annotations. See Load and Store with Virtual Resource Annotations section in <i>AI Engine Kernel and Graph Programming Guide</i> (UG1079)	Single kernel accesses buffers on the same bank. Or a single kernel has multiple accesses on one buffer on the same bank. (a cycle can have two loads and one store)
Multiple kernels on adjacent AI Engine tiles	Multiple buffers on one bank	Memory stall	- Dispatch buffers to different banks (memories include system memory, RTP, window buffer, DMA, and FIFO). - BufferOptLevel. - If memory banks are exhausted, do profile and AI Engine stall analysis to find better solution with less kernel execution time or less stall percentage.	Multiple kernels accessing multiple buffers on the same bank.

DMA Stall Analysis

DMA can be configured in AI Engine memory modules or AI Engine-ML memory tiles to transfer data to and from streams. The DMA write or read operation requires the lock of the write or read buffer to be acquired before the operation starts. If the lock of the DMA buffer is not acquired, the DMA operation will be stalled.

Following picture shows a DMA Stalls view. The DMA stalls can be cross-probed across graph, array and trace views.

Figure 40: DMA Stalls View



Each DMA stall has the following information:

- **NAME:** The stream stall is named `DMALS_<NUM>`. The earlier the stall happens, the smaller the number.
- **TILE:** The DMA location.
- **DMA CHANNEL:** The DMA channel.
- **STALLED INSTANCE:** The stalled instance.
- **STALLED PORT:** The port where the stall happens.
- **START (PS):** The start time that the stall happens.
- **DURATION (PS):** The duration of the stall.
- **RELATED STALLS:** Other stalls that might cause this stall, or the stalls this stall could cause.
- **BUFFER:** The buffer related to the DMA operation.
- **LOCK HOLDER:** The source that is holding the lock of the buffer.

The following table lists some possible DMA stall scenarios and solutions:

Table 22: DMA Stall Scenarios and Solutions

Source	DMA Channel	Destination	Stalled Instance	Possible Cause or Solution
PL Interface	DMA Write	Tile Buffer	PL Interface	Tile buffers are full. Improve kernel performance.
PL Interface	DMA Write	Memory Tile	PL Interface	Memory tile reading is not completed. Use Ping-Pong buffers of memory tile.
Tile Buffer	DMA Read	PL Interface	PL Interface	Tile buffer is not ready. If it occurs in the very beginning, it's ok. If it occurs between kernel executions, try to improve kernel performance.
Memory Tile	DMA Read	Tile Buffer	Kernel	Data transfer is slow, or multiple read ports of memory tile affect each other. Try to improve data transfer speed comparing to kernel execution.

FIFO Depth Visualization in the Vitis IDE

The DMA FIFO size and stream switch FIFO size can be viewed in the timeline in the Vitis IDE using VCD-based analysis. The IDE shows the FIFO depth that was used in the simulation. The IDE can help analyze design stall issues, and optimize the FIFO size needed. This helps optimize the design performance.

For enabling FIFO size visualization, the VCD dump option of `aiesimulator` should be enable. Also, the simulation run result can be opened in the IDE, for example:

```
aiesimulator --pkg-dir=./Work --online -wdb -text
vitis -a aiesimulator_output/default.aierun-summary
```

For more options on how to run simulator and open run results, see [AI Engine Stall Analysis in the Vitis IDE](#).



TIP: If design hangs in simulation, use the `--simulation-cycle-timeout=<cycles>` option to stop the `aiesimulator` simulation at a set time.

DMA FIFO Depth Visualization

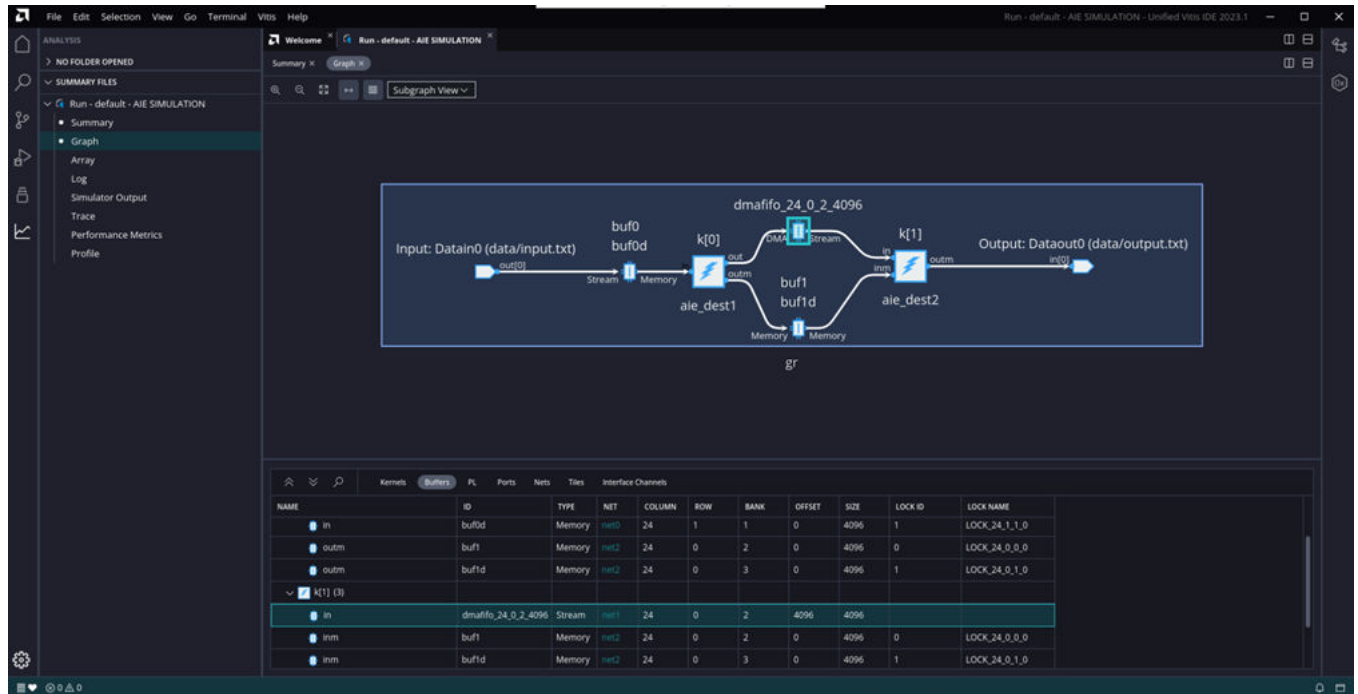
For DMA FIFO depth visualization, in the IDE, select the **Graph** view, and select the DMA FIFO in the view.



TIP: Hover the mouse on the DMA FIFO to show information about the DMA FIFO, such as the FIFO depth in words.

The following figure shows an example of DMA FIFO in Graph view:

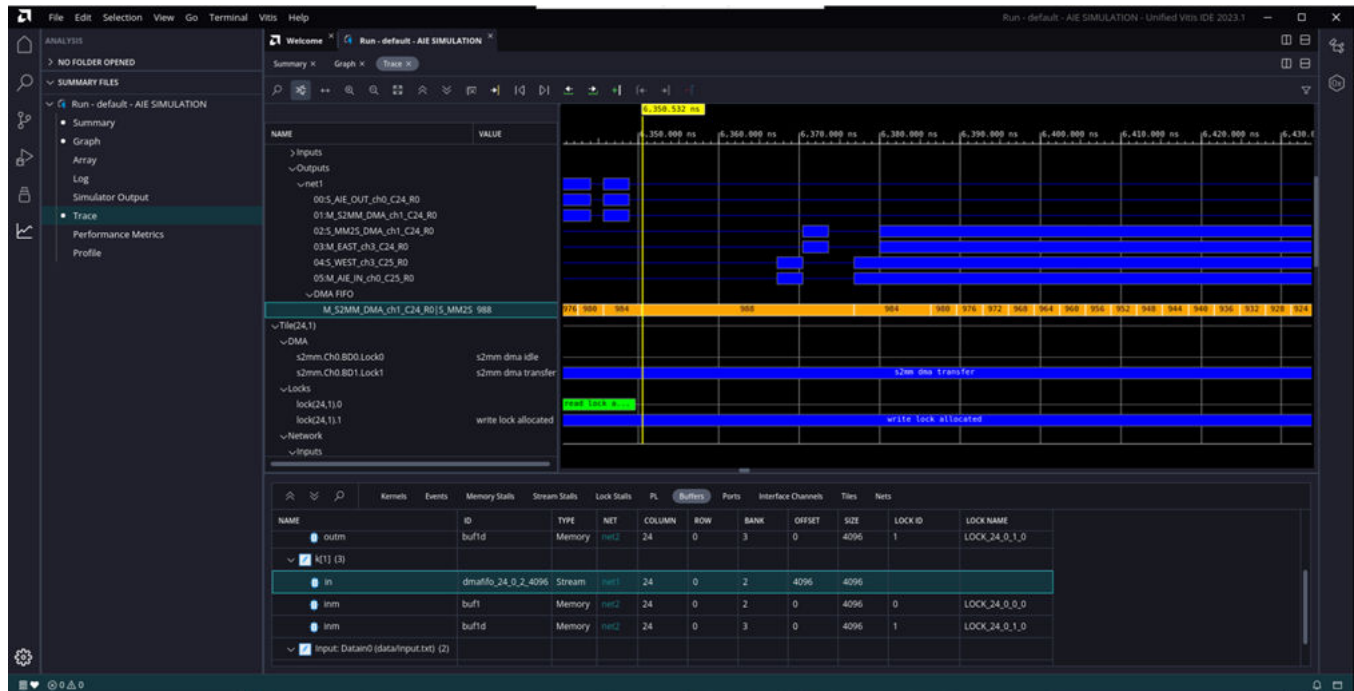
Figure 41: DMA FIFO in the Graph View



The selected DMA FIFO in the Graph view is also highlighted in the Trace view. Click **Trace** to switch to the Trace view.

The DMA FIFO size in words is shown in the timeline, as shown in the following figure. In the Events view, the size information for the `DMA_FIFO_SIZE` event is shown.

Figure 42: DMA FIFO in the Trace View



Note: The DMA FIFO size is shown as multiples of four, because it only increments after it successfully writes four words (128 bits) in a memory bank at a time. If the last words is less than four, the last words is in held inside the S2MM channel, and is never read by the consumer. To make sure that no data is held inside the S2MM channel, the stream must be either continuous or a multiple of four words.

The DMA FIFO information can be used together with other design analyzing techniques to help with design optimization. For additional design analysis techniques, see [AI Engine Stall Analysis in the Vitis IDE](#).

Stream Switch FIFO Visualization

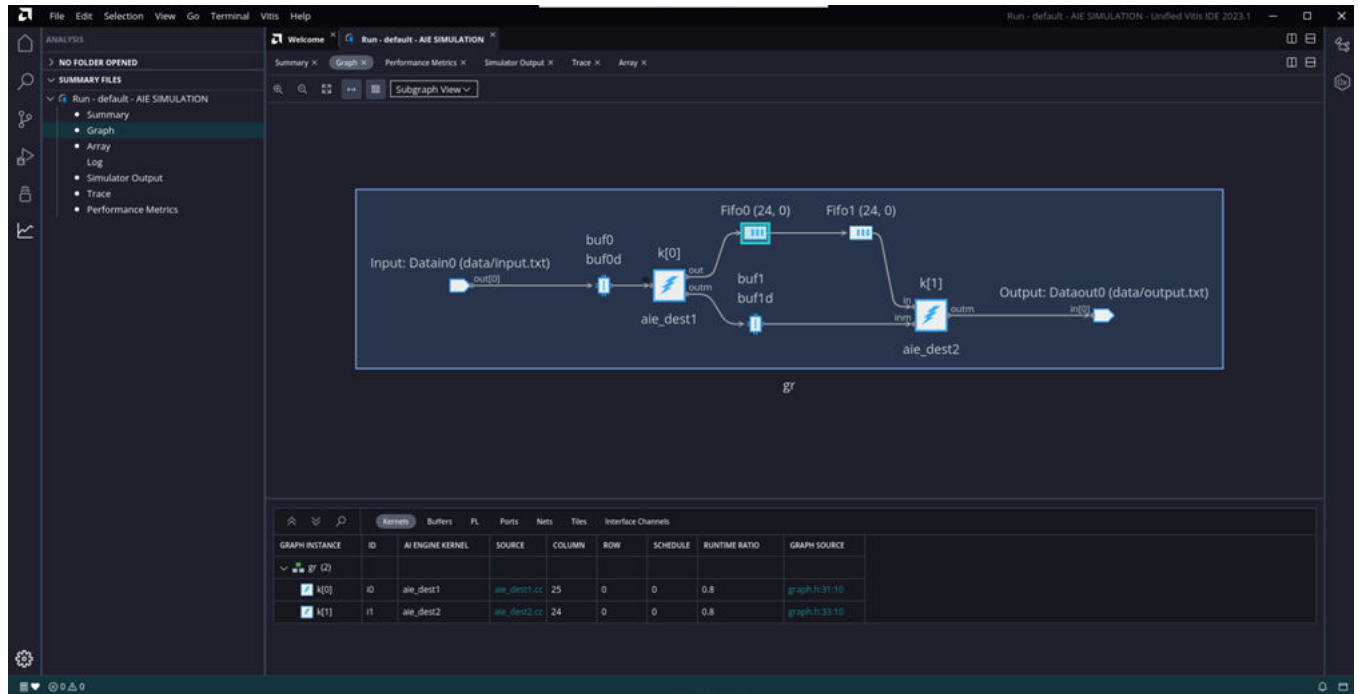
For stream switch FIFO visualization, in the IDE, select the **Graph** view and select the stream switch FIFO in the view.



TIP: Hover the mouse on the stream switch FIFO to show information about the FIFO location.

The following figure shows an example of DMA FIFO in Graph view:

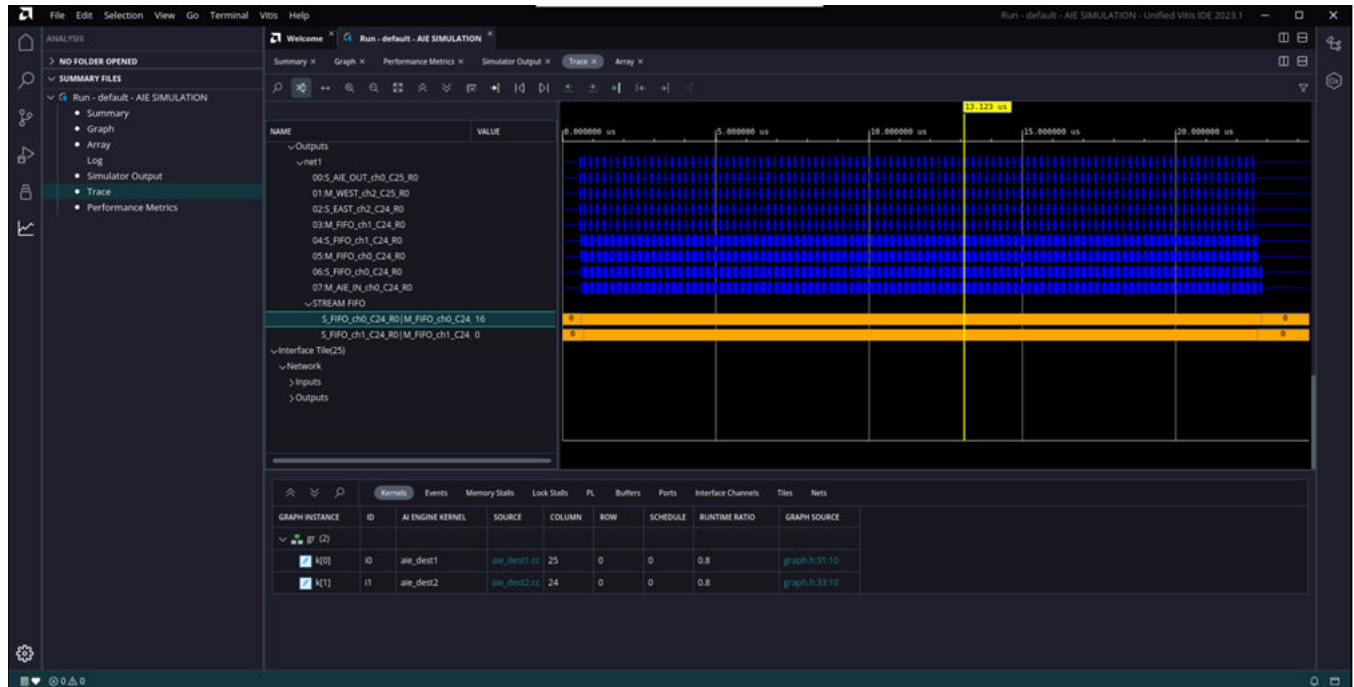
Figure 43: Stream Switch FIFO in the Graph View



The selected stream switch FIFO in the Graph view is also highlighted in the Trace view.

The stream switch FIFO size in words is shown in the timeline, as shown in the following figure. In the Events view, the size information for the `STREAM_SWITCH_FIFO_SIZE` event is shown.

Figure 44: Stream Switch FIFO in the Trace View



Note: When multiple stream switch FIFOs are chained, not all FIFOs are shown in the Trace view. Only the stream switch FIFO that have a FIFO size larger than one will be shown.

The stream switch FIFO information can be used together with other design analyzing techniques to help design optimization. For additional design analysis techniques, see [AI Engine Stall Analysis in the Vitis IDE](#).

Performance Analysis of AI Engine Graph Application on Hardware

A system-level view of program execution can be helpful in identifying problems during program execution including functional correctness and performance related challenges. While system level hardware emulation helps during the development phase of the system, AMD offers functional and performance debug flows in hardware as well. The AI Engine architecture has support for generation, collection, and streaming of profile related data, and events as trace data during hardware execution.



TIP: To facilitate the testing of AI Engine graphs on VCK190, AMD provides a ready-to-use test harness. This test harness is designed to provide a very simple and natural transition from the AI Engine simulation environment to testing on hardware. With the test harness, AI Engine graphs running in simulation can be taken to hardware in a few minutes and with only a few minor modifications. More details about the AI Engine test harness can be found here: <https://xilinx.github.io/AI-Engine-Test-Harness/>

Analyzing AI Engine Status in Hardware

The AI Engine provides the ability to output a summary of AI Engine status, which include error events that have occurred. Warnings are also issued when a deadlock is detected in the hardware. The status and warnings can be further analyzed in the AMD Vitis™ IDE for debug purposes.

There are two ways to output AI Engine status when running designs in hardware.

- Automated and periodic AI Engine status output: After initial setup in `xrt.ini`, this method requires minimal user intervention, because the tool will output periodic status at specified time intervals.
- Manual output the AI Engine status: You must run a command each time you want a status output report.

You can then open the status output in the IDE for further analysis.

This section covers the steps needed to obtain status output, and analysis of that output. For more details on understanding stalls, see [AI Engine Stall Analysis in the Vitis IDE](#).

Generating AI Engine Status

Periodic AI Engine Status Output Using XRT

You can enable the run-time deadlock detection and status output using the `xrt.ini` file. This is a one-time set up, which results in periodic output of status data, including deadlock detection. To turn on this feature, add the following to the `xrt.ini` file:

```
[Debug]
aie_status=true
```

To specify the interval of probing and analysis the AI Engine status:

```
[Debug]
aie_status=true
aie_status_interval_us=1000
```

The AI Engine status is copied to the following files when the host program is running:

- `xrt.run_summary`: Run summary information that can be used by the IDE.
- `aie_status_<device>_<current time>.json`: Status of AI Engine, AI Engine memory and AI Engine Interface tiles.
- `summary.csv`: Always created for other profiling capabilities, like guidance.

If a potential deadlock is detected, an warning message is reported, such as the following:

```
[XRT] WARNING: Potential deadlock/hang found in AI Engines. Graph : gr
[XRT] WARNING: Potential stuck cores found in AI Engines. Graph : gr Tile :
(25,1) Status 0x81 : Enable,Lock_Stall_W
```

Besides the AI Engine status, if any error event occurred, it is recorded in the JSON file.

You are responsible for determining the severity of the error, and whether it is recoverable. Use the summary and JSON files for analyzing in the IDE.

Manual AI Engine Status Output

You can output a single snapshot of the AI Engine status to the JSON file at any time after the device has been loaded. This is a static snapshot of the AI Engine running status, along with the events that happened before.

Use the `xbutil` command with option `-f json` to output status to the JSON file. For example:

```
xbutil examine -r aie -d 0 -f json -o aie_status_xbutil.json
```

Related Information

[Analyzing the Manual Status Output](#)

Analyzing AI Engine Status

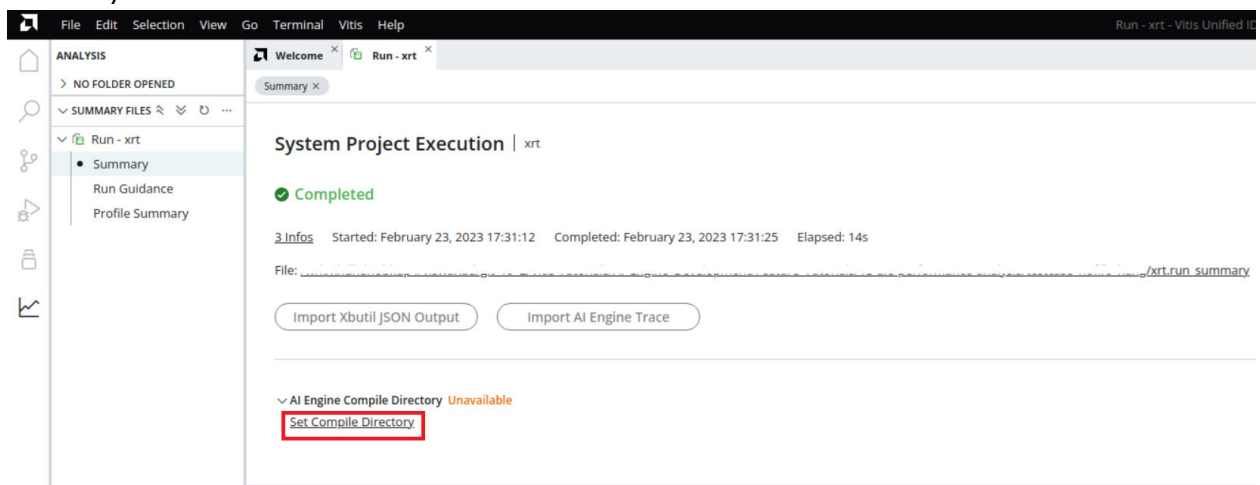
Analyzing the Automated Status Output

In the Vitis IDE, you can analyze the AI Engine status output that is automatically generated as follows:

1. Open the run summary file, `xrt.run_summary`, with the following command:

```
vitis -a xrt.run_summary
```

2. Click **Set Compile Directory** in the summary view to set the AI Engine compile/work directory.



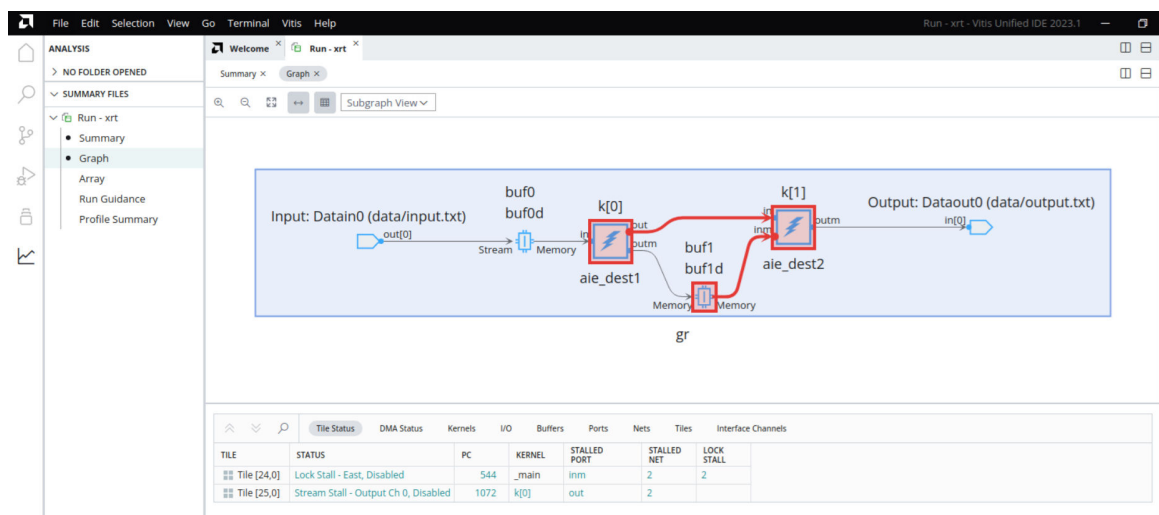
3. In the prompted dialog box, click the ... button, and select the AI Engine compile summary file, such as `./Work/graph.aiecompile_summary`.

The summary data displays in the IDE.

4. Click **Graph** or **Array** to display the summary data as desired. Note that all stalls are shown in red.
5. At the bottom of the summary data, you can access the associated views related to the AI Engine status, some of which are defined as follows.
 - **Tile Status:** Shows AI Engine status, including stream, cascade, lock stalls. Columns are explained as follows:
 - **PC:** Shows the current PC value. It can be cross-probed with source code by clicking it.
 - **Status:** Shows AI Engine status. It can be:
 - **Done:** All done.
 - **Memory Stall - South:** AI Engine stalled on accessing memory that is located south of the AI Engine. See Array view for the physical placement of AI Engine's and buffers.
 - **Memory Stall - West:** AI Engine stalled on accessing memory that is located west of the AI Engine.

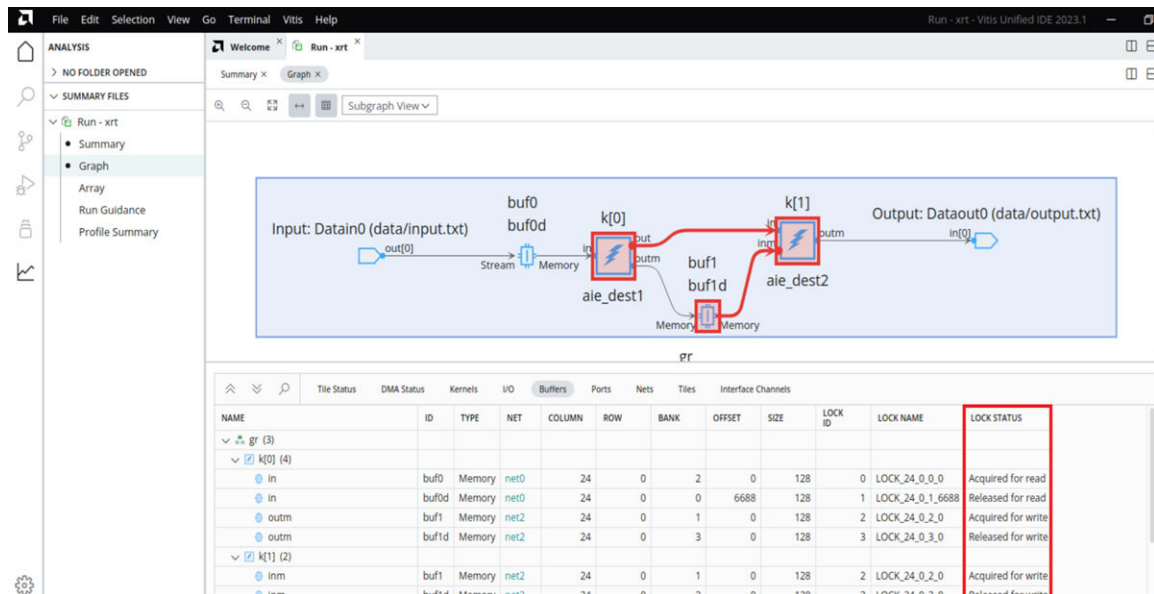
- **Memory Stall - North:** AI Engine stalled on accessing memory that is located north of the AI Engine.
- **Memory Stall - East:** AI Engine stalled on accessing memory that is located east of the AI Engine.
- **Lock Stall - South:** AI Engine stalled on acquiring lock of buffer, that is located south of the AI Engine.
- **Lock Stall - West:** AI Engine stalled on acquiring lock of buffer, that is located west of the AI Engine.
- **Lock Stall - North:** AI Engine stalled on acquiring lock of buffer, that is located north of the AI Engine.
- **Lock Stall - East:** AI Engine stalled on acquiring lock of buffer, that is located east of the AI Engine.
- **Stream Stall - Input Ch 0:** AI Engine stalled on reading from stream input channel 0.
- **Stream Stall - Input Ch 1:** AI Engine stalled on reading from stream input channel 1.
- **Stream Stall - Output Ch 0:** AI Engine stalled on writing to stream output channel 0.
- **Stream Stall - Output Ch 1:** AI Engine stalled on writing to stream output channel 1.
- **Cascade Stall - Input:** AI Engine stalled on reading from cascade input stream.
- **Cascade Stall - Output:** AI Engine stalled on writing to cascade output stream.
- **Debug Halt:** AI Engine stalled by enabling debug mode.
- **ECC Error Stall:** AI Engine stalled when ECC error happens.
- **ECC Scrubbing Stall:** AI Engine stalled on ECC Scrubbing error.
- **Error Halt:** AI Engine stalled on error.
- **Enabled:** AI Engine is enabled.
- **Reset:** In reset.

Figure 45: Stalls and Status in the Graph View



- **DMA Status:** Shows the Memory module DMA channel status in the design. This is an internal DMA channel control state machine which is asynchronous to application data movement and should not be used to determine if the channel is idle.
- **Error Events:** Shows error events that have happened before the status was generated. Further details are available in the following tables: [Table 28: floating_point Metric](#), [Table 76: CORE Module Error Events](#), and [Table 77: MEMORY Module Error Events](#).
- **Buffers:** Shows buffer objects in the design with the current status. Something to note:
 - Lock Status: Lock status of the buffer object:
 - Acquired for read: The buffer has been acquired for read by the consumer kernel.
 - Released for read: The buffer has been released for read by the producer kernel.
 - Acquired for write: The buffer has been acquired for write by the producer kernel.
 - Released for write: The buffer has been released for write by the consumer kernel.

Figure 46: Buffers with Lock Status



TIP: Hover the mouse on objects in the Graph view. It will show more information about the object.

Related Information

[AI Engine Stall Analysis in the Vitis IDE](#)

Analyzing the Manual Status Output

After manually generating the AI Engine status, as described in [Generating AI Engine Status](#), you can analyze the data in the Vitis IDE as follows:

1. After opening the IDE, click on **Import Xbutil/Xsdb JSON output**.

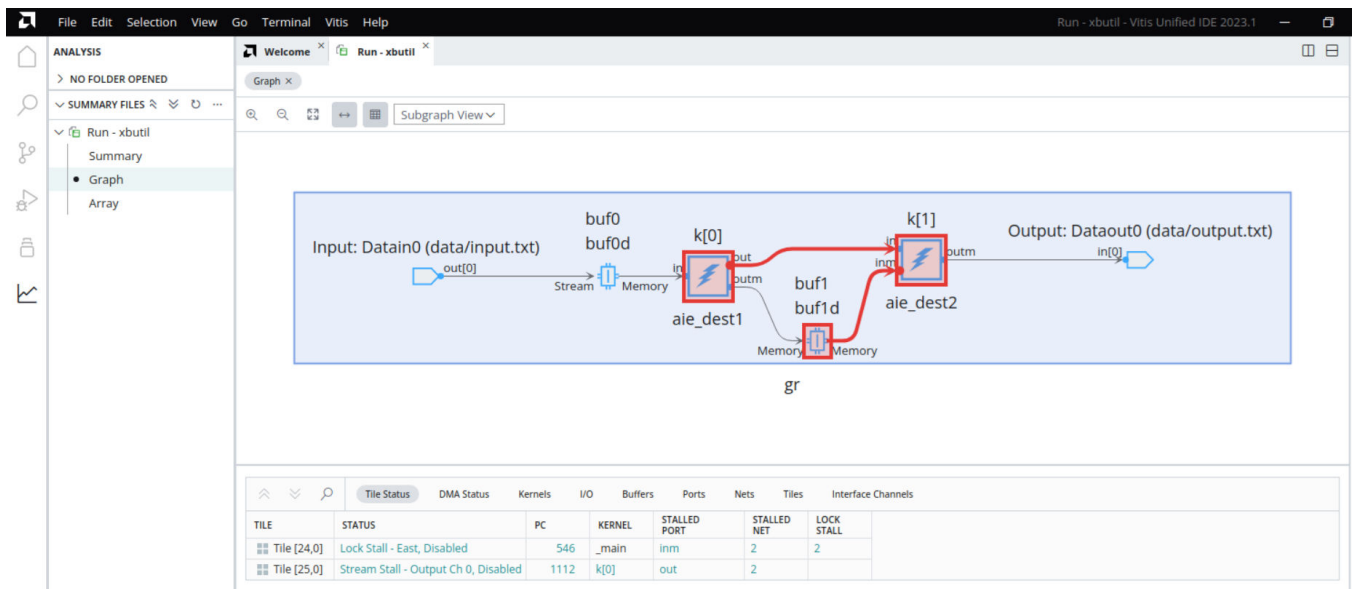
2. In the prompted window, set the following options.

- **Xbutil JSON File:** Select the JSON file that was manually generated with the `xbutil` command. For example, select the file `aie_status_xbutil.json` file.
- **AI Engine Compile Summary:** Select the AI Engine compile summary file. For example:
`./Work/graph.aiecompile_summary`
- **Run Summary:** The run summary to be written. The run summary can be used to reload the analysis next time by **Analysis**→**Open Summary** or **File**→**Open Recent**→**Summary**.

The Graph and Array views are shown in the IDE. The following figure shows a manual AI Engine status example.

Note: If you load the JSON file before loading the compile work directory, the information will not be displayed because there is no graph report. An imported JSON file overrides any other JSON file previously set.

Figure 47: AI Engine Status Example



Related Information

[AI Engine Stall Analysis in the Vitis IDE](#)

Generating AI Engine Status using XSDB

You can examine the status of the AI Engine using XSDB on both Linux and Bare metal operating systems. This feature allows you to debug applications and detect the status of the AI Engine in situations where the board is in a deadlock or hung state. Unlike the `xbutil` command which requires XRT, the XSDB command is run independent of XRT.

XSDB reports the AI Engine status in `.json` file format using the `aiestatus examine` command. You can use this command before, during and after running the application.

Details about the XSDB command `aiestatus examine` and options are explained below.

Command

```
aiestatus examine
```

Supported Options

```
aiestatus examine [-graphs] <graph-list> [-work-dir] <dir-path> [-aie-version] <version> [-file] <file-name> [-run-summary] [-target-name] <target-name> [-tiles] <tile-list>
```

The support options are defined as follows:

Table 23: Supported Options

Options	Description
<code>-graphs</code>	Specify the list of one or more graphs used in the AI Engine application. If this option is not specified, the command targets all graphs in the design.
<code>-work-dir</code>	Specify the AI Engine project Work directory. This is a mandatory option.
<code>-aie-version</code>	Specify the AI Engine hardware generation number, available as <code>hw_gen</code> value in <code>Work/ps/c_rts/aie_trace_config.json</code> file. Note: Needed only when no work directory is provided. The default value used is 1.
<code>-file</code>	Specify the name of the output file in <code>.json</code> format generated by the command. This is not a mandatory option.
<code>-run-summary</code>	Enables option to generate a run summary file in addition to the <code>.json</code> file. This is not a mandatory option.
<code>-target-name</code>	Specify the name of target to connect. Note: Default target is <code>Versal</code> . To see different targets available, type the command <code>connect</code> followed by <code>targets</code> in XSDB terminal.
<code>-tiles</code>	Specify a list of one or more tiles used by the AI Engine application and that will be monitored by the command. Tiles can be in the format of <code>col,row</code> (ex. 0,0; 0,1). All the tiles in a row or column can be specified using wildcard <code>'*'</code> (ex. 0,*; *,1; *,*)

Command Examples

Source the `aie_status` Tcl script in XSDB terminal as shown in the two examples below.

```
xsdb% source $::env(XILINX_VITIS)/scripts/vitis/util/aie_status.tcl
```

- **Example 1:** Specifying graph and work directory.

```
aiestatus examine -work-dir ./Work -graphs dut
```

This example examines and reports the status of the graph named `dut`.

- **Example 2: Specifying tiles.**

```
aiestatus examine -tiles 10,2;5,*
```

This example examines and reports the status of tile (10,2) and all tiles in column 5.

The commands above can be run anytime before, during or after running the application, where the board is running the design or is in a deadlocked or hung state. The command produces a JSON file, `<aie_status_<date_time>.json>`, which is located in the directory where you run the command. An example file name is `aie_status_2022_08_11_113542.json`. This file can be opened in AMD Vitis™ IDE, and the status can be viewed and analyzed. To read the status registers of AI Engine tiles before running the application, run the `aiestatus` command prior to running the host application on the board.

The sample XSDB console after running the `aiestatus examine` command is shown below.

Figure 48: XSDB Console

```
xsdb% aiestatus examine -work-dir ./Work
INFO: Targeting all graphs in design.
Reading status registers for AI Engine tiles in graph all...
AIE Tiles: 24,1 24,2 25,1 25,3 25,2
```

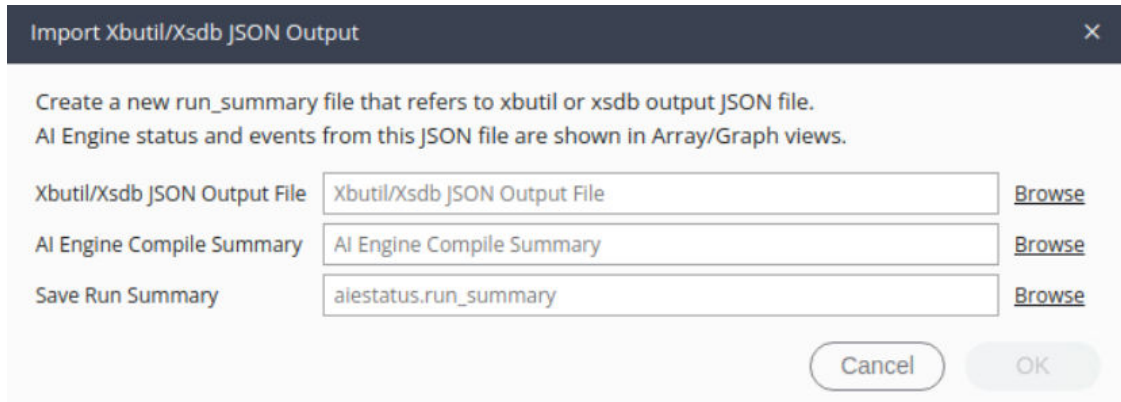
Note: In the sample XSDB console above, the option `-graphs` is not specified and the command targets all graphs in the AI Engine application.

Also observe the command returns the status registers of all the AI Engine tiles corresponding to the graph specified in the command.

Opening the JSON file in the Vitis IDE

1. Open the Vitis IDE, and select the **Import Xbutil/Xsdb JSON output** option.
2. In the dialog box, specify the `.json` file, as well as the **AI Engine Compile Summary file** and **Run Summary file** to be generated.

Figure 49: Import Xbutil/Xsdb JSON Output



Import Xbutil/Xsdb JSON Output

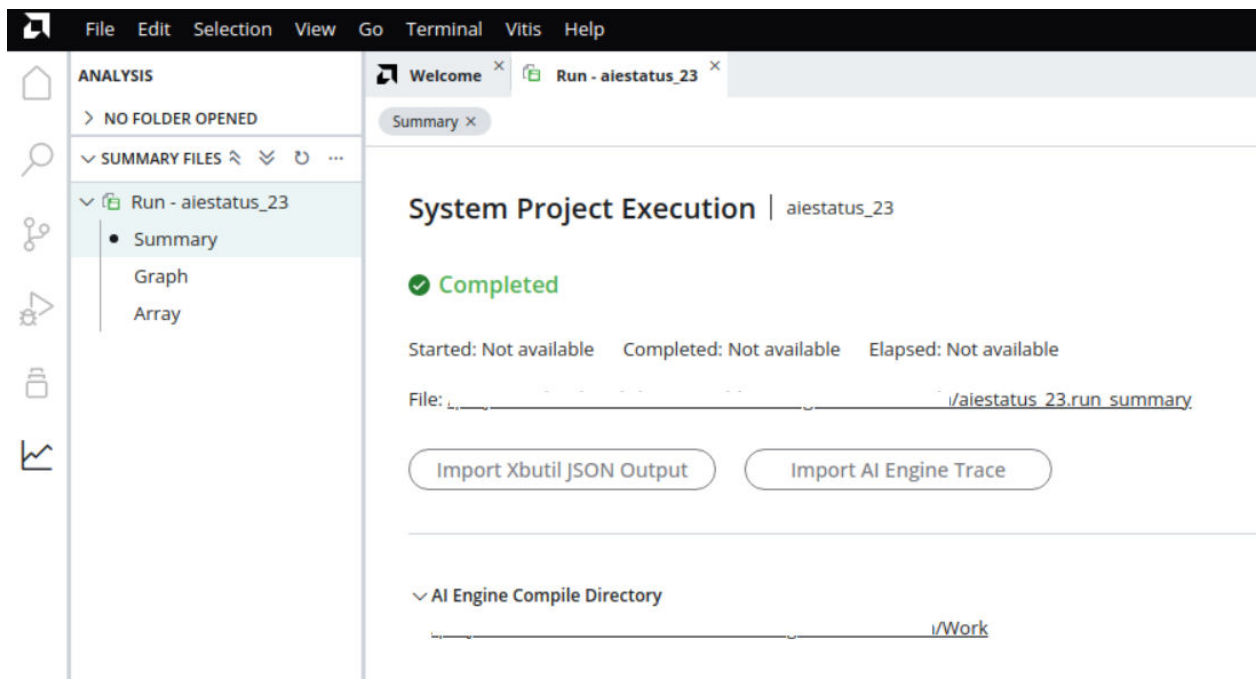
Create a new run_summary file that refers to xbutil or xsdb output JSON file.
AI Engine status and events from this JSON file are shown in Array/Graph views.

Xbutil/Xsdb JSON Output File	Xbutil/Xsdb JSON Output File	Browse
AI Engine Compile Summary	AI Engine Compile Summary	Browse
Save Run Summary	aiestatus.run_summary	Browse

[Cancel](#) [OK](#)

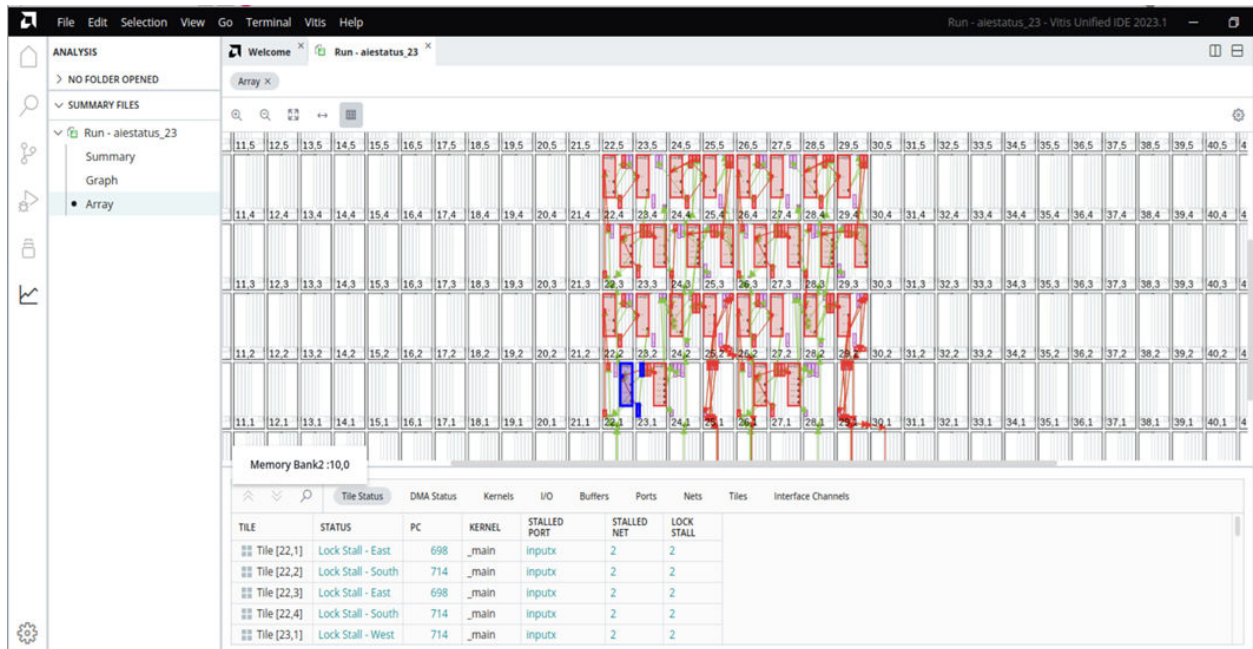
- After you click **OK**, a new summary file is generated in current working directory. The Analysis view opens the summary file as shown below.

Figure 50: Summary View



You can navigate to the Graph or Array view, and observe the status of the AI Engine tiles. The following figure shows the status of AI Engine tiles that are stalled.

Figure 51: Array View



This `aie_status.run_summary` file can be used to view and analyze the status results in Vitis IDE.

Alternatively, you can also run the `aiestatus` command with the `-run-summary` option as shown below.

```
aiestatus examine -work-dir ./Work -graphs dut -run-summary
```

This command generates the `aie_status.run_summary` file along with the `.json` file, which can be opened in the IDE.



RECOMMENDED: Use `-run-summary` option along with `-work-dir` option to seamlessly open and analyze the summary file in the Vitis IDE.

Profiling the AI Engine

Profiling is a form of dynamic program analysis that measures the input/output throughput, the space (memory), time complexity of a program, the usage of specific instructions, or the frequency and duration of function calls. Most commonly, profiling information serves to aid program optimization, and more specifically, performance tuning.

You can obtain profiling data when you run your design in simulation or in hardware at run-time. Analyzing this data helps you gauge the efficiency of the kernels, the stall and active times associated with each AI Engine, and pinpoint the AI Engine kernel whose performance might not be optimal. This also allows you to collect data on design latency, throughput, and bandwidth by using run-time event APIs in your PS host code.

You have two options to gather this information:

1. Use performance counters built into the hardware to monitor AI Engine and memory module events. This feature can be used in hardware and hardware emulation.
2. Use event APIs to profile the AI Engine graph inputs and outputs from the graph host application. This feature can be used in simulation and hardware flows.

Profiling the AI Engine in Hardware

An AI Engine event gives information about the system at a specific instant in time. An event that is associated with a timestamp, type and set of data values is referred to as a payload. The interpretation of the payload depends on the type of the event. A timestamp allows ordering of events, computation of causality relationships, and implementation of verifiers on a sequence of events.

For event modeling purposes, the key modules of the AI Engine array are processor, DMA, lock modules, memory, and I/O streams. Each module can be viewed as an event generator/responder. Each module receives an event and responds to the event. As a response, new events could be generated. Events are classified based on the generator of the events. The timestamp is not mentioned explicitly in the event definitions. Each event is described with a payload, which is a couple of values associated with the event. Each AI Engine, memory, DMA, or lock is addressable by a two-dimensional index <col, row>, which is the column and row index in the AI Engine array.

Note: Profiling is also supported on BDC platforms vck_190_bdc, as described in [Vitis Base Platform for the VCK190 Board](#), and vek_280_bdc, as described in [Vitis Base Platform for the VEK280 Pre-production Board](#).

Profiling the AI Engine, Memory Modules and Interface Tiles

There are four types of performance counters, run-time event performance counters for the AI Engine modules, run-time memory counters for memory modules, run-time interface counters for AI Engine-PL interface tiles and run-time memory counters for memory tiles (AI Engine-ML only). These performance counters can be configured to track a variety of events in the AI Engine, the memory module and the interface tile. Various features like error-correction code (ECC) scrubbing, event trace and profiling can use these performance counters. Performance counters count occurrences of a given event in a profile configuration. The profile feature offers several different configurations of these performance counters that can be dynamically applied at run-time to collect various profiling statistics.

No changes are required in PS host code when using performance counters. These counters can be configured, read and collected at run-time while the design is executing in hardware. The following table lists the number of performance counters that are available at different configurations.

Table 24: Available Performance Counters

Event Trace Used?	Counters Available for Profiling			
	Core Module	Memory Module	PL Interface	Memory Tile (AI Engine-ML)
No	3	2	2	4
Yes	2	1	2	3

When performance counters are used for event trace and profiling in the same execution, allocated performance counters cannot meet the requirements of all the requested features at the same time. The following warning messages indicate this situation.

Figure 52: Warning Message

```

Loading: 'a.xclbin'
XAIEFAL: INFO: Resource group Avail is created.
XAIEFAL: INFO: Resource group Static is created.
XAIEFAL: INFO: Resource group Generic is created.
XAIEFAL: INFO: Resource group Avail is created.
XAIEFAL: INFO: Resource group Static is created.
XAIEFAL: INFO: Resource group Generic is created.
XRT build version: 2.12.0
Build hash: 2719b6027e185000fc49783171631db03fc0ef79
Build date: 2021-10-09 04:14:02
Git branch: 2021.2
PID: 1234
UID: 0
[Tue Nov 23 13:34:03 2021 GMT]
HOST: versal-rootfs-common-2021_2
EXE: /media/sd-mmchlk0p1/host.exe
[XRT] WARNING: Only 1 out of 4 metrics were available for aie core module profiling due to resource constraints. AIE profiling uses performance counters which could be already used by AIE trace, ECC, etc.
Available metrics : ACTIVE_CORE
Unavailable metrics : GROUP_CORE_STALL_CORE INSTR_VECTOR_CORE GROUP_CORE_PROGRAM_FLOW
[XRT] WARNING: Only 0 out of 2 metrics were available for aie memory module profiling due to resource constraints. AIE profiling uses performance counters which could be already used by AIE trace, ECC, etc.
Available metrics :
Unavailable metrics : GROUP_MEMORY_CONFLICT_MEM GROUP_ERRORS_MEM

```

All the chosen metric sets should be grouped in the `xrt.ini` file under the `[Debug]` and `[AIE_profile_settings]` keywords in the XRT flow or specified as flags when using `aieprofile` in the XSDB flow.

Profiling for AI Engine

The following tables list the pre-defined metric set configurations available for AI Engine, in order of priority by which they are assigned to the available counters. In the `xrt.ini` file all these metric names should be in lower case and assigned to one of the following metric selectors:

- `tile_based_aie_metrics`

- `graph_based_aie_metrics`

Table 25: **heat_map Metric**

Metric Name	Description
Active Time	The time AI Engine is active because being enabled.
Stall Time	The time AI Engine is stalled. This stall includes AI Engine memory, stream, cascade, and lock stalls.
Vector Instruction Time	The time AI Engine spent executing instructions in the vector processor.
Cumulative Instruction Time	The time AI Engine spent executing load/store, stream get/put, lock acquire/release instructions.
Active Utilization	The time AI Engine is actively executing instructions and not stalling. The percentage is relative to active time.
Vector Instruction Utilization	The time AI Engine is executing vector instructions. The percentage is relative to active utilization time (active - stalls).

These indicators help you understand the efficiency of the kernels that are implemented in the AI Engines. You can compare stall time with active time to determine if there is a data communication issue for each AI Engine.

Table 26: **stalls Metric**

Metric Name	Description
Memory Stall Time	The time the AI Engine is not active due to a memory stall.
Stream Stall Time	The time the AI Engine is not active due to a stream stall.
Lock Stall Time	The time the AI Engine is in a lock stall.
Cascade Stall Time	The time the AI Engine is in a cascade stall.

A stall in an AI Engine can occur in various situations:

- A memory stall happens when multiple accesses to the same memory bank are requested from one core, multiple cores, and/or DMAs.
- Stream stalls occur when data production and consumption on a stream do not have the same rate, leading to input stream starvation or output stream overflow.
- A cascade stall is generated when the cascade writer does not have the same rate as the cascade reader.
- A lock stall happens if the window data producer does not have the same iteration rate as the window consumer.

Table 27: **execution Metric**

Metric Name	Description
Vector Instruction Time	The time spent by the AI Engine on vector instructions: vector processor instruction and vector data load/store.
Load Instruction Time	The time spent by the AI Engine on load instructions (move data from memory to registers).

Table 27: **execution Metric** (cont'd)

Metric Name	Description
Store Instruction Time	The time spent by the AI Engine on store instructions (move data from registers to memory).
Cumulative Instruction Time	The time spent by the AI Engine on memory and stream accesses and lock acquire/release.

All these indicators allow you to estimate the efficiency of your kernel. To increase efficiency, you should optimize data access, favor vector instructions over scalar instructions, and use 128-bit access to streams whenever possible.

Table 28: **floating_point Metric**

Metric Name	Description
Floating-Point Overflow Exception	The number of floating-point overflow exceptions generated by AI Engine.
Floating-Point Underflow Exception	The number of floating-point underflow exceptions generated by AI Engine.
Floating-Point Invalid Exception	The number of floating-point Invalid exceptions generated by AI Engine.
Floating-point Divide by Zero Exception	The number of floating-point divide by zero exceptions generated by AI Engine.

Floating-point exceptions lead to erroneous results. You might have to recode your floating-point algorithm if you get too many exceptions, or even a single in a critical area of the code.

Table 29: **stream_put_get Metric**

Metric Name	Description
Cascade Read Instruction Time	Time AI Engine spent executing read instructions on the cascade stream.
Cascade Write Instruction Time	Time AI Engine spent executing write instructions on the cascade stream.
Stream Read Instruction Time	Time AI Engine spent executing read instructions on the stream.
Stream Write Instruction Time	Time AI Engine spent executing write instructions on the stream.

This metric is useful for cascade and stream usage analysis.

Table 30: **aie_trace Metric**

Metric Name	Description
AI Engine Trace Word Count	The amount of AI Engine trace produced.
AI Engine Trace Stall Count	The amount of AI Engine trace back-pressure events produced.
Memory Module Trace Word Count	The amount of Memory Module trace produced.
Memory Module Trace Stall Count	The amount of Memory Module trace back-pressure events produced.

These metrics, particularly the stall counts, help in defining the right number of streams to transmit AI Engine and Memory Module events to the Programmable Logic.

Table 31: write_throughputs Metric

Metric Name	Description
Active Time	Time AI Engine was active because it was enabled.
Stream Write Instruction Time	Time AI Engine spent executing write instructions on data streams.
Cascade Write Instruction Time	Time AI Engine spent executing write instructions on the cascade stream.
Stall Time	Time AI Engine was stalled. This stall includes AI Engine memory, stream, cascade, and lock stalls.
Stream Write Throughput (MB/s)	Write throughput of Stream Ports in MB/s
Cascade Write Throughput (MB/s)	Write throughput of Cascade Ports in MB/s

These metrics are useful to evaluate the overall streams and cascades output write throughput of the system.

Table 32: read_throughputs Metric

Metric Name	Description
Active Time	Time AI Engine was active because it was enabled.
Stream Read Instruction Time	Time AI Engine spent executing read instructions on data streams.
Cascade Read Instruction Time	Time AI Engine spent executing read instructions on the cascade stream.
Stall Time	Time AI Engine was stalled. This stall includes AI Engine memory, stream, cascade, and lock stalls.
Stream Read Throughput (MB/s)	Read throughput of Stream Ports in MB/s
Cascade Read Throughput (MB/s)	Read throughput of Cascade Ports in MB/s

These metrics are useful to evaluate the overall streams and cascades input read throughput of the system.

s2mm_throughputs and mm2s_throughputs Metrics

These metrics are automatically selected, replacing other metrics that might have been set, when the corresponding Memory Module metric sets are specified by the user as AI Engine metrics will contribute to the throughputs calculation for these memory module metric sets.

Profiling for Memory Modules

The following tables list the pre-defined metric set configurations available for memory modules. In the `xrt.ini` file all of these metric names should be in lower case and assigned to one of the following metric selectors:

- `tile-based_aie_memory_metrics`
- `graph-based_aie_memory_metrics`

Table 33: conflicts

Metric Name	Description
Memory Conflict	The time taken due to data memory conflicts on any of the 8 banks of the memory module. Note: The hardware view is 8 banks of 128-bit width. The software view is 4 banks of 256-bit width.
Cumulative Memory Errors	The time taken due to ECC errors in any of the Data Memory banks, as well as the 2x MM2S and the 2x S2MM DMAs.

Memory conflicts happen when two memory chunks reside in the same memory bank and are accessed either by the same AI Engine (using the two read ports) or by two different AI Engines. A potential solution is to constrain the locations of these memories to different banks. In order to get more details about which bank is causing these conflicts, you should analyze the events from an AI Engine simulation.

Table 34: dma_locks

Metric Name	Description
Cumulative DMA Activity	The time taken due to stalled lock acquires on both the MM2S and S2MM channels of the DMA.
Cumulative DMA Lock Count	The lock stall count on the DMA channels.

The four DMA channels (2xS2MM and 2xMM2S) are driven by buffer descriptors. The Cumulative DMA Activity is a count of the time taken due to stalled lock acquire events on all channels. All these DMA events will help you understand why some connections through the device are slower than expected.

Table 35: dma_stalls_s2mm

Metric Name	Description
S2MM Channel 0 Stalls	The time S2MM channel 0 is stalled on lock acquire.
S2MM Channel 1 Stalls	The time S2MM channel 1 is stalled on lock acquire.

Table 36: dma_stalls_mm2s

Metric Name	Description
MM2S Channel 0 Stalls	The time the MM2S channel 0 stalled on lock acquire.
MM2S Channel 1 Stalls	The time the MM2S channel 1 stalled on lock acquire.

Each AI Engine memory module contains two input streams to memory map (S2MM) DMA, two memory map to output DMA streams (MM2S) channels. The `s2mm_throughputs` and `mm2s_throughputs` metrics profile the throughput of the S2MM and MM2S DMA channels respectively.

Table 37: **s2mm_throughputs** of AI Engine

Metric Name	Description
DMA S2MM Channel 0 BD Packet Count	The number of BD packets written over DMA S2MM channel 0.
DMA S2MM Channel 1 BD Packet Count	The number of BD packets written over DMA S2MM channel 1.
DMA S2MM Channel 0 Throughput (MB/s)	The throughput of DMA S2MM channel 0.
DMA S2MM Channel 1 Throughput (MB/s)	The throughput of DMA S2MM channel 1.

The `write_throughputs` metric is deprecated, and `s2mm_throughputs` is used instead.

Table 38: **mm2s_throughputs** of AI Engine

Metric Name	Description
DMA MM2S Channel 0 BD Packet Count	The number of BD packets written over DMA MM2S channel 0.
DMA MM2S Channel 1 BD Packet Count	The number of BD packets written over DMA MM2S channel 1.
DMA MM2S Channel 0 Throughput (MB/s)	The throughput of DMA MM2S channel 0.
DMA MM2S Channel 1 Throughput (MB/s)	The throughput of DMA MM2S channel 1.

The `read_throughputs` metric is deprecated, and `mm2s_throughputs` is used instead.



IMPORTANT! For both `mm2s_throughputs` and `s2mm_throughputs` this setting will override any AI Engine metric setting since corresponding AI Engine metrics contribute to the throughput calculation.

Table 39: **s2mm_throughputs** of AI Engine-ML

Metric Name	Description
DMA S2MM Channel Lock Stall Time/%	The time DMA S2MM channel 0 is stalled on a lock. This must be used with <code>s2mm_throughputs</code> in an AI Engine module.
DMA S2MM Channel Lock Stall Time/%	The time DMA S2MM channel 1 is stalled on a lock. This must be used with <code>s2mm_throughputs</code> in an AI Engine module.
DMA S2MM Memory Backpressure Time/%	The time DMA S2MM channel 0 is inactive due to memory backpressure. This must be used with <code>s2mm_throughputs</code> in an AI Engine module.
DMA S2MM Memory Backpressure Time/%	The time DMA S2MM channel 1 is inactive due to memory backpressure. This must be used with <code>s2mm_throughputs</code> in an AI Engine module.
DMA S2MM Channel 0 Throughput (MB/s)	The throughput of DMA S2MM channel 0. This must be used with <code>s2mm_throughputs</code> in the AI Engine.
DMA S2MM Channel 1 Throughput (MB/s)	The throughput of DMA S2MM channel 1. This must be used with <code>s2mm_throughputs</code> in the AI Engine.

Note: These metrics are only available on AI Engine-ML devices.

Table 40: **mm2s_throughputs** of AI Engine-ML

Metric Name	Description
DMA MM2S Stream Backpressure Time/%	The time DMA MM2S channel 0 is inactive due to stream backpressure. This must be used with <code>mm2s_throughputs</code> in an AI Engine module.

Table 40: mm2s_throughputs of AI Engine-ML (cont'd)

Metric Name	Description
DMA MM2S Stream Backpressure Time/%	The time DMA MM2S channel 1 is inactive due to stream backpressure. This must be used with <code>mm2s_throughputs</code> in an AI Engine module.
DMA MM2S Memory Starvation Time/%	The time DMA MM2S channel 0 is inactive due to memory starvation. This must be used with <code>mm2s_throughputs</code> in an AI Engine module.
DMA MM2S Memory Starvation Time/%	The time DMA MM2S channel 1 is inactive due to memory starvation. This must be used with <code>mm2s_throughputs</code> in an AI Engine module.
DMA MM2S Channel 0 Throughput (MB/s)	The throughput of DMA MM2S channel 0. This must be used with <code>mm2s_throughputs</code> in the AI Engine.
DMA MM2S Channel 1 Throughput (MB/s)	The throughput of DMA MM2S channel 1. This must be used with <code>mm2s_throughputs</code> in the AI Engine.

Note: These metrics are only available on AI Engine-ML devices.



IMPORTANT! For both `mm2s_throughputs` and `s2mm_throughputs` this setting will override any AI Engine-ML metric setting since corresponding AI Engine-ML metrics contribute to the throughput calculation.

Profiling Memory Tiles

The following tables list the pre-defined metric set configurations available for memory tiles. In the `xrt.ini` file, all of these metric names should be in lowercase and assigned to one of the following metric selectors:

- `tile_based_memory_tile_metrics`
- `graph_based_memory_tile_metrics`

Metric name changes are as follows:

- `input_channels` is now `s2mm_channels`.
- `input_channels_details` is now `s2mm_channels_details`.
- `output_channels` is now `mm2s_channels`.
- `output_channels_details` is now `mm2s_channels_details`.
- `input_throughputs` is now `s2mm_throughputs`.
- `output_throughputs` is now `mm2s_throughputs`.

Table 41: s2mm_channels (input_channels)

Metric Name	Description
Channel Transfer Time/%	The time the memory tile channel transferred data.
Channel Stalled Time/%	The time the memory tile channel stalled.
Channel AXI Packet Count	The number of AXI4-Stream packets transferred over memory tile channel.

Table 41: s2mm_channels (input_channels) (cont'd)

Metric Name	Description
Input Channel Packet Count	The number of BD packets transferred over memory tile input channel.
Input Channel Idle Time/%	The time the memory tile channel is idle.

Table 42: s2mm_channels_details (input_channels_details)

Metric Name	Description
Channel Transfer Time/%	The time the memory tile channel is transferring data.
Input Channel Memory Backpressure Time/%	The time the memory tile input channel backpressured from the memory.
Input Channel Lock Stall Time/%	The time the memory tile input channel is stalled on a lock.
Input Channel Stream Starvation Time/%	The time the memory tile input channel is starved for data on input stream.

Table 43: mm2s_channels (output_channels)

Metric Name	Description
Channel Transfer Time/%	The time the memory tile channel transferred data.
Channel Stalled Time/%	The time the memory tile channel is stalled.
Channel AXI Packet Count	The number of AXI4-Stream packets transferred over memory tile channel.
Output Channel Packet Count	The number of BD packets transferred over memory tile output channel.
Output Channel Idle Time/%	The time the memory tile channel is idle.

Table 44: mm2s_channels_details (output_channels_details)

Metric Name	Description
Channel Transfer Time/%	The time the memory tile channel transferred data.
Output Channel Stream Backpressure Time/%	The time the memory tile output channel backpressured from the output stream.
Output Channel Memory Starvation Time/%	The time the memory tile output is starved by the memory.
Output Channel Lock Stall Time/%	The time the memory tile output channel is stalled on a lock.

Table 45: memory_stats

Metric Name	Description
Group_Memory_Conflict	This event is triggered when any memory bank has a conflict.

Table 45: **memory_stats** (cont'd)

Metric Name	Description
Group_Errors	Any of the following events triggers this event: - Scrubber has fixed a 1-bit ECC. - ECC scrubber detects 2-bit ECC error. 1-bit ECC error is detected during access. 2-bit ECC error is detected during access. - Any error in any of the 6 S2MM channels (including task queue overflow). - Any error in any of the 6 MM2S channels (including task queue overflow). - Any parity error at stream endpoints. - Parity error in the packet header. - Any error processing ctrl-pkt (slave error). - AXI-MM port access results in slave error - Underflow or overflow in locks. - Any channel stalled due to task-complete-token backpressure, or from FoT count FIFO full.
Group_Lock	Any of the following events for eight selected locks will trigger this event: - For selected lock <0...7>, granted acquire-equal request. - For selected lock <0...7>, granted acquire-greater-equal request. - For selected lock <0...7>, completed release request. - Selected lock <0...7> reached value set in register.
Group_Watchpoint	This event triggered by a memory access to a configured address, sets in register four specific registers.

Table 46: **s2mm_throughputs (input_throughputs)**

Metric Name	Description
S2MM Channel Transfer Time/%	The amount of words transferred over selected input channel.
S2MM Channel Stream Starvation Time/%	The time the memory tile input channel is starved for data on input stream.
S2MM Channel Memory Backpressure Time/%	The time the memory tile input channel is inactive due to memory backpressure.
S2MM Channel Lock Stall Time/%	The time the memory tile input channel is stalled on a lock.
S2MM Channel Throughput(MB/s)	Throughput used by memory tile channel (MB/s).

Table 47: **mm2s_throughputs (output_throughputs)**

Metric Name	Description
MM2S Channel Transfer Time/%	The amount of words transferred over selected output channel.
MM2S Channel Stream Backpressure Time/%	The time the memory tile output channel is inactive due to stream backpressure.
MM2S Channel Memory Starvation Time/%	The time the memory tile output channel is inactive due to memory starvation.

Table 47: **mm2s_throughputs (output_throughputs)** (cont'd)

Metric Name	Description
MM2S Channel Lock Stall Time/%	The time the memory tile output channel is stalled on a lock.
MM2S Channel Throughput(MB/s)	The throughput used by memory tile channel (MB/s).

The `conflict_stats1` metric monitors the number of clock cycles during which memory conflicts occurred on banks 0-3.

Table 48: **conflict_stats1**

Metric Name	Description
Bank 0 Conflict Count	The number of memory bank 0 conflicts.
Bank 1 Conflict Count	The number of memory bank 1 conflicts.
Bank 2 Conflict Count	The number of memory bank 2 conflicts.
Bank 3 Conflict Count	The number of memory bank 3 conflicts.

The `conflict_stats2` metric monitors the number of clock cycles during which memory conflicts occurred on banks 4-7.

Table 49: **conflict_stats2**

Metric Name	Description
Bank 4 Conflict Count	The number of memory bank 4 conflicts.
Bank 5 Conflict Count	The number of memory bank 5 conflicts.
Bank 6 Conflict Count	The number of memory bank 6 conflicts.
Bank 7 Conflict Count	The number of memory bank 7 conflicts.

The `conflict_stats3` metric monitors the number of clock cycles during which memory conflicts occurred on banks 8-11.

Table 50: **conflict_stats3**

Metric Name	Description
Bank 8 Conflict Count	The number of memory bank 8 conflicts.
Bank 9 Conflict Count	The number of memory bank 9 conflicts.
Bank 10 Conflict Count	The number of memory bank 10 conflicts.
Bank 11 Conflict Count	The number of memory bank 11 conflicts.

The `conflict_stats4` metric monitors the number of clock cycles during which memory conflicts occurred on banks 12-15.

Table 51: **conflict_stats4**

Metric Name	Description
Bank 12 Conflict Count	The number of memory bank 12 conflicts.
Bank 13 Conflict Count	The number of memory bank 13 conflicts.
Bank 14 Conflict Count	The number of memory bank 14 conflicts.
Bank 15 Conflict Count	The number of memory bank 15 conflicts.

Profiling for Interface Tiles

The following tables list the pre-defined metric set configurations available for interface tiles. In the `xrt.ini` file all of these metric names should be in lower case and assigned to the metric selector `tile_based_interface_tile_metrics` or `graph_based_interface_tile_metrics`.

All these metric sets are targeted to a single channel in each PLIO which is optionally specified right after the metric set. For example, `tile_based_interface_tile_metrics=all:input_throughputs:2` will profile all the channels numbered 2 which are used in all the input PLIOs across the AI Engine interface. If not specified in the parameter list, the selected channel will be the latest appearing in the list of used channels generated by the `aiecompiler`.

Table 52: **input_throughputs (mm2s_throughputs)**

Metric Name	Description
Stalled Time/%	The time spent in stalled state (for example, master is ready, slave is stalled).
Transfer Time/%	The time spent transferring data.
Idle Time/%	The time spent in idle state (for example, master is idle).
Throughput (MB/s)	Throughput used by master/slave (MB/s).

Table 53: **output_throughputs (s2mm_throughputs)**

Metric Name	Description
Stalled Time/%	The time spent in stalled state (for example, master is ready, slave is stalled).
Transfer Time/%	The time spent in transferring data.
Idle Time/%	The time spent in idle state (for example, master is idle).
Throughput (MB/s)	Throughput used by master/slave (MB/s).

Table 54: **input_stalls (mm2s_stalls) for AI Engine**

Metric Name	Description
DMA MM2S Channel 0 Stall Time/%	DMA MM2S Channel 0 Stall Time/%
DMA MM2S Channel 0 Idle Time/%	DMA MM2S Channel 0 Idle Time/%

Table 55: output_stalls (s2mm_stalls) for AI Engine

Metric Name	Description
DMA S2MM Channel 0 Stall Time/%	DMA S2MM Channel 0 Stall Time/%
DMA S2MM Channel 0 Idle Time/%	DMA S2MM Channel 0 Idle Time/%

Table 56: input_stalls (mm2s_stalls) for AI Engine-ML

Metric Name	Description
DMA MM2S Channel 0 Stall Time/%	DMA MM2S Channel 0 Stall Time/%
DMA MM2S Channel 0 Idle Time/%	DMA MM2S Channel 0 Idle Time/%
DMA MM2S Channel 0 Stream Backpressure Time/%	DMA MM2S Channel 0 Stream Backpressure Time/%
DMA MM2S Channel 1 Stream Backpressure Time/%	DMA MM2S Channel 1 Stream Backpressure Time/%
DMA MM2S Channel 0 Memory Starvation Time/%	DMA MM2S Channel 0 Memory Starvation Time/%
DMA MM2S Channel 1 Memory Starvation Time/%	DMA MM2S Channel 1 Memory Starvation Time/%

Table 57: output_stalls (s2mm_stalls) for AI Engine-ML

Metric Name	Description
DMA S2MM Channel 0 Stall Time/%	DMA S2MM Channel 0 Stall Time/%
DMA S2MM Channel 0 Idle Time/%	DMA S2MM Channel 0 Idle Time/%
DMA S2MM Channel 0 Memory Backpressure Time/%	DMA S2MM Channel 0 Memory Backpressure Time/%
DMA S2MM Channel 1 Memory Backpressure Time/%	DMA S2MM Channel 1 Memory Backpressure Time/%
DMA S2MM Channel 0 Lock Stall Time/%	DMA S2MM Channel 0 Lock Stall Time/%
DMA S2MM Channel 1 Lock Stall Time/%	DMA S2MM Channel 1 Lock Stall Time/%

Table 58: packets

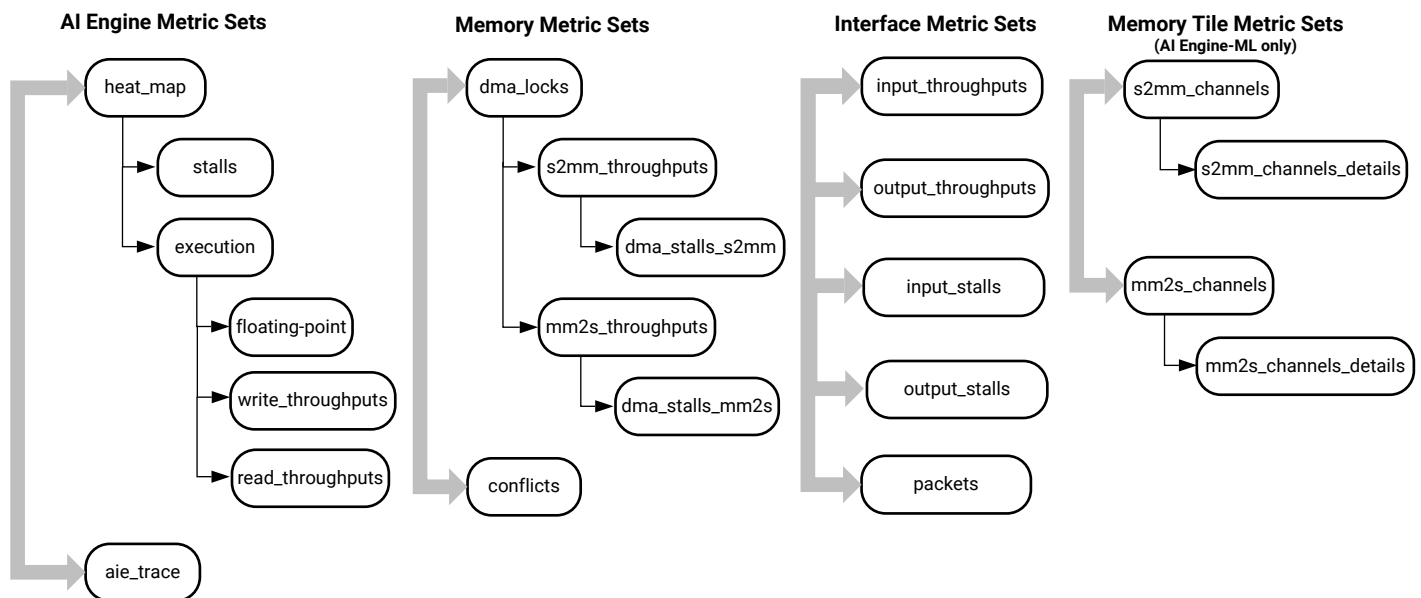
Metric Name	Description
TLAST event count on port 0	Reports the number of input packets
TLAST event count on port 1	Reports the number of output packets

These values are referring to a specific input channel of all the PLIOs (PL to AI Engine) among the 8 available (0 to 7).

Metric Sets

The following figure shows metric sets for the AI Engine (core) module, the memory module, and the interface module. These metric sets are generally mutually exclusive. A metric from one module type (for example, AI Engine) can be used in a combination with a metric from another module type (for example, memory), based on the requirements of your design. One exception is the use of *mm2s_throughputs* and *s2mm_throughputs* in which the same metric is used for both AI Engine and Memory module. Initially, AMD recommends that you use the *heat_map* metric for the AI Engine module and *conflicts* metric for memory modules. These settings are specified at run-time and therefore, you can run them as many times as you would like with your preferences. However, some of these metrics sets are interconnected because some use group events and others use individual events. For example, the *heat_map* metric set contains a metric that groups all kinds of stall events in a single metric along with other metrics that group data transfer events (load/store, streams, cascade, etc.) and vector instructions. To get a better view of which stall type(s) are prevalent, re-run with the *stalls* metric set. To better understand execution, re-run with the *execution* metric set.

Figure 53: Metric Sets



X26534-052224

Profiling Flows

The two different flows for configuring and capturing AI Engine profile counters are as follows:

- **XSDB Flow:** This flow provides a low-overhead method of communicating with the board, regardless of the operating system used on the host.
- **XRT Flow:** This flow is seamlessly integrated into Xilinx Runtime (XRT).

XSDB Flow

Set up `xsdb` as described in the following steps to connect to the device hardware.

1. When running the application, the profile data is captured in counters that can be retrieved by the debugging and profiling IP. To capture and evaluate this data, you must connect to the hardware device using `xsdb`. This command is typically used to program the device and debug applications. Connect your system to the hardware platform or device over JTAG, launch the `xsdb` command in a command shell, and run the following sequence of commands:

```
xsdb% connect
xsdb% ta 1
xsdb% source $::env(XILINX_VITIS)/scripts/vitis/util/aie_profile.tcl
xsdb% aieprofile start -graphs myGraph -work-dir ./Work \
-graph-based-aie-metrics "dut:kernell1:heat_map" \
-tile-based-aie-metrics "all:stalls" \
-graph-based-aie-memory-metrics "dut:all:write_throughputs" \
-tile-based-aie-memory-metrics "{4,1}:{6,2}:conflicts;
{8,3}:dma_locks" \
-tile-based-interface-tile-metrics "2:10:input_throughputs:3" \
-interval 20 -samples 100
```

where:

- `connect`: Launches the `hw_server` and connects `xsdb` to the device.
 - `source $::env(XILINX_VITIS)/scripts/vitis/util/aie_profile.tcl`: Sources the Tcl trace command to set up the `xsdb` environment.
2. Run the design on hardware to generate hardware profile data. The XSDB `aieprofile` script should still be running and sampling while the design application runs and until application finishes in order to capture valid statistics. The `-samples` and `-interval` switches can be adjusted accordingly to ensure this.
 3. Use `vitis_analyzer` to import and analyze the data, as described [Viewing Profile Results in the Vitis IDE](#).

aieprofile Command Synopsis

```
aieprofile start [options] -tiles <tile-list>
```

```
aieprofile start [options] -graphs <graph-list>
```

Table 59: aieprofile Options

Option Name	Description
<code>-config-file <json-file></code>	The JSON file generated by the AI Engine compiler at <code>Work/ps/c_rts/aie_trace_config.json</code> with profile configuration data. This file can also be created for custom profile configuration. You can alternatively specify the <code>-work-dir</code> option instead of <code>config-file</code> .
<code>start</code>	AI Engine profiling starts. Instructs the DPA IP to begin capturing profile data based on configuration.

Table 59: aieprofile Options (cont'd)

Option Name	Description
-graphs <graph-list>	This option allows you to set one or more graphs used in the AI Engine application. If this option is not specified, profiling is enabled for all available graphs.
-tiles <tile-list>	This option allows you to select one or more tiles to profile. Tiles can be in the format of col, row (for example, 0,1 or 2,5). All the tiles in a row or column can be specified using wildcard '*' (for example, 0,* to select all tiles in column 0). <pre># This example profiles tile (10,2) and all tiles in column 5 xsdb% aieprofile start [options] -tiles 10,2;5,*</pre> Note: The column index starts at 0. The row index starts at 1 and not at 0.
-work-dir <dir-path>	This option lets you set project Work directory. This is a mandatory option if config-file is not specified.
-interval	This option lets you set the number of sample interval in milliseconds (default 20).
-samples	This option lets you set the exact number of counter samples (default 100) used in profiling the application.
-graph-based-aie-metrics <graph name all>:<kernel name all>:<metric_set>	This option allows you to set the AI Engine counter metrics set for specified graphs and kernels. Valid values are heat_map, stalls, execution, floating-point, stream_put_get, s2mm_throughputs, mm2s_throughputs and aie_trace. Note: You can specify multi-level hierarchy graphs, as follows: -graph-based-aie-metrics <graph.subgraph1.subgraph2/all>:<kernel_name all>:<metric_set>
-tile-based-aie-metrics <{<column>,<row>} all>:<metric_set>	Sets the AI Engine counter metrics set for specified tiles. Valid metrics as above.
-tile-based-aie-metrics {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}>:<metric_set>	Sets the AI Engine counter metrics set for specified tiles in a range. Valid metrics as above.
-graph-based-aie-memory-metrics <graph name all>:<kernel name all>:<metric_set>	This option allows you to set the AI Engine memory module counter metrics set for specified graphs and kernels. Valid values are conflicts, dma_locks, dma_stalls_s2mm, dma_stalls_mm2s, s2mm_throughputs and mm2s_throughputs.
-tile-based-aie-memory-metrics = <{<column>,<row>} all>:<metric_set>	This option allows you to set the AI Engine memory module counter metrics set for specified tiles. Valid metrics as above.
-tile-based-aie-memory-metrics = {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}>:<metric_set>	Sets the AI Engine memory module counter metrics set for specified tiles in a range. Valid metrics as above.
-graph-based-interface-tile-metrics = <graph name all>:<port name all>:<metric_set>	This options allows you to set the AI Engine Array Interface counter metrics set for specified graph port name. Valid metrics are input_throughputs (mm2s_throughputs), output_throughputs (s2mm_throughputs), input_stalls (mm2s_stalls), output_stalls (s2mm_stalls), packets.

Table 59: aieprofile Options (cont'd)

Option Name	Description
<code>-tile-based-aie-metrics <column all>:<metric_set></code>	This option allows you to set the AI Engine interface tile counter metrics set for specified tiles. Valid metrics as above.
<code>-tile-based-aie-metrics <mincolumn>:<maxcolumn>:<metric_set></code>	This option allows you to set the AI Engine interface tile counter metrics set for specified tiles. Valid metrics as above.
AIE-ML Specific Options	
<code>-graph-based-memory-tile-metrics "<graph name all>:<buffer name all>:<metric_set>[:<channel>]"</code>	This option configures the Memory Tile metric to apply to one or more buffers in one or more graphs for a specified channel ID. If the channel ID is omitted, a default channel will be taken. Valid metrics are <code>input_channels</code> (<code>s2mm_channels</code>), <code>output_channels</code> (<code>mm2s_channels</code>), <code>input_channels_details</code> (<code>s2mm_channels_details</code>), <code>output_channels_details</code> (<code>mm2s_channels_details</code>), <code>memory_stats</code> , <code>input_throughputs</code> (<code>s2mm_throughputs</code>), <code>output_throughputs</code> (<code>mm2s_throughputs</code>), <code>conflict_stats1</code> to <code>conflict_stats4</code> .
<code>-tile-based-memory-tile-metrics "<column all>:<metric set>[:<channel>]"</code>	This option configures the Memory Tile metric to apply to one or more columns for a specified channel ID. If the channel ID is omitted, a default channel will be taken. Valid metrics as above.
<code>-tile-based-memory-tile-metrics "<mincolumn>:<maxcolumn>:<metric set>[:<channel>]"</code>	This option configures the Memory Tile metric to apply for a range of columns for a specified channel ID. If the channel ID is omitted, a default channel will be taken. Valid metrics as above.

XRT Flow

1. Burn the generated `sd_card.img` to the physical SD card.
2. Create the `xrt.ini` file in the same location where the PS host application is located. An example of `xrt.ini` file is as follows:

```
[Debug]
#
# Profile Counters
#
aie_profile = true

[AIE_profile_settings]
# Sample interval (in usec)
interval_us = 100
# All tiles
tile_based_aie_metrics = all:heat_map
tile_based_aie_memory_metrics = all:conflicts
tile_based_interface_tile_metrics = all:s2mm_throughputs:0
```

where:

- `[Debug]`: Specifies debug section for XRT, this is case sensitive.
- `aie_profile`: Enables profile configuration.
- `[aie_profile_settings]`: Specifies profile settings for XRT.

- `aie_profile_interval_us`: Profiles data collection interval in micro seconds.
 - `tile_based_aie_metrics`: Configures metric to be applied to the AI Engine on a tile basis.
 - `tile_based_aie_memory_metrics`: Configures memory metric to be applied on a tile basis.
 - `tile_based_interface_tile_metrics`: Configures interface metric to be applied on a tile basis.
3. Run the design on hardware to capture profile data.
 4. Copy the generated profile files, `aie_profile_*.csv`, `summary.csv`, and `xrt.run_summary`, from the SD card to your design at the same level as the `Work` directory. These generated files are at the same location as the host application on the SD card.
 5. Use the Vitis IDE to import and analyze the data as described in [Viewing Profile Results in the Vitis IDE](#).

xrt.ini Specification

```
[Debug]
#
# Profile Counters
#
aie_profile = true

# Subsection for AIE profile settings
[AIE_profile_settings]
# Interval in between reading counters (in us)
interval_us = 1000

# Graph/Kernel name
graph_based_aie_metrics = <graph name|all>:<kernel name|all>:<off|heat_map|
stalls|execution|floating_point|s2mm_throughputs|mm2s_throughputs|
aie_trace>
graph_based_aie_memory_metrics = <graph name|all>:<kernel name|all>:<off|
conflicts|dma_locks|dma_stalls_s2mm|dma_stalls_mm2s|s2mm_throughputs|
mm2s_throughputs>

# AI Engine Core Metrics : Configuration can be used only once : Multiple
values can be specified on a single line separated with ';'
# Single or all tiles
tile_based_aie_metrics = <{<column>,<row>}|all>:<off|heat_map|stalls|
execution|floating_point|mm2s_throughputs|s2mm_throughputs|aie_trace>
# Range of tiles
tile_based_aie_metrics = {<mincolumn>,<minrow>}:{<maxcolumn>,<maxrow>}:<off|
heat_map|stalls|execution|floating_point|mm2s_throughputs|s2mm_throughputs|
aie_trace>

# AI Engine Memory Metrics : Configuration can be used only once : Multiple
values can be specified on a single line separated with ';'
# Single or all tiles
tile_based_aie_memory_metrics = <{<column>,<row>}|all>:<off|conflicts|
dma_locks|dma_stalls_s2mm|dma_stalls_mm2s|s2mm_throughputs|mm2s_throughputs>
# Range of tiles
tile_based_aie_memory_metrics = {<mincolumn>,<minrow>}:<
```

```

{<maxcolumn>,<maxrow>}:<off|conflicts|dma_locks|dma_stalls_s2mm|
dma_stalls_mm2s|s2mm_throughputs|mm2s_throughputs>

# Interface Tiles
#Graph port name
graph_based_interface_tile_metrics = <graph name|all>:<port name|all>:<off|
s2mm_throughputs|mm2s_throughputs|packets>
# Single or all columns
tile_based_interface_tile_metrics = <column|all>:<off|s2mm_throughputs|
mm2s_throughputs|packets>[:<channel>]
# Range of columns
tile_based_interface_tile_metrics = <mincolumn>:<maxcolumn>:<off|
s2mm_throughputs|mm2s_throughputs|packets>[:<channel>]

# Memory Tile Metrics : Configuration can be used only once : Multiple
values can be specified on a single line separated with ';'
# AIE-ML only
#     Graph/kernel based
graph_based_memory_tile_metrics = <graph name|all>:<buffer name|all>:<off|
s2mm_channels|s2mm_channels_details|mm2s_channels|mm2s_channels_details|
memory_stats|s2mm_throughputs|mm2s_throughputs|conflict_stats1|...|
conflict_stats4>[:<channel>]
#     Single or all columns
tile_based_memory_tile_metrics = <column|all>:<off|s2mm_channels|
s2mm_channels_details|mm2s_channels|mm2s_channels_details|memory_stats|
s2mm_throughputs|mm2s_throughputs|conflict_stats1|...|
conflict_stats4>[:<channel>]
#     Range of columns
tile_based_memory_tile_metrics = <mincolumn>:<maxcolumn>:<off|s2mm_channels|
s2mm_channels_details|mm2s_channels|mm2s_channels_details|memory_stats|
s2mm_throughputs|mm2s_throughputs|conflict_stats1|...|
conflict_stats4>[:<channel>]

```

Table 60: XRT Profiling Options

Option Name	Description
[Debug]	This option specifies debug section for XRT, this is case sensitive.
aie_profile	Enables profile configuration.
[AIE_profile_settings]	This option specifies profile settings for XRT, this is case sensitive.
interval_us	This option lets you set the number of sample interval in micro-seconds (default 20).
graph_based_aie_metrics = <graph name all>:<kernel name all>:<metric_set>	This option configures the AI Engine metric to be applied for all or a specific kernel in all or a specific graph. The metric is applied to the tile, even if the kernel is not alone in it.
graph_based_aie_memory_metrics = <graph name all>:<kernel name all>:<metric_set>	This option configures memory metric to be applied for all or a specific kernel in all or a specific graph. The metric is applied to the tile, even if the kernel is not alone in it.
graph_based_interface_tile_metrics = <graph name all>:<port name all>:<metric_set>	This option configures the AI Engine Array Interface counter metrics set for specified graph port name.
tile_based_aie_metrics = <{<column>,<row>} all>:<metric_set>	This option configures the AI Engine metric to be applied for a single tile or all tiles.
tile_based_aie_metrics = {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}>:<metric_set>	This option configures the AI Engine metric to be applied for all tiles in a range.

Table 60: XRT Profiling Options (cont'd)

Option Name	Description
<code>tile_based_aie_memory_metrics = <{<column>,<row>} all>:<metric_set></code>	This option configures the AI Engine memory metric to be applied for a single tile or all tiles.
<code>tile_based_aie_memory_metrics = {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}:<metric_set></code>	This option configures the AI Engine memory metric to be applied for all tiles in a range.
<code>tile_based_interface_tile_metrics = <column all>:<metric_set>[:<channel>]</code>	This option configures AI Engine Interface metric to be applied for a single tile or all tiles on a specified channel ID. If the channel ID is omitted, a default channel will be taken.
<code>tile_based_interface_tile_metrics = <mincolumn>:<maxcolumn>:<metric_set>[:<channel>]</code>	This option configures AI Engine Interface metric to be applied for all tiles on a range for a specified channel ID. If the channel ID is omitted, a default channel will be taken.
AI Engine-ML Only	
<code>graph_based_memory_tile_metrics = <graph name all>:<buffer name all>:<metric_set>[:<channels>]</code>	This option configures the Memory Tile metric to be applied for all or a specific buffer in all or a specific graph for a specified channel ID. If the channel ID is omitted, a default channel will be taken.
<code>tile_based_memory_tile_metrics = <column all>:<metric_set>[:<channels>]</code>	This option configures the Memory Tile metric to be applied for a single column or all columns for a specified channel ID. If the channel ID is omitted, a default channel will be taken.
<code>tile_based_memory_tile_metrics = <mincolumn>:<maxcolumn>:<metric_set>[:<channels>]</code>	This option configures the Memory Tile metric to be applied for a range of columns for a specified channel ID. If the channel ID is omitted, a default channel will be taken.

xrt.ini Examples

In the following example, no tile or graph selection is done before extracting profiling information. All used tiles will be profiled with default metric sets:

```
[AIE_profile_settings]
tile_based_aie_metrics = all:heat_map
tile_based_aie_memory_metrics = all:conflicts
tile_based_interface_tile_metrics = s2mm-throughputs:0
```

In this example, bounding box and tile selection is demonstrated:

```
[AIE_profile_settings]
tile_based_aie_metrics = {4,1}:{6,2}:stalls; {10,4}:execution
tile_based_aie_memory_metrics = {4,1}:dma_locks
```

In this example the kernel selection is based on graph names. Multiple selections must be made on a single line separated by a semicolon:

```
[AIE_profile_settings]
graph_based_aie_metrics = tx_chain_0:all:execution;
tx_chain_1:all:floating_point
graph_based_aie_memory_metrics = tx_chain_2:all:s2mm-throughputs
```

This example demonstrates how to profile multi-level hierarchical graphs:

```
[AIE_profile_settings]
graph-based_aie_metrics = dut.dlbf0.bf1:all:execution
```

Note: If the total number of performance counters used exceeds the number of available performance counters, the API will not acquire a performance counter and fail with the following error message displayed on console.

```
[AIE ERROR]: Failed to request resource 0
[AIE WARNING]: Unable to request resources. RscType: 0
XAIEFAL: WARN: perfcount _reserve (6,1) Expect Mod= 1 resource not
available.
...
```

Note: If different metric sets must be applied to various kernel/tile selections, the metric sets must be concatenated in a single line separated by semicolons.

Viewing Profile Results in the Vitis IDE

To view the profile results obtained using the XRT flow, use the following command:

```
vitis -a xrt.run_summary
```

To launch the Vitis IDE to view the profiling information in the XSDB flow, use the following command.

```
vitis -a aie_trace_profile.run_summary
```

Performance Annotation in Graph View

Depending on what is profiled in the hardware, the performance data can be contained in specific tables in the graph view. By hovering the mouse over the corresponding object, the performance is displayed.

Following example shows how output throughput is measured in the hardware and where is it shown in the graph view.

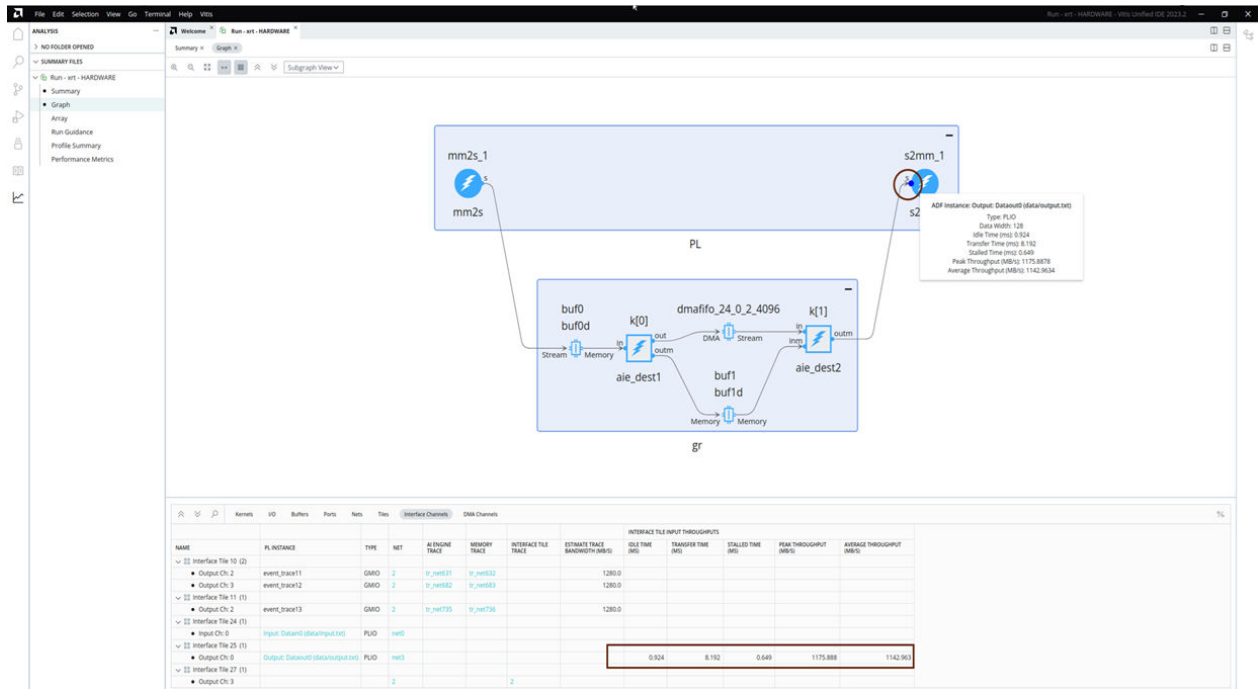
1. Add the content to `xrt.ini`:

```
[Debug]
aie_profile=true
[AIE_profile_settings]
tile-based-interface_tile_metrics=25:25:output_throughputs
```

2. And then run application to get profiling result, and view it in the Vitis IDE.

The performance data is contained in `Interface Channels` table. By hovering the mouse on the profiled port, the performance data is displayed, as shown in the following figure.

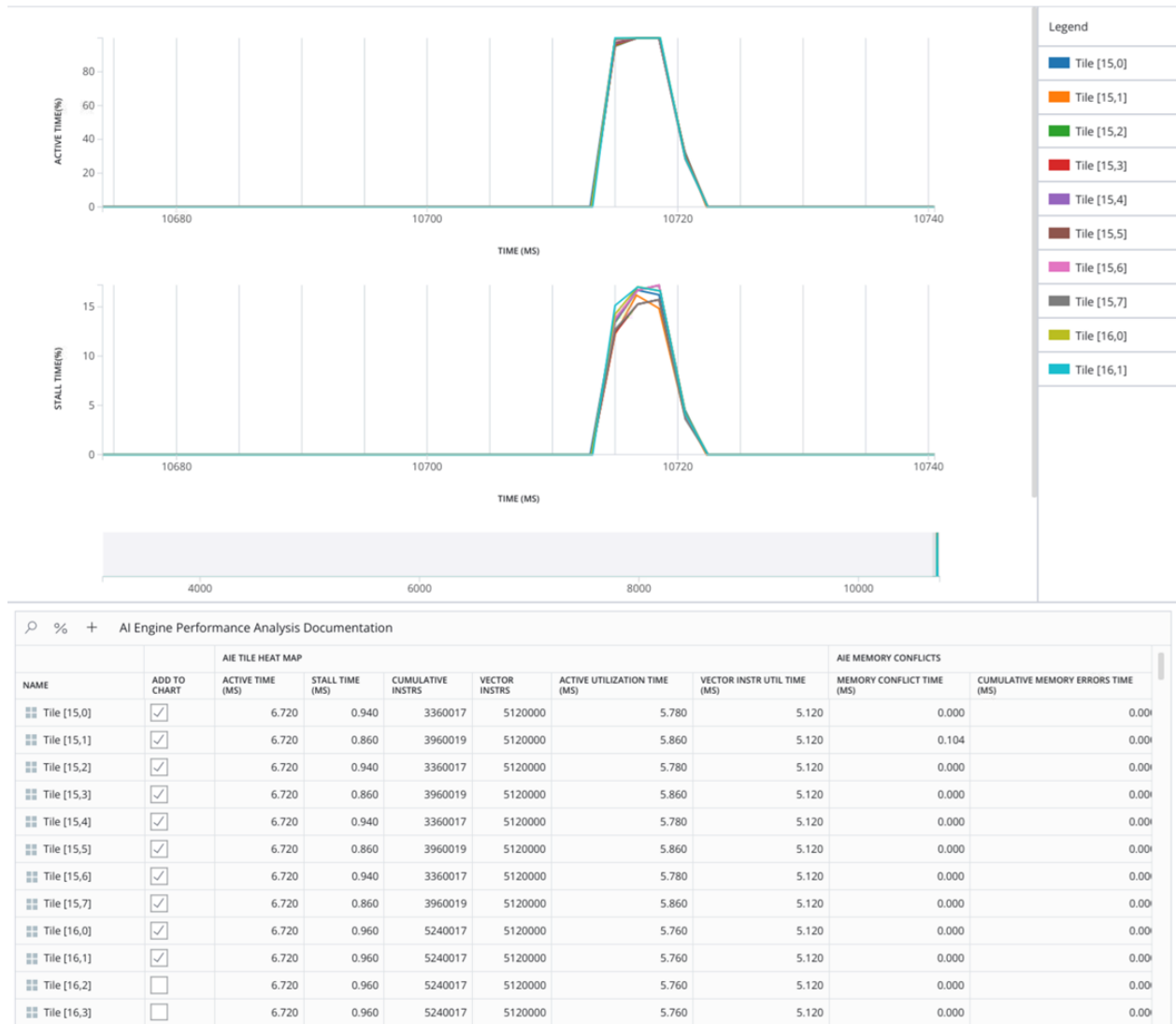
Figure 54: Performance Annotation on Output Throughput



Example of heat_map Core Metrics and conflicts Memory Metrics

The following image shows the design's active time, stall time, cumulative instruction count, and vector instruction count as part of `heat_map` metric and memory conflict time, as well as cumulative memory error time of `conflicts` metrics for ten tiles of an example design.

Figure 55: Example of heat_map and conflicts Metrics

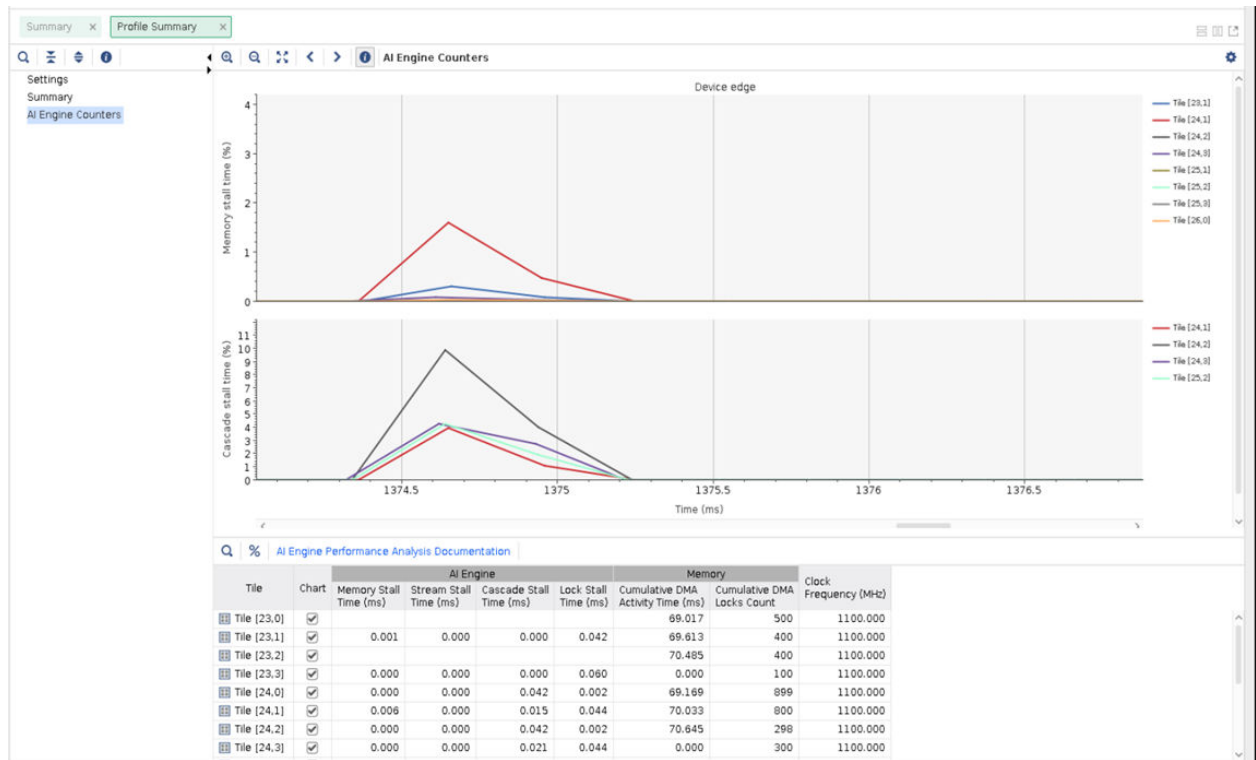


Consider the AI Engine located in (15,0). During the active utilization time (5.120 ms) it performs 5120000 vector instructions which represents 87% of the active time. This is an excellent performance that indicates a well optimized core.

Example of stalls Core Metrics and dma_locks Memory Metrics

The following image shows the design's memory stall time, stream stall time, cascade stall time, and lock stall time as part of `stalls` metrics and cumulative DMA activity time, as well as cumulative DMA locks count of `dma_locks` metrics for ten tiles of an example design.

Figure 56: Example of stalls and dma_locks Metrics



On the core (24,2), the DMA has been active for 70.645 ms (77.8 millions instructions), but has been stalled 298 times.

Example of execution Core Metrics and conflicts Memory Metrics

The following image shows the design's cumulative instruction count, vector instruction count, load instruction count, and store instruction count as part of `execution` metrics and memory conflict time, as well as cumulative memory error time of `conflicts` metrics for ten tiles of an example design.

Figure 57: Example of execution and conflicts Metrics



Although they are minor, core (15,1) suffers from some memory conflicts that must be identified. The occurrence being very small might be due to some DMA or some other kernel access interference.

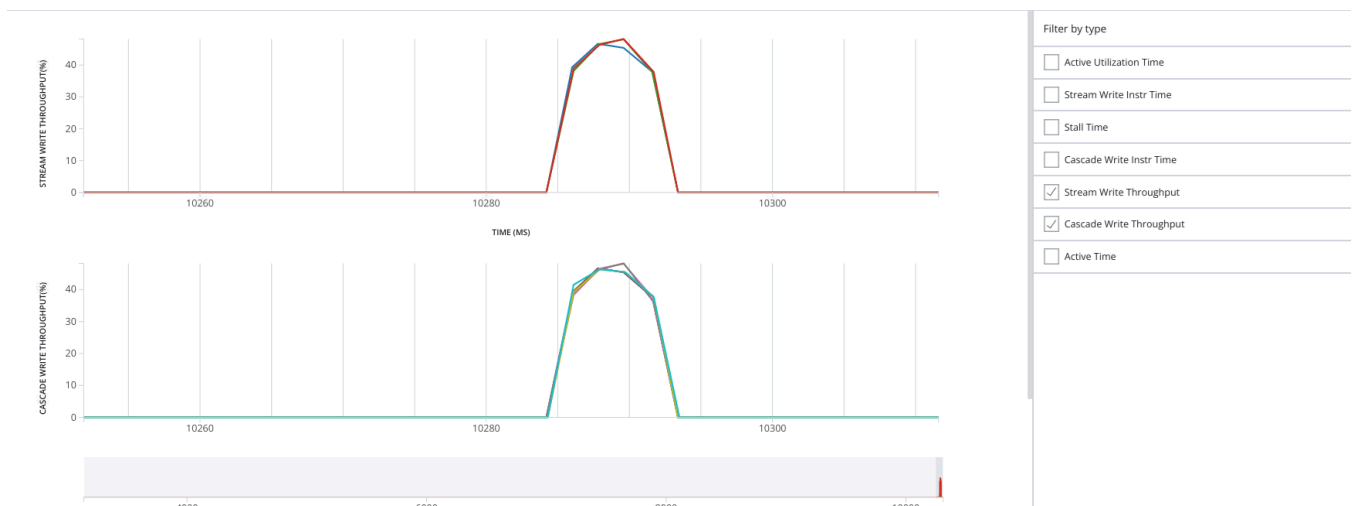
Example of read_throughputs and write_throughputs AI Engine Metrics and dma_stalls_s2mm and dma_stalls_mm2s AI Engine Memory Metrics

The following image shows the design's stream and cascade read and write instruction count as part of read_throughputs and write_throughputs metrics and s2mm and mm2s channel0 and channel1 stalls time of dma_stalls_s2mm and dma_stalls_mm2s metrics for ten tiles of an example design.

Figure 58: Data Table for read and write throughputs of the AI Engines and stalls of all mm2s and s2mm Channels of the AI Engine Memories

AI Engine Performance Analysis Documentation												
NAME	ADD TO CHART	AIE TILE HEAT MAP			AIE TILE STREAM PUT GET		AIE TILE WRITE THROUGHPUTS		AIE MEMORY WRITE THROUGHPUTS			
		ACTIVE TIME (MS)	STALL TIME (MS)	ACTIVE UTILIZATION TIME (MS)	STREAM WRITE INSTRS	CASCADE WRITE INSTRS	STREAM WRITE THROUGHPUT (MB/S)	CASCADE WRITE THROUGHPUT (MB/S)	DMA S2MM CH 0 PACKETS	DMA S2MM CH 1 PACKETS	DMA S2MM CH 0 THROUGHPUT (MB/S)	DMA S2MM CH 1 THROUGHPUT (MB/S)
Tile [15,0]	☒	6.720	0.940	5.780	0	2560000	0.000	22412.041	0	0	0.000	0.000
Tile [15,1]	☒	6.720	0.860	5.860	2560000	0	1867.048	0.000	0	0	0.000	0.000
Tile [15,2]	☒	6.720	0.940	5.780	0	2560000	0.000	22388.650	0	0	0.000	0.000
Tile [15,3]	☒	6.720	0.860	5.860	2560000	0	1923.697	0.000	0	0	0.000	0.000
Tile [15,4]	☒	6.720	0.940	5.780	0	2560000	0.000	23087.891	0	0	0.000	0.000
Tile [15,5]	☒	6.720	0.860	5.860	2560000	0	1924.144	0.000	0	0	0.000	0.000
Tile [15,6]	☒	6.720	0.940	5.780	0	2560000	0.000	23142.354	0	0	0.000	0.000
Tile [15,7]	☒	6.720	0.860	5.860	2560000	0	1927.819	0.000	0	0	0.000	0.000
Tile [16,0]	☒	6.720	0.960	5.760	0	2560000	0.000	22232.967	0	0	0.000	0.000
Tile [16,1]	☒	6.720	0.960	5.760	0	2560000	0.000	22234.406	0	0	0.000	0.000
Tile [16,2]	☐	6.720	0.960	5.760	0	2560000	0.000	22229.607	0	0	0.000	0.000
Tile [16,3]	☐	6.720	0.960	5.760	0	2560000	0.000	22224.996	0	0	0.000	0.000
Tile [16,4]	☐	6.720	0.960	5.760	0	2560000	0.000	22226.303	0	0	0.000	0.000

Figure 59: Charts for Cascade Read and Stream read Instruction Time in Percent

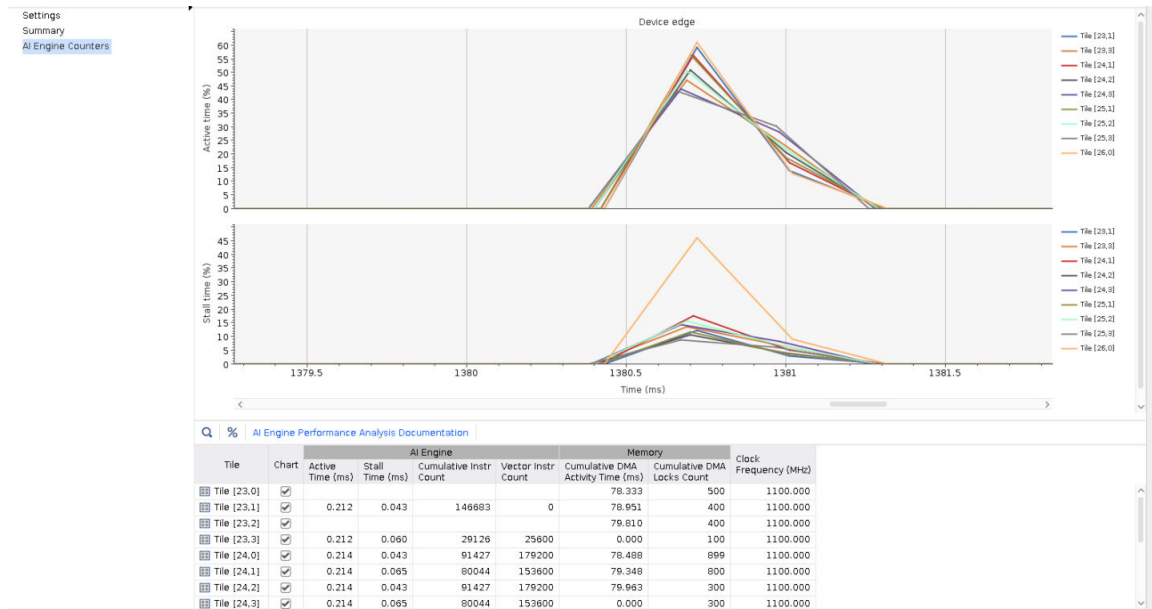


In the figure above, there is a cascade write and a stream write more than 45% of the time in the AI Engine kernels. This is necessary to keep the AI Engine active because the stream throughputs is much less than the memory throughput.

Example of heat_map Core Metrics and dma_locks Memory Metrics

The following image shows the design's active time, stall time, cumulative instruction count and vector_instruction_count as part of `heat_map` metrics and cumulative DMA activity time, as well as cumulative DMA locks count of `dma_lock` metrics for ten tiles of an example design.

Figure 60: Example of heat_map and dma_locks Metrics

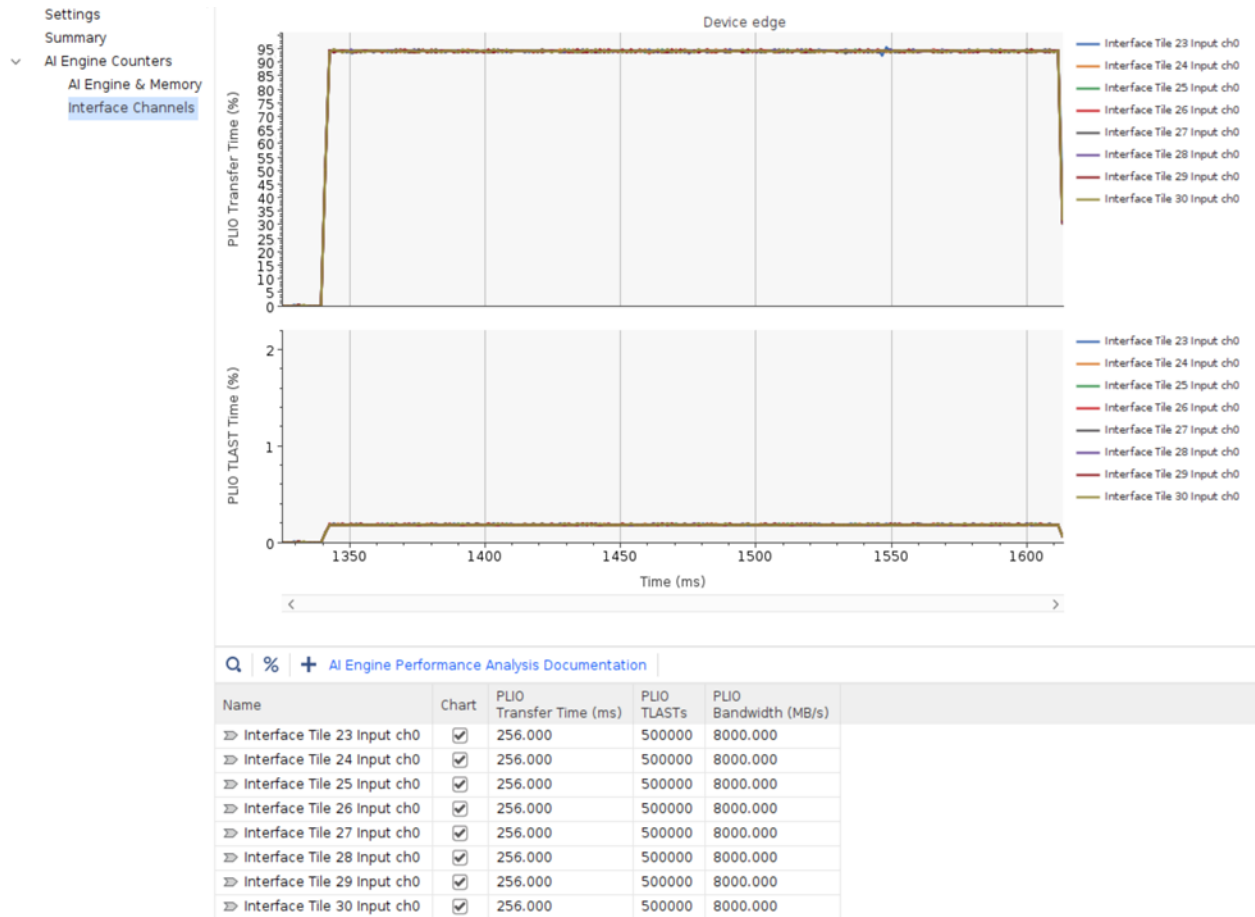


The cumulative DMA Activity time jointly with the Cumulative DMA Locks count allows you to see if there is any discrepancy between lock acquisition number and the number of data transferred through the DMAs. The relative number of locks count can also be used to interpret the relative number of iterations of each core.

Example of input_throughputs Interface Metrics

The following image shows the design's input throughput at the PLIO level as part of `input_throughputs:0` metric in a 8 x 8 cascaded tiles design.

Figure 61: Example of input_throughputs:0 Interface Metrics



In this graph, the channel 0 throughput for all input PLIOs is approximately 95% which is close to the achievable maximum. After this profiling step, verify that the AI Engines are not starving for data.

Report Consolidation in the Vitis IDE

During the profiling stage, not all metrics can be used at the same time during runtime. You can run the design in hardware multiple times by rebooting the board each run using different profile metric sets in `xrt.ini`. Typically, for AI Engine interface throughput profiling, a single channel (the same for all PLIOs) can be profiled during runtime. Multiple channel profiling will necessitate multiple runs.

In the Vitis IDE, you can consolidate multiple reports from different runs of the same design. That enables you to display the throughput of multiple interface channels, for example. While `vitis -a` is run with the `xrt.run-summary` of a specific run of the design, other `xrt.run-summary` reports can be opened by clicking the **+** toolbar button in the main toolbar and a window toolbar, as shown below.

Figure 62: The Add (+) Button in the Main Toolbar

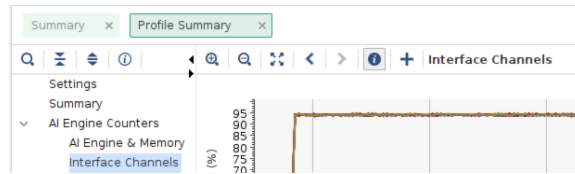
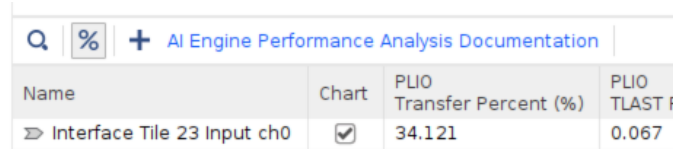


Figure 63: The Add (+) Button in a Window Toolbar



After consolidating the profiling data for input PLIOs channels 0 and 4, and output PLIOs channel 0, the Vitis IDE can display the following table:

Figure 64: Channel 0 and 4 Input and Channel 0 Output PLIO Throughput

Name	Chart	PLIO Transfer Time (ms)	PLIO TLASTs	PLIO Bandwidth (MB/s)
➤ Interface Tile 23 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 23 Input ch4	☐	256.000	500000	7586.198
➤ Interface Tile 24 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 24 Input ch4	☐	256.000	500000	7583.512
➤ Interface Tile 25 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 25 Input ch4	☐	256.000	500000	7586.771
⏪ Interface Tile 25 Output ch0	☐	256.000	0	7604.266
➤ Interface Tile 26 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 26 Input ch4	☐	256.000	500000	7585.827
➤ Interface Tile 27 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 27 Input ch4	☐	256.000	500000	7589.033
➤ Interface Tile 28 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 28 Input ch4	☐	256.000	500000	7593.244
➤ Interface Tile 29 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 29 Input ch4	☒	256.000	500000	7588.190
➤ Interface Tile 30 Input ch0	☒	256.000	500000	8000.000
➤ Interface Tile 30 Input ch4	☒	256.000	500000	7588.719
⏪ Interface Tile 21 Output ch0	☐	256.000	0	7583.766

Event Profile APIs for Graph Inputs and Outputs

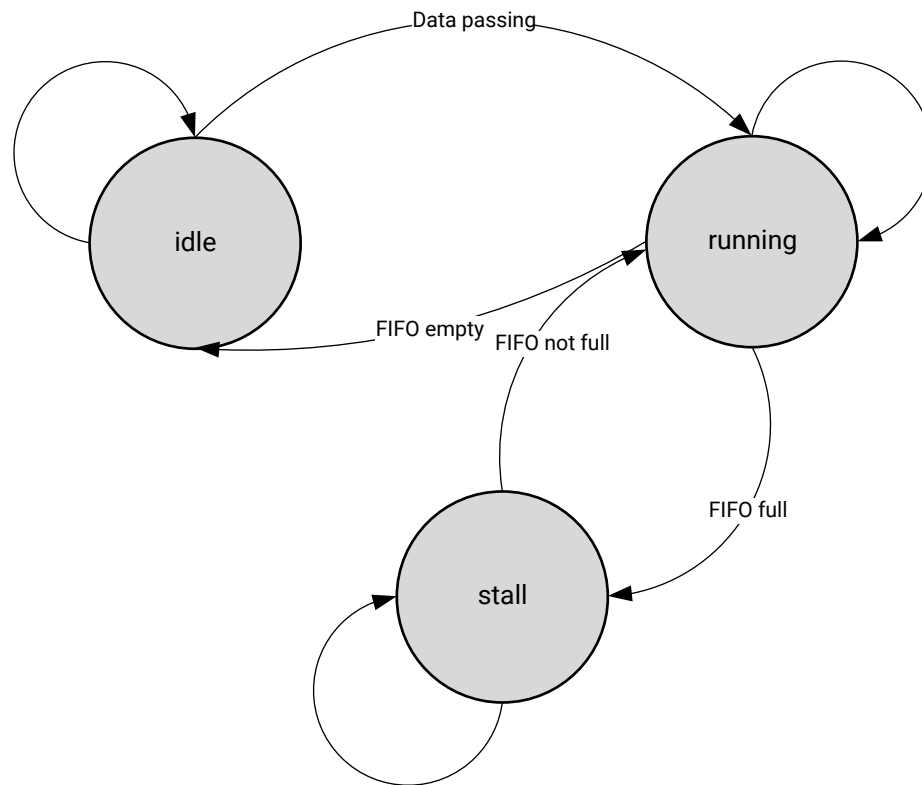
You can collect profile statistics of your design by calling event APIs in your PS host code. These event APIs are available both during simulation and when you run the design in hardware.

Note: The event APIs introduced in this section should be considered incompatible with profiling methods in the previous section. The event APIs are inserted in the host code. The designer is responsible to control when the event starts or how it stops.

The AI Engine has hardware performance counters and can be configured to count hardware events for measuring performance metrics. You can use the event API together with the graph control API to profile certain performance metrics during a controlled period of graph execution. The event API supports only platform I/O ports (PLIO & GMIO) to measure performance metrics such as platform I/O port bandwidth, graph throughput, and graph latency.

The event API tracks the events that occurred on the stream switch of the nets that cross the AI Engine - PL interfaces. The events on the stream switch of the nets include `idle`, `running` and `stall`, as shown in following figure.

Figure 65: Events on Nets

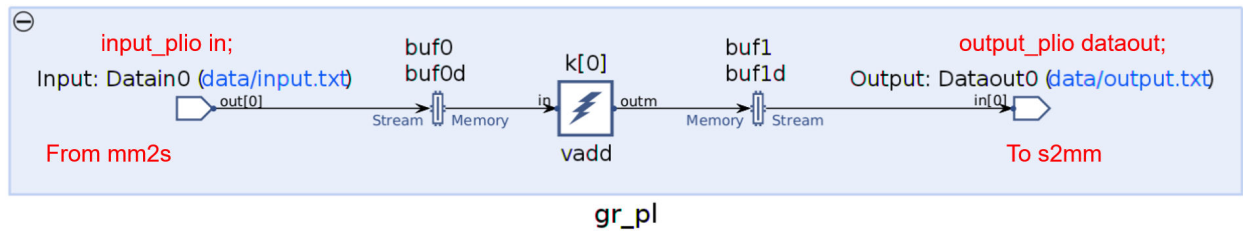


X26641-051122

- When there is no data passing the stream switch, the stream switch is in an `idle` state.
- When there is data passing the stream switch, the stream switch is in a `running` state.
- When all the FIFOs on the net are full, the stream switch is in a `stall` state.
- When the transfer of data resumes, the stream switch returns to a `running` state.

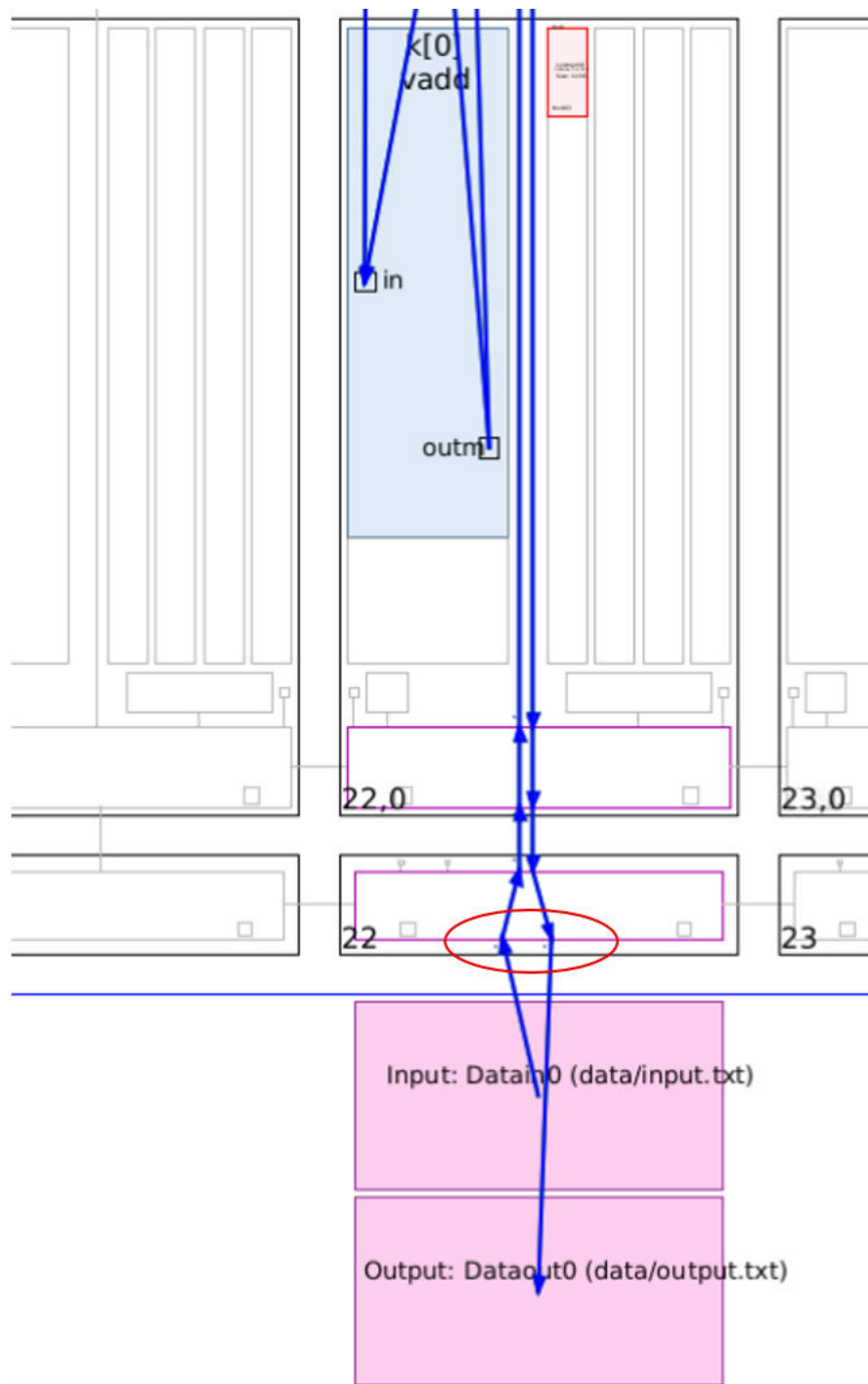
The following graph shows an example of data being sent from the `mm2s` PL kernel to the AI Engine. It also shows the graph sending data from the AI Engine to the `s2mm` PL kernel.

Figure 66: Example Graph



Different ports can go through the same AI Engine - PL interface column, which shares the performance counters in the interface. You can check the Array view in Vitis IDE to see which columns are those ports routing through. The following picture shows an array view of the above example, and note that the stream switches in red circle are where the event API is monitoring.

Figure 67: Example Array View



Note:

- The input buffer `buf0` in the AI Engine is ready to accept data from the `mm2s` PL kernel after the graph has been initialized. As soon as the `mm2s` PL kernel starts, it will sequentially fill up the PING-PONG buffers and FIFOs inside the stream switch connected to the `buf0`. The data transported to and from these buffers do not depend on `graph.run()`.
- Each column of the AI Engine - PL interface has two performance counters. Because there are limited performance counters, `event::stop_profiling()` can be used to release the performance counters.
- There is some overhead when calling the graph and profiling APIs. The profiling results can be read using `event::read_profiling()`. The profiling results can vary if the performance counters are not stopped before `event::read_profiling()`.



RECOMMENDED: For hardware emulation and hardware flows, AMD recommends using XRT-based event APIs. For more information about usage of XRT APIs, see [XRT Support for Event APIs](#).



IMPORTANT! The ADF APIs can also be used in AI Engine simulation, hardware emulation, and hardware flows. In hardware emulation and hardware flows, `adf::registerXRT` is required before using the event profile ADF APIs. For example:

```
#include "adf/adf_api/XRTConfig.h"
.....
auto device = xrt::device(0); //device index=0
auto uuid = device.load_xclbin(xclbinFilename);
auto dhdl = xrtDeviceOpenFromXcl(device);
adf::registerXRT(dhdl, uuid.get());
event::handle handle = event::start_profiling(.....);
```

Profiling Running Event and Graph Throughput

AMD provides `event::io_stream_running_event_count` enumeration to count the running event, which corresponds to the number of samples sent through the net. When the `event::start_profiling()` is run, the performance counter starts. The performance counter increments each time the data sample passes through the AI Engine - PL interface. The value read back by `event::read_profiling()` is the number of samples that have been sent through that interface.

Method Used to Count Samples Sent and Received

This method can be used to count the number of samples sent or received prior to the graph stall. The following example can be used to count the number of samples received from the AI Engine port prior to a graph stall:

```
event::handle handle = event::start_profiling(gr_pl.dataout,
event::io_stream_running_event_count);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
```

```
gr_pl.run(iterations);
sleep(2);//Wait for enough time
long long cycle_count = event::read_profiling(handle);
printf("Sample number: %d\n", cycle_count);
event::stop_profiling(handle);//Performance counter is released and cleared
```

The following example can be used to count the number of samples sent to the AI Engine port prior to a graph stall:

```
event::handle handle = event::start_profiling(gr_pl.in,
event::io_stream_running_event_count);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
auto mm2s_run = mm2s(nullptr, OUTPUT_SIZE_MM2S);//After start profiling,
send data from mm2s
gr_pl.run(iterations);

sleep(2);//Wait for enough time
long long cycle_count = event::read_profiling(handle);
printf("Sample number: %d\n", cycle_count);
event::stop_profiling(handle);//Performance counter is released and cleared
```

Profile Port Throughput

Port throughput is defined as the number of samples transferred in a specific time period. After the graph runs, the following code can be inserted in the host code to measure the port throughput. In order to profile port throughput for your design in steady state, you must ensure the data transfer is in a stable state prior to profiling the port throughput.

```
gr_pl.run(iterations); // The graph can also have been started during
device boot-up
usleep(100); // Wait enough time (here 100us) to be in a steady state IO
activity
int wait_time_us=20000;
event::handle handle = event::start_profiling(gr_pl.dataout,
event::io_stream_running_event_count);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
long long count0 = event::read_profiling(handle);
usleep(wait_time_us);
long long count1 = event::read_profiling(handle);
event::stop_profiling(handle);
long long samples = count1 - count0;
std::cout << "num running samples: " << samples << std::endl;
std::cout << "Throughput: " << (double)samples / wait_time_us << " MSPS "
<< std::endl;
```

AMD recommends that you run the design for many iterations in hardware to ensure accuracy. The accuracy of this method can vary in hardware emulation.

For the AI Engine simulator, this profiling method applies too. You need to replace `usleep` with the `wait` function in SystemC, and the wait time can be smaller depending on the simulation time, because it is running slower in simulation. For example, the `usleep` function in the preceding code can be replaced with following function call for the AI Engine simulator, with reduced simulation time.

```
wait(20, SC_US);
```

Profiling Graph Throughput

Graph throughput can be defined as the average number of bytes produced (or consumed) per second. The `event::io_stream_start_to_bytes_transferred_cycles` enumeration can be used to record the number of cycles taken to transfer a certain amount of data.

After `event::start_profiling()`, two performance counters `performance counter 0` and `performance counter 1` work together. `performance counter 0` starts incrementing a counter after it receives the first data. `performance counter 1` increments after it receives data. When `performance counter 1` equals the amount of data specified in `event::start_profiling`, it generates an event that notifies `performance counter 0` to stop. The value read back by `event::read_profiling()` is the `performance counter 0` value. After `performance counter 0` stops, the value of the counter represents the number of cycles taken to transfer the data.

If the specified amount of data in `event::start_profiling` has not been transferred, `performance counter 0` does not stop. After the specified amount of data has been transferred, the `performance counter 0` will stop. This technique is useful when you want to profile the time taken to transfer a known amount of data. However, if additional data is transferred, the `performance counter 0` continues counting.



WARNING! For any method to profile graph throughput, the overhead of the graph latency or graph and kernel API calls might not be negligible if the iteration number is too small. It is recommended to run a large number of iterations to minimize the impact of such overhead.

Profile Graph Throughput Using the Graph Output

The following example shows how to profile graph throughput using graph output:

```
auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE);
const int WINDOW_SIZE_in_bytes=8192;
int iterations=999;
//Third parameter is the amount of data to be transferred (in bytes).
event::handle handle = event::start_profiling(gr_pl.dataout,
event::io_stream_start_to_bytes_transferred_cycles,
WINDOW_SIZE_in_bytes*iterations);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
```

```
gr_pl.run(iterations);
s2mm_run.wait();//performance counter 0 stops, assuming s2mm able to
receive all data
long long cycle_count = event::read_profiling(handle);
double throughput = (double)WINDOW_SIZE_in_bytes*iterations / (cycle_count
* 1e-9); //bytes per second
event::stop_profiling(handle);//Performance counter is released and cleared
```

Note that in above code, the run waits for s2mm to complete to ensure that all data are transferred through PLIO.

When using the API in the AI Engine simulation flow, you can use `graph.wait()` instead. Note that after `graph.wait()`, the API still requires additional cycles to transfer data from the window buffer to PLIO. One solution is use large enough number of iterations, so that the overhead is small enough to be negligible. Another solution is to use `graph.wait(<NUM_CYCLES>)` for a number of cycles that is long enough to make sure all data is transferred through PLIO.

Profile Graph Throughput Using the Graph Input

The stream from PLIO to kernel and the DMA for the input buffers are ready to receive data right after configuration. When the PL kernel `mm2s` is asserted, the input net can start receiving data even before `graph::run`. One way to profile a PLIO input is to assert the PL after `event::start_profiling()`. The following example shows how to profile graph throughput using graph input:

```
const int WINDOW_SIZE_in_bytes=8192;
int iterations=999;
//Third parameter is the amount of data to be transferred (in bytes).
event::handle handle = event::start_profiling(gr_pl.in,
event::io_stream_start_to_bytes_transferred_cycles,
WINDOW_SIZE_in_bytes*iterations);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
gr_pl.run(iterations);
auto mm2s_run = mm2s(nullptr, OUTPUT_SIZE_MM2S);//After start profiling,
send data from mm2s
gr_pl.wait();//performance counter 0 stops, assuming s2mm able to receive
all data
long long cycle_count = event::read_profiling(handle);
double throughput = (double)WINDOW_SIZE_in_bytes*iterations / (cycle_count
* 1e-9); //bytes per second
event::stop_profiling(handle);//Performance counter is released and cleared
```

If the amount of data transferred is not known, it is also possible to estimate graph throughput with this method. For example, if PL kernels are free-running and if the graph output AI Engine - PL interface column is running out of performance counters, you can still profile the graph throughput via graph input:

```
const int WINDOW_SIZE_in_bytes=8192;
int iterations=999;
//Third parameter is the amount of data to be transferred (in bytes).
event::handle handle = event::start_profiling(gr_pl.in,
event::io_stream_start_to_bytes_transferred_cycles,
WINDOW_SIZE_in_bytes*iterations);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
gr_pl.run(iterations);
gr_pl.wait();//performance counter 0 does not stop
//Read performance counter value immediately
//Assuming that overhead can be negligible if iteration is large enough
long long cycle_count = event::read_profiling(handle);
double throughput = (double)WINDOW_SIZE_in_bytes*iterations / (cycle_count
* 1e-9); //bytes per second
event::stop_profiling(handle);//Performance counter is released and cleared
```

Profiling Graph Latency

The `event::io_stream_start_difference_cycles` enumeration can be used to measure the latency between two PLIO or GMIO ports. After `event::start_profiling()` API, two performance counters starts incrementing each cycle, waiting two independent nets to receive their first data. After the first data passes either net, the corresponding performance counter will stop. The value read back by `event::read_profiling()` is the number difference between the two performance counters.

After `event::stop_profiling()`, the performance counter is cleared and released.

Profile Graph Latency

Graph latency can be defined as the time spent from receiving the first input data to producing the first output data. It is not dependent on the number of iterations the graph is run for. The following examples shows how to profile graph latency using the event API in AI Engine simulation flow and hardware/hardware emulation flows.

Note: `event::start_profiling()` has two different PLIO parameters.

In AI Engine simulation:

```
event::handle handle = event::start_profiling(gr_pl.in, gr_pl.dataout,
event::event::io_stream_start_difference_cycles);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
```

```

}
gr_pl.run(iterations); //Data transfer starts after graph.run()
gr_pl.wait();
long long cycle_count = event::read_profiling(handle);
printf("Latency cycles=: %d\n", cycle_count);
event::stop_profiling(handle); //Performance counter is released and cleared

```

In hardware and hardware emulation flows:

```

auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE);
event::handle handle = event::start_profiling(gr_pl.in, gr_pl.dataout,
event::event::io_stream_start_difference_cycles);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
gr_pl.run(iterations);
auto mm2s_run = mm2s(nullptr, OUTPUT_SIZE_MM2S); //input data transfer
starts
s2mm_run.wait(); //make sure both ports have data transferred
long long cycle_count = event::read_profiling(handle);
printf("Latency cycles=: %d\n", cycle_count);
event::stop_profiling(handle); //Performance counter is released and cleared

```

Note: Input data transfers starts immediately after the PL kernel `mm2s` starts. To avoid any overhead that `graph.run()` might introduce in the profiling graph latency, in the profiling code, start PL kernel `mm2s` after `event::start_profiling`, and after `graph.run()`.

Profile Latency Difference Between Two Ports

This method is not limited to profile latency between input port and output port of the same graph. It can be used to profile latency between any two ports. For example, it can profile latency between two output ports that have a common input port.

AI Engine Simulation

Example code in this simulation flow is as follows:

```

event::handle handle = event::start_profiling(gr_pl.dataout,
gr_pl.dataout2, event::event::io_stream_start_difference_cycles);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
gr_pl.run(iterations);
gr_pl.wait();
long long cycle_count = event::read_profiling(handle);
printf("Latency cycles=: %d\n", cycle_count);
event::stop_profiling(handle); //Performance counter is released and cleared

```


Hardware Emulation and Hardware

Example code in these flows is as follows:

```
auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE); event::handle handle =
event::start_profiling(gr_pl.dataout, gr_pl.dataout2,
event::event::io_stream_start_difference_cycles);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
gr_pl.run(iterations);
auto mm2s_run = mm2s(nullptr, OUTPUT_SIZE_MM2S);
s2mm_run.wait();//make sure both ports have data transferred
long long cycle_count = event::read_profiling(handle);
printf("Latency cycles=: %d\n", cycle_count);
event::stop_profiling(handle);//Performance counter is released and cleared
```

where, a positive number indicates that the data arrives `gr_pl.dataout2` later than `gr_pl.dataout`, while a negative number indicates that data arrives `gr_pl.dataout2` earlier than `gr_pl.dataout`.

Profiling Graph Bandwidth

The `event::io_total_stream_running_to_idle_cycles` profiling event can be used to track the running and stall events that occurs on the profiled AI Engine - PL interface. This means that it will track the number of cycles that the interface is active (data flowing through) and the number of cycles the interface is stalled. It does not track the number of cycles that the interface is in idle state.

After `event::start_profiling()`, the performance counter will wait for the running data to start, and it will be paused if the stream is idle. After the performance counter is paused, it will resume when data flow is resumed. After `event::stop_profiling()`, the performance counter will be cleared and released. This API reports how well the AI Engine and PL kernels utilize the available bandwidth. It is not a measure of best-case utilization of the port bandwidth.

Profile Graph Bandwidth Using the Input Port

The bandwidth of the graph can be defined as a percentage of the time that the graph can accept data.

An example code to measure the graph bandwidth via graph input port is as follows:

```
const int WINDOW_SIZE_in_bytes=8192;
int iterations=999;
event::handle handle = event::start_profiling(gr_pl.in,
event::io_total_stream_running_to_idle_cycles);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
}
```

```

    return 1;
}
auto mm2s_run = mm2s(nullptr, OUTPUT_SIZE_MM2S);
gr_pl.run(iterations);
gr_pl.wait();
long long cycle_count = event::read_profiling(handle);
double bandwidth = (double) (WINDOW_SIZE_in_bytes*iterations/4) /
cycle_count;
event::stop_profiling(handle); //Performance counter is released and cleared

```

where, the total running cycles can be calculated from how many bytes are transferred. With four bytes a cycle, the total running cycles are `WINDOW_SIZE_in_bytes*iterations/4`. The total running and stalled cycles are read from the performance counter by `event::read_profiling()`.

If the profiled bandwidth is 1, it means that the graph is running faster than the PL kernel `mm2s`, the input port has not been stalled.

If the profiled bandwidth is less than 1, it means that PL kernel `mm2s` can send data faster than the graph or PL kernel `s2mm` can receive. You might need to evaluate if the bandwidth drop is caused by the graph or PL kernel `s2mm`.

Profile Graph Bandwidth Using the Output Port

The bandwidth of the graph can be defined as percentage of the time that the graph can send data. If the profiled bandwidth is 1, it means that the graph is not blocked by PL kernel `s2mm`. If the profiled bandwidth is less than 1, it means that the graph is blocked in some percentage by `s2mm` due to back pressure. An example code to profile graph bandwidth via graph output port is as follows:

```

const int WINDOW_SIZE_in_bytes=8192;
int iterations=999;
event::handle handle = event::start_profiling(gr_pl.dataout,
event::io_total_stream_running_to_idle_cycles);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-PL
interface tile\n");
    return 1;
}
gr_pl.run(iterations);
gr_pl.wait();
long long cycle_count = event::read_profiling(handle);
double bandwidth = (double) (WINDOW_SIZE_in_bytes*iterations/4) /
cycle_count;
event::stop_profiling(handle); //Performance counter is released and cleared

```

Run-Time Event API Performance Counters Usage Summary

Run-time event APIs use the performance counters in the AI Engine-PL interface tiles and the AI Engine-NoC interface tiles. There are two performance counters in each column of the interface tiles. This section lists the number of performance counter - used by each run-time event API. If the total number of performance counters used exceeds the availability of performance counters in a column of the interface tile, the API that cannot acquire the performance counter fails with the following error message in AI Engine simulator.

```
[AIE WARNING]: Unable to request resources. RscType: 0
ERROR: event::start_profiling: Failed to request performance counter
resources.
```

For hardware emulation or hardware flows, the following error message is used.

```
[XRT] ERROR: ERROR: event::start_profiling: Failed to request performance
counter resources.: Resource temporarily unavailable
```

Table 61: Run-Time Event API Performance Counters Usage

Run-time Event Enumeration	Number of Performance Counters
event::io_total_stream_running_to_idle_cycles	1
event::io_stream_start_to_bytes_transferred_cycles	2
event::io_stream_start_difference_cycles	1 for input port, 1 for output port
event::io_stream_running_event_count	1

Note: There are four performance counters available in total. Performance counters are released after event::stop_profiling. The run-time event API can acquire the same performance counters again after they are released.

Note: When multiple graph ports are mapped into the same interface tile, if run-time event APIs are used on these ports, they will compete for the performance counters in the same interface tile.

XRT Support for Event APIs

XRT API is recommended for event profiling for hardware emulation and hardware flows. The XRT API header file `xrt/xrt_aie.h` defines class `xrt::aie::profiling` to support event profiling. The `xrt::aie::profiling` class member functions `start`, `read`, and `stop` are used to start profiling, read profiling results, and stop profiling.

The profiling modes are defined in XRT as:

```
enum class profiling_option : int
{
    io_total_stream_running_to_idle_cycles = 0,
    io_stream_start_to_bytes_transferred_cycles = 1,
    io_stream_start_difference_cycles = 2,
    io_stream_running_event_count = 3
};
```

The `start` function has the following parameters:

```
int start(profiling_option option, const std::string& port1_name, const
std::string& port2_name, uint32_t value) const; //meaning of value depends
on option.
```

An example host code to use XRT API to profile follows:

```
#include "xrt/xrt_aie.h"
.....
auto ghdl=xrt::graph(device,uuid,"gr");
xrt::aie::profiling handle(device);
handle.start(xrt::aie::profiling::profiling_option::io_stream_start_to_bytes
_transferred_cycles, "gr.dataout", "", output_size_in_bytes);
.....
long long cycle_count = handle.read();
handle.stop();
```

For more information about PS host coding and compilation, see [Chapter 7: Programming the PS Host Application](#).

Profiling the NoC

Introduction

The AMD Versal programmable Network on Chip (NoC) is an AXI-interconnecting network used for sharing data between IP endpoints in the programmable logic (PL), the processing system (PS), AI Engine, and other integrated blocks. This device-wide infrastructure is a high-speed, integrated data path with dedicated switching. The NoC system is a large-scale interconnection of instances of NoC master units (NMUs), NoC slave units (NSUs), and NoC packet switches (NPSs), each controlled and programmed from a NoC programming interface (NPI). There are 16 NMUs/NSUs on the VC1902 device, each one is capable of 16 GB/s of throughput in each direction.

For more information on NoC architecture, see [NoC Architecture](#) in the *Versal Adaptive SoC Programmable Network on Chip and Integrated Memory Controller LogiCORE IP Product Guide (PG313)*.

The main data streams to and from the AI Engine are PL streaming interface (PLIO) and GMIO. A GMIO is used to make external memory-mapped connections to or from the global memory through the AI Engine - NoC master unit. This feature lets you monitor the run-time traffic between the DDR memory and the AI Engine. This section provides you with a methodology to profile the NoC master unit in terms of latency and throughput of GMIO ports that read and write data to/from AI Engine to the DDR memory.

Network performance of the NoC interconnecting network can be monitored by the Vperf utility. Vperf is a Vitis-based tool that profiles the NoC and DDRMC in applications built using a v++ flow. You do not need to add any compile level options to use the utility.

Profiling the NMUs Connected to AI Engine

The steps below provides a procedure to profile the NoC. It is expected that you have generated the hardware boot image (`sd_card.img`) before proceeding further.

1. Setup the Vitis tool environment by sourcing the `settings64.csh` script.

```
<Vitis Installation Path>/Vitis/<202x.x>/settings64.sh
```

2. Launch the Chipscopy server using the command below.

```
cs_server &
```

Figure 68: Chipscopy Server

```
***** Xilinx cs_server v20: .0
***** Build date : Jul 24 2023-20:33:47
**** Build number : 2023.1690211027
** Copyright 2017-2023 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

Connect to TCP: <IP>:3042...
```

3. Run the hardware server from the computer that connects to target board. To do so, launch hardware server from the computer that has JTAG connection to the vck190 board. Note the computer Name or IP address as highlighted above for `cs_server`.
4. Boot up the board using the `sd_card` image.
5. In the local Linux terminal, navigate to the directory where the v++ build output is present.
6. Configure the options to the `vperf` command as shown below.

```
vperf --link_summary <Design>.xsa.link_summary --hw_url
<TCP:ComputerName>/<IP>:3121 --cs_url <TCP:ComputerName>/<IP>:3042 --
verbose
```

You must provide the `--link_summary` option, which should be set to the `link_summary` output generated from the v++ `--link` step. Set the `--hw_url` option to the hardware server location, and the `--cs_url` option to the Chipscopy location.

The output below is generated when the command is executed.

```
[INFO] Successfully connected to hw_server and cs_server
[INFO] Debug IP layout found at : <Vitis Build Path>/_x/link/int/
debug_ip_layout.rtd
[INFO] HW Device : xilinx_vck190_base_<>
1 NOC_NMU128_X0Y6 : CIPS_0/FPD_CCI_NOC_0
2 NOC_NMU128_X0Y7 : CIPS_0/FPD_CCI_NOC_1
3 NOC_NMU128_X0Y8 : CIPS_0/FPD_CCI_NOC_2
```

```

4 NOC_NMU128_X0Y9      : CIPS_0/FPD_CCI_NOC_3
5 NOC_NMU128_X0Y4      : CIPS_0/FPD_AXI_NOC_0
6 NOC_NMU128_X0Y5      : CIPS_0/FPD_AXI_NOC_1
7 NOC_NMU128_X0Y3      : CIPS_0/LPD_AXI_NOC_0
8 NOC_NMU128_X0Y2      : CIPS_0/PMC_NOC_AXI_0
9 NOC_NMU128_X7Y10     : ai_engine_0/M03_AXI
10 NOC_NMU128_X10Y10    : ai_engine_0/M00_AXI
11 NOC_NMU128_X12Y10    : ai_engine_0/M01_AXI
12 NOC_NMU128_X8Y10     : ai_engine_0/M05_AXI
13 NOC_NMU128_X9Y10     : ai_engine_0/M04_AXI
14 NOC_NMU128_X11Y10    : ai_engine_0/M02_AXI
15 DDRMC_X0Y0           : DDRMC/DDRMC_X0Y0
16 DDRMC_X3Y0           : DDRMC/DDRMC_X3Y0
17 DDRMC_X1Y0           : DDRMC/DDRMC_X1Y0

```

The `vperf` command reads the metadata from the link summary and populates all the nodes corresponding to the NoC NMUs used by the design. For example, the output above lists the NoC NMu units that corresponds to the Control, Interrupt and Processing subsystem, AI Engine GMIO interface ports, and DDR memory controller channel ports.

The `vperf` profiling control commands can be used to filter out the nodes of interest and start profiling.

Below is list of the control commands that are available to start/end profiling the nodes of interest.

Table 62: vperf Control Commands

Command	Description	
s/start_profile	Starts profiling.	
e/end_profile	Ends profiling.	
q/quit	End profiling and quits the <code>vperf</code> command.	
n/set_npi_sample_period	Sets the sample period that signifies the period over which values are sampled. Issuing this command discovers the debug cores that are available and displays the supported sample periods. You can choose sample period using the ID.	
	ID	Sample Period
	0	56 ms
	1	112 ms
	2	224 ms
	3	447 ms
	4	895 ms
	5	1790 ms
	6	3579 ms
	7	7158 ms
t/set_tslide	Represents 2 ^{TSLIDE} clock cycles as one count. This is useful when overflow occurs while profiling the NoC.	

Table 62: vperfc Control Commands (cont'd)

Command	Description	
f/set_filter	This command allows you to choose the nodes to profile. You can specify the list of nodes separated by comma or specify the range.	
	Filter String	Nodes Selected
	1,3,4,5,6,8	1,3,4,5,6,8
	1-5,8	1,2,3,4,5,8
	2,4,9-10	2,4,9,10
	It is recommended that you profile up to four nodes at a time. This is due to the slower sampling intervals over JTAG. Profiling more than four nodes could result in missing samples and consequently inaccurate results.	
c/clear_filter	Clears all the filters and profiles all nodes(default)	
p/print	This prints the configuration for easy reference before starting the profile.	

Below is an example of setting `npi_sample_period`, `Tslide` options.

```
> n
Discovering debug cores...Done!

Enumerating NoC (warning this can take 60-80s on xcvc1902)...Done
Discovered Nodes:
{'enabled': ['DDRMC_X0Y0', 'DDRMC_X1Y0', 'DDRMC_X3Y0',
'NOC_NMU128_X0Y3', 'NOC_NMU128_X0Y2', 'NOC_NMU128_X0Y4',
'NOC_NMU128_X0Y5', 'NOC_NMU128_X0Y6', 'NOC_NMU128_X0Y7',
'NOC_NMU128_X0Y8', 'NOC_NMU128_X0Y9', 'NOC_NMU128_X11Y10',
'NOC_NMU128_X12Y10', 'NOC_NMU128_X9Y10', 'NOC_NMU128_X10Y10',
'NOC_NMU128_X7Y10', 'NOC_NMU128_X8Y10', 'NOC_NSU128_X6Y6'], 'disabled':
[], 'invalid': []}
Select ID from the list:
ID Sample Period
0 : 56ms
1 : 112ms
2 : 224ms
3 : 447ms
4 : 895ms
5 : 1790ms
6 : 3579ms
7 : 7158ms

>> 0
[INFO] Sample period changed to 56ms

> t
Select Tslide between 0 and 27:
>> 1
[INFO] Tslide changed to: 1.
```

- Once all configurations are set, you can start the profiling using `s/start_profile`.
- Navigate to the hardware Linux console and run the application. Once the run is complete you can end the profiling using `e/end_profile`.
- When you quit from the `vperfc` utility using the `q/quit` command, a new `vperfc.run_summary` file gets generated which can be opened in the Vitis IDE.

In the example below the AI Engine application includes three GMIO input ports and six GMIO output ports. When you use option `f` command on the `vperf` utility to filter the nodes of interest for profiling, it displays all the available nodes as shown below.

```
> f
List of detected nodes in the design:
1  NOC_NMU128_X0Y6      : CIPS_0/FPD_CCI_NOC_0
2  NOC_NMU128_X0Y7      : CIPS_0/FPD_CCI_NOC_1
3  NOC_NMU128_X0Y8      : CIPS_0/FPD_CCI_NOC_2
4  NOC_NMU128_X0Y9      : CIPS_0/FPD_CCI_NOC_3
5  NOC_NMU128_X0Y4      : CIPS_0/FPD_AXI_NOC_0
6  NOC_NMU128_X0Y5      : CIPS_0/FPD_AXI_NOC_1
7  NOC_NMU128_X0Y3      : CIPS_0/LPD_AXI_NOC_0
8  NOC_NMU128_X0Y2      : CIPS_0/PMC_NOC_AXI_0
9  NOC_NMU128_X7Y10     : ai_engine_0/M03_AXI
10 NOC_NMU128_X10Y10    : ai_engine_0/M00_AXI
11 NOC_NMU128_X12Y10    : ai_engine_0/M01_AXI
12 NOC_NMU128_X8Y10     : ai_engine_0/M05_AXI
13 NOC_NMU128_X9Y10     : ai_engine_0/M04_AXI
14 NOC_NMU128_X11Y10    : ai_engine_0/M02_AXI
15 DDRMC_X0Y0           : DDRMC/DDRMC_X0Y0
16 DDRMC_X3Y0           : DDRMC/DDRMC_X3Y0
17 DDRMC_X1Y0           : DDRMC/DDRMC_X1Y0
```

One important point to observe here is that the design has a total of nine I/O ports. However, there are only six NMU nodes (ID: 9-14) corresponding to the AI Engine. This is because, for each AI Engine to NMU connection through the interface tile, there are two MM2S and two S2MM channels supported.

There is a high possibility that read and write happens through the same interface tile but on different channels. Hence, they use the same NMU node for both read and write transfers, and the same NMU node appears in both NoC counters in the Vitis IDE as shown in the following figure.

Information on GMIO port (both input and output) interface tile and channel mapping can be found in the graph compile summary report in the Vitis IDE.

Figure 69: Interface Channels

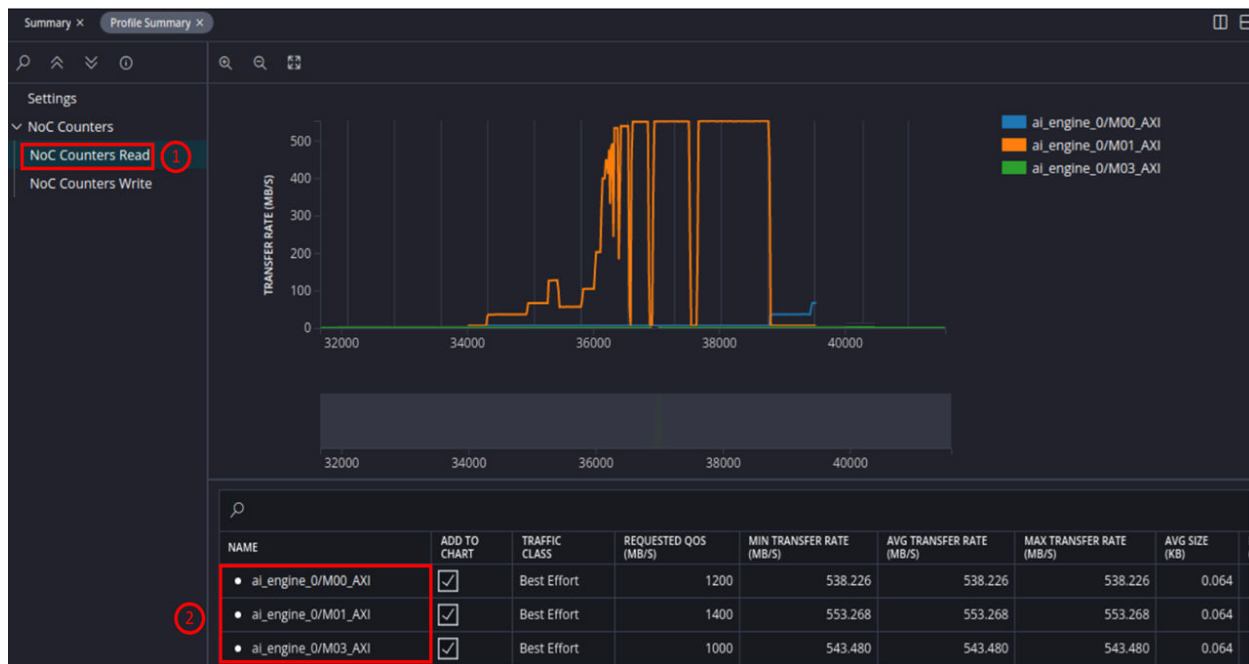
	Kernels	I/O	Buffers	Ports	Nets	Tiles	Interface Channels	DMA Channels
NAME	PL INSTANCE	TYPE	NET	AI ENGINE TRACE	MEMORY TRACE	INTERFACE TILE TRACE	ESTIMATE TRACE BANDWIDTH (MB/S)	
▼ Interface Tile 18 (2)								
• Input Ch: 3	in0x	GMIO	net0					
• Output Ch: 2	upscale_out0	GMIO	net4					
▼ Interface Tile 19 (1)								
• Output Ch: 2	data_shuffle0	GMIO	net6					
▼ Interface Tile 26 (1)								
• Output Ch: 2	data_shuffle1	GMIO	net13					
▼ Interface Tile 27 (2)								
• Input Ch: 3	in1x	GMIO	net7					
• Output Ch: 2	upscale_out1	GMIO	net11					
▼ Interface Tile 34 (1)								
• Output Ch: 2	upscale_out2	GMIO	net18					
▼ Interface Tile 35 (2)								
• Input Ch: 3	in2x	GMIO	net14					
• Output Ch: 2	data_shuffle2	GMIO	net20					

In the following example, the configuration is set to profile three NoC NMUs, 9-11 (considering the JTAG bandwidth limitations).

```
Enter the filter to select Nodes from the above list:
>> 9,10,11
[INFO] Selected nodes according to the filter :
1 NOC_NMU128_X7Y10 : ai_engine_0/M03_AXI
2 NOC_NMU128_X10Y10 : ai_engine_0/M00_AXI
3 NOC_NMU128_X12Y10 : ai_engine_0/M01_AXI
```

The NoC profile output in Vitis IDE is shown below.

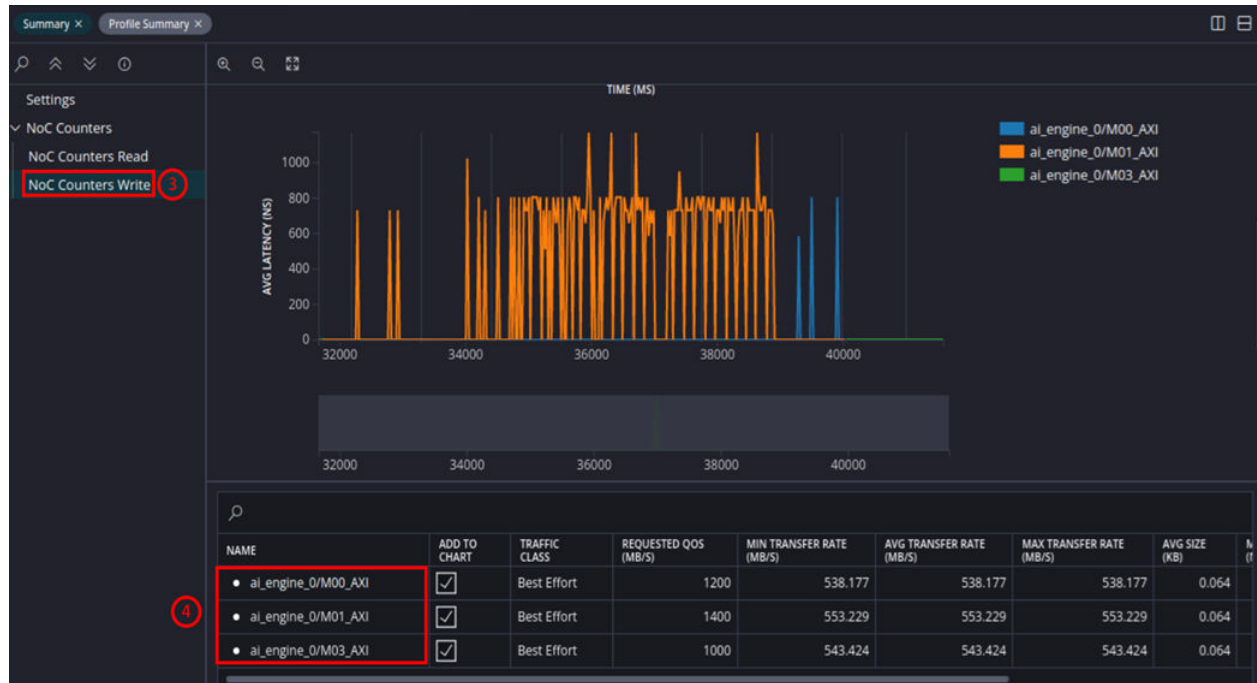
Figure 70: NoC Counters Read



- NoC counters read group, which represents the traffic going through NMUs from DDR to AI Engine.
- NMU nodes corresponding to the GMIO input read.

The NoC profile output in the Vitis IDE contains NMU write groups and the corresponding nodes.

Figure 71: NoC Counters Write



- c. NoC Counter write group, which represents the traffic going through NMUs from AI Engine to DDR.
- d. NMU nodes corresponding to the GMIO output write.

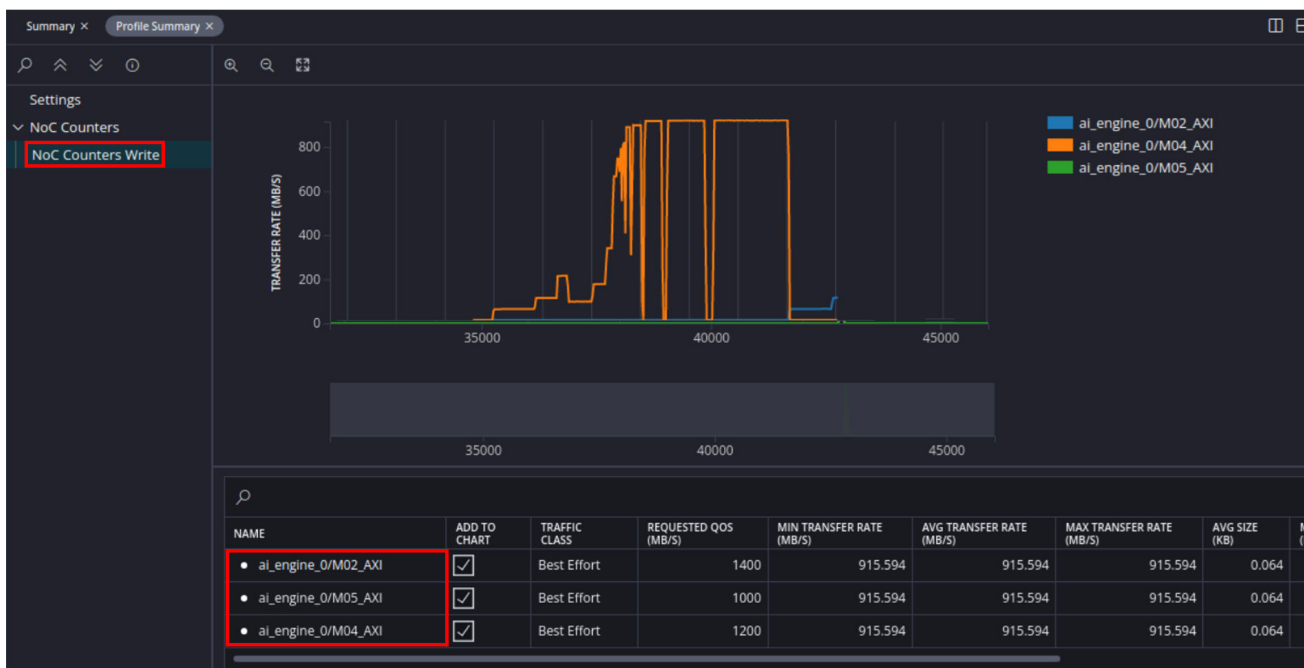
The configuration is set to profile NoC NMUs, 12-14 (considering the JTAG bandwidth limitations).

```
Enter the filter to select Nodes from the above list:
>> 12,13,14
[INFO] Selected nodes according to the filter :
1 NOC_NMU128_X8Y10 : ai_engine_0/M05_AXI
2 NOC_NMU128_X9Y10 : ai_engine_0/M04_AXI
3 NOC_NMU128_X11Y10 : ai_engine_0/M02_AXI
```

To get profiling for other NMU nodes:

1. Re-configure the `npi_sample_period`, `TSLIDE` values and filter the required nodes.
2. Start the profiling.
3. Re-run the application on hardware.
4. End the profiling and quit to generate the new `vperf.run_summary` file.

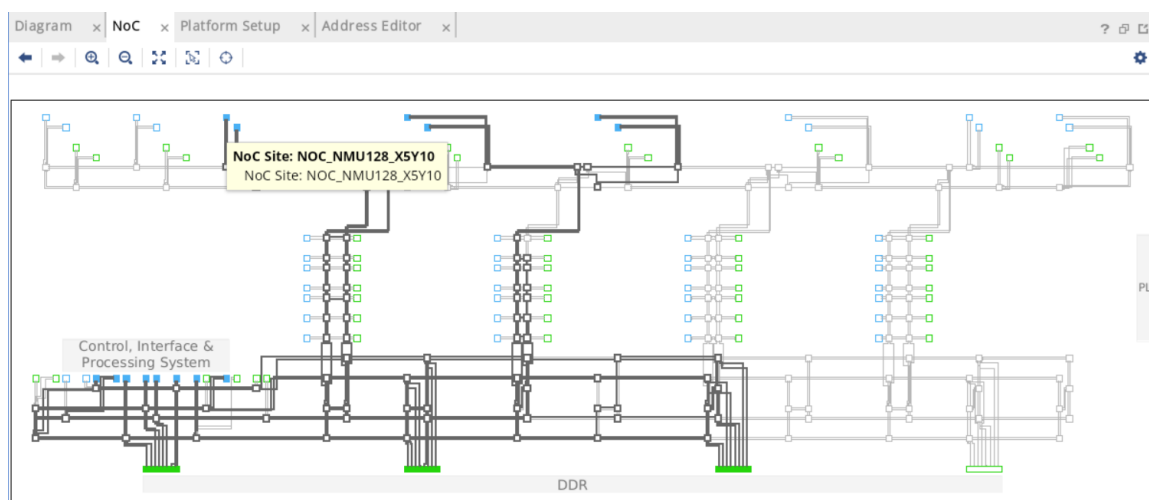
Figure 72: Only NoC Counter Write



In the preceding figure, only the NoC counters write group is in the profile summary and no read group. This is because the three nodes correspond to the GMIO output, and no GMIO input is mapped to that interface tile.

You can also open the Vivado project generated by the v++ link (`_x/link/vivado/vpl/prj/prj.xpr`) step to view the NoC Interconnect network and locate the NMUs that are connected for any design that have traffic going through the NoC.

Figure 73: NoC Vivado



Profile Summary Report Analysis

As observed in the section above, the profile summary provides you with many details about different NMU nodes.

The parameters used to configure the NMU nodes are described in the table.

Table 63: Profile Summary Report Analysis

Parameter	Description
Name	Represents the NMU node.
Traffic Class	This parameter is configured to best effort for VCK190 base platform. The other options available are low latency and Isochronous, which can be configured in Vivado IP design.
Requested QoS	This is the quality of service requested in GMIO API specification in graph code.
Transfer rate	The amount of data transferred within a sampling window.
Latency	Accumulated (summed up) latency during the measurement window.
Avg. size	Total KB transferred / Total burst count Burst count is the number of transactions seen during the measurement period.

Limitations

Below are some limitations to profiling the NoC:

- This utility only supports profiling NMU nodes. Profiling support for NSU nodes is not available at this time.
- This utility uses JTAG to collect profiling data which results in relatively slower sampling intervals.
- This utility is not suitable for profiling the design for short time bursts. The `vperf` utility performs better if NoC traffic is active over longer periods of time (few seconds minimum).
- It is recommended that you limit the number of nodes being profiled to 4 to avoid missing profiling sample data.

Event Tracing in Hardware

AI Engine event trace tools provide an in-depth investigation into the operation and performance of a design. In support of this, a number of settings are required to capture trace data at runtime. In hardware, you must prepare the design when compiling the AI Engine graph application to ensure the `libadf.a` supports capturing trace data at runtime. In order to trace events in hardware, use the `--event trace` option when compiling the AI Engine graph. This option sets up the hardware device to capture AI Engine runtime trace data.

The event trace flow consists of three parts:

1. Event trace build flow.
2. Running the design in hardware and capturing trace data at runtime.
3. Viewing and analyzing the trace data using the Vitis IDE.

Note: Event trace is also supported on BDC platforms vck_190_bdc, as described in [Vitis Base Platform for the VCK190 Board](#), and vek_280_bdc, as described in [Vitis Base Platform for the VEK280 Pre-production Board](#)

Event Trace Build Flow

The event trace build flow is as follows:

1. Compile the graph with `--event-trace` and other appropriate flags.

An example of the AI Engine compiler command for event tracing is as follows:

```
v++ --c --mode aie --verbose --pl-freq=100 --workdir=./myWork \
--event-trace-port=gmio --event-trace=runtime \
--num-trace-streams=8 --xlopt=0 --include= "./" \
--include= "./src" --include= "./src/kernels" --include= "./data" \
./src/graph.cpp
```

For examples using the unified command line interface, see [v++ Mode AI Engine](#) in the *Vitis Reference Guide* (UG1702).

Note:

- The preceding example illustrates compiling the design with `--event-trace=runtime` configuration. When you use this option, you can configure the type of events that AI Engine captures during runtime.
- The `--event-trace-port=gmio` option uses GMIO to capture event trace data. This option uses the AI Engine-to-NoC event trace pathway. The alternative is to use PLIO which uses the AI Engine-PL pathway to capture event trace data. This uses programming logic resources to capture data from AI Engine to DDR.



RECOMMENDED: AMD recommends using `gmio` option as the `event-trace-port` configuration. This avoids the usage of programming logic resources and avoids timing errors caused by the PL resource usage.

- `gmio` is the AI Engine NoC event pathway. GMIO is the default event trace port configuration.
- `plio` is the AI Engine to PL event trace pathway. Event data get transferred to the PL where it is stored within BRAMs and URAMs resources. This PL resource add-on can induce timing errors on the hardware design side.

Note: The compiler option `--graph-iterator-event` should be used to delay the event trace data based on the graph iteration. For more information on delaying the event trace, see [Table 66: XSDB Trace Options](#) in *XSDB Flow*, or [Table 67: XRT Trace Options](#) in *XRT Flow*.

2. Compile and link the design using the Vitis compiler.

After compiling the AI Engine graph application, you must build the other elements of the system, as described in [Building and Running the System](#) in the *Embedded Design Development Using Vitis* (UG1701). With `--event-trace` enabled in the `libadf.a` file from the AI Engine compiler, the system hardware generated by the Vitis compiler includes the compiled ELF file for the PS application, the compiled ELF files for the AI Engine processors, and the XCLBIN file for the PL. These are the elements you need to run the system on hardware.

3. After linking to create the device binary, run the Vitis compiler `--package` step to create the `sd_card` folder and files needed to boot the device, as described in [Integrate System](#) in the *Embedded Design Development Using Vitis* (UG1701). This step packages everything needed to build the `BOOT.BIN` file for the system. When packaging the boot files for the device, you must also specify the `--package.defer_aie_run` to load the AI Engine application with the ELF file, but wait to run it until `graph.run` directs it, as described in [Graph Execution Control](#) in the *AI Engine Kernel and Graph Programming Guide* (UG1079).

A feature of the AI Engine compiler `--event-trace` option is that you can compile your design to capture data using the `runtime` argument. Using this option, you can compile the AI Engine graph to be set up for event trace, and specify the type of profile data to capture at runtime: `functions`, `partial_stalls`, and `all_stalls`.

Table 64: Events Supported in Predefined Event Trace Levels

Event Type	Predefined Event Trace Level		
	functions	partial_stalls	all_stalls
Functions Calls/Returns	Captured	Captured	Captured
Stream Stalls	NA	Captured	Captured
Cascade Stalls	NA	Captured	Captured
Lock Stalls	NA	Captured	Captured
Memory Stalls	NA	NA	Captured

- **Functions Calls/Returns:** Event generated when kernel functions are being invoked and returned.
- **Stream Stalls:** Event generated when core gets stalled. This can be due to either NO data at Input or backpressure on the stream output from the core.
- **Cascade Stalls:** Event generated when core gets stalled. This can be due to either NO data at cascade stream Input or due to backpressure on the cascade stream output from the core.
- **Lock Stalls:** Event generated when the core gets stalled due to the lock already being acquired.
- **Memory Stalls:** Event generated when core gets stalled due to a memory conflict.

Note: Metrics `<functions_all_stalls>` and `<functions_partial_stalls>` are renamed to `all_stalls` and `partial_stalls` respectively for runtime configuration.

Running the Design in Hardware and Capturing Trace Data at Run-Time

Xilinx Runtime (XRT) and Xilinx Software Debugger (XSDB) are two ways to run the design on the Arm® processor in hardware and capture trace data at run time. XRT is supported on the Linux platform, whereas XSDB is supported on both bare metal and Linux platforms. The table below highlights the features supported in both flows.

Table 65: XRT versus XSDB

	Bare Metal	PetaLinux	Bandwidth per Trace Stream (bits/s)	Ease of Use
XSDB	Yes	Yes	PL clock-rate * W (with W=32, 64 or 128)	This flow involves manually setting up the environment, sourcing scripts and running the event trace flow. (Details can be found in the XSDB section below)
XRT	No	Yes	PL clock-rate * W (with W=32, 64 or 128)	This flow is easier than XSDB, because event trace can be set up in the <code>xrt.ini</code> file.

Note: This table refers to PLIO data transfers and the platform default clock will be used to connect trace streams in PL.

XSDB Flow

The XSDB flow is as follows:

1. Set up `xsdb` as described in the following steps to connect to the device hardware.

When running the application, the trace data is stored in DDR memory by the debugging and profiling IP. To capture and evaluate this data, you must connect to the hardware device using `xsdb`. This command is typically used to program the device and debug bare-metal applications. Connect your system to the hardware platform or device over JTAG, launch the `xsdb` command in a command shell, and run the following sequence of commands:

```
xsdb% connect
xsdb% ta
xsdb% ta 1
xsdb% source $::env(XILINX_VITIS)/scripts/vitis/util/aie_trace.tcl
xsdb% aietrace start -graphs mygraph -work-dir ./Work -link-summary
$PROJECT/xsa.link_summary -base-address 0x900000000 -depth 0x800000 -
tile-based-aie-tile-metrics "all:functions; {4,1}:{6,2}:all_stalls"

# Execute the PS host application (.elf) on Linux
## After the application completes processing.
xsdb% aietrace stop
```

where, the `source $::env(XILINX_VITIS)/scripts/vitis/util/aie_trace.tcl` command sources the Tcl trace command to set up the `xsdb` environment.

2. Run the design on hardware to trace hardware events.
3. Offload the captured trace data.
4. Launch the Vitis IDE to import and analyze data with this command.

```
vitis -a aie_trace_profile.run_summary
```

Table 66: XSDB Trace Options

Option	Description
start	Start the event trace and initializes the DPA IP to begin capturing trace data.
stop	Stop the event trace and instructs the DPA IP to offload the trace event data from the DDR memory. This command must wait until after the application completes. The data is written to the <code>event_trace<N>.txt</code> file in the current working directory from where <code>xsdb</code> was launched. An <code>aie_trace_profile.run_summary</code> file is also created. It can be opened in the Vitis IDE as explained in Viewing the Run Summary in the Vitis IDE. TIP: If you do not remove the <code>event_trace<N>.txt</code> when running the graph again, the old files will be overwritten by the new run results.
-link-summary	Specify the path to the link summary generated during design linking phase. This is a mandatory option.
-start-type time iteration kernel_event0	Specifies different trace delaying approaches like time, iteration or user-defined event. To use the <code>-start-type kernel_event0</code> option, you must add <code>event0()</code> intrinsic in the kernel code that generates core event 0 for profiling. <code>-start-type time iteration</code> option should be combined with the <code>-start-time</code> and <code>-start-iteration</code> options respectively as explained below. For more information on delayed event trace, refer to the topic Using the Delayed Event Trace .
-start-time	Specifies the start delay of event trace in terms of either AI Engine clock cycles or time in sec, ms, us or ns. If no units are specified, the value is assumed to be in AI Engine clock cycles. This option should be used in combination with <code>-start-type time</code> .
-start-iteration	Starts the event trace based on the graph iteration count. To use this type of event trace start, you must re-compile the AI Engine design with an <code>graph-iterator-event</code> option. This option should only be used in combination with <code>-start-type iteration</code> .
-work-dir <Work Directory>	Specify working directory. This is a mandatory option.
-base-address <address>	Starting address in the DDR from which the Trace data will be stored. IMPORTANT! The DDR memory address used in <code>-base-address</code> must be a high address to limit any chance of running into memory conflicts with the OS on the Versal platform or the application. For a custom platform, make sure you know how much DDR memory is being used and plan accordingly.
-depth <size>	Length of the memory that will be used in the DDR. This is a required option.
-tile-based-aie-tile-metrics	Sets which AI Engine event to store for specified tiles.
-graph-based-aie-tile-metrics	Sets which AI Engine event to store for specified kernels/graphs.
-graph-based-memory-tile-metrics	Note: This option only applies to AI Engine-ML devices. Sets which Memory Tile metrics to store for specified buffer/graph

Table 66: XSDB Trace Options (cont'd)

Option	Description
-tile-based-memory-tile-metrics	Note: This option only applies to AI Engine-ML devices. Sets which Memory Tile metrics to store for specified Memory Tiles.

For the event specifications each option is essentially a combination string of all settings separated by semi-colon. For example:

```
# Sets event trace to "all_stall" for the tile range column 4 to
# 6 and row 1 to 2, and overrides to "functions" for tile (4,1)
-tile-based-aie-metrics "{4,1}:{6,2}:all_stalls; {4,1}:functions"

# Sets event trace to "functions" for kernel k1 in graph myGraph
-graph-based-aie-tile-metrics "myGraph:k1:functions"

# Sets event trace to "functions" for all kernels in hierarchical graph
# myGraph->subgraph1->subgraph2
-graph-based-aie-tile-metrics "myGraph.subgraph1.subgraph2:all:functions"

#Examples on Delayed event trace

# Sets event trace to "functions" for kernel k1 in graph myGraph delayed
# by 1.5ms
-start-type time -start-time 1.5ms -graph-based-aie-tile-metrics
"myGraph:k1:functions"

# Sets event trace to "functions" for kernel k1 in graph myGraph delayed
# by 10 graph iterations
-start-type iteration -start-iteration 10 -graph-based-aie-tile-metrics
"myGraph:k1:functions"

# Sets event trace to "functions" for kernel k1 in graph myGraph delayed
# by user defined event
-start-type kernel_event0 -graph-based-aie-tile-metrics
"myGraph:k1:functions"
```

Note: Metrics <functions_all_stalls> and <functions_partial_stalls> are renamed to all_stalls and partial_stalls respectively for runtime configuration.

XRT Flow

The XRT flow is as follows:

1. Burn the generated `sd_card.img` to the physical SD card.
2. Create the `xrt.ini` file in the `sd_card` folder as described in this section to enable xrt flow.

An example `xrt.ini` file is shown below.

```
# Main switch to turn on aie trace
[Debug]
aie_trace = true
# Continuous trace knobs
[AIE_trace_settings]
reuse_buffer = true
periodic_offload = true
# Time to wait between trace reads
buffer_offload_interval_us = 100
# Total amount of device memory shared between trace streams
buffer_size = 16M
# granularity
graph_based_aie_tile_metrics = all:all:functions
```

3. Run the design on hardware to trace hardware events.
4. Copy the captured trace data from the `sd_card` folder to your design at same level as the design `Work` directory. The trace data is generated in the same location as the host application on the SD card. They are `xrt.run_summary`, `aie_event_runtime_config.json`, and `aie_trace_N.txt`.
5. Use the Vitis IDE to import and analyze data with this command.

```
vitis -a xrt.run_summary
```

```
[Debug]
aie_trace = true

# Section for AIE trace settings
[AIE_trace_settings]

# Size of AIE trace buffer in DDR (Format: <Integer>[K|k|M|m|G|g]; Default: 1M)
buffer_size = 100M

# Graph/Kernel name
graph_based_aie_tile_metrics = <graph name|all>:<kernel_name|all:<off|functions|partial_stalls|all_stalls>

# AI Engine Tiles
# Single or all tiles
tile_based_aie_tile_metrics = <{<column>,<row>}|all>:<off|functions|partial_stalls|all_stalls>

# Range of tiles
tile_based_aie_tile_metrics = {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}:<off|functions|partial_stalls|all_stalls>
```

Table 67: XRT Trace Options

Option	Description
<code>aie_trace = true</code>	Enables AI Engine event trace during application execution.
<code>buffer_size = 100M</code>	Sets the size of the event trace buffer in DDR memory.

Table 67: XRT Trace Options (cont'd)

Option	Description
<code>reuse_buffer = true</code>	Enables the reuse of trace buffer in DDR memory. When this option is enabled, the DDR trace buffer is treated as a circular buffer, and trace data is continuously offloaded from XRT. This option only applies to event trace data being captured using PLIO trace. This option cannot be used with GMIO trace. The default is <code>false</code> .
<code>periodic_offload = true / false</code>	Enables offloading of trace data from DDR to XRT at regular intervals while the application is running. If this option is not set (<code>false</code>), the trace data is offloaded only at the end of the run. The default option setting is <code>true</code> . This option enables you to offload trace data at regular periodic intervals even if there are application crashes. In such cases, you can examine trace information up to the previous periodic download of trace data. This option can be used with the <code>offload_interval_us</code> option to capture GMIO and PLIO trace in periodic offload intervals. This option, when used in combination with the <code>reuse_buffer</code> option (applicable only to PLIO trace), enables continuous offloading of trace data.
<code>offload_interval_us = 10</code>	Specify the frequency, in milliseconds, of how often the trace events data is read from the trace buffers in DDR memory to the buffers in XRT. This option is effective only when <code>periodic_offload=true</code> . Default is 100
<code>file_dump_interval_s = 3</code>	Specify the frequency, in seconds, of how often the trace events data is read from the trace buffer in XRT and appended to the event trace files in the SD card. This option is effective only when <code>periodic_offload=true</code> . Default is 5
<code>start_type = time iteration kernel_event0</code>	To effectively use the finite trace buffer available in DDR, this option enables the capability of delaying the trace based on time, iteration or user-defined event. To use the <code>start_type = kernel_event0</code> option, you must add the <code>event0()</code> intrinsic in the kernel code that generates core event 0 for profiling. <code>start_type = time iteration</code> option should be combined with the <code>start_time</code> and <code>start_iteration</code> options respectively as explained below. For more information on delayed event trace, refer to the topic Using the Delayed Event Trace .
<code>start_time = <1000 1s 1ms 1us 1ns></code>	This option specifies the start delay of event trace in terms of either AI Engine clock cycles or time in sec, ms, us or ns. If no units are specified, the value is in AI Engine clock cycles. This option should only be used in combination with the <code>start_type = time</code> option.
<code>start_iteration = <int></code>	This option allows to start the event trace based on the graph iteration count. To use this type of event trace start, you must re-compile the AI Engine design with an option <code>graph-iterator-event</code> . This option should only be used in combination with <code>start_type = iteration</code> . If no value is specified, the default iteration is taken as 1.
<code>graph_based_aie_tile_metrics = <graph name all>:<kernel name all>:<off functions partial_stalls all_stalls></code>	This option configures the AI Engine event trace metric to be applied for all kernels in all or a specific graph. The metric is applied to the tile, even if there are multiple kernels running on the tile.
<code>tile_based_aie_tile_metrics = <{<column>,<row>} all>:<off functions partial_stalls all_stalls></code>	This option configures the AI Engine event trace metric to be applied for a single tile or all tiles.
<code>tile_based_aie_tile_metrics = {<mincolumn>,<minrow>}: {<maxcolumn>,<maxrow>}:<off functions partial_stalls all_stalls></code>	This option configures the AI Engine event trace metric to be applied for all tiles in a range.

Table 67: XRT Trace Options (cont'd)

Option	Description
<code>graph_based_interface_tile_metrics = <graph name all>:<port name all>:<off input_ports output_ports input_ports_stalls output_ports_stalls input_ports_details output_ports_details></code>	This option configures the event trace of the interface tile metric applied for all or specific ports in all or a specific graph.
<code>tile_based_interface_tile_metrics = <column all>:<off input_ports output_ports input_ports_stalls output_ports_stalls input_ports_details output_ports_details>[:<channel 1>][:<channel 2>]</code>	This option configures the event trace of the interface tile applied for a single tile or all tiles.
<code>tile_based_interface_tile_metrics = <mincolumn>:<maxcolumn>:<off input_ports output_ports input_ports_stalls output_ports_stalls input_ports_details output_ports_details>[:<channel 1>][:<channel 2>]</code>	This options configures the event trace of the interface tile applied for all tiles in a range.
<code>graph_based_memory_tile_metrics = <graph name all>:<buffer name all>:<off input_channels input_channels_details output_channels output_channels_details memory_stats mem_trace>[:<channel>]</code>	This option configures the event trace of the Memory Tile metric applied for a set of graph/buffers. Note: This option is applicable to AI Engine-ML devices only.
<code>tile_based_memory_tile_metrics = <column all>:<off channels input_channels_stalls output_channels_stalls>[:<channel 1>][:<channel 2>]</code>	This option configures the event trace of the Memory Tile metric for a single or all tiles. Note: This option is applicable to AI Engine-ML devices only.
<code>tile_based_memory_tile_metrics = <mincolumn>:<maxcolumn>:<off input_channels input_channels_stalls output_channels output_channels_stalls>[:<channel 1>][:<channel 2>]</code>	This option configures the event trace of the Memory Tile metric for all tiles in a range. Note: This option is applicable to AI Engine-ML devices only.

For example,

```
# Example 1 : trace function events for all tiles used in the AI Engine
array
[AIE_trace_settings]
tile_based_aie_tile_metrics = all:functions

# Example 2 : trace function events in all the kernels in all graphs
(Similar to the example above)
[AIE_trace_settings]
graph_based_aie_tile_metrics = all:all:functions
```

```

# Example 3 : trace all function stalls events on all used tiles
[AIE_trace_settings]
tile_based_aie_tile_metrics = all:all_stalls

# Example 4 : trace function events within the bounding box of tiles or
# on specific tiles
[AIE_trace_settings]
tile_based_aie_tile_metrics = {4,1}:{6,2}:functions_all_stalls;
{4,1}:functions

# Example 5 : trace events specified by graph name
[AIE_trace_settings]
graph_based_aie_tile_metrics = chain_0:all:functions;
chain_1:all:all_stalls; chain_2:all:functions_partial_stall

# Example 6 : trace functions_all_stalls events all kernels in all graphs
# and trace function events occurring in all kernels in the graph chain_0
[AIE_trace_settings]
graph_based_aie_tile_metrics = all:all:all_stalls; chain_0:all:functions

# Example 7 : Trace graphs and all kernels function events and override
# tile 4,1 with functions all stalls
[AIE_trace_settings]
graph_based_aie_tile_metrics = all:all:functions
tile_based_aie_tile_metrics = {4,1}:all_stalls

# Example 8 : Trace all graphs and all kernels function events and turn off
# trace in tile 4,1
[AIE_trace_settings]
graph_based_aie_tile_metrics = all:all:functions
tile_based_aie_tile_metrics = {4,1}:off

# Example 9 : Trace multi-level hierarchical graph
[AIE_trace_settings]
graph_based_aie_tile_metrics = MyGraph.sub_graph1.sub_graph2:all:functions

#Examples on Delayed event trace

# Example 10 : Trace function events in all the kernels in all graphs,
# delayed by 1ms.
[AIE_trace_settings]
graph_based_aie_tile_metrics = all:all:functions
start_type = time
start_time = 1ms

# Example 11 : Trace function events for all tiles used in the AI Engine
# array, delayed by 10 graph iterations.
[AIE_trace_settings]
tile_based_aie_tile_metrics = all:functions
start_type = Iteration
start_iteration = 10

# Example 12 : Trace user defined event in the kernel 'add' in graph
# 'MyGraph'.
[AIE_trace_settings]
graph_based_aie_tile_metrics = MyGraph:add:functions
start_type = kernel_event0

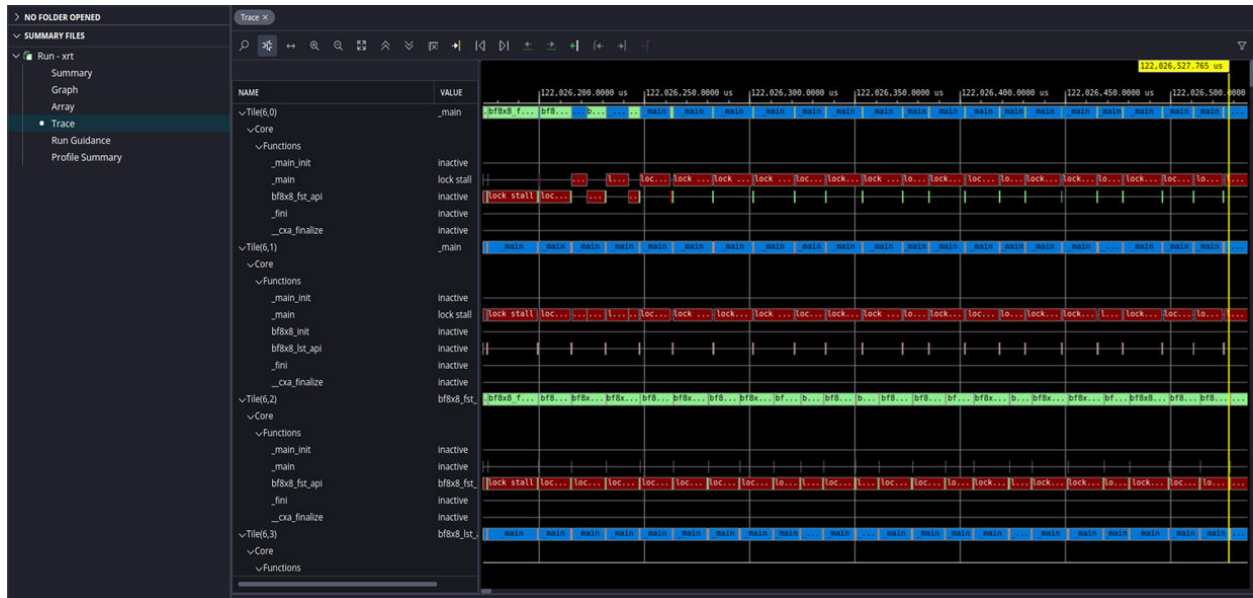
```

Note: Metrics `<functions_all_stalls>` and `<functions_partial_stalls>` are renamed to `all_stalls` and `partial_stalls` respectively for runtime configuration.

Viewing and Analyzing the Trace Data in the Vitis IDE

The Vitis IDE must be used to view and analyze trace data. After the trace data has been captured either using XRT or XSDB, you should have all the data needed to open the Event Trace view in the Vitis IDE.

Figure 74: Event Trace



Open the run summary file to view event trace data. An example of the XSDB flow is as follows:

```
vitis -a ./aie_trace_profile.run_summary
```

An example of the XRT flow is as follows:

```
vitis -a ./xrt.run_summary
```

Limitations

- Due to limited resources, overruns can be seen from event trace. Refer to [Using Multiple Event Trace Streams](#) to configure the number of trace streams to minimize the overruns issue.
- For detailed event trace information, the `--xlopt=0` option must be specified in compile. If omitted, the default setting `--xlopt=1` is used which might cause functions to be in-lined and limits debug capability.
- Host code is required to invoke `graph.end()` to ensure that the XRT flow completes properly.

Using the Delayed Event Trace

When the trace buffer available in the DDR is full, you have much less control on the event trace that can be viewed and analyzed. To address this, you can delay the event trace based on the time, iteration or user-defined event.

Consider a scenario of viewing an event trace of a complex design that runs for long time and the trace buffer is not sufficient enough to hold the complete event trace information. In that case, you can open the initial trace and estimate the approximate time the kernel is in ideal state. Based on this, you can specify the start time of the event trace, using the `start_time` option as explained in [Table 67: XRT Trace Options](#) and [Table 66: XSDB Trace Options](#) with an example. You can iterate over the value in `xrt.ini` file, run the application on hardware, and view the trace information in the Vitis IDE.

Consider another scenario where the graph is running for 100 iterations and as explained above, the buffer is only sufficient enough to hold the trace information of 10 iterations. In that case, to get the event trace information at 90th iteration, you can delay the start of the event trace based on the iteration and specify the value of the iteration using the option `start_iteration=90`. To retrieve the trace information based on the iteration value, you need to:

- Re-compile the graph using the option `--graph-iterator-event`.
- Re-package the updated `libadf.a` using `v++` package, and
- Re-generate the `sd_card` image.

You can use the XRT or XSDB flows to generate the event trace.

You can also delay the event trace based on the user-defined event specified in the kernel code. For example, the code below uses the `event0()` intrinsic.

```
void kernel_w_2048_1(input_window_cint16 * in, output_window_cint16 * out)
{
    cint16 c1, c2, cout;
    for (unsigned i=0; i<2048; i++)
    {
        c1 = in.data();
        c2 = out.data();
        c2.real = c1.real+c1.imag;
        c2.imag = c1.real-c1.imag;
        if(c2.real == 50)
            event0();
    }
}
```

As shown above, the `event0()` is triggered when the real value of `c2` equals 50. To view the trace information corresponding to this user-defined event, specify the value `kernel_event0` to the option `start_type` either in the XSDB or XRT flow, and generate the event trace.

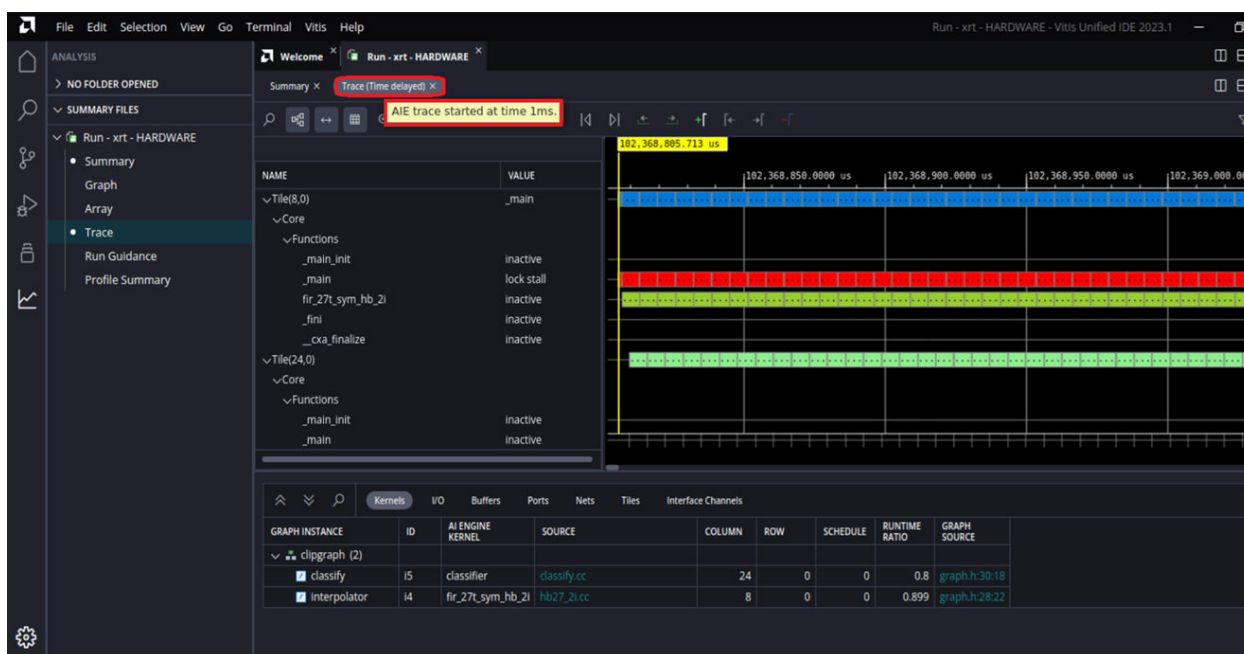
For more information about the `event0()` intrinsic, refer to [Events](#) in the *AI Engine Intrinsic User Guide* ([UG1078](#)).

Only one `event0()` can be used per AI Engine tile. Multiple events can be specified on multiple kernels that are mapped to different AI Engine tiles.

Note: Both iteration-based delay and event based delay uses the `event0()` intrinsic. So, to use the `event0()` intrinsic for event based delay, be sure to remove the compile option `--graph-iterator-event` and re-generate the `sd_card` image with updated `libadf.a`.

A sample delayed event trace, for which the `start_time` is specified as 1 ms, can be visualized in the Vitis IDE, as shown below. Notice the highlighted portion at the top-left corner which signifies the trace being viewed is delayed in this case. Hover your mouse over that heading to see more information.

Figure 75: Delayed Event Trace



Using Multiple Event Trace Streams

As AI Engine designs grow larger, tracking the events produced while running the design can be useful to identify performance bottlenecks as well as understanding how the overall AI Engine is operating for the design. Of course, with larger designs more and more events will be produced causing a bottleneck of the events being recorded by the trace IP being used. To capture all this data effectively, and quickly, you should consider instantiating multiple event trace streams. These streams will spread out the event data coming from the AI Engine, letting it store them correctly and in a timely manner.

To increase the trace streams in a design, use the `--num-trace-streams` option when you compile, which can have a value in the range of 1 to 16. The following table provides guidance on the number of trace streams to use, depending on the size of the design.

Table 68: Number of Event Trace Streams Methodology

Number of AI Engines	Recommended Number of Streams
Less than 10	1
Between 10 and 20	2
Between 20 and 40	4
Between 40 and 80	8
Larger than 80	16

Notes:

1. It is recommended to only use up to 16 trace streams due to the resource utilization impact on the PL and DMA channel resources.

After the change to the AI Engine compiler option, recompile and re-link the XCLBIN file and `libadf.a` using the Vitis compiler with a `config` file as described in [Linking the System](#) in the *Embedded Design Development Using Vitis (UG1701)*.

```
v++ -l --config system.cfg ...
```

Event Trace Offload using High Speed Debug Port

The high-speed debug port (HSDP) debug port provides debugging and trace capability for programmable logic (PL), processing system (PS), and AI Engines through a dedicated Aurora interface and a high-speed debug cable like SmartLynq+. The HSDP leverages the high-speed gigabit transceivers to make debug less intrusive to the system configuration.

The SmartLynq+ module is a high-speed debug and trace module, primarily targeting the AMD Versal™ adaptive SoC. It drastically improves configuration and trace speed. For trace capture, the SmartLynq+ module is capable of speeds up to 10 Gb/s by means of its high-speed debug port (HSDP), which is 100 times faster than standard JTAG. Faster iterations and repetitive downloads increase development productivity and reduce the design cycle.

In the traditional hardware event trace, the trace information is stored in DDR memory available in the Versal device initially, and offloaded to SD card after the application run completes. This imposes limitations on the amount of trace information that can be stored and analyzed. AI Engine trace offload via HSDP has more DDR memory in the SmartLynq+ module and supports analyzing large quantities of trace information for complex designs. In addition, the SmartLynq+ module offers high bandwidth connectivity to offload trace information via HSDP which is faster than standard JTAG connection. HSDP bandwidth is lower than direct DDR storage but allows much larger trace data set to be stored and analyzed.

Trace Offload Methodology

This section lists out the requirements and explain how to build a system that uses the SmartLynq+ module and the HSDP.

Compile, Link, and Package

- Compile the AI Engine design using the `-event-trace-port=plio` option. This sets the event tracing port to PLIO.

Note: If the event tracing port is set to GMIO, the AI Engine trace cannot be offloaded via HSDP.

- After the AI Engine graph and the C/C++ kernels are compiled, and any RTL kernels are packaged, the Vitis `v++ --link` command links them with the target platform to build the platform file (XSA). For offloading the AI Engine trace via HSDP, it is required to add the `-profile.aie_trace_offload=HSDP` option to the `v++ --link` command.

With this, a new HSDP IP gets instantiated for AI Engine trace offload and all the PLIO event trace streams are connected to the HSDP IP.

- Only the host program with XRT APIs controlling the AI Engine graph should be used for offloading AI Engine trace via HSDP.
- Package the HSDP enabled XSA file generated during linking step and `libadf.a` to generate a `sd_card` image.

Setup SmartLynq+ Module, and Connect to Versal Device

- You must download and install the latest microSD card image and SmartLynq+ application package to set up the SmartLynq+ module correctly. For more information on this installation process, see [Downloading and Installing SW Tools](#). Once the setup of SmartLynq+ is done, you need to connect the Versal device evaluation board.
- The SmartLynq+ module can be accessed by a host system using the USB 3.0 connection. The driver setup information for both Windows and Linux environments is provided in the [USB 3.0 Host Connection](#).
- For more information on connecting the Versal device evaluation board, see [Connection to Versal Evaluation Boards SmartLynq+ Module User Guide \(UG1514\)](#).

Note: [Embedded Design Tutorial](#) walks you through a system design example for High-Speed Debug Port with SmartLynq+ module.

- Additional information on overview of the SmartLynq+ module for the Versal adaptive SoCs is available in *SmartLynq+ Module User Guide (UG1514)*.

Launch XSDB, and Offload Trace Information

After loading the HSDP enabled Linux image on Versal Evaluation board:

1. Power on the Versal Evaluation board and SmartLynq+ module.
2. Observe the Linux boot messages in the `minicom` application (SmartLynq+ module can be used as a serial terminal to remotely view the UART output from the Versal board using the pre-installed minicom application).
3. Launch XSDB from your local directory, where the AI Engine design `Work/` directory is located.

- Source the `aie_trace.tcl` script using the command `source $::env(XILINX_VITIS)/scripts/vitis/util/aie_trace.tcl`.
- Start the AI Engine trace using the following command:

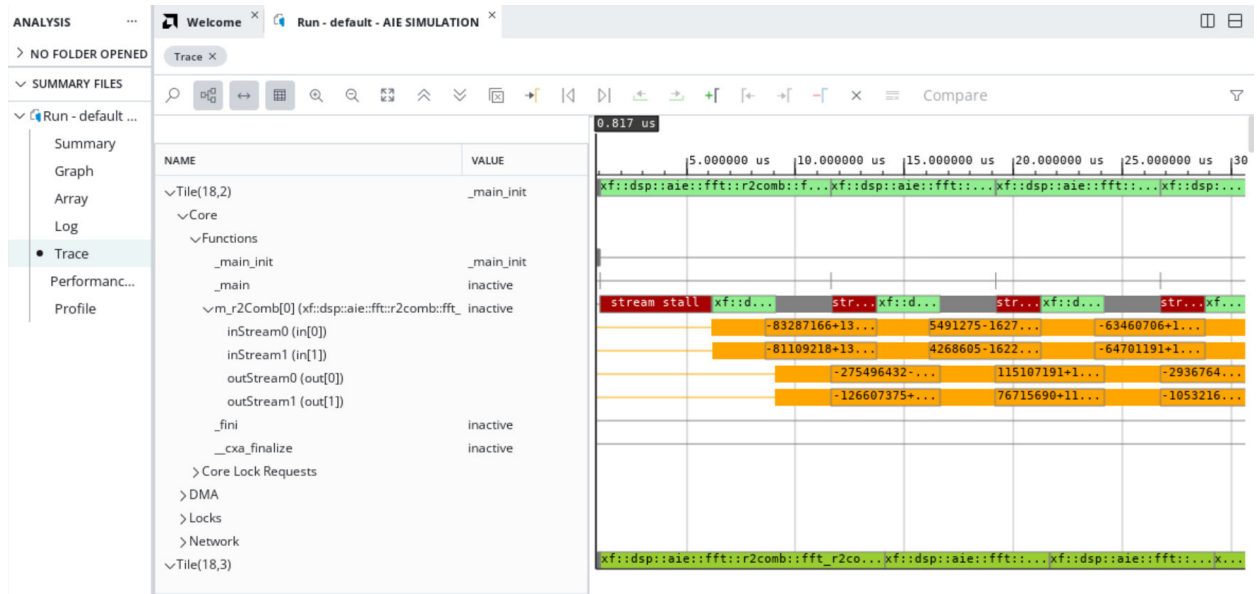
```
aietrace start -graphs <GRAPH_NAME> -graph-based-aie-tile-metrics
all:all:all_stalls -work-dir <WORK_DIR> -link-summary
<*.xsa.link_summary>
```

Note: You should be able to use all the options specified in [XSDB Flow](#). However, the options `-base-address` and `-depth` are not applicable for HSDP offload. XSDB issues an error if you try to use these options.

- Run the application on the hardware. Make sure `xrt.ini` file does not contain any AI Engine trace configurations which conflicts with the configuration given by XSDB.
- Stop AI Engine trace using the command `aietrace stop`. You should see the file `hstdp_event_trace0.txt` corresponding to the AI Engine trace and a JSON file that contains event configuration.
- Now, the dumped trace should be processed using the `hw_analyze` command to generate the `.wdb` file which can be used in Vitis IDE.

```
hwanalyze --pkg-dir=<WORK_DIR> --trace=hstdp_event_trace_aie.txt --
trace-config=<PATH_TO_JSON> -wdb
```

- Launch the Vitis IDE and open the `.wdb` file to analyze the event trace.
- The trace information looks as shown below.



Note: AI Engine trace offload via HSDP is also supported on BDC platforms `vck_190_bdc`, as described in [Vitis Base Platform for the VCK190 Board](#), and `vek_280_bdc`, as described in [Vitis Base Platform for the VEK280 Pre-production Board](#).

Limitations

- The AI Engine trace offload via HSDP is supported only in Hardware and not for Hardware emulation.
- Supported only for non-DFX platform.
- Supports only when the `-event-trace-port` is set to `PLIO` and not `GMIO`.
- XRT-based trace offload is not supported.

Troubleshooting Event Trace in Hardware

Table 69: Troubleshooting Event Trace in Hardware

Issue	Resolution
Trace packets are being dropped on certain streams. (This can be determined by observing black bars on event trace streams in the Vitis IDE.)	Option 1: Increase the number of trace streams coming out of AI Engine using the option below in build time. <pre>v++ -c --mode aie --aie.num-trace-streams=<N></pre> Option 2: If using PLIO event trace streams, use the <code>--aie.trace-plio-width=128</code> option for a 128-bit PLIO stream width. This ensures the PLIO stream width matches the AI Engine data rate. Option 3: Reduce the number of tiles connected per trace stream. You can configure the AI Engine event trace metric to be applied on specific graphs or tiles. More details on these options can be found in Table 66: XSDB Trace Options and Table 67: XRT Trace Options. Option 4: <code>aiecompiler</code> allocates trace streams for every tile in a horizontal fashion. To ensure optimal trace nets routing it is recommended to use location constraints to ensure AI Engines are placed vertically wherever possible so that all active AI Engines equally distributed to trace streams.
Kernel function names are not visible from trace.	This could be because the kernel is inlined either by the compiler or via an attribute. Apply <code>--xlopt=0</code> to compile the design or specify kernel functions with <code>__attribute__((noinline))</code> attribute to disable kernel in-lining.
Kernels start time are off by more than one hundred cycles.	Apply <code>--broadcast-enable-core=true</code> option to the compiler to ensure the design starts all kernels in a few clock cycles range.

Table 69: Troubleshooting Event Trace in Hardware (cont'd)

Issue	Resolution
Tool issues a warning message indicating trace buffer is full.	Increase event trace buffer size. For the XSDB flow, specify larger value with the <code>-depth</code> option in the AI Engine trace start command. <pre>%xsdb aietrace start -graph-based-aie-tile-metrics dut:all:all_stalls - work-dir ./Work -link-summary ./bf_hw.xsa.xclbin.link_summary -base- address 0x900000000 -depth 0x8000000</pre> For the XRT flow, update <code>xrt.ini</code> file and <code>aie_trace_buffer_size</code> line. <pre>[Debug] aie_trace=true [AIE_trace_settings] buffer_size=100M graph-based-aie-tile-metrics = dut:all:all_stalls</pre> Note: The following warning message is issued by both XSDB and XRT when the trace buffer is full. Warning message: <pre>AI Engine Trace Buffer size is full, Device trace could be incomplete.</pre> Alternatively, you can also delay the event trace start based on time, iteration or user-defined event using the trace option <code>-start-type</code> under Table 66: XSDB Trace Options and Table 67: XRT Trace Options topics.
Need to know if the PL is sending or receiving data to/from the AI Engine as expected.	Add monitors on PL kernels and their memory-mapped AXI4 masters. For more information on adding Acceleration Monitors and AXI Performance Monitors, see Enabling Trace in Your Application in the <i>Embedded Design Development Using Vitis (UG1701)</i>.

System Level Trace

In order to simplify system level debug when running applications on the board, the System Level Trace provides a single view for all the events from AI Engine, Programmable Logic, and Processing System. Vitis Analyzer provides such a view with all the tools available to navigate across all events.

Event trace must be enabled for AI Engine and PL so that all events can be captured and displayed on the same time line. PS trace is also enabled at when running the application on hardware.

AI Engine

As in the previous sections, event trace must be set to `runtime` during compilation. This can be done using a configuration file enabled in the `v++` command line:

```
v++ -c --mode aie --include "./src" --include "./src/kernels" --
aie.workdir ./Work ${GRAPH} --platform ${PLATFORM} --config
aiecompiler_trace_options.cfg
```

The file `aiecompiler_trace_options.cfg` contains the options that will allow event trace through PLIOs or GMIOs (default):

```
[aie]
event-trace=runtime
broadcast-enable-core=true

# For PLIO runtime Trace
event-trace-port=plio
num-trace-streams=16
trace-plio-width=128

xlopt=0
```

```
[aie]
event-trace=runtime
broadcast-enable-core=true

# For GMIO runtime Trace
event-trace-port=gmio

xlopt=0
```

`broadcast-enable-core` is true by default. Options can be set to start tracing at some event or at a specific iteration.

Programmable Logic

PL trace is enable during system linking. This is done in the command line adding the `--profile.data` option:

```
v++ -g -l --platform ${PLATFORM} ${XOS} ${LIBADF} -t hw --save-temps --
verbose --config ${VPP_SPEC} --profile.data all:all:all -o XCLBIN_File
```

The `--profile` option can be used to profile many different activities. For more information, see [--profile Options](#).

`--profile.data all:all:all` will monitor data on all kernels and compute units.

Runtime Trace

When the application is run on hardware, using XRT on Linux, the trace is captured using the following `xrt.ini` file specification,.

```
# Debug group for the aie, ps and pl
[Debug]
aie_trace = true
device_trace=fine
host_trace=true

# PL Trace buffer
trace_buffer_size = 32M
trace_buffer_offload_interval_ms = 5

[AIE_trace_settings]
# PLIO
tile_based_aie_tile_metrics = all:all:functions_all_stalls

# Global system timeline
enable_system_timeline = true

[Runtime]
verbosity = 10
```

`enable_system_timeline` is true by default.

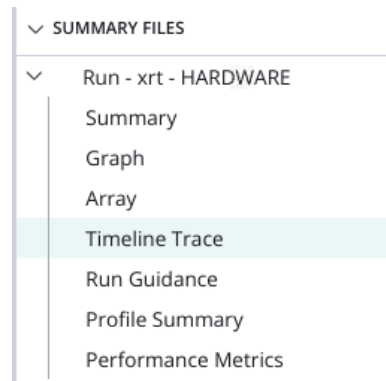
More details on the `xrt.ini` file for PL and PS Trace, see [xrt.ini File](#).

Vitis Unified IDE

After the application is run on the board, a number of files are generated containing trace information for the different domains of the system: AI Engine, PS and PL. Here is an example of files generated by on-board trace:

```
device_trace_0.csv
2-device_trace_0.csv
3-device_trace_0.csv
aie_event_runtime_config.json
aie_event_timestamps.bin
aie_trace_0_0.txt
aie_trace_0_1.txt
aie_trace_0_2.txt
...
aie_trace_0_13.txt
aie_trace_0_14.txt
aie_trace_0_15.txt
native_trace.csv
summary.csv
xrt.run_summary
```

Vitis Analyzer must be run with `xrt.run_summary` as a parameter to visualize the entire system in the Timeline view:

Figure 76: Left Menu in Vitis IDE

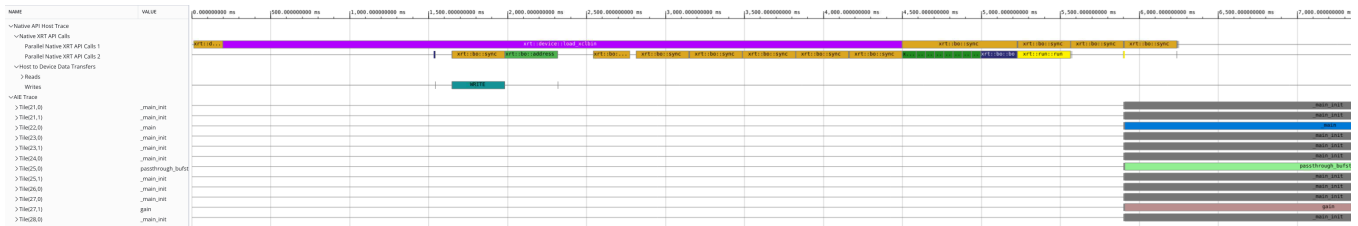
On the left side of the Timeline view there is the list of all traces that have been captured during application execution in hardware:

Figure 77: System Trace

NAME	VALUE
Native API Host Trace	
Native XRT API Calls	
Parallel Native XRT API Calls 1	xrt::device::device
Parallel Native XRT API Calls 2	
Host to Device Data Transfers	
Reads	
Writes	
AIE Trace	
Tile(21,0)	
Tile(21,1)	
Tile(22,0)	
Tile(23,0)	
Tile(23,1)	
Tile(24,0)	
Tile(25,0)	
Tile(25,1)	
Tile(26,0)	
Tile(27,0)	
Tile(27,1)	
Tile(28,0)	
edge	
aie_pl_hw.xclbin	
Compute Unit gen2s_16_128_1	
Compute Unit gen2s_16_128_2	
Compute Unit gen2s_16_128_3	
Compute Unit gen2s_16_128_4	
Compute Unit s2mm_16_128_1	
Compute Unit s2mm_16_128_2	
Compute Unit s2mm_16_128_3	
Compute Unit s2mm_16_128_4	

To view the details in this timeline view zooming is required:

Figure 78: System Timeline Overview



Zooming in the XRT API rows will allow you to view the PL kernel launch and AI Engine RTP update:

Figure 79: Programmable Logic Kernel Launch

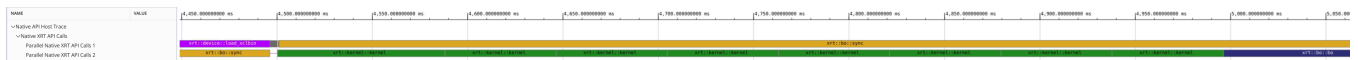
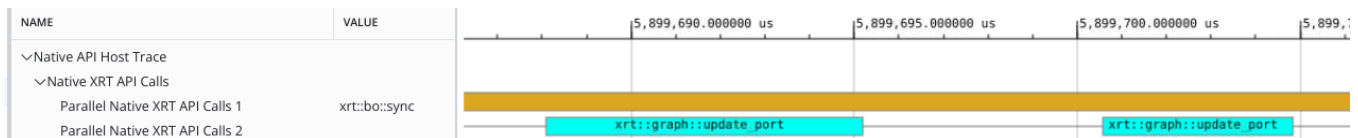
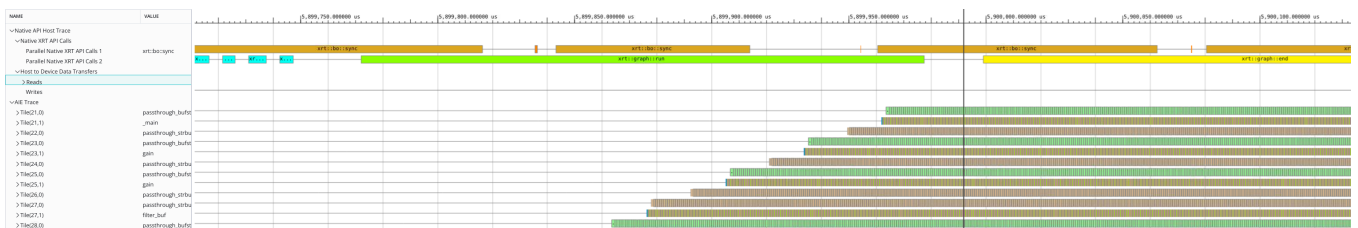


Figure 80: AI Engine RTP Update



The Graph Run and End can be seen on the graph, as well as, how the API correlates with effective AI Engine graph execution:

Figure 81: Graph Run and Graph End API



Viewing Guidance in the Vitis IDE

Viewing design guidance helps to evaluate design performance and highlights areas of the design where performance tuning is recommended. In the Vitis IDE, you can view the design guidance generated by tool as follow:

1. Open the run summary file, `xrt.run_summary`, with the following command:

```
vitis -a xrt.run_summary
```

2. In the IDE, click **Run Guidance** in the summary view to view guidance information.

NAME	THRESHOLD	ACTUAL	DETAILS	RESOLUTION
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [32, 5] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [19, 4] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_UTILIZATION	> 50,000	0.088	AI Engine tile [19, 4] on device edge was utilized 0.088% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [27, 6] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [27, 6] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [16, 1] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_MEMORY_CONFLICTS	<= 0	0.000	AI Engine tile [16, 1] on device edge had memory conflicts for 0.000% of the time.	Reduce memory conflicts in AI Engine tile. Click here.
AIE_UTILIZATION	> 50,000	0.088	AI Engine tile [16, 1] on device edge was utilized 0.088% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [18, 0] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_MEMORY_CONFLICTS	<= 0	0.002	AI Engine tile [18, 0] on device edge had memory conflicts for 0.002% of the time.	Reduce memory conflicts in AI Engine tile. Click here.
AIE_UTILIZATION	> 50,000	0.088	AI Engine tile [18, 0] on device edge was utilized 0.088% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [26, 2] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [26, 2] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [29, 5] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [29, 5] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [31, 1] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.

3. From Run Guidance, there are recommended predefined threshold values and actual result from the design under reviewed.
4. Click on the guidance name for better description of the guidance rule. For example, **AIE_MEMORY_CONFLICTS** has the description of The rule checks the amount of memory conflicts for AI Engine tiles.
5. The predefined values can be adjusted by clicking on the predefined value shown in blue and update with preferred value. This filters out rules in the guidance view for the design.
6. Click on percentage value of the guidance rule to open the Profile Summary of the design.

NAME	THRESHOLD	ACTUAL	DETAILS	RESOLUTION
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [32, 0] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_MEMORY_CONFLICTS	<= 0	0.002	AI Engine tile [32, 0] on device edge had memory conflicts for 0.002% of the time.	Reduce memory conflicts in AI Engine tile. Click here.
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [32, 0] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.

AI Engine Performance Analysis Documentation									
NAME	ADD TO CHART	ACTIVE TIME (NS)	STALL TIME (NS)	CUMULATIVE INSTRUCTIONS	VECTOR INSTRUCTIONS	ACTIVE UTILIZATION TIME (NS)	VECTOR INSTRUCTIONS TIME (NS)	MEMORY CONFLICT TIME (NS)	CUMULATIVE MEMORY ERRORS TIME (NS)
Tile [31,6]	☐	6.624	0.864	5240017	5120000	5.760	5.120	0.000	0.000
Tile [31,7]	☐	6.624	0.864	5240017	5120000	5.760	5.120	0.000	0.000
Tile [32,0]	☒	6.624	0.764	3960019	5120000	5.840	5.120	0.171	0.000
Tile [32,1]	☐	6.624	0.844	3960017	5120000	5.760	5.120	0.000	0.000

7. Click on highlighted hyperlink, as shown for guidance resolution from the tool, to help improve design performance.

Tile: [19, 1] (2)				
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [19, 1] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_UTILIZATION	> 50,000	0.088	AI Engine tile [19, 1] on device edge was utilized 0.088% of the time.	
Tile: [16, 4] (2)				
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [16, 4] on device edge used vector instructions for 0.067% of the time.	
AIE_UTILIZATION	> 50,000	0.088	AI Engine tile [16, 4] on device edge was utilized 0.088% of the time.	
Tile: [32, 0] (3)				
AIE_VECTOR_INSTRUCTIONS	> 1,000	0.067	AI Engine tile [32, 0] on device edge used vector instructions for 0.067% of the time.	Use more vector instructions to improve performance of AI engines. Click here.
AIE_MEMORY_CONFLICTS	<= 0	0.002	AI Engine tile [32, 0] on device edge had memory conflicts for 0.002% of the time.	Reduce memory conflicts in AI Engine tile. Click here.
AIE_UTILIZATION	> 50,000	0.087	AI Engine tile [32, 0] on device edge was utilized 0.087% of the time.	Ensure AI Engine tile is adequately utilized. Click here.

Resolution

Reducing memory conflicts can improve AI Engine performance. For detailed explanation and recommendation: [AIE_MEMORY_CONFLICTS](#)

Memory Tile Event Trace for AI Engine-ML-based Devices

Trace data can be captured from Memory Tiles using Xilinx Runtime (XRT) and Xilinx Software Debugger (XSDB) the same way it can be done for AI Engine Tiles, AI Engine Memory, and AI Engine Interface. These options are specific to the AI Engine-ML.

The channels metric set focuses on the start and end of buffer descriptors while the other metric sets add stalls to trace.

Table 70: Memory Tile Event Trace

Event Type / Metric Set	input_channels	input_channels_stalls	output_channels	output_channels_stalls
S2MM channel buffer descriptors transfers	Yes 2x	Yes	-	-
S2MM channel lock stalls	-	Yes	-	-
S2MM channel stream starvation	-	Yes	-	-
S2MM channel memory backpressure	-	Yes	-	-
MM2S channel buffer descriptors transfers	-	-	Yes 2x	Yes
MM2S channel lock stalls	-	-	-	Yes
MM2S channel stream backpressure	-	-	-	Yes
MM2S channel memory starvation	-	-	-	Yes

Table 71: XSDB Trace Options

Option	Description
-tile-based-memory-tile-metrics	Sets which AI Engine-ML Memory Tile event to store for specified tiles.

Table 71: XSDB Trace Options (cont'd)

Option	Description
-graph-based-memory-tile-metrics	Sets which AI Engine-ML Memory Tile event to store for specified buffers/graph.

Here are some examples of event tracing for Memory Tiles using a XSDB flow:

```
# Sets event trace to "input_channels" for the buffer named buffer2 of
graph1 (channels 1 and 2) and "input_channels_stalls" for buffer4 in graph3
for channels 3 and 4.
-graph-based-memory-tile-metrics "graph1:buffer2:input_channels:1:2;
graph3:buffer4:input_channels_stalls:3:4"

# Sets event trace to "output_channels" for columns 1 to 4 only for channel
1 and 2, and "output_channels_stalls" for column 5 and channels 3 and 4.
-tile-based-memory-tile-metrics "1:4:output_channels:1:2;
5:output_channels_stalls:3:4"
```

Table 72: XRT Flow Trace Options for Memory Tiles

Option	Description
tile_based_memory_tile_metrics = <column all>:<off input_channels input_channels_stalls output_channels_stalls>[:<channel 1>][:<channel 2>]	Configures the AI Engine Memory Tile event trace metric to be applied for a single column or all columns.
tile_based_memory_tile_metrics = <mincolumn><maxcolumn>:<off input_channels input_channels_stalls output_channels output_channels_stalls>[:<channel 1>][:<channel 2>]	Configures the AI Engine Memory Tile event trace metric to be applied for all columns in a range.
graph_based_memory_tile_metrics = <graph name all>:<buffer name all>:<off input_channels input_channels_stalls output_channels output_channels_stalls>[:<channel 1>][:<channel 2>]	Configures the AI Engine Memory Tile event trace metric to be applied for one or all buffers in a specific or all graphs.

Following are some xrt.ini file examples:

```
# Sets event trace to "input_channels" for the buffer named buffer2 of
graph1 (channels 1 and 2) and "input_channels_stalls" for buffer4 in graph3
for channels 3 and 4.
graph_based_memory_tile_metrics=graph1:buffer2:input_channels:1:2;
graph3:buffer4:input_channels_stalls:3:4

# Sets event trace to "output_channels" for columns 1 to 4 only for channel
1 and 2, and "output_channels_stalls" for column 5 and channels 3 and 4.
tile_based_memory_tile_metrics=1:4:output_channels:1:2;
5:output_channels_stalls:3:4
```

DMA Event Trace in AI Engine-ML Tiles

The DMA transfer metric `all_dma` enables tracing DMA activity from memory tiles on AI Engine. This metric adds functions and activity events for all four DMA channels to trace, while the `all_stalls_dma` metric set includes stall events to trace.

These metrics provide information on each channel, such as the amount of data being transferred. In the `xrt.ini` file, metric sets are supported for AI Engine tiles only.

Table 73: DMA Transfer Metric Set

Event Type / Metric Set	<code>all_dma</code>	<code>all_stalls_dma</code>
Functions	Captured	Captured
Stalls (stream/lock/cascade/memory)	NA	Captured
Running activity of all four DMA Channels	Captured	Captured

Note: Stalls are grouped in case of `all_stalls_dma` metric, so stall type is not shown.

Table 74: XRT Flow Trace Options for AI Engine Tiles

Option	Description
<code>tile_based_aie_tile_metrics = {<mincolumn>,<minrow>}: {<maxcolumn>,<maxrow>}:<all_dma all_stalls_dma></code>	Configures the AI Engine tile DMA event trace metric to be applied for a range of tiles.
<code>tile_based_aie_tile_metrics = <{<column>,<row>} all>:<all_dma all_stalls_dma></code>	Configures the AI Engine tile DMA event trace metric to be applied for single or all tiles.
<code>graph_based_aie_tile_metrics = <graph name all>:<kernel_name all>:<off all_dma all_stalls_dma></code>	Configures the AI Engine tile DMA event trace metric to be applied for all DMA in a specific or all graphs.

Following are `xrt.ini` file examples

```
#Sets event trace to "all_stalls_dma" for the kernel2 of graph1 and
"all_dma" for all kernels in graph3.
graph_based_aie_tile_metrics=graph1:kernel2:all_stalls_dma;
graph3:all:all_dma
```

```
#Sets event trace to "all_dma" for column 4 and row 1 only.
tile_based_aie_tile_metrics={4,1}:all_dma
```

Table 75: XSDB Flow Trace Options for AI-Engine Tiles

Option	Description
<code>-tile-based-aie-tile-metrics</code>	Sets which AIE-ML tile event to store for specified tiles.
<code>-graph-based-aie-tile-metrics</code>	Sets which AI Engine-ML tile event to store for specified buffers/graph.

Following are examples of event tracing for AI Engine tiles using a XSDB flow:

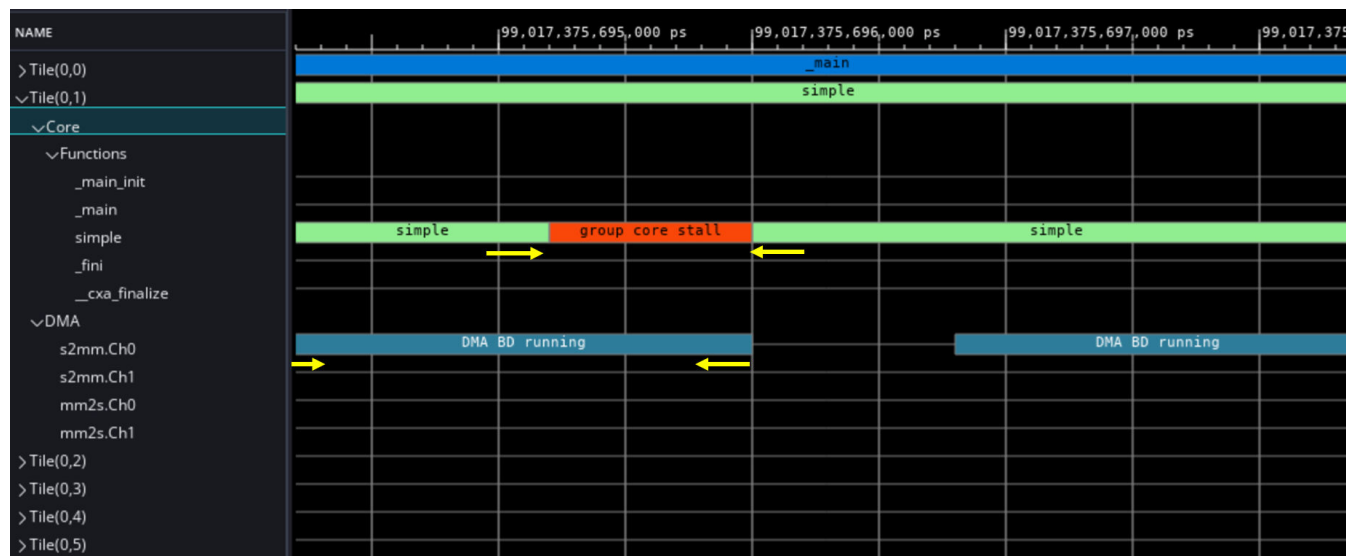
```
# Sets event trace to "all_stalls_dma" for the kernel2 of graph1 and
"all_dma" for all kernels in graph3.
-graph-based-aie-tile-metrics "graph1:kernel2:all_stalls_dma;
graph3:all:all_dma"
```

```
#Sets event trace to "all_dma" for column 4 and row 1 only.
-tile-based-aie-tile-metrics "4:1:all_dma"
```

Figure 82: Trace using the all_dma Metric



Figure 83: Trace using the all_stalls_dma Metric



Programming the PS Host Application

In [Creating a Data Flow Graph \(Including Kernels\)](#) in *AI Engine Kernel and Graph Programming Guide (UG1079)*, the discussion is centered around a very simple AI Engine graph application. The top-level application initialized the graph, ran the graph, and ended the graph. However, for actual AI Engine graph applications the host code must do much more than those simple tasks. The top-level PS application running on the Cortex®-A72, controls the graph and PL kernels: manage data inputs to the graph, handle data outputs from the graph, and control any PL kernels working with the graph.

In addition, AI Engine graph applications can be run on Linux operating systems or on bare-metal systems. The requirements for programming in these two systems are significantly different as outlined in the following topics. AMD provides drivers used by the API calls in the host program to control the graph and PL kernels based on the operating system. In Linux this is provided by the XRT API, in bare metal the AI Engine kernels are controlled using the graph APIs, and PL kernels are controlled using `libUIO` driver calls.

Preventing Multiple Graph Executions

In cases where your graph is implemented in a PS-based host application, you must define a conditional pragma (`#ifdef`) for your `graph.cpp` code to ensure the graph is only initialized once, or run only once. The following example code is the simple application defined in [Creating a Data Flow Graph \(Including Kernels\)](#) in *AI Engine Kernel and Graph Programming Guide (UG1079)* with the additional guard macro `__AIESIM__` and `__X86SIM__`.

```
#include "project.h"

simpleGraph mygraph;

#ifdef __AIESIM__ || defined(__X86SIM__)

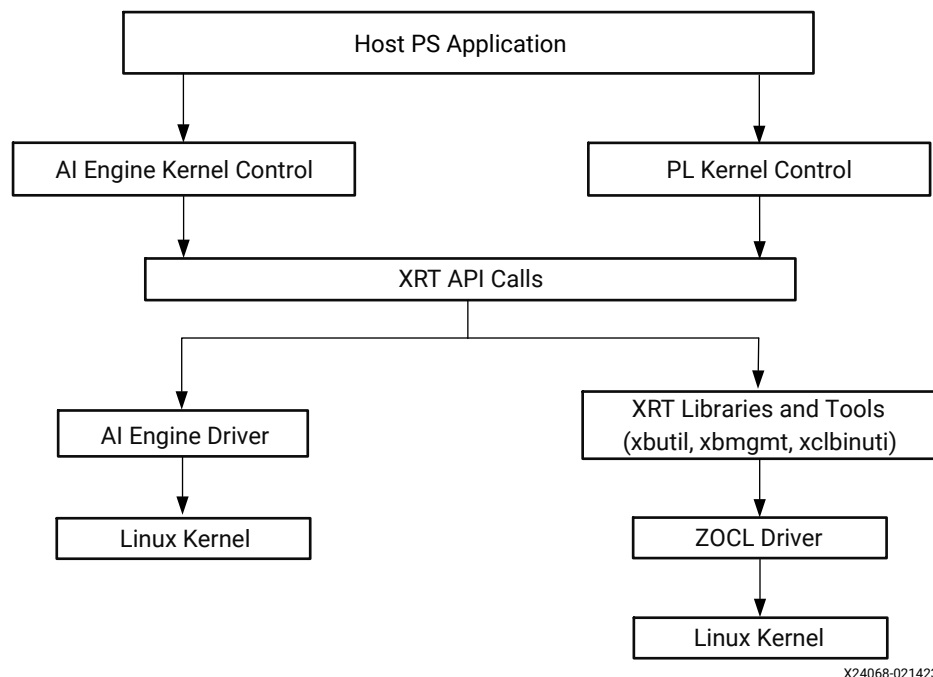
int main(void) {
    mygraph.init();
    mygraph.run(<number_of_iterations>);
    mygraph.end();
    return 0;
}
#endif
```

This conditional directive compiles the application only for use with the AI Engine simulator. It prevents the graph from being initialized or run multiple times, from both the graph and PS host application. The directive lets the `graph.cpp` run in the AI Engine simulator and the x86 simulator. However when running in hardware and hardware emulation, the graph is initialized and run from the PS application, rather than from the `graph.cpp`.

Host Programming on Linux

In Linux operating systems, the XRT API controls the AI Engine graph and PL kernels. The following figure shows the APIs and drivers required in this system.

Figure 84: AI Engine XRT Software Stack



X24068-021423

Controlling the Application with the XRT C++ API

XRT provides C and C++ APIs to control PL kernels and AI Engine graphs.

The execution model for the XRT API controlling PL kernels and AI Engine graphs is as follows:

1. Open the device, and load XCLBIN. Get the UUID as needed.
2. Allocate buffer objects and map-to-host memory. Process and transfer data from the host memory to the device memory.
3. Get PL kernel handles, set arguments for kernels, and launch kernels.

4. Get AI Engine graphs, and run graphs.
5. Wait for the completion of the graphs.
6. Wait for the completion of the kernels.
7. Transfer data from the global memory in the device back to the host memory.
8. The host code continues processing using the new data in the host memory.

Note: There are two ways to start the AI Engine graph. The AI Engine graph can be auto-started when the board is booted and runs forever after download. The package setting `defer_aie_run` determines this behavior. PL kernels, and AI Engine graphs can also be started in the host application and the application code can determine whether to wait for a specific kernel's completion or a specific graph's completion. This behavior can vary depending on the design.

XRT provides class `graph` in the name space `xrt` and its member functions to control the AI Engine graph.

Example code to control the AI Engine graph and PL kernels using the XRT C++ API is as follows:

```
// Including the xrt header files below is mandatory
#include "xrt/xrt_graph.h"
#include "xrt/xrt_kernel.h"

size_t output_size_in_bytes = OUTPUT_SIZE * sizeof(int);

// Open xclbin
auto device = xrt::device(0); //device index=0
//load the xclbin application which may contain PL kernels and AI Engine
graphs
auto uuid = device.load_xclbin(xclbinFilename);

// PL control
// Get the handle to s2mm & random_noise PL kernel
auto s2mm = xrt::kernel(device, uuid, "s2mm");
auto random_noise = xrt::kernel(device, uuid, "random_noise");
// allocate output memory for data from s2mm kernel
auto out_bo = xrt::bo(device, output_size_in_bytes, s2mm.group_id(0));
auto host_out=out_bo.map<std::complex<short>*>();
//run the s2mm and random_noise PL kernels
auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE); //start s2mm
auto random_noise_run = random_noise(nullptr, OUTPUT_SIZE);

//AI Engine Graph Control
//Initialize run time parameter data
int coeffs_readback[12];
int narrow_filter[12] = {180, 89, -80, -391, -720, -834, -478, 505, 2063,
3896, 5535, 6504};
int wide_filter[12] = {-21, -249, 319, -78, -511, 977, -610, -844, 2574,
-2754, -1066, 18539};
//get the handle to the graph called "gr"
auto ghdl=xrt::graph(device,uuid,"gr");
// update run time parameter in the graph
ghdl.update("gr.fir24.in[1]",narrow_filter);
//run the graph for 16 iterations
ghdl.run(16);
// wait for graph to complete running 16 iterations
ghdl.wait();
```

```
//read value from a run time parameter
ghdl.read("gr.fir24.inout[0]",coeffs_readback);//Read after graph::wait.
RTP update effective
// update run time parameter in the graph
ghdl.update("gr.fir24.in[1]",wide_filter);
//run the graph for 16 iterations
ghdl.run(16);
ghdl.read("gr.fir24.inout[0]", coeffs_readback);//Async read
ghdl.wait();

// wait for the s2mm PL kernel to be done
auto state = s2mm_run.wait();
std::cout << "s2mm completed with status(" << state << ")\n";
out_bo.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

//Post-processing...
...
```

Controlling Data Transfers between AI Engine and Global Memory

The AI Engine graph allows memory-mapped connections to and from the global memory. The global memory can be either High Bandwidth Memory (HBM) on the device or DDR memory external to the device. Linear access to data between AI Engine and global memory through memory-mapped connections is possible through GMIO objects. To learn how to use GMIO in the AI Engine graph code, refer to [Configuring input_gmio/output_gmio](#) in the *AI Engine Kernel and Graph Programming Guide* (UG1079).

An alternative to GMIO objects is supported in AI Engine-ML devices. The alternative to GMIO objects for memory-mapped connections between global memory AI Engine, in the graph, is external buffer objects. Unlike GMIOs, which access data in a linear manner, external buffers can use advanced and intricate data access patterns, using tiling parameters, in the graph. For more information on external buffer usage, refer to [AI Engine-ML External Memory Access](#) in the *AI Engine-ML Kernel and Graph Programming Guide* (UG1603).

The host code to control data transfers remains the same whether using GMIOs or external buffers to access data in global memory. Both methods support synchronous and asynchronous data transfer. For asynchronous data transfer, the `async` API of the `xrt::aie::bo` object manages the transfer and requires the GMIO or external buffer object name to be specified as the first parameter.

Following example code shows an asynchronous buffer transfer. This can be performed using either GMIO or external buffers with XRT API:

```
char* xclbinFilename = argv[1];
// Open xclbin
auto device = xrt::device(0); //device index=0
auto uuid = device.load_xclbin(xclbinFilename);

//Only non-cacheable buffer is supported
auto din_buffer = xrt::aie::bo(device, BLOCK_SIZE_in_Bytes,
xrt::bo::flags::normal, /*memory group*/0);
int* dinArray= din_buffer.map<int*>();
```

```

//Only non-cacheable buffer is supported
auto dout_buffer = xrt::aie::bo (device, BLOCK_SIZE_in_Bytes,
xrt::bo::flags::normal, /*memory group*/0);
int* doutArray= dout_buffer.map<int*>();

int ret=0;
int error=0;

//Initialization
for(int i=0;i<ITERATION*1024/4;i++){
    dinArray[i]=i;
}

// Parameter "gr.gmPortIn" can be the name of a GMIO/external buffer object
din_buffer.async( "gr.gmPortIn",XCL_BO_SYNC_BO_GMIO_TO_AIE,BLOCK_SIZE_in_Bytes,
/*offset*/0);

auto ghdl=xrt::graph(device,uuid, "gr");
ghdl.run(ITERATION);

// Parameter "gr.gmPortOut" can be the name of a GMIO/external buffer object
auto
out_buffer_run=dout_buffer.async( "gr.gmPortOut",XCL_BO_SYNC_BO_AIE_TO_GMIO,B
LOCK_SIZE_in_Bytes,/*offset*/0);

ghdl.wait();//Wait for graph to complete
dout_buffer_run.wait();//Wait for gmioOut to complete

// Post-processing
...

```

Note: Only non-cacheable buffers are supported for AI Engine GMIO and external buffer buffers.

XRT API introduces the `xrt::aie::buffer` object to represent the buffer of the AI Engine. The following code uses this object to perform asynchronous buffer transactions similar to the code above:

```

//Only non-cacheable buffer is supported
auto din_buffer = xrt::aie::bo (device, BLOCK_SIZE_in_Bytes,
xrt::bo::flags::normal, /*memory group*/0);

//Only non-cacheable buffer is supported
auto dout_buffer = xrt::aie::bo (device, BLOCK_SIZE_in_Bytes,
xrt::bo::flags::normal, /*memory group*/0);

//"gr.gmioIn" is the name of the buffer which represents GMIO/External Buffer
xrt::aie::buffer bufIn(device, uuid, "gr.gmioIn");
bufIn.async(din_buffer, XCL_BO_SYNC_BO_GMIO_TO_AIE, BLOCK_SIZE_in_Bytes, 0);

xrt::aie::buffer bufOut(device, uuid, "gr.gmioOut");
bufOut.async(dout_buffer, XCL_BO_SYNC_BO_AIE_TO_GMIO, BLOCK_SIZE_in_Bytes, 0);

bufOut.wait();

```

For more details on the buffer object API support, refer to the [xrt_aie.h](#) file in the XRT Repository.

Controlling AI Engine Partitions

There can be multiple partitions in an AI Engine array. Each of these AI Engine partitions contain independent graphs. The `xclbin` file generated by the `v++ packager` includes the metadata for all partitions. For information about the `v++` building flow, see [Compiling AI Engine Graph for Independent Partitions](#).

A handle to a graph in a partition can be retrieved from the device after loading the `xclbin`. You can use this handle to control the graph. To do so, the graph name is prefixed with the partition name in the `xrt::graph(device, uuid, "pr0-gr")` API. Following is example code used to control the `pr0` and `pr1` partitions:

```
// Open xclbin
auto device = xrt::device(0); //device index=0
auto uuid = device.load_xclbin(xclbinFilename);
//Create handle for graph "gr" in partition "pr0"
auto ghdl1=xrt::graph(device,uuid, "pr0-gr");
ghdl1.run(iterations);
//Create handle for graph "gr" in partition "pr1"
auto ghdl2=xrt::graph(device,uuid, "pr1-gr");
ghdl2.run(iterations);

ghdl1.wait();
ghdl2.wait();
```

Multi-Process and Multi-Thread Control of AI Engine Graphs

Multi-Process Access Control

Xilinx Runtime (XRT) APIs provide multi-process support for controlling the AI Engine array and graphs. It supports three operating modes on AI Engine array and graphs.

- **Exclusive Mode:** Provides full access to the AI Engine array or graph. No other process can access it.
- **Primary Mode:** Provides full access to the AI Engine array or graph. Other processes can gain non-destructive access to the AI Engine array or graph. Non-destructive access includes any AIE control functions that do not change the status of the AI Engine, such as asynchronous RTP read.
- **Shared Mode:** Provides only non-destructive access to the AI Engine array or graph. Non-destructive access includes any AIE control functions that do not change the status of the AI Engine, such as asynchronous RTP read.

In summary, the three modes offer different levels of access control for multiple processes:

- **Exclusive Mode:** Single process with full control.

- **Primary Mode:** Single process with full control, other processes can read without modifying the state.
- **Shared Mode:** Multiple processes can read, but no modifications are allowed.

The XRT C++ API extends the `xrt::aie::device` class to open device with access mode in `xrt/xrt_aie.h`:

- `xrt::aie::access_mode::exclusive` (Exclusive mode)
- `xrt::aie::access_mode::primary` (Primary mode)
- `xrt::aie::access_mode::shared` (Shared mode)

The XRT C++ API extends the `xrt::graph` class to open graph with access mode in `xrt/xrt_graph.h`:

- `xrt::graph::access_mode::exclusive` (Exclusive mode)
- `xrt::graph::access_mode::primary` (Primary mode)
- `xrt::graph::access_mode::shared` (Shared mode)

Graphs must be closed before closing the array. After all open AI Engine arrays and graphs are closed, they can be reopened, but some restrictions apply as listed below.

The following restrictions apply for the AI Engine array multi-process support.

- Only one process can open AI Engine array in exclusive mode. After opening in exclusive mode, the AI Engine cannot be opened again in any mode by the same or other processes.
- Only one process can open AI Engine array in the primary mode. After opening in primary mode, it cannot be opened again in exclusive mode or primary mode, but it can be opened in shared mode again.

The following restrictions apply for the AI Engine graph multi-process support.

- A graph cannot be opened multiple times within a single process.
- If an AI Engine graph is opened in exclusive mode, it cannot be reopened in any mode again.
- Multiple processes can open a graph in shared mode, but only one process can open a graph in primary mode.

Multi-Thread Access Control



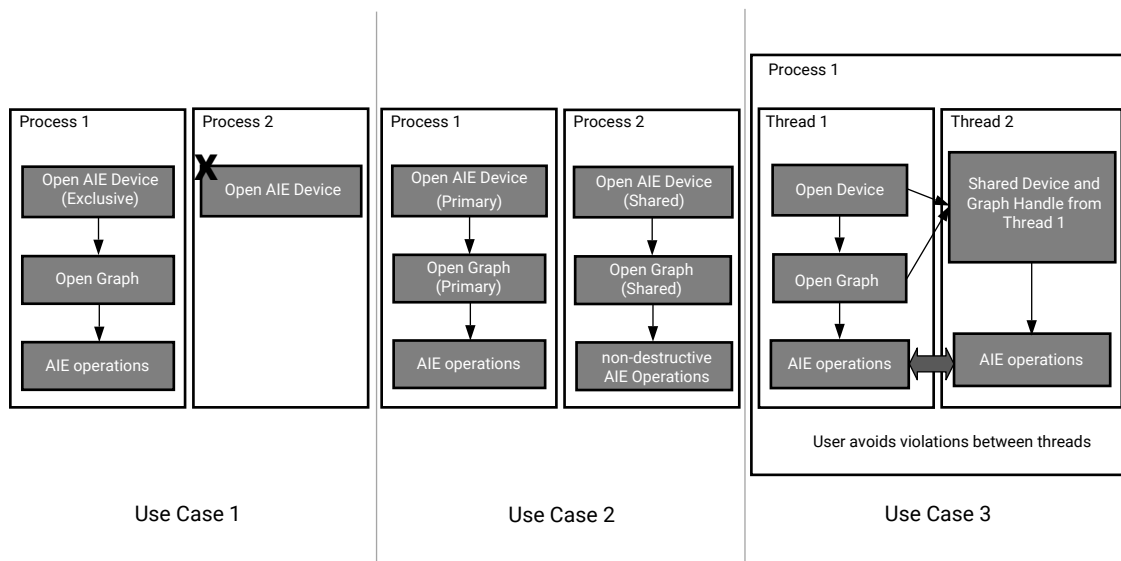
RECOMMENDED: Although the restrictions for multi-process access also apply to multi-threaded access, it is important to remember that AI Engine device handles and graph handles are sharable between threads. This means you can use the same handle between multiple threads. However, the host application is responsible for ensuring proper synchronization of the AI Engine array and graph states between threads. This is particularly crucial when multiple threads have exclusive or primary ownership of the AI Engine array or graph.

It is recommended that multiple threads use the same model as multiple processes. However, because the AI Engine device handle and graph handle are sharable between threads, it is legal to use the same device handle or graph handle between threads. The host application is responsible for synchronizing the AI Engine array state and graph state between threads, especially when multiple threads are the exclusive or primary owner of the AI Engine array or graph.

Use Cases

The following figure shows multiple-process and multiple-thread use cases based on the access mode restrictions:

Figure 85: Multi-Process and Multi-Thread Use Cases



X30072-110524

The figure illustrates how different processes can access the AI Engine devices and graphs.

Use Case 1:

- Process 1 opens `xrt::aie::device` or `xrt::graph` in exclusive mode.
- Process 2 cannot open them because exclusive mode grants full control to Process 1. No other process can access the resource.

Use Case 2:

- Process 1 opens `xrt::aie::device` or `xrt::graph` in primary mode.
- Process 2 can open them in shared mode only.
- In shared mode, Process 2 can only perform non-destructive operations on the AI Engine, such as asynchronous RTP read. Non-destructive operations are those that do not change the state of the AI Engine.

Use Case 3:

- Thread 1 opens `xrt::aie::device` or `xrt::graph` without any mode restrictions.
- Both Thread 1 and Thread 2 can operate on the AI Engine.
- However, you must ensure that thread violations are handled appropriately to avoid conflicts between the threads.

A sample code to demonstrate Use Case 2 is as follows.

```
#include <stdlib.h>
#include <fstream>
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>
#include "adf/adf_api/XRTConfig.h"
#include "experimental/xrt_aie.h"
#include "experimental/xrt_graph.h"
#include "experimental/xrt_kernel.h"

//8192 matches 32 iterations of graph::run
#define OUTPUT_SIZE 8192
int value1[16] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
int value2[16] = {-1,-2,-3,-4,-5,-6,-7,-8,-9,-10,-11,-12,-13,-14,-15,-16};

using namespace adf;

int run(int argc, char* argv[],int id){
    std::cout<< "Child process "<<id<< " start"<<std::endl;

    //TARGET_DEVICE macro needs to be passed from gcc command line
    if(argc != 2) {
        std::cout << "Usage: " << argv[0] << " <xclbin>" << std::endl;
        return EXIT_FAILURE;
    }
    char* xclbinFilename = argv[1];
    std::string graph_name=std::string("gr["+std::to_string(id)+"]");
    std::string rtp_inout_name=std::string("gr["+std::to_string(id)
+std::string(".k.inout[0]");

    int ret;
    int value_readback[16]={0};
    if(fork()==0){ //child child process
        xrt::aie::device device{0, xrt::aie::device::access_mode::shared};
        auto uuid = device.load_xclbin(xclbinFilename);
        xrt::graph graph{device, uuid, graph_name,
xrt::graph::access_mode::shared};

        graph.read(rtp_inout_name, value_readback);
        std::cout<< "Add value read back are:";
        for(int i=0;i<16;i++){
            std::cout<<value_readback[i]<< ",\t";
        }
        std::cout<<std::endl;
        std::cout<< "child child process exit"<<std::endl;
        exit(0);
    }

    xrt::aie::device device{0}; // default primary context
    auto uuid = device.load_xclbin(xclbinFilename);
    xrt::graph graph{device, uuid, graph_name}; // default primary context

    std::string rtp_in_name=std::string("gr["+std::to_string(id)
```

```

+std::string( ".k.in[1]");
graph.update(rtp_in_name, value1);
graph.run(16); // 16 iterations

graph.wait(0); // wait 0 => wait till graph is done
std::cout<< "Graph wait done"<<std::endl;

//second run
graph.update(rtp_in_name.data(), value2);
graph.run(16); // 16 iterations;

while(wait(NULL)>0){//Wait for child child process
}

graph.wait(0); // wait 0 => wait till graph is done
std::cout<< "Child process:"<<id<< " done"<<std::endl;
return 0;
}

int main(int argc, char* argv[])
{
    try {
        for(int i=0;i<GRAPH_NUM;i++){
            if(fork()==0){//child
                auto match = run(argc, argv,i);
                std::cout << "TEST child " <<i<< (match ? " FAILED" : "
PASSED") << "\n";
                return (match ? EXIT_FAILURE : EXIT_SUCCESS);
            }else{
                size_t output_size_in_bytes = OUTPUT_SIZE * sizeof(int);
                //TARGET_DEVICE macro needs to be passed from gcc command
                line
                if(argc != 2) {
                    std::cout << "Usage: " << argv[0] << " <xclbin>" <<
std::endl;
                    return EXIT_FAILURE;
                }
                char* xclbinFilename = argv[1];

                int ret;
                // Open xclbin
                auto device = xrt::device(0); //device index=0
                auto uuid = device.load_xclbin(xclbinFilename);

                // s2mm & data_generator kernel handle
                std::string s2mm_kernel_name=std::string( "s2mm:{s2mm_ "
+std::to_string(i+1)+std::string( "}")");
                xrt::kernel s2mm = xrt::kernel(device, uuid,
s2mm_kernel_name.data());
                std::string
data_generator_kernel_name=std::string( "data_generator:{data_generator_ "
+std::to_string(i+1)+std::string( "}")");
                xrt::kernel data_generator = xrt::kernel(device, uuid,
data_generator_kernel_name.data());

                // output memory
                auto out_bo=xrt::bo(device,
output_size_in_bytes,s2mm.group_id(0));
                auto host_out=out_bo.map<int*>();
                auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE);//1st
run for s2mm has started
                auto data_generator_run = data_generator(nullptr,
OUTPUT_SIZE);

```

```

        // wait for s2mm done
        std::cout<< "Waiting s2mm to complete"<<std::endl;
        auto state = s2mm_run.wait();
        std::cout << "s2mm "<< " completed with status(" << state <<
        ") "<<std::endl;
        out_bo.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

        int match = 0;
        int counter=0;
        for (int i = 0; i < OUTPUT_SIZE/2/16; i++) {
            for(int j=0;j<16;j++){
                if(host_out[i*16+j]!=counter+value1[j]){
                    std::cout<< "ERROR: num="<<i*16+j<< "
out="<<host_out[i*16+j]<<std::endl;
                    match=1;
                    break;
                }
                counter++;
            }
        }
        for(int i=OUTPUT_SIZE/2/16;i<OUTPUT_SIZE/16;i++){
            for(int j=0;j<16;j++){
                if(host_out[i*16+j]!=counter+value2[j]){
                    std::cout<< "ERROR: num="<<i*16+j<< "
out="<<host_out[i*16+j]<<std::endl;
                    match=1;
                    break;
                }
                counter++;
            }
        }

        std::cout << "TEST " <<i<< (match ? " FAILED" : " PASSED")
<< "\n";

        while(wait(NULL)>0){//Wait for all child process
        }
        std::cout<<"all done"<<std::endl;
        return (match ? EXIT_FAILURE : EXIT_SUCCESS);
    }
}
}
catch (std::exception const& e) {
    std::cout << "Exception: " << e.what() << "\n";
    std::cout << "FAILED TEST\n";
    return 1;
}
}

```

AI Engine Error Reporting

XRT provides error reporting APIs and tools. The error can be categorized into two types:

- **Synchronous error:** Errors that can be detected during the XRT run-time function call.
- **Asynchronous error:** Errors from the underneath driver, system, hardware, etc.

A synchronous error handling example:

```
auto ghdl=xrt::graph(device,uuid, "gr");
try{
    ghdl.update( "gr.fir24.in[1]",narrow_filter);
    ghdl.run(16);
    ghdl.read( "gr.fir24.inout[0]", coeffs_readback);//Async read
} catch(std::exception const& e){
    std::cout<< "Graph Execution Error"<<std::endl;
    return 1;
}
```

An asynchronous error might not be related to the current XRT function call or the application that is running. Asynchronous errors are cached in driver subsystems and can be accessed by the user application through the asynchronous error reporting APIs. Cached errors are persistent until explicitly cleared. Persistent errors are not necessarily indicative of the current system state, for example, a board might have been reset and be functioning correctly while previously cached errors are still available. To avoid current state confusion, asynchronous errors have a timestamp attached indicating when the error occurred. The timestamp can be compared to, for example, the timestamp for last `xbutil` reset.

The errors cached by the driver contain a system error code and additional meta data as defined in the [xrt_error_code.h](#) file in the XRT Repository, which is shared between the user space and the kernel space.

The XRT error handling APIs can refer to `experimental/xrt_error.h`. An asynchronous error handling example:

```
xrt::error error(device, XRT_ERROR_CLASS_AIE);
auto errCode = error.get_error_code();
auto timestamp = error.get_timestamp();
auto err_str = error.to_string();
/* code to deal with this specific error */
std::cout<<"Async error: "<< err_str << std::endl;
```

An example asynchronous error output:

```
Error Number (6): AIE_ACCESS
Error Driver (4): DRIVER_AIE
Error Severity (3): SEVERITY_CRITICAL
Error Module (3): MODULE_AIE_CORE
Error Class (2): CLASS_AIE
Timestamp: 1637342412366664740
```

XRT maintains the latest error for each class and an associated timestamp for when the error was generated. The information of error can be interpreted from the [xrt_error_code.h](#) file in the XRT Repository. For example, Error Module (3): `MODULE_AIE_CORE` corresponds to `XRT_ERROR_MODULE_AIE_CORE` in enumeration `xrtErrorModule`.

`xbutil` can be used to report errors. The error report accumulates *all* the errors from the various classes and sorts them by timestamp. The report queries the drivers as to when the last reset was requested.

```
$ xbutil examine -r error -d 0

Asynchronous Errors
Time Class Module Driver Severity Error Code
Fri Nov 19 17:19:42 2021 GMT CLASS_AIE MODULE_AIE_CORE DRIVER_AIE
SEVERITY_CRITICAL AIE_ACCESS

$ xbutil examine -r error -f json -o <OUTPUT_FILE> -d 0
{
  "schema_version": {
    "schema": "JSON",
    "creation_date": "Fri Nov 19 17:58:09 2021 GMT"
  },
  "devices": [
    {
      "interface_type": "pcie",
      "device_id": "0000:00:00.0",
      "asynchronous_errors": [
        {
          "time": {
            "epoch": "1637342382770339700",
            "timestamp": "Fri Nov 19 17:19:42 2021 GMT"
          },
          "class": "CLASS_AIE",
          "module": "MODULE_AIE_CORE",
          "severity": "SEVERITY_CRITICAL",
          "driver": "DRIVER_AIE",
          "error_code": {
            "error_id": "6",
            "error_msg": "AIE_ACCESS"
          }
        }
      ]
    }
  ]
}
```

`xbutil` can also be used to report AI Engine running status and read registers for debug purposes. For example, the following command reads the status of kernels after the graph has executed.

```
$ xbutil examine -r aie -d 0

-----
1/1 [0000:00:00.0] : edge
-----

Aie
Aie_Metadata
GRAPH[ 0] Name : gr
      Status : unknown
      SNo. Core [C:R] Iteration_Memory [C:R] Iteration_Memory_Addresses
[ 0] 23:1 23:1 16388
[ 1] 23:2 23:0 6980
[ 2] 23:3 23:1 4
[ 3] 24:1 24:0 4
```

```

[ 4] 24:2 24:2 4
[ 5] 24:3 24:1 4
[ 6] 25:1 25:1 4

Core [ 0]
Column : 23
Row : 1
Core:
  Status : disabled, core_done
  Program Counter : 0x00000308
  Link Register : 0x00000290
  Stack Pointer : 0x000340a0
DMA:
  MM2S:
    Channel:
      Id : 0
      Channel Status : idle
      Queue Size : 0
      Queue Status : okay
      Current BD : 0

      Id : 1
      Channel Status : idle
      Queue Size : 0
      Queue Status : okay
      Current BD : 0

    S2MM:
      Channel:
        Id : 0
        Channel Status : idle
        Queue Size : 0
        Queue Status : okay
        Current BD : 0

        Id : 1
        Channel Status : idle
        Queue Size : 0
        Queue Status : okay
        Current BD : 0

  Locks:
    0 : released_for_write
    1 : released_for_write
    2 : released_for_write
    3 : released_for_write
    4 : released_for_write
    5 : released_for_write
    6 : released_for_write
    7 : released_for_write
    8 : released_for_write
    9 : released_for_write
    10 : released_for_write
    11 : released_for_write
    12 : released_for_write
    13 : released_for_write
    14 : released_for_write
    15 : released_for_write

  Events:
    core : 1, 2, 5, 22, 23, 24, 28, 29, 31, 32, 35, 36, 38, 39, 40, 44, 45,

```

```
47, 68
memory : 1, 43, 44, 45, 106, 113

.....

Core [ 6]
Column : 25
Row : 1
Core:
  Status : enabled, east_lock_stall
  Program Counter : 0x000001e6
  Link Register : 0x000000b0
  Stack Pointer : 0x00030020
DMA:
  MM2S:
    Channel:
      Id : 0
      Channel Status : stalled_on_requesting_lock
      Queue Size : 0
      Queue Status : okay
      Current BD : 2

      Id : 1
      Channel Status : idle
      Queue Size : 0
      Queue Status : okay
      Current BD : 0

    S2MM:
      Channel:
        Id : 0
        Channel Status : running
        Queue Size : 0
        Queue Status : okay
        Current BD : 0

        Id : 1
        Channel Status : idle
        Queue Size : 0
        Queue Status : okay
        Current BD : 0

  Locks:
    0 : acquired_for_write
    1 : released_for_write
    2 : released_for_write
    3 : released_for_write
    4 : released_for_write
    5 : released_for_write
    6 : released_for_write
    7 : released_for_write
    8 : released_for_write
    9 : released_for_write
    10 : released_for_write
    11 : released_for_write
    12 : released_for_write
    13 : released_for_write
    14 : released_for_write
```



```
15 : released_for_write

Events:
  core : 1, 2, 5, 22, 26, 28, 29, 31, 32, 35, 38, 39, 44
  memory : 1, 20, 21, 23, 35, 43, 44, 106, 113
```

The following command can be used to read specific registers for debug purposes.

```
$ xbutil advanced --read-aie-reg -d 0 0 25 Core_Status
Register Core_Status Value of Row:0 Column:25 is 0x00000201
```

For AI Engine register definitions, see the *Versal Adaptive SoC AI Engine Register Reference* (AM015). For details on `xbutil` command use, see [Xilinx Runtime \(XRT\) Architecture](#). For error analysis in the Vitis IDE, see [Analyzing AI Engine Status](#).

AI Engine Error Events

This section provides error and related debug information for the errors obtained using the XRT error reporting APIs described previously. These are errors propagated from the AI Engine array and can be used to debug application specific errors in hardware.

For errors with class `XRT_ERROR_CLASS_AIE`, as found in the `xrt_error_code.h` file in the XRT Repository, you can obtain additional information by enabling the `dmesg` logs, which provide the causes of the error (and are described in the following tables). An example log is shown here:

```
[18.462615] aie aie0: Asserted tile error event 56 at col 6 row 7
[18.471397] aie aie0: Asserted tile error event 60 at col 25 row 1
```

Note: Note the tile location is indicated by the `col` and `row` number. Row 0 is the AI Engine interface tile. AI Engines start from row 1.

The following tables list the various categories of error, in addition to the exact error number, description, and tips on the next steps to debug and resolve the errors.

Table 76: CORE Module Error Events

Error Group	No.	Name	Description	Debug Tips
Instruction Errors	59	Instruction Decompression Error	Event generated when AI Engine cannot decompress instruction fetched. This can happen if the program instructions are corrupt. Validate ELF generation.	Regenerate the ELF file with the Vitis compiler (V++) <code>--package</code> command. If the issue persists, contact AMD support.

Table 76: CORE Module Error Events (cont'd)

Error Group	No.	Name	Description	Debug Tips
Access Errors	55	PM Reg Access Failure	This error can happen on bank access conflict to PM by the memory mapped AXI interface and AI Engine.	Contact AMD support.
	60	DM address out of range	Event generated if AI Engine tries to access a memory location outside of 0x20000 – 0x3FFFF.	Run AI Engine simulator (<code>aiesimulator</code>) with <code>--enable-memory-check</code> that will flag any access violations. Alternatively run <code>x86simulator</code> with <code>--valgrind</code> that will flag any access violations.
	65	PM address out of range	Event generated if PC is out of range	Run AI Engine simulator (<code>aiesimulator</code>) with <code>--enable-memory-check</code> that will flag any access violations. Alternatively run <code>x86simulator</code> with <code>--valgrind</code> that will flag any access violations.
	66	DM access to unavailable	Event generated if AI Engine issues an access to the isolated tile in neighborhood.	Check if the kernel runs on AI Engine accesses data memory of an isolated tile (a different partition). If the issue persists, contact AMD support.
Bus Errors	58	AXI MM Slave Error	Event generated if the memory mapped AXI interface slave read/write request is for an address which does not exist in the AI Engine tile.	If the PL IP is accessing the AI Engine registers using the memory mapped AXI interface, check the PL IP to see if it access invalid registers. If the issue persists, contact AMD support.
Stream Errors	54	TLAST in WSS words 0-2	Event generated if TLAST is not on the fourth word of a wide stream.	If PL IP is used to generate the stream, check if it generates TLAST correctly. For example, if the kernel reads stream in wide mode (read 128 bits a time), it requires that the tlast is asserted in the last word of the 128 bits. Otherwise, if the tlast is asserted in the 0~2 words of the 128 bits, the event 54 will occur. If the issue persists, contact AMD support.
	56	Stream Pkt Parity Error	Event generated if there is any parity error in the packet header.	Check the data source such as PL IP which generates the packets to see if the packet is valid and if the parity bit is correctly calculated. If the data is from PL IP, check the packet header generated from the PL IP.
	57	Control Pkt Error	Control Packet Error	Check the data source, such as PL IP which generates the packets to see if it generates the packets correctly. If the issue persists, contact AMD support.
ECC Errors	64	PM ECC Error 2 bit	Event generated when 2 bit ECC error is detected	Re-run the application. If the issue persists, contact AMD support.
	62	PM ECC Error Scrub 2 bit	Event generated if ECC scrubber detects 2 Bit ECC error	Re-run the application. If the issue persists, contact AMD support.

Table 76: CORE Module Error Events (cont'd)

Error Group	No.	Name	Description	Debug Tips
Lock Errors	67	Lock Access to unavailable	Event generated if AI Engine issues an access to the isolated tile in neighborhood.	Run AI Engine simulator (<code>aiesimulator</code>) with <code>--enable-memory-check</code> that will flag any access violations. If the issue persists, contact AMD support. Alternatively run <code>x86simulator</code> with <code>--valgrind</code> that will flag any access violations.

Notes:

1. CORE refers to the AI Engine in the AI Engine tile.

Table 77: MEMORY Module Error Events

Errors Group	No.	Name	Description	Debug Tips
ECC Errors	88	DM ECC Error Scrub 2 bit	Event generated when ECC scrubber detects 2-bit ECC error in bank 0 or bank 1 of DM.	Re-run the application. If the issue persists, contact AMD support.
	90	DM ECC Error 2 bit	Event generated when 2-bit ECC error is detected during access to bank 0 or 1 of DM. This data memory ECC error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.

Table 77: MEMORY Module Error Events (cont'd)

Errors Group	No.	Name	Description	Debug Tips
Memory Parity Errors	91	DM Parity Error Bank 2	Event generated when a parity error is detected during access to DM bank 2. This data memory parity error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.
	92	DM Parity Error Bank 3	Event generated when a parity error is detected during access to DM bank 3. This data memory parity error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.
	93	DM Parity Error Bank 4	Event generated when a parity error is detected during access to DM bank 4. This data memory parity error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.
	94	DM Parity Error Bank 5	Event generated when a parity error is detected during access to DM bank 5. This data memory parity error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.
	95	DM Parity Error Bank 6	Event generated when a parity error is detected during access to DM bank 6. This data memory parity error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.
	96	DM Parity Error Bank 7	Event generated when a parity error is detected during access to DM bank 7. This data memory parity error can be caused by DM access from the AI Engine, tile DMA, or memory mapped AXI interface.	Re-run the application. If the issue persists, contact AMD support.

Table 77: MEMORY Module Error Events (cont'd)

Errors Group	No.	Name	Description	Debug Tips
DMA Errors	97	DMA S2MM 0 Error	This error can be caused by writing to the BD task queue of S2MM channel 0 when it is full.	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If the issue persists, contact AMD support.
	98	DMA S2MM 1 Error	This error can be caused by writing to the BD task queue of S2MM channel 1 when it is full.	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If the issue persists, contact AMD support.
	99	DMA MM2S 0 Error	This error can be caused by writing to the BD task queue of MM2S channel 0 when it is full.	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If the issue persists, contact AMD support.
	100	DMA MM2S 1 Error	This error can be caused by writing to the BD task queue of MM2S channel 1 when it is full.	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If the issue persists, contact AMD support.

Table 78: AI Engine Interface Tile Module Error Events

Error Group	No.	Name	Description	Debug Tips
Bus Errors	62	AXI MM Slave Tile Error	Event generated if a memory mapped AXI interface slave request comes to an interface tile but the address is invalid.	If using the PL IP to access the AI Engine register with the memory mapped AXI interface, check if the IP tries to access the wrong address. If the issue persists, contact AMD support.
	64	AXI MM Decode NSU Error	The memory mapped AXI interface traffic internally has responded with a DECERR. For example, if a column, set of tiles are clock gated, a decode error is generated internally and travels on the memory mapped AXI interface to the interface tile to generate this event.	If using the PL IP to access the AI Engine register using the memory mapped AXI interface, check if the IP tries to access tile which is gated. If the issue persists, contact AMD support.
	65	AXI MM Slave NSU Error	The memory mapped AXI interface traffic internally has responded with a SLVERR. For example, an AI Engine tile in that interface tile column has responded with a slave error. That slave error will travel over the memory mapped AXI interface to the interface tile as a slave error.	If using the PL IP to access the AI Engine register with the memory mapped AXI interface, check if the IP tries to access wrong address. If the issue persists, contact AMD support.
	66	AXI MM Unsupported Traffic	The memory mapped AXI interface from the NoC has made a request that the AI Engine does not support.	If using the PL IP to access the AI Engine register with the memory mapped AXI interface, check if the IP generates unsupported memory mapped AXI interface requests.
	67	AXI MM Unsecure Access in Secure Mode	The memory mapped AXI interface from the NoC is violating the secure mode (trying to route unsecured traffic when AI Engine only supports secure traffic).	Check if the AI Engine array is configured in secure mode.
	68	AXI MM Byte Strobe Error	The memory mapped AXI interface from the NoC is writing with non-complete 32-bit words (within a 32-bit word all byte strobes must be set).	If the PL IP is accessing the AI Engine using the memory mapped AXI interface, check if all byte strobes are set for a 32-bit word.
Stream Error	63	Control Pkt Error	Control Packet Error	If the PL IP is generating the control packets, check if the IP generates packets properly. If the issue persists, contact AMD support.

Table 78: AI Engine Interface Tile Module Error Events (cont'd)

Error Group	No.	Name	Description	Debug Tips
DMA Error	69	DMA S2MM 0 Error	This DMA error is for DMA S2MM channel 0. It can be caused by: - writing to the BD task queue when it is full; - decode error when it tries to access the memory - slave error when it tries to access the memory	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If you manage buffer descriptors in your application, check if the memory address sent to the interface tile DMA buffer descriptor is invalid. If the issue persists, contact AMD support.
	70	DMA S2MM 1 Error	This DMA error is for DMA S2MM channel 1. It can be caused by: - writing to the BD task queue when it is full; - decode error when it tries to access the memory - slave error when it tries to access the memory	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If you manage buffer descriptors in your application, check if memory address sent to the interface tile DMA buffer descriptor is invalid. If the issue persists, contact AMD support.
	71	DMA MM2S 0 Error	This DMA error is for DMA MM2S channel 0. It can be caused by: - writing to the BD task queue when it is full; - decode error when it tries to access the memory - slave error when it tries to access the memory	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If you manage buffer descriptors in your application, check if memory address sent to the interface tile DMA buffer descriptor is invalid. If the issue persists, contact AMD support.
	72	DMA MM2S 1 Error	This DMA error is for DMA MM2S channel 1. It can be caused by: - writing to the BD task queue when it is full; - decode error when it tries to access the memory - slave error when it tries to access the memory	If you manage buffer descriptors in your application, verify that you are not pushing new buffer descriptors when the queue is full. If you manage buffer descriptors in your application, check if memory address sent to the interface tile DMA buffer descriptor is invalid. If the issue persists, contact AMD support.

Compiling and Linking Host Code for Linux

You can compile the embedded application by following the typical cross-compilation flow for the Arm Cortex-A72 processor.

In order to compile and link the host application to run on the Versal architecture board you must use the Vitis software.

The expectation is that you have completed the following setup:

1. Install the Vitis software. The Vitis release includes all the embedded base platforms, including the VCK190 base platforms.
2. In addition, ensure you have downloaded the Common Images for Embedded Vitis Platforms from this link <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-platforms/2024-2.html>.

The common image package contains a prebuilt Linux kernel and root file system that can be used with the Versal architecture board for embedded design development using Vitis.

3. Unzip the Versal common image package.
4. In a Bash shell, run the following script:

```
/Common Images Dir/xilinx-versal-common-v2024.1/environment-setup-cortexa72-cortexa53-xilinx-linux
```

This script sets up the `SDKTARGETSYSROOT` and `CXX` variables.

If the script is not present, you must run `/Common Images Dir/xilinx-versal-common-v2024.1.2/sdk.sh`.

Now you are ready to compile and link the host application.

Compiling the Host Application

Compiling the PS application uses the `CXX` variable. The following is an example command for compiling the PS application:

```
$CXX -std=c++17 -I${SDKTARGETSYSROOT}/usr/include/xrt/ -g -Wall -c -fmessage-length=0 -o host.o host.cpp
```

Many of the options in the preceding command are standard and can be found in a description of the `g++` command. The more important options are listed as follows.

- `-std=c++17`
- `-I${SDKTARGETSYSROOT}/usr/include/xrt/`
- `-o host.o host.cpp`

Linking the Host Application

Linking the PS application uses the `CXX` variable. The following is an example command for linking the PS application:

```
$CXX -lxrt_coreutil -o host.exe host.o
```

`xrt_coreutil`: Required library for linking for XRT API.



IMPORTANT! The steps to integrate the host code with the AI Engine graph application and the PL kernels into the hardware platform requires running the `v++ --package` command. Details on building and running the Versal system using the Vitis tool flow is described in [Building and Running the System in the Embedded Design Development Using Vitis \(UG1701\)](#).

Compiling and Linking Host Code with ADF API

When adaptive data flow (ADF) APIs or event profile APIs are used in the host code, you must:

- Use the `registerXRT()` API before using the ADF APIs or the event profile APIs in the host code.
- Compile and link the host code with `Work/ps/c_rts/aie_control_xrt.cpp`.

The following are example commands for compiling and linking the PS application:

```
//Compiling host.cpp
$CXX -std=c++17 -O0 -g -Wall -c -I$SDKTARGETSYSROOT/usr/include/xrt/ --sysroot=$SDKTARGETSYSROOT -I./ -I./src -I$XILINX_HLS/include/ -I$XILINX_VITIS/aietools/include -o sw/host.o sw/host.cpp

//Compiling aie_control_xrt.cpp
-std=c++17 -O0 -g -Wall -c -I$SDKTARGETSYSROOT/usr/include/xrt/ --sysroot=$SDKTARGETSYSROOT -I./ -I./src -I$XILINX_HLS/include/ -I$XILINX_VITIS/aietools/include -o sw/aie_control_xrt.o Work/ps/c_rts/aie_control_xrt.cpp

//Linking the host application
$CXX -ladf_api_xrt -lgcc -lc -lpthread -lrt -ldl -lcrypt -lstdc++ -lxrt_coreutil -L$SDKTARGETSYSROOT/usr/lib/ --sysroot=$SDKTARGETSYSROOT -L$XILINX_VITIS/aietools/lib/aarch64.o -o sw/host.exe sw/host.o sw/aie_control_xrt.o
```

Where `-ladf_api_xrt` is the required library for the ADF API in Linux.

The following is an example that uses ADF API and event profile API in the host code:

```
.....
#include "adf/adf_api/XRTConfig.h"
#include "xrt/xrt_kernel.h"

#include "graph.cpp"

int run(int argc, char* argv[]){
    .....
    char* xclbinFilename = argv[1];

    int ret=0;
    // Open xclbin
    auto device = xrt::device(0); //device index=0
    auto uuid = device.load_xclbin(xclbinFilename);
    auto dhdl = xrtDeviceOpenFromXcl(device);

    // s2mm & mm2s kernel handle
    auto s2mm = xrt::kernel(device, uuid, "s2mm");
    auto mm2s = xrt::kernel(device, uuid, "mm2s");
```

```

// output memory
auto out_bo = xrt::bo(device, output_size_in_bytes, s2mm.group_id(0));
auto host_out=out_bo.map<int*>();

// input memory
auto in_bo = xrt::bo(device, output_size_in_bytes, mm2s.group_id(0));
auto host_in=in_bo.map<int*>();

// Initialization input memory
.....

//kernel run
auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE);//1st run for s2mm
has started
auto mm2s_run = mm2s(in_bo, nullptr, OUTPUT_SIZE);

// Use registerXRT before ADF API
adf::registerXRT(dhdl, uuid.get());
std::cout<< "Register XRT"<<std::endl;

event::handle handle = event::start_profiling(gr.dataout,
event::io_stream_start_to_bytes_transferred_cycles, output_size_in_bytes);
if(handle==event::invalid_handle){
    printf("ERROR:Invalid handle. Only two performance counter in a AIE-
PL interface tile\n");
    return 1;
}

gr.run(iterations);
gr.end();
s2mm_run.wait();
// Wait graph for some cycles
long long cycle_count = event::read_profiling(handle);
std::cout<< "cycle count:"<<cycle_count<<std::endl;
event::stop_profiling(handle);
double throughput = (double)output_size_in_bytes / (cycle_count *0.8 *
1e-3); //Every AIE cycle is 0.8ns in production board
std::cout<< "Throughput of the graph: "<<throughput<< " MB/s"<<std::endl;
out_bo.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

//post processing
.....

return ret;
}

int main(int argc, char* argv[])
{
    try {
        auto match = run(argc, argv);
        std::cout << "TEST " << (match ? "FAILED" : "PASSED") << "\n";
        return (match ? EXIT_FAILURE : EXIT_SUCCESS);
    }
    catch (std::exception const& e) {
        std::cout << "Exception: " << e.what() << "\n";
        std::cout << "FAILED TEST\n";
        return 1;
    }
}

```

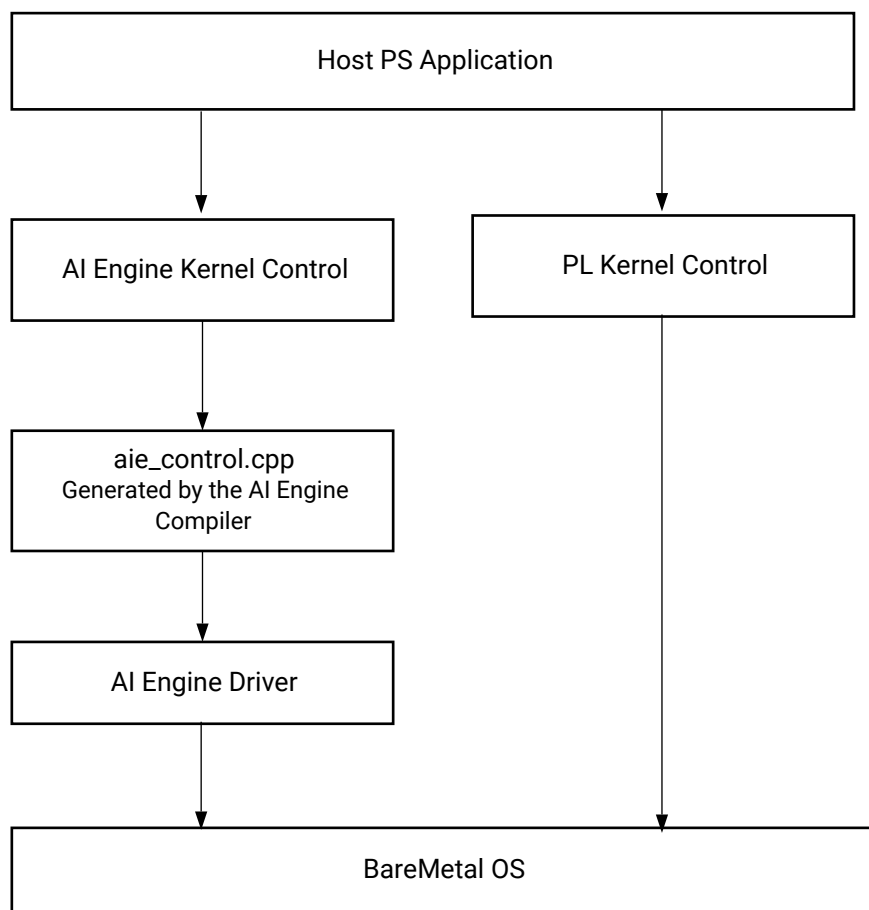
Host Programming for Bare-Metal

In a bare-metal/standalone environment, AMD provides standalone board support package (BSP), drivers, and libraries for applications to use to reduce development effort. As described in [Host Programming on Linux](#), the top-level application for bare-metal systems must also integrate and manage the AI Engine graph and PL kernels.



TIP: The steps to integrate a bare-metal system with the AI Engine graph and PL kernels is described in [Creating a Bare-Metal System](#) in the Vitis Reference Guide ([UG1702](#)).

Figure 86: AI Engine Bare-Metal Software Stack



X26401-031622

The following is an example top-level application (`main.cpp`) for a bare-metal system:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <unistd.h>
#include "platform.h"
#include "xparameters.h"
```

```

#include "xil_io.h"
#include "xil_cache.h"
#include "graph.cpp"
.....
#define MM2S_BASE XPAR_MM2S_S_AXI_CONTROL_BASEADDR
#define S2MM_BASE XPAR_S2MM_S_AXI_CONTROL_BASEADDR
#define MEM_OFFSET 0x10
#define SIZE_OFFSET 0x1C
#define CTRL_OFFSET 0x0

int RunTest(uint64_t mm2s_base, uint64_t s2mm_base, int32_t* in, int32_t*
golden,
    int32_t* out, int input_size, int output_size)
{
    int i;
    int errCount = 0;
    uint64_t memAddr = (uint64_t)in;
    uint64_t mem_outAddr = (uint64_t)out;

    printf("Starting test w/ cu\n");
    printf("Starting mm2s & s2mm\n");
    Xil_Out32(mm2s_base + MEM_OFFSET, (uint32_t) memAddr);
    Xil_Out32(mm2s_base + MEM_OFFSET + 4, 0);
    Xil_Out32(s2mm_base + MEM_OFFSET, (uint32_t) mem_outAddr);
    Xil_Out32(s2mm_base + MEM_OFFSET + 4, 0);
    Xil_Out32(mm2s_base + SIZE_OFFSET, input_size);
    Xil_Out32(s2mm_base + SIZE_OFFSET, output_size);
    Xil_Out32(mm2s_base + CTRL_OFFSET, 1);
    Xil_Out32(s2mm_base + CTRL_OFFSET, 1);

    printf("GRAPH INIT\n");
    clipgraph.init();

    printf("GRAPH RUN\n");
    clipgraph.run();

    //Wait for s2mm to be done
    while(1) {
        uint32_t v = Xil_In32(s2mm_base + CTRL_OFFSET);
        if(v & 6) {
            break;
        }
    }

    printf("PLIO IP DONE!\n");

    for(i = 0; i < output_size; i++) {
        if((((int32_t*)out)[i] != ((int32_t*)golden)[i]) ) {
            printf("Error found in sample %d != to the golden %d\n", i+1,
                ((int32_t*)out)[i], ((int32_t*)golden)[i]);
            errCount++;
        }
        else
            printf("%d\n ", ((int32_t*)out)[i]);
    }

    printf("Ending test w/ cu\n");
    return errCount;
}

int main()
{
    int i;

```

```

    int32_t* out;
    int errCount;

    Xil_DCacheDisable();
    init_platform();
    sleep(1);

    printf("Beginning test\n");
    //Preparing data
    .....

    //Run PL kernels & AIE & Verification
    errCount = RunTest(MM2S_BASE, S2MM_BASE, (int32_t*)cint16input,
int32golden, out, INPUT_SIZE, OUTPUT_SIZE);

    //Post-Processing
    if(errCount == 0)
        printf("Test passed. \n");
    else
        printf("Test failed! Error count: %d \n",errCount);

    cleanup_platform();
    return errCount;
}

```

The following are the steps in the code example:

- The `main()` function initializes the platform, data, runs the test, verifies the return code, and return the error code.
- `#include "graph.cpp"` includes the code to instantiate the graph object `clipgraph`. The `graph.cpp` is same as used in AI Engine compilation and simulation, but can be guarded by macro `__AIESIM__` and `__X86SIM__`. The usage of these macros is found in [Macros](#).
- `Xil_DCacheDisable()` disable data cache. Essential to make sure data is synchronized between APIs.
- `RunTest()` passes necessary data to the kernel to process and return a result.
- `Xil_Out32` writes to registers to control PL kernels. More details are available in the following section.
- `clipgraph.init()` initializes the graph.
- `clipgraph.run()` starts the graph.

The preceding code example references `xparameters.h` which is automatically generated from the bare-metal BSP. The application needs to ensure the bare-metal BSP is properly generated so that the memory mapped addresses for all drivers are correctly assigned.

`xil_io.h` contains general driver I/O APIs. This is the preferred method for accessing drivers.

Addressing Kernels in Bare-Metal Applications

For bare-metal applications, when addressing the PL kernels from the embedded application, you must use the control registers, or read and write to the kernel at the appropriate base address and offset as it is implemented in hardware. Looking at the application discussed earlier, the embedded application can deliver data to the MM2S kernel, to introduce it to the AI Engine graph for the Interpolator and Classifier kernels, and read data from the S2MM kernel to continue processing in the embedded application. In this case, address the MM2S and S2MM kernels as they are implemented in the PL region of the fixed platform.

The `main.cpp` of the example shows the `#define` statements for the kernel base address and the address offset for specific registers. For example:

```
#define MM2S_BASE XPAR_MM2S_S_AXI_CONTROL_BASEADDR
#define S2MM_BASE XPAR_S2MM_S_AXI_CONTROL_BASEADDR

#define MEM_OFFSET 0x10
#define SIZE_OFFSET 0x1C
#define CTRL_OFFSET 0x0
```

To determine the address and offsets for the kernels, examine some of the files in the fixed platform. The location of the base address for the implemented kernels is located in the fixed platform `xparameters.h` file, that is located in the `<platform_name>/standalone_domain/bsp/include/include` folder. For the example design, use the following entries in `xparameters.h` to determine the base addresses of these kernels.

```
/* Definitions for peripheral MM2S */
#define XPAR_MM2S_S_AXI_CONTROL_BASEADDR 0xA4020000
#define XPAR_MM2S_S_AXI_CONTROL_HIGHADDR 0xA402FFFF

/* Definitions for peripheral S2MM */
#define XPAR_S2MM_S_AXI_CONTROL_BASEADDR 0xA4030000
#define XPAR_S2MM_S_AXI_CONTROL_HIGHADDR 0xA403FFFF
```

Note: The `xparameters.h` file is generated and addressing is dynamic. It is best to reference the address macros for kernels than to hard code them.

The location of the address offset is located in the `<kernel_driver>_hw.h` file of the `_x/<kernel>/<kernel>/<kernel>/solution/impl/ip/drivers/<kernel>_v1_0/src` folder of the compiled kernel produced by the AMD Vitis™ compiler. For example, the MM2S kernel driver, `xmm2s_mm2s_hw.h` displays the following data.

```
#define XMM2S_MM2S_CONTROL_ADDR_AP_CTRL 0x00
#define XMM2S_MM2S_CONTROL_ADDR_GIE 0x04
#define XMM2S_MM2S_CONTROL_ADDR_IER 0x08
#define XMM2S_MM2S_CONTROL_ADDR_ISR 0x0c
#define XMM2S_MM2S_CONTROL_ADDR_MEM_V_DATA 0x10
#define XMM2S_MM2S_CONTROL_BITS_MEM_V_DATA 64
#define XMM2S_MM2S_CONTROL_ADDR_SIZE_DATA 0x1c
#define XMM2S_MM2S_CONTROL_BITS_SIZE_DATA 32
```

Use these offsets when reading or writing to the kernels. For instance, from the example application `main.cpp` file use the following to write to the memory location.

```
Xil_Out32(MM2S_BASE + MEM_OFFSET, (uint32_t) memAddr);
```

Compiling and Linking Host Code for Bare-Metal

To build the PS application for the bare-metal flow, use the platform generated in the prior step. You need the PS application (`main.cpp`), and the bare-metal AI Engine control file (`aie_control.cpp`), which is created by the AI Engine compiler and can be found in the `./Work/ps/c_rts` folder.

Compile the `main.cpp` file using the following command:

```
aarch64-none-elf-gcc -I. -I. -I../src \
-I./vck190_baremetal/export/vck190_baremetal/sw/vck190_baremetal/
standalone_domain/bspinclude/include \
-g -c -std=c++17 -o main.o main.cpp
```

Note: You must include the BSP include files for the generated custom base-platform platform, located at `./vck190_baremetal/export/vck190_baremetal/sw/vck190_baremetal/standalone_domain/bspinclude/include`

where, `vck190_baremetal` is the user-defined platform name.

Compile the `aie_control.cpp` file using the following command:

```
aarch64-none-elf-gcc -I. -I. -I../src \
-I./vck190_baremetal/export/vck190_baremetal/sw/vck190_baremetal/
standalone_domain/bspinclude/include \
-g -c -std=c++17 -o aie_control.o ../Work/ps/c_rts/aie_control.cpp
```

Link the PS application using the two compiled object files:

```
aarch64-none-elf-gcc main.o aie_control.o -g -mcpu=cortex-a72 -Wl,-T -Wl,./
lscript.ld \
-L./vck190_baremetal/export/vck190_baremetal/sw/vck190_baremetal/
standalone_domain/bsplib/lib \
-ladf_api -Wl,--start-group, -lxil, -lgcc, -lc, -lstdc++, --end-group -o main.elf
```

Note: You also need the BSP `libxil.a` located at `./vck190_baremetal/export/vck190_baremetal/standalone_domain/bsplib/lib` during linking. This assumes the AI Engine is enabled during the platform management controller (PMC) boot.

Note: The linker script `lscript.ld` can be obtained from the Vitis IDE, as described in [Creating a Bare-Metal System](#) found in [Vitis Reference Guide \(UG1702\)](#). Additional details can be found in [A to Z Bare-metal Flow](#) in the AMD GitHub AI Engine Vitis Tutorials.

Note: If the graph or kernel code is designed for multiple architectures and it must use the `__AIE_ARCH__` macro to guard parts of the code specific to an architecture. You must define the macro in the host compilation. For example, add `-D__AIE_ARCH__=10` for host compilation for AI Engine devices, and `-D__AIE_ARCH__=20` for AI Engine-ML devices.



IMPORTANT! The steps to integrate the host code with the AI Engine graph application and the PL kernels into the hardware platform requires running the `++ --package` command. Details on building and running the Versal device system using the Vitis tool flow is described in [Building and Running the System](#) in the *Embedded Design Development Using Vitis (UG1701)*.

Host Programming Support Comparison Between Linux and Bare-Metal

The following table compares the host programming features supported by PetaLinux and Bare-metal operating systems.

Table 79: Host Programming Support Comparison

Features	Bare-Metal	PetaLinux
Host application heap/stack size configuration	Manual adjustment required	Auto adjusted from OS
Direct access to device registers	Supported	Supported
Host application <code>printf()</code> support	Supported	Supported
XRT support	Not supported	Supported
<code>xbutil</code> support	Not supported	Supported
<code>sysfs</code> support	Not supported	Supported
Multi-thread host application support	Not supported	Supported
Multi-process host application support	Not supported	Supported
Event trace/profile XSDB flow	Supported	Supported
Event trace/profile XRT flow	Not supported	Supported
Operating system support	Not supported	Supported

Resetting and Reloading Flow of the AI Engine Array

The AI Engine reset and reload flow enables you to run the application on hardware multiple times without rebooting the board and the Linux operating system. This flow supports resetting and reloading the AI Engine array and all PL logic within the platform dynamic region block design under control of the RST_PL RESET0 register. The RST_PL register is described in the *Versal Adaptive SoC Register Reference* ([AM012](#)). This reset and reload flow does as follows:

- Resets the AI Engine array that is configured with an AI Engine graph application, such as, kernels programmed in specific tiles, DMAs, and FIFOs between tiles and memories.
- Resets the AI Engine array on a non-DFX platform.
- Resets the AI Engine array without rebooting the board.
- Reloads the AI Engine array with the same graph or a different graph without rebooting the board.

The flow is supported on Linux XRT.

Consecutive resets and reloads of the application must use the same AI Engine-PL interfaces. You can reload the AI Engine array with the same graphs, or using different graphs.

Use this flow to:

- Avoid resetting the system and rebooting the board to run the AI Engine application multiple times.
- Avoid long reboot times, in case the system runs into abnormal situation running on the board, and the application must be rerun again.
- Avoid resetting the system and rebooting the board to run a modified AI Engine kernel(s).

The reset and reload flow requires that you compile the AI Engine graphs and PL kernels into separate XCLBIN files. The host application will need to use these XCLBIN files. You will then use the xbutil utility to reset the PL and AI Engine Kernels between consecutive application runs.

Event trace and profile of AI Engine design in hardware is not supported when the design is reset and reloaded in hardware.

Prerequisites to the Reset and Reload Flow

Run through these steps to prepare the application to supports the reset and reload flow:

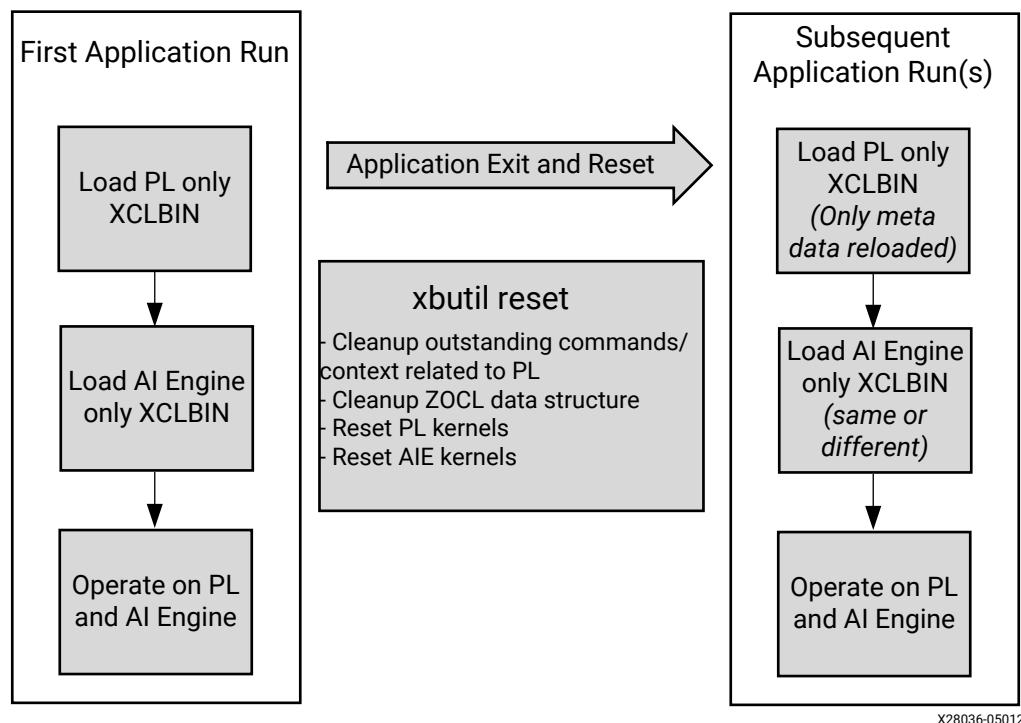
1. Compile AI Engine graphs.
2. Compile PL kernels.

3. Use v++ linker to target the HW flow and generate the XSA file.
4. Use v++ packager to generate the PL-only XCLBIN. Refer to [Generate AI Engine-only and PL-only XCLBIN](#).
5. Use v++ packager to generate the AI Engine-only XCLBIN. Refer to [Generate AI Engine-only and PL-only XCLBIN](#).
6. Use v++ packager to include the above generated AI Engine-only XCLBIN and PL-only XCLBIN into `sd_card` image using the `--package.sd_file <>` option.
7. Ensure the host code loads the AI Engine-only XCLBIN, and the PL-only XCLBIN that was generated in the previous step. Refer to [Host Application for Reset and Reload Flow](#).

Reset and Reload Flow Steps

1. Run the application on the board. Refer to [Chapter 6: Performance Analysis of AI Engine Graph Application on Hardware](#).
2. Reset the application using the `xbutil reset` command. Refer to [Reset Before Consecutive Application Runs](#).
3. Rerun the application on the board.

Figure 87: AI Engine Reset and Reload Flow



If different kernels are used in the AI Engine array in consecutive runs, the incremental flow of the AI Engine compiler can be used to ensure that the same AI Engine-PL interfaces are used. You will need to:

1. Recompile the newer kernels using the `v++ -c --mode aie` command with the platform XSA-generated in step 3 of the prerequisite section above.
2. Use `v++` packager to generate the newer AI Engine-only XCLBIN.
3. Use `v++` packager to include the newer AI Engine-only XCLBIN into `sd_card` image using the `--package.sd_file <>` option.

Generate AI Engine-only and PL-only XCLBIN

You will compile and link the AI Engine graphs and PL kernels using `v++` for the reset and reload flow as is typically done. However, during packaging you must generate AI Engine-only and PL-only XCLBIN files. The XCLBIN inputs are defined below.

- AI Engine-only XCLBIN: Contains AI Engine metadata, AI Engine PDI, and PL metadata. It allows XRT to reset and reload the AI Engine array.
- PL-only XCLBIN: Contains only PL metadata, and it allows XRT to reload PL information.

Generate AI Engine-only XCLBIN

1. Add the following constraints to a configuration file (`package_aie_only.cfg`):

```
[advanced]
param=package.generateAiePdi=1
```

2. Run with `v++` packager which takes the configuration file as input, as shown:

```
v++ -p -t hw -f ${PLATFORM} \
--config package_aie_only.cfg \
--package.defer_aie_run \
--package.rootfs ${ROOTFS} \
--output aie_only.xclbin \
libadf.a <HW xsa by v++ linker>
```

Generate PL-only XCLBIN

1. Add the following constraints to a configuration file (`package_pl_only.cfg`):

```
[advanced]
param=package.generateFlatPlVersalXclbin=1
```

2. Run with `v++` packager which takes the configuration file as input, as shown:

```
v++ -p -t hw -f ${PLATFORM} \
--config package_pl_only.cfg \
--output pl_only.xclbin \
<HW xsa by v++ linker>
```

Host Application for Reset and Reload Flow

The reset and reload flow requires that the host application uses the AI Engine XCLBIN and PL only XCLBIN generated in the previous steps. Every XCLBIN file has a universal unique identifier (UUID). The UUID can be obtained using XRT `load_xclbin` API. The host application needs to load the PL only XCLBIN and AI Engine only XCLBIN by obtaining the UUIDs associated with the XCLBIN files.

The host application code includes the following sequence of operations:

1. Open the device.
2. Load PL-only XCLBIN and get PL UUID.
3. Load AI Engine-only XCLBIN and get the AI Engine UUID.
4. Use PL UUID with XRT API operating on the PL kernels.
5. Use AI Engine UUID with XRT API operating on the AI Engine graphs.
6. Operate on the PL kernels and the AI Engine graphs.

An example host code is as follows:

```
int open_xclbin(char* pl_xclbinFilename, char*
aie_xclbinFilename, xrt::device &device, xrt::uuid &pl_id, xrt::uuid &aie_id)
{
    int ret=0;
    // Open the device
    device = xrt::device(0); //device index=0
    //Load PL-only XCLBIN and get PL UUID.
    pl_id = device.load_xclbin(pl_xclbinFilename);
    //Load AI Engine-only XCLBIN and get AI Engine UUID.
    aie_id = device.load_xclbin(aie_xclbinFilename);
    return ret;
}

int run_graph(const xrt::device &device, const xrt::uuid &pl_id, const
xrt::uuid &aie_id, int iterations){
    int output_size_in_bytes=iterations*WINDOW_SIZE_in_Bytes;
    int OUTPUT_SIZE=output_size_in_bytes/4;

    //Use PL UUID with XRT API operating on PL kernels.
    auto s2mm = xrt::kernel(device, pl_id, "s2mm");
    auto mm2s = xrt::kernel(device, pl_id, "datagen");

    // output memory
    auto out_bo = xrt::bo(device, output_size_in_bytes, s2mm.group_id(0));
    auto host_out=out_bo.map<int*>();

    auto s2mm_run = s2mm(out_bo, nullptr, OUTPUT_SIZE); //1st run for s2mm has
started
    auto mm2s_run = mm2s(nullptr, OUTPUT_SIZE);

    //Use AI Engine UUID with XRT API operating on AI Engine graphs.
    auto gr=xrt::graph(device, aie_id, "gr-pl");
    gr.run(iterations);

    int match=0;
```

```

s2mm_run.wait();
out_bo.sync(XCL_BO_SYNC_BO_FROM_DEVICE);
for(int i=0;i<OUTPUT_SIZE;i++){
    if(host_out[i]!=i+1){
        match=1;
    }
}
return match;
}

int main(int argc, char* argv[]) {
    if(argc < 4) {
        std::cout << "Usage: " << argv[0] << " <pl_xclbin> <aie_xclbin>
<iterations>" << std::endl;
        return EXIT_FAILURE;
    }
    char* pl_xclbinFilename=argv[1];
    char* aie_xclbinFilename=argv[2];
    int iterations=atoi(argv[3]);

    xrt::device device;
    xrt::uuid pl_id,aie_id;
    int ret;

    ret=open_xclbin(pl_xclbinFilename,aie_xclbinFilename,device,pl_id,aie_id);
    if(ret==0){
        std::cout<< "Download PL Xclbin successfully"<<std::endl;
        std::cout<< "Download AIE Xclbin successfully"<<std::endl;
    }

    ret=run_graph(device,pl_id,aie_id,iterations);
    std::cout << "TEST " << (ret ? " FAILED" : " PASSED") << std::endl;

    return (ret ? EXIT_FAILURE : EXIT_SUCCESS);
}

```

Reset Before Consecutive Application Runs

Consecutive reset and reload of the application in hardware requires that you run the `xutil reset` command. This command must be run between consecutive runs, as follows:

1. Kill the user application, if it hangs.
2. Reset the AI Engine and PL kernels into clean status from previous runs using the `xutil reset -d 0` command. It makes sure that:
 - The PL kernels, AI Engine, and FIFOs, data width converter IP's inserted by v++ automatically are reset.
 - The software parts of XRT, like XRT ZOCL data structures, are reset.
3. Run the application with PL-only XCLBIN and AI Engine-only XCLBIN again.

Mapper/Router Methodology

Design Convergence

This section describes the process of handling a failure in the AI Engine compiler during the mapper (place) or router steps.

Mapping Solution Not Found

Mapper failures typically have two modes. Either the failure happens during a pre-check phase or it might happen during the actual mapping phase.

Pre-check failures have explicit error messages which point to exact reason for failure, as shown in the following example.

```
ERROR: [aiecompiler 47-772] Inst g.kernel_a is in conflicting pblocks:(0,0)
(5,5) and (20,0) (25,5).
```

You can trace such errors to the design element or the constraints.

Mapping phase failures typically have an error message that looks like the following.

```
ERROR: [aiecompiler 47-51] AIE Mapper failed to find a legal solution.
Please try to relax constraints and/or try alternate strategies like
disableFloorplanning.
```

In this case, use the following steps to either narrow down the cause of the failure if it is design-related or help the tool to find a solution if the failure is tool-related.

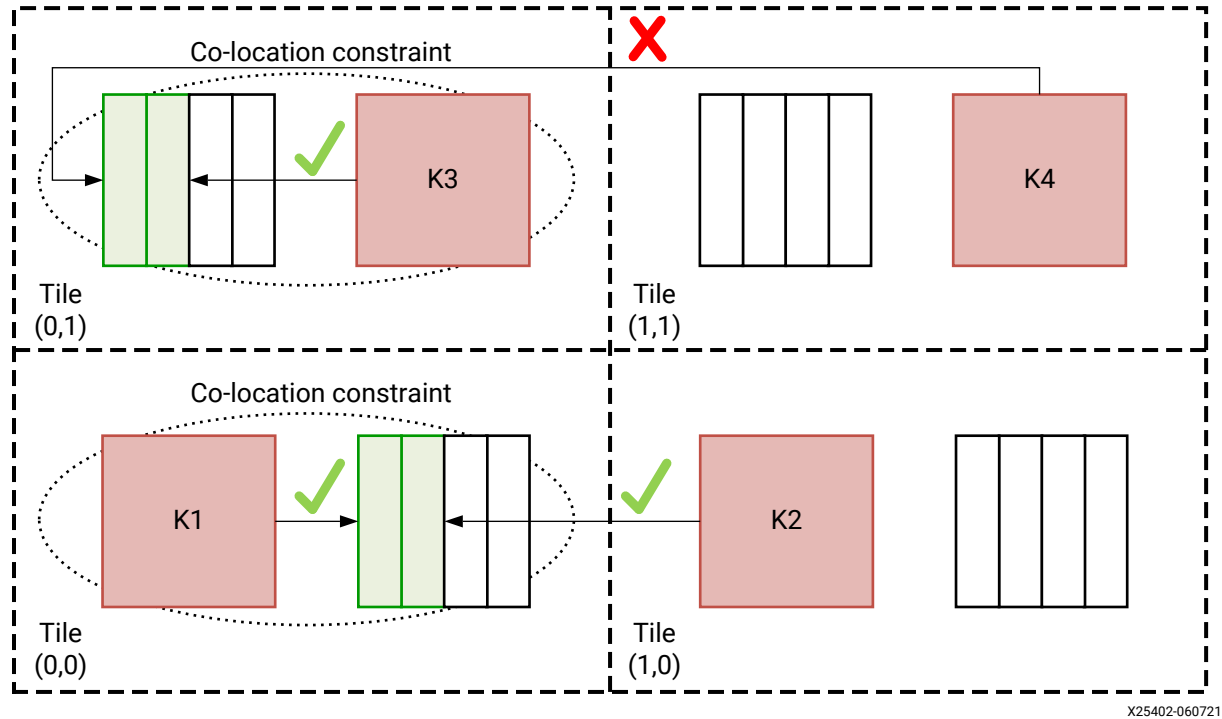
Checking User-Defined Constraints

Incorrectly defined user constraints can cause mapper failure. Some of these will be caught by the mapper pre-checker. However, not all of them can be caught in the pre-check phase. In such cases, check for the following conditions.

1. If you have a large number of absolute location or co-location constraints in the graph, check that these constraints do not give conflicting directives to the mapper. This might occur because of the checkerboard nature of the AI Engine array as shown in the following diagram.

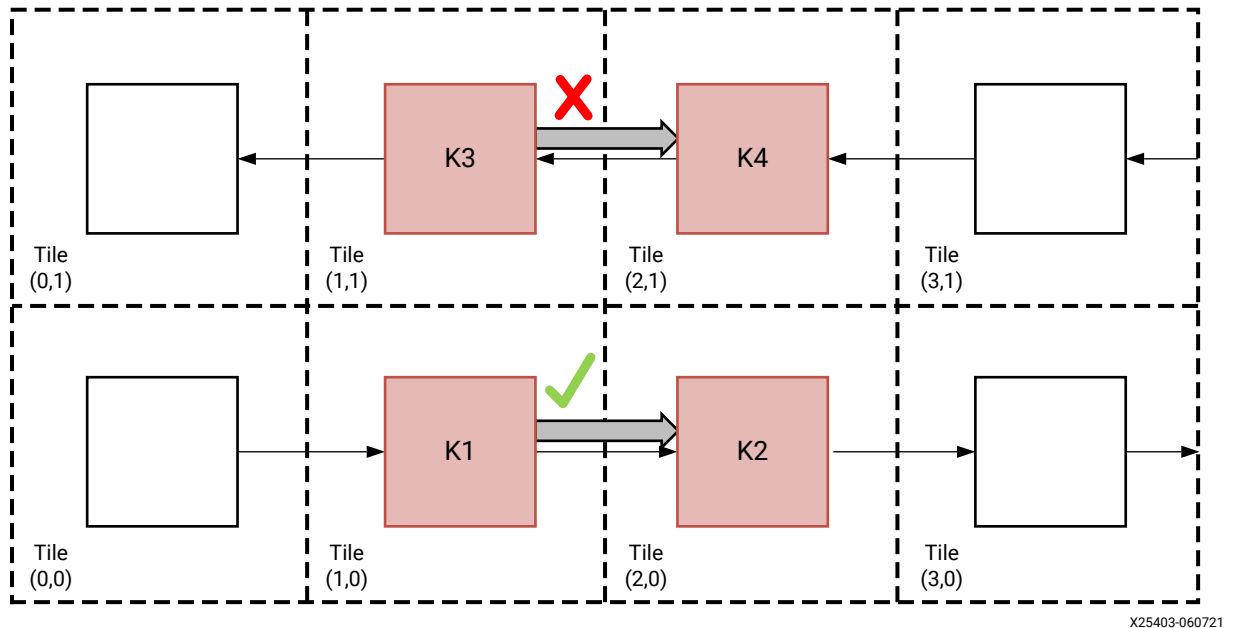
In this diagram, the kernels in red have absolute location constraints while the window buffer (green) between them has a co-location constraint with the first kernel. This will result in mapper failure.

Figure 88: **Conflicting Absolute Location/Co-location Constraints**



- If you have absolute location constraints for kernels that are part of a cascade chain, check that these constraints are compatible with the architecture. In AI Engine architecture, the direction of the cascade changes in each row. If you have absolute constraints and cascades as shown in the following diagram, it will cause mapper failure.

Figure 89: Conflicting Cascade Direction



3. In some cases the size of the buffers being constrained to a particular tile might exceed the memory capacity of the tile (32 KB in AI Engine architecture). This will result in mapper failure.
4. Each tile in the AI Engine architecture has two input and two output DMA channels. If you have constrained buffers in such a way that a particular tile needs more than this number of DMA channels, it will result in mapper failure.

Reducing Window Buffer Sizes for Very High Memory Density Designs

One of the main considerations when determining the window sizes for a design is that the number of cycles required for data loading is balanced with the number of compute cycles required by the kernel. This helps to pipeline the ping and pong buffer data loading with the kernel compute. For very high memory density designs, it makes sense to have smaller window sizes which can still balance the kernel compute because having larger window sizes might lead to mapper failure.

The following table shows the number of cycles required for the matrix multiplication of two matrices with 16-bit data. Example 1 and Example 2 have different matrix sizes, but both have their compute and data loading balanced. Note that only the larger of the A or B matrix size determines the data loading time whereas the time of kernel compute is determined by both sizes. This shows that Example 1 has smaller window sizes than Example 2, but the compute and data loading are balanced and can be pipelined.

Table 80: Matrix Multiplication Examples

	Matrix A Size	Matrix B Size	# of Multiplication Operations (MultOps)	#Cycles for Compute 32 ops/ cycle	#Cycles for Data Loading 32 bits/ cycle
Example 1	16x64	64x16	16384	512 (16384/32)	512 (64x16x16/32)
Example 2	16x64	64x32	32768	1024 (32768/32)	1024 (64x32x16/32)

Providing User Guidance to Mapper

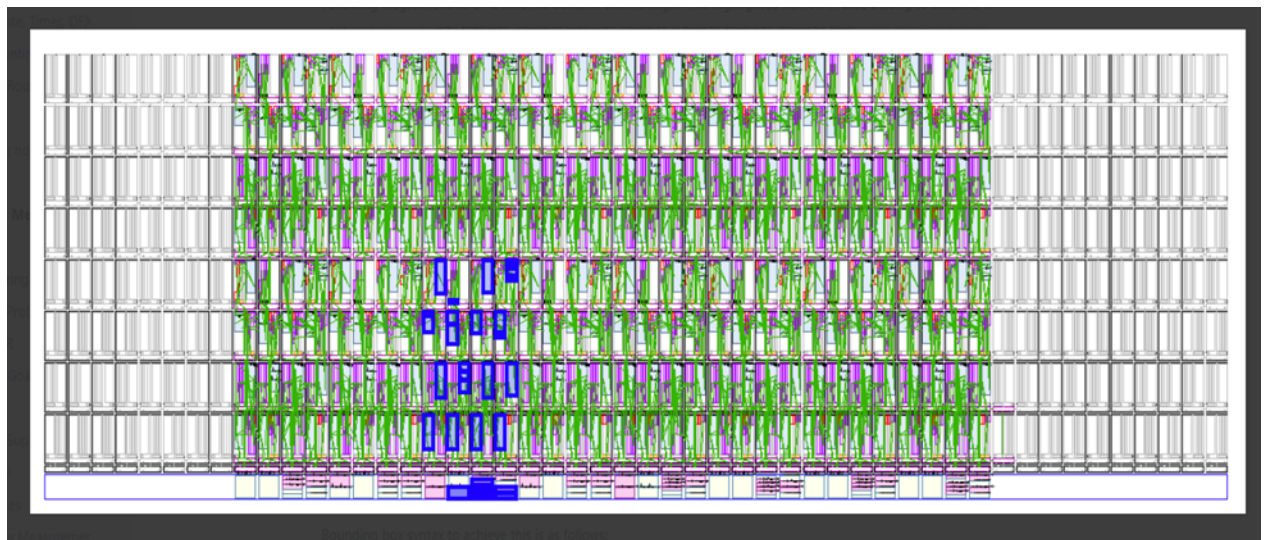
In some cases, the mapper error might be due to limitations of the tool. In such cases, it might be useful to provide guidance to the tool.

1. Turn off automatic AI Engine compiler floorplanning using the command line switch.

```
--Xmapper=disableFloorplanning
```

2. Create your own floorplanning by using either bounding box constraints in the graph or `areaGroup` constraints in the constraints file. This is particularly useful if there are multiple disjoint graphs in the design. Each of these separate graphs can be constrained to a particular region of the array. This technique not only helps design convergence but can also help improve performance by minimizing interference between different graph buffers. The following diagram shows an example using a 16 antenna transmit chain design. The highlighted kernels in blue belong to antenna 4.

Figure 90: Kernels in Antenna 4



The constraints file syntax to achieve this is as follows.

```
{
  "GlobalConstraints": {
    "areaGroup": {
      "name": "ant4_cores",
      "nodeGroup": [ "tx_chain4.*" ],
      "tileGroup": [ "(16,0):(19,3)" ]
    }
  }
}
```

The bounding box syntax to achieve this is as follows.

```
location<graph>(tx_chain4) = bounding_box(16,0,19,3);
```

3. Add co-location or absolute location constraints if necessary. If this guidance does not get the design to converge, you can try adding co-location or absolute location constraints. Co-location constraints can be added between kernels and buffers or system memory that you expect to be mapped to the same tile, as shown in the following example.

```
location<buffer>(kernel_1.out[0]) = location<kernel>(kernel_1);
location<stack>(kernel_1) = location<kernel>(kernel_1);
```

Absolute location constraints can also be added to certain key kernels or buffers to act as anchors and guide the mapper's placement of other components, as shown in the following example.

```
location<kernel>(kernel_1) = tile(20, 0);
location<buffer>(kernel_1.in[0]) =
    { address(19, 0, 0x0),
      address(19, 0, 0x2000) }; // double buffer needs two locations
```

Routing Solution Not Found

This section identifies how to check router congestion to see whether the mapper has put the router into an impossible situation, and how to determine if packet switching disable is affecting the congestion.

Initially, check the user-defined constraints, which include the FIFO depth constraints and the area group constraints and then check the mapping results.

FIFO Depth Constraints

There are limited switch and DMA FIFOs on the device. When deciding on `fifo_depth` constraints it is important to consider the amount of FIFOs you specify for an area. This includes taking into account if the nets that have `fifo_depth` constraints also have area group constraints. In this case make sure that all `fifo_depth` constraints can be met within the specified area.

If there is a high contention for switch FIFOs, consider moving to DMA FIFOs. Without changing the `fifo_depth` you can specify the DMA FIFO type using the following constraint.

```
location<fifo>(net1) = { dma_fifo() }
```

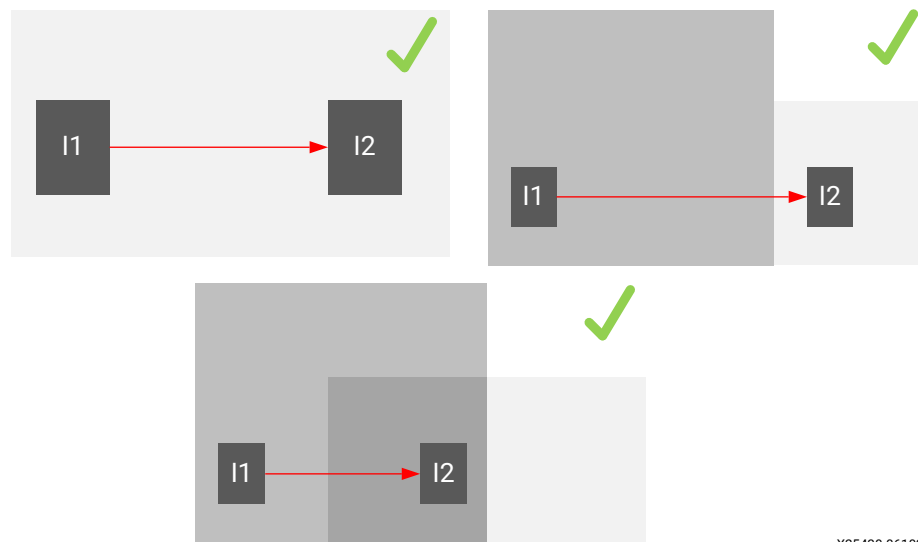
With careful consideration FIFO locations constraints can be applied, as shown in the following example.

```
location<fifo>(net2) = { dma_fifo(aie_tile, 15, 0, 0x3100, 32) };
location<fifo>(net3) = { ss_fifo(shim_tile, 16, 0, 0), dma_fifo(aie_tile,
17, 0, 0x3100, 48) }
```

Area Group Constraints

Sometimes the placement of objects is only considered when defining the area group constraints. This can leave routing without the ability to form all its connections. The following image shows a variety of area group constraints that all allow routing to form connections. In all three of these cases the routing never has to leave the defined area groups to complete its routing.

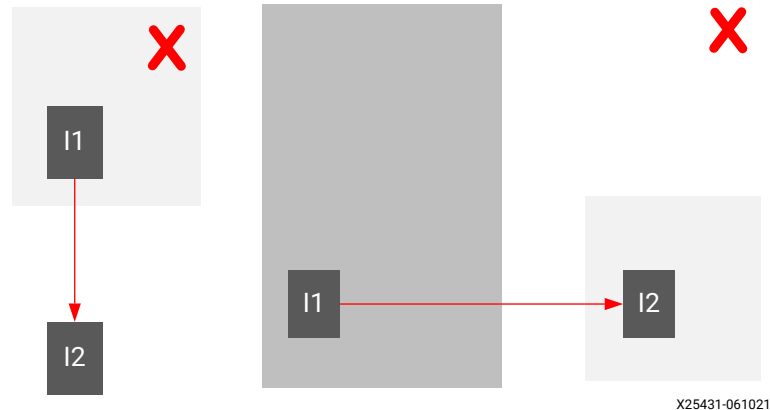
Figure 91: Routing in Defined Area Groups



X25430-061021

In contrast there are two common errors with routing and area group constraints. The left side of the following image shows a missing area group for an object. The second case is one where all objects are contained within separate area groups but the two groups are not adjacent. In this case the router has no way to complete the routing of its nets without violating an area group constraint and the router will fail to find a legal solution.

Figure 92: Routing Errors



Relative Constraints

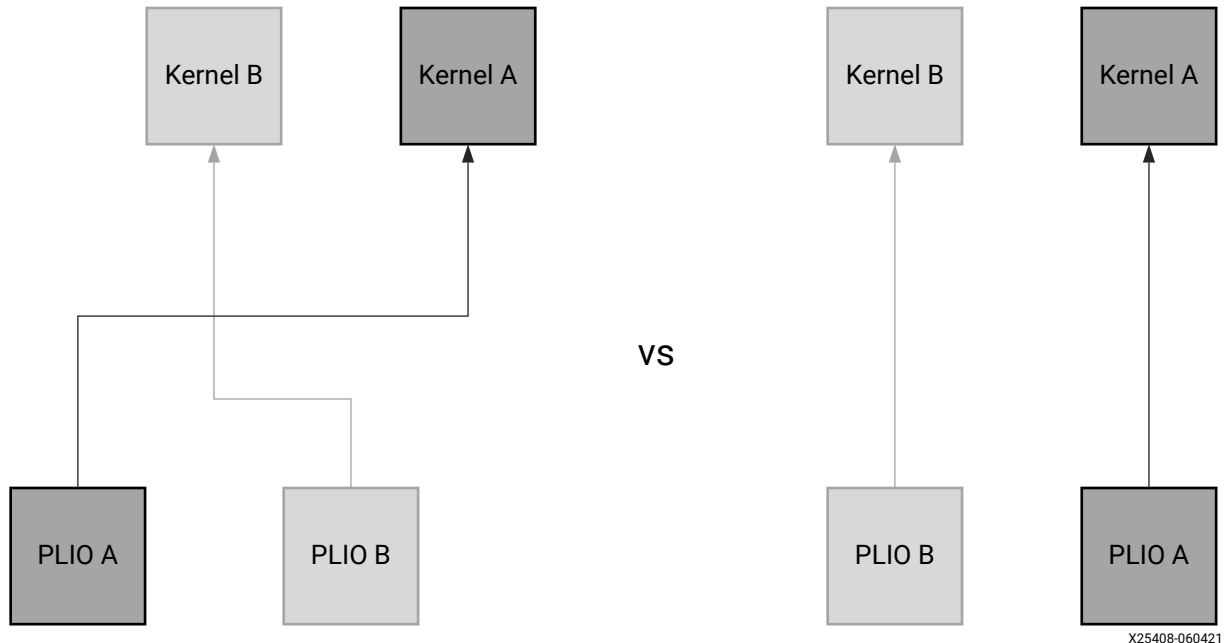
Controlling the placement of kernels relative to one another to help with guiding the mapper is needed for some use cases. To best utilize the AI Engine resources it is important that the AI Engines have a specific relative placement, and the user knows this placement and would like to pass it the router. Often these patterns are repeated many times in the array, so this is a request to allow the user to create these patterns using relative constraints. The idea is to have an anchor AI Engine and then allow for placement constraints relative to this engine, just as, (+/-X, +/-Y).

Checking Mapping Results

Check PLIO placement to see if it is causing routing congestion. The most common congestion area for router is in the interface tile region. There are more PLIO channels than there are Interface Channels to connect them into the AI Engine array.

1. If PLIOs are being constrained make sure adequate resources into the AI Engine array are open to handle the locked PLIOs.
2. If possible limit PLIO placement near shadow regions to decrease congestion of routing resources.

3. Check PLIO placement/constraints to verify that crossing such as shown in the following image is not occurring.



X25408-060421

Handling Large Designs

For larger AI Engine designs the placement stage of the AI Engine compiler where it attempts to find suitable placement in the AI Engine array can be challenging and time consuming. Suggestions for the AI Engine compiler to handle such large designs are as follows:

- When mapping large designs, keep the physical resources of the target device in mind, such as, the number of AI Engine cores, the size of buffers allocated, and the broadcast or merge points that can lead to congestion.
- AI Engine design development can start by targeting a subset of the AI Engine arrays. Then, development can be expanded to all AI Engine arrays.
- The full design can be divided into smaller sub-blocks, each of which can be constrained to specific areas using location constraints.
- Relative constraints can be used to provide guidance while maintaining flexibility about the exact position. When there are multiple instances of the same sub-graph in the AI Engine design, use the `Stamp` and `Repeat` constraints to ensure more identical sub-graph placement. This also helps provide more deterministic AI Engine compiler behavior and performance.

These suggestions help reduce the search space problem for the AI Engine compiler, resulting in better placement convergence.

Event Trace Routing

Enabling the event trace feature on designs can impact the routing of the AI Engine application. Extra resources are required to collect event information and pass it outside the AI Engine block. The routing phase occurs in two steps: the data-flow nets are routed first; and then the event trace signals are routed among the remaining unused paths.

In case all the trace nets could not be routed for a design with event trace enabled, the default behavior is as follows:

- The AI Engine compiler issues a critical warning to indicate that all requested event trace signals could not be routed.
- The AI Engine compiler then errors out.

An example of the Critical Warning is shown below, indicating the trace nets that failed to route.

```
CRITICAL WARNING: [aiecompiler 35-4476] Trace Net tr_net352 driven by
AIE_ML_MEM_TILE_X9Y2/AIE_MEMGRP_M_TRACE_PIN failed to route. Design will
not pass post route drc check.
CRITICAL WARNING: [aiecompiler 35-4476] Trace Net tr_net306 driven by
AIE_ML_MEM_TILE_X9Y3/AIE_MEMGRP_M_TRACE_PIN failed to route. Design will
not pass post route drc check.
```

To successfully route as many event trace nets as possible, AMD recommends using the option `Xrouter=skipunroutabletracenets`. When this option is used, the AI Engine compiler does not error out. It continues to route those trace nets that can be routed. A critical warning is provided for every trace net that cannot be routed.

Improving Design Performance

Design throughput can be negatively impacted by memory or stream stalls and also by large skew between the graph output. These situations can be identified by viewing the simulator output in the AMD Vitis™ IDE. The following sections discuss techniques to be followed in each of these cases.

Memory Stalls

The objective of the mapper is to prevent buffer conflicts, where possible. It also has different buffer optimization levels that try to increase the size of buffers to prevent conflicts. These buffer optimization levels range from 0 (default) to 9. They are invoked with the `Xmapper` option, `--Xmapper=BufferOptLevel<level_num>`. At the highest buffer optimization level (9), no two buffers can be placed in the same bank. However it is important to know that at the higher buffer optimization levels it might become impossible for the mapper to find a solution and it will error out. So the first option if you see a large number of memory stalls is to cycle through the `BufferOptLevel` options to see if fewer memory stalls are seen at higher `bufferOptLevels`.

Another possibility is that you can explicitly inform the mapper not to place two buffers in the same bank. If your simulation analysis indicates that a significant throughput degradation is caused by memory stall resulting from a bank conflict between buffer `kernel_0.in[0]` and `kernel_1.out[0]`, you can provide a directive to the mapper to not place these buffers in the same bank as follows.

```
not_equal(location<buffer>(kernel_0.in[0]),
location<buffer>(kernel_1.out[0]));
```

Note: While the buffers can be placed in different banks, they might not be on the same AI Engine tile.

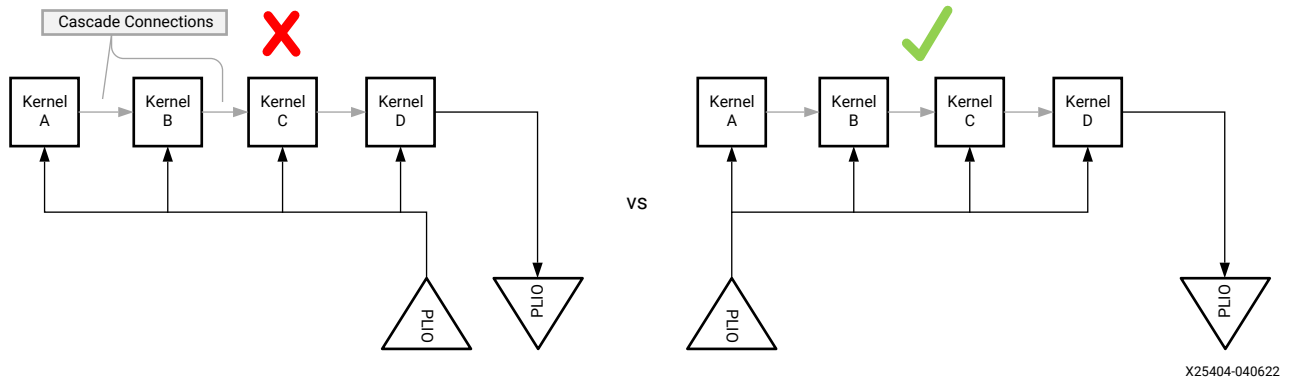
If DMA FIFOs are used in the design and they are placed in the same bank as other buffers then the `Xrouter` option `DMAFIFOsInFreeBankOnly` can force the router to place these FIFOs in free banks. This eliminates memory conflicts with the DMA FIFOs. If it is not possible to reserve an entire free bank for the DMA FIFO then location constraints can be used in coordination with outside knowledge of memory buffers. In this case it is important to have knowledge of which buffers might cause stalls when conflicting with DMA FIFOs. The constraints can look like the following.

```
location<fifo>(net2) = { dma_fifo(aie_tile, 15, 0, 0x3100, 32) };
```

Stream Stalls

The driver of a cascade chain should be placed closer to the head of the cascade chain, if possible. This reduces routing latency for these long latency paths and can reduce the need for expensively large switch/DMA FIFOs.

Figure 93: Cascade Chain



1. Make sure `fifo_depth` constraints have been specified for nets that require buffering.
2. Minimize streams and favor windows.

If `fifo_depth` has been specified you can check the log for the FIFO report section in `AICompiler.log`.

Large Skew Between Identical Graph Outputs

If a design has multiple instantiations of the same graph, consider using the stamp and repeat flow in the mapper. In this flow, you provide input to the mapper that all the graphs should be mapped in an exactly identical manner in the area group region that you have provided for each graph. This not only simplifies the problem for the mapper, it also significantly reduces skew between the outputs of different graphs. This is particularly important for wireless communications designs with multiple antennas. Steps to use the stamp and repeat flow are as follows.

1. Define an area group for each graph. For example in the following add constraints in the `aiecost` file.

```
"GlobalConstraints": {
  "areaGroup": {
    "name": "ant0_cores",
    "nodeGroup": [ "tx_chain0.*" ],
    "tileGroup": [ "(0,0):(3,3)" ]
  },
  "areaGroup": {
    "name": "ant1_cores",
    "nodeGroup": [ "tx_chain1.*" ],
    "tileGroup": [ "(4,0):(7,3)" ]
  }
}
```

or add constraints in the graph:

```
location<graph>(tx_chain0) = bounding_box(0,0,3,3);
location<graph>(tx_chain1) = bounding_box(4,0,7,3);
```


2. Define stamp and repeat constraint in the `aiecost` file.

```
{
  "GlobalConstraints": {
    "isomorphicGraphGroup": {
      "name": "isoGroup",
      "referenceGraph": "tx_chain0",
      "stampedGraphs": [ "tx_chain1" ]
    }
  }
}
```

Note that the graph that is designated as the reference graph can also be given additional constraints such as co-location or absolute location constraints. These are automatically applied to other graphs with an appropriate offset.

Note: For more information see [Mapping Constraints](#) in *AI Engine Kernel and Graph Programming Guide* (UG1079).

Event Trace Reference

Simulation Event Trace

An event in the AI Engine simulator gives information about a system at a specific instant in time. An event that is associated with a time stamp, type and set of data values, is referred to as a payload. The interpretation of the payload depends on the type of the event. A time stamp allows ordering of events, computation of causality relationships, and implementation of verifiers on a sequence of events.

For event modeling purposes, the key modules of the AI Engine array are processor cores, DMA, lock modules, memory and I/O streams. Each module can be viewed as an event generator/responder. Each module receives an event and responds to the event. As a response, new events could be generated. Events are classified based on the generator of the events. The time stamp is not mentioned, explicitly, in the event definitions. Each event is described with a payload, which is a couple of values associated with the event. Each AI Engine, memory, DMA, or lock is addressable by a 2-dimensional index <col, row>, which is the column and row index in the AI Engine array. Some of the AI Engine events are shown in the following table. All events have a time stamp, which is omitted from the table.

Table 81: AI Engine Events

Event Name	Values	Description
CORE_RESET	<col, row>	Core <col, row> is out of reset and starts executing code from PC0.
CORE_WAIT	<col, row>	Core <col, row> is waiting for lock, stream input, or memory response.
CORE_READY	<col, row>	Core <col, row> is ready after a stall is serviced.
ACQ_READ_LOCK_REQ ACQ_WRITE_LOCK_REQ	<col, row, lockid, dir, pc, info>	Core <col, row> requests for a lock to one of its neighboring memory blocks. The <pc> is the current program counter. The <info> is the symbolic information.
REL_LOCK_REQ	<col, row, lockid, dir>	Core <col, row> releases a lock in a neighboring lock module.

Table 81: AI Engine Events (cont'd)

Event Name	Values	Description
CORE_READ_REQ	<col, row, address, dir, port, bank, data, pcinfo, addrinfo>	Core <col, row> requests a read at a memory address. The address is a 15-bit address local to the memory. Together, the <dir, address> forms a 17-bit address. The <dir> field specifies [E, W, N, S] direction. The <port> specifies the load port (A,B), and <bank> specifies access to an odd or even bank. Symbolic information about program variables and the <pc> value is available in the event.
CORE_WRITE_REQ	<col, row, address, dir, bank, data, size.pcinfo, addrinfo>	Core <col, row> writes data at a particular 15-bit address. The <dir> specifies the direction of the memory being accessed. Symbolic information about the <pc> value and data symbol are available in the event.
PC_CHANGE	<col, row, pc, info>	Core <col, row> shows the change in the <pc> value. It is useful for identifying loops.
DMA_S2MM_ACQ_LOCK	<col, row, ch>	DMA requests a lock. The <ch> field represents the DMA channel number. The supported channels are 0 and 1.
DMA_S2MM_IDLE	<col, row, bd, ch>	DMA in an idle state. <bd> is the buffer descriptor.
DMA_S2MM_START	<col, row, bd, ch, start>	Start DMA transfer at the start address
DMA_S2MM_DONE	<col, row, bd, ch>	End DMA transfer at the end address
DMA_S2MM_LOCKSTALL	<col, row, bd, ch>	DMA is stalled to get a lock
DMA_S2MM_LOCKSTALL_RELEASE	<col, row, bd, ch>	DMA gets a lock and stall is released
DMA_MM2S_ACQ_LOCK	<col, row, ch>	DMA <col, row> requests a lock
DMA_MM2S_IDLE	<col, row, ch>	DMA <col, row> in an idle state
DMA_MM2S_START	<col, row, bd, ch, start>	Start DMA transfer at <col, row> at the start address, the <bd> is the buffer descriptor.
DMA_MM2S_DONE	<col, row, bd, ch>	End DMA transfer at <col, row> at the end address, the <bd> is the buffer descriptor.
DMA_MM2S_LOCKSTALL	<col, row, bd, ch>	DMA is stalled to get a lock
DMA_MM2S_LOCKSTALL_RELEASE	<col, row, bd, ch>	DMA gets a lock and stall is released
IO	<dir, id, data >	An I/O event represents new data on an input stream. The <dir> [E, W, N, S] and <id> [0-15] together represent a physical stream, the data field represents 32-bit data appearing on the stream.
DATA_HEAD	<col, row, name, netid, pktid, idx>	Core <col, row> transfers packet switched data header with routing <name>, on net<netid>. This packet has packet ID <pktid>. <idx> is reserved.
DATA_START	<col, row, name, netid, pktid, idx>	Data payload (either packet switched or circuit switched) start. All fields follow the meaning of DATA_HEAD fields.
DATA_RESUME	<col, row, name, netid, pktid, idx>	Data payload resumed after being stalled. All fields follow the meaning of DATA_HEAD fields.
DATA_STALL	<col, row, name, netid, idx>	Data payload stalled within packet payload. All fields follow the meaning of DATA_HEAD fields.
DATA_END	<col, row, name, netid, idx>	Data payload end (either packet switched or circuit switched). All fields follow the meaning of DATA_HEAD fields.
PL_TO_SHIM, SHIM_TO_PL	<name, col, channelId, data0, data1, tlast>	Data movement between the PL and AI Engine array interface (SHIM). Name shows the PL block name.

Table 81: AI Engine Events (cont'd)

Event Name	Values	Description
PL2PL, PL2PL_E	<name, port, data0, data1, tlast, tkeep>	PL port activity start and end. Name and port show PL block name/port.
DM_READ_REQ	<col, row, portname>	A read request event on a port of a memory module. A read port can be a core, DMA, or memory-mapped AXI4 interface read request port.
DM_WRITE_REQ	<col, row, portname>	A write request event on a port of a memory module. A write port can be a core, DMA, or memory-mapped AXI4 interface write request port.
DM_BANK_CONFLICT	<col, row, bankid, bank>	A bank conflict event shows a bank conflict because multiple requests are sent to the same bank of a memory module.

Hardware Event Trace

The following table contains all the possible events produced during hardware event tracing.

An example of how to read the event is if the `functions` event is selected only the `FUN_CALL` and `FUN_RETURN` are produced and provide details about the tile the function resides on and the program counter (PC), and whether the event is calling the function, or if the function is being called.

Table 82: Hardware Event Trace

Event Name	Values	Description
FUN_CALL	<col, row, PC, val, info>	PC: function call instruction PC Val: 0 or 2 Info: calling function name with val 0 called function name with val 2
FUN_RETURN	<col, row, PC, val, info>	PC: function return instruction PC Val: 0 or 2 Info: Returning function name with val 0 Returned function name with val 2
MEM_STALL	<col, row, val, info>	Val: 23 (fixed) Info: Function name
MEM_STALL_RELEASE	<col, row, val, info>	Val: 2 (fixed) Info: Function name
STREAM_STALL	<col, row, val, info>	Val: 24 (fixed) Info: Function name

Table 82: Hardware Event Trace (cont'd)

Event Name	Values	Description
STREAM_STALL_RELEASE	<col, row, val, info>	Val: 2 (fixed) Info: Function name
CASCADE_STALL	<col, row, val, info>	Val: 25 (fixed) Info: Function name
CASCADE_STALL_RELEASE	<col, row, val, info>	Val: 2 (fixed) Info: Function name
LOCK_STALL	<col, row, val, info>	Val: 26 (fixed) Info: Function name
LOCK_STALL_RELEASE	<col, row, val, info>	Val: 2 (fixed) Info: Function name
DMA_S2MM_RUNNING	<col, row, ch, val>	Ch: channel ID Val: 0 (activity ends) 1 (activity starts)
DMA_MM2S_RUNNING	<col, row, ch, val>	Ch: channel ID Val: 0 (activity ends) 1 (activity starts)
OVERRUN	<col, row>	Overrun occurred in tile <col, row>

Event API

The event API provides functions to configure AI Engine hardware resources for performance profiling and event tracing. In this release, a subset of performance profiling use cases are supported.

Enumeration

```
enum io_profiling_option
{
    io_total_stream_running_to_idle_cycles,
    io_stream_start_to_bytes_transferred_cycles,
    io_stream_start_difference_cycles,
    io_stream_running_event_count
};
```

The `io_profiling_option` contains the enumerated options for performance profiling using PLIO and GMIO objects. The `io_total_stream_running_to_idle_cycles` option represents the total accumulated clock cycles in between the stream running event and the stream idle event of the corresponding stream port in the interface tile. This option can be used to profile platform I/O bandwidth.

The `io_stream_start_to_bytes_transferred_cycles` option represents the clock cycles in between the first stream running event to the event that the specified number of bytes are transferred through the stream port in the interface tile. This option can be used to profile graph throughput.

The `io_stream_start_difference_cycles` option represents the clock cycles elapsed between the first stream running events of the two platform I/O objects. This option can be used to profile graph latency.

The `io_stream_running_event_count` option represents the number of stream running events. This option can be used to profile the graph throughput during a period of time for streaming applications.

Member Functions

```
static handle start_profiling(IoAttr& io, io_profiling_option option,
uint32 value = 0);
```

This function configures the performance counters in the AI Engine and starts profiling. `io` is the platform GMIO or PLIO object. `option` is one of the `io_profiling_option` enumerations described in the previous topic. If the `io_stream_start_to_bytes_transferred_cycles` option is used, the number of bytes can be specified in the `value` parameter. This function should be called after `graph::init()`. It returns a handle to be used by `read_profiling` and `stop_profiling`. If the specification is incorrect or there is insufficient hardware resources to perform the profiling, an `invalid_handle` is returned.

```
static handle start_profiling(IoAttr& io1, IoAttr& io2, io_profiling_option
option, uint32 value = 0);
```

This function configures the performance counters in the AI Engine and starts profiling. Parameters `io1` and `io2` specify the two platform I/O objects. This function should be called after `graph::init()`. It returns a handle to be used by `read_profiling` and `stop_profiling`. If the specification is incorrect or there is insufficient hardware resources to perform the profiling, an `invalid_handle` is returned.

```
static long long read_profiling(handle h);
```

This function returns the current performance counter value associated with the `handle`.

```
static void stop_profiling(handle h);
```

This function stops the performance profiling associated with the `handle` and releases the corresponding hardware resources.

Encrypting AI Engine Kernels

The methodology for encrypting the AI Engine source code helps the IP developer to deliver IP sources to the end-user in a more secure way. The feature does not deliver AI Engine source file in clear text, and does not expose any kernel source content during compilation.

This feature aims for a trust model based upon the concerns of the IP author. The IP author determines how their secured IP can be viewed and used. The trust model can be summed up by the following: Encryption of source code is a mandate by the IP author that they expect their IP to be secure. Tool behavior defaults to the maximum reasonable protection that is available, while still accomplishing tool goals.

The AI Engine source encryption feature is available from a lounge website: https://www.xilinx.com/member/ai_engine_encryption.html (Registration required). Available on the lounge are the license and Synopsys keys that are necessary to encrypt and decrypt the AI Engine source code. Supporting documentation is also available.

Mapper/Router Design Rule Checks

The following tables list the Design Rule Check IDs that are part of the AI Engine mapper and router. When a rule check is violated, the corresponding ID and error message displays in log files generated during AI Engine compilation. The description tab provides a brief summary about the rule ID.

Table 83: AIE-PRE-MAPPER DRC Rules

ID	Error Message	Resolution
AIE-PRE-MAPPER-1	Area group is out of device bounds.	Review applied area group constraints.
AIE-PRE-MAPPER-2	AI Engine interface tile instance Area group violation, conflicting location constraints.	Review the applied interface tile constraints.
AIE-PRE-MAPPER-3	Instance Area group violation, conflicting location constraints.	Review the applied area group constraints.
AIE-PRE-MAPPER-4	Colocated buffers cannot fit within a single bank.	Reduce buffer size or relax colocation constraint requirement.
AIE-PRE-MAPPER-5	Insufficient PLIO capacity violation.	Contact AMD support.
AIE-PRE-MAPPER-6	Insufficient PLIO channel capacity violation.	Contact AMD support.
AIE-PRE-MAPPER-7	Insufficient GMIO channel capacity violation.	Contact AMD support.
AIE-PRE-MAPPER-9	Area group PLIO capacity violation.	Contact AMD support.
AIE-PRE-MAPPER-11	Area group PLIO 128 bit capacity violation.	Contact AMD support.
AIE-PRE-MAPPER-12	Area group PLIO capacity violation.	Contact AMD support.
AIE-PRE-MAPPER-13	Buffer location constraint violation, out of device bounds.	Contact AMD support.
AIE-PRE-MAPPER-14	AI Engine interface tile instance location constraint violation, out of device bounds.	Review the applied interface location constraint.
AIE-PRE-MAPPER-15	AI Engine interface tile instance location constraint does not have a valid AI Engine interface tile site.	Review the applied interface location constraint.
AIE-PRE-MAPPER-16	GMIO instance location constraint violation, location constraint to channel in a column with no GMIO support.	Review the applied GMIO location constraint.
AIE-PRE-MAPPER-17	GMIO instance location constraint violation.	Review the applied GMIO location constraint.
AIE-PRE-MAPPER-18	Instance location constraint violation.	
AIE-PRE-MAPPER-19	Illegal Y coordinate offset specified.	Check supplied (x,y) coordinate offset, contact AMD support.
AIE-PRE-MAPPER-20	Illegal Y coordinate offset specified.	Check supplied (x,y) coordinate offset, contact AMD support.
AIE-PRE-MAPPER-21	Y coordinate offset violation.	Check supplied (x,y) coordinate offset, contact AMD support.

Table 83: AIE-PRE-MAPPER DRC Rules (cont'd)

ID	Error Message	Resolution
AIE-PRE-MAPPER-22	Illegal Y coordinate offset specified.	Check supplied (x,y) coordinate offset, contact AMD support.
AIE-PRE-MAPPER-23	Illegal Y coordinate offset specified.	Check supplied (x,y) coordinate offset, contact AMD support.
AIE-PRE-MAPPER-24	Illegal Y coordinate offset specified.	Check supplied (x,y) coordinate offset, contact AMD support.
AIE-PRE-MAPPER-25	Illegal Y coordinate offset specified.	Check supplied (x,y) coordinate offset, contact AMD support.
AIE-PRE-MAPPER-26	DMA FIFO size violation.	Contact AMD support.
AIE-PRE-MAPPER-27	DMA FIFO start address alignment violation.	Contact AMD support.
AIE-PRE-MAPPER-28	Outgoing DMA channels violation.	Contact AMD support.
AIE-PRE-MAPPER-29	Incoming DMA channels violation.	Contact AMD support.
AIE-PRE-MAPPER-30	Incoming buffers with DMA usage violation.	Contact AMD support.
AIE-PRE-MAPPER-31	Outgoing buffers with DMA usage violation.	Contact AMD support.
AIE-PRE-MAPPER-32	Buffer offset violation.	Contact AMD support.
AIE-PRE-MAPPER-33	Buffer bank violation.	Contact AMD support.
AIE-PRE-MAPPER-34	Incoming GMIOs location constraints violation.	Contact AMD support.
AIE-PRE-MAPPER-35	Outgoing GMIOs location constraints violation.	Contact AMD support.
AIE-PRE-MAPPER-36	AI Engine interface tile incoming location constraint violation.	Contact AMD support.
AIE-PRE-MAPPER-37	AI Engine interface tile outgoing location constraint violation.	Contact AMD support.
AIE-PRE-MAPPER-39	Location constraint on non registered channel.	Contact AMD support.
AIE-PRE-MAPPER-40	Location constraint on non registered channel.	Contact AMD support.
AIE-PRE-MAPPER-41	Location constraint on non registered channel.	Contact AMD support.
AIE-PRE-MAPPER-42	Reserved memory violation.	Contact AMD support.
AIE-PRE-MAPPER-43	Ping-pong buffers location constraints.	Contact AMD support.
AIE-PRE-MAPPER-44	Ping-pong buffers offset constraints.	Contact AMD support.
AIE-PRE-MAPPER-45	Overallocation of tile memory.	Contact AMD support.
AIE-PRE-MAPPER-46	Cascade location constraint violation.	Contact AMD support.
AIE-PRE-MAPPER-47	Cascade fit violation.	Contact AMD support.
AIE-PRE-MAPPER-48	Cascade length violation.	Contact AMD support.
AIE-PRE-MAPPER-49	Overlapping buffers.	Contact AMD support.
AIE-PRE-MAPPER-50	Buffer size violation.	Contact AMD support.
AIE-PRE-MAPPER-51	Overallocation of device memory.	Contact AMD support.

Table 84: AIE-MAPPER DRC Rules

ID	Error Message
AIE-MAPPER-1	Cluster not placed.
AIE-MAPPER-2	Instance not placed.

Table 84: AIE-MAPPER DRC Rules (cont'd)

ID	Error Message
AIE-MAPPER-3	Buffer not placed.
AIE-MAPPER-4	Buffer offset violation.
AIE-MAPPER-5	AI Engine interface tile (Shim) instance out of Area group(s).
AIE-MAPPER-6	Instance out of Area group(s).
AIE-MAPPER-7	PLIO/GMIO instance location constraint violation.
AIE-MAPPER-8	Instance location constraint violation.
AIE-MAPPER-9	Buffer location constraint violation.
AIE-MAPPER-10	Colocated cluster-buffer placement violation.
AIE-MAPPER-11	Colocated cluster-packet control node placement violation.
AIE-MAPPER-12	Colocated Buffers not placed in same tile.
AIE-MAPPER-13	Overlapping buffers found.
AIE-MAPPER-14	Tile HW lock threshold exceeded.
AIE-MAPPER-15	Cascaded placement violation.
AIE-MAPPER-16	Stamping violation, mismatching instances.
AIE-MAPPER-17	Packet control node not placed.
AIE-MAPPER-18	Ping and pong buffers are placed in different columns.
AIE-MAPPER-19	Ping and pong buffers are placed in different rows.
AIE-MAPPER-20	PLIO instance placed in channel which does not support BLI registers.
AIE-MAPPER-21	Buffers overlaps with location set for DMA FIFO.
AIE-MAPPER-22	Stamping violation Buffers in isomorphic area groups do not match.
AIE-MAPPER-23	Stamping violation Buffer does not have expected DMA duplicate buffer.

For assistance in resolving these DRC errors, contact AMD Support.

Table 85: AIE-ROUTER DRC Rules

ID	Error Message
AIE-ROUTER-1	Net has no routing.
AIE-ROUTER-3	Demand exceeding the sum total of net utilization on a node.
AIE-ROUTER-4	Net FIFO requirement violation.
AIE-ROUTER-5	Router overlapping buffer offsets.
AIE-ROUTER-6	Trace net routing should not overlap with data net.
AIE-ROUTER-7	Router DMA FIFO size violation.
AIE-ROUTER-8	Router DMA FIFO address alignment.
AIE-ROUTER-9	Route tree for net is not connected to driver %s.
AIE-ROUTER-10	The route tree for net has gaps.
AIE-ROUTER-11	Load node is not connected to route tree.
AIE-ROUTER-12	Trace net has no valid status.

For assistance in resolving these DRC errors, contact AMD support.

Related Information

[Design Rule Check Options](#)

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to <https://docs.amd.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

References

These documents provide supplemental material useful with this guide:

1. The AI Engine Intrinsic User Guide ([UG1078](#))
2. AI Engine-ML Intrinsic User Guide ([UG1583](#))
3. AI Engine Kernel and Graph Programming Guide ([UG1079](#))
4. AI Engine-ML Kernel and Graph Programming Guide ([UG1603](#))
5. Data Center Acceleration using Vitis ([UG1700](#))
6. Embedded Design Development Using Vitis ([UG1701](#))
7. Vitis Reference Guide ([UG1702](#))
8. Vitis Software Platform Release Notes ([UG1742](#))
9. Versal Adaptive SoC AI Engine Architecture Manual ([AM009](#))
10. Versal Adaptive SoC Register Reference ([AM012](#))
11. Versal Adaptive SoC AI Engine Register Reference ([AM015](#))
12. Versal Adaptive SoC AIE-ML Architecture Manual ([AM020](#))
13. Versal Adaptive SoC AIE-ML Register Reference ([AM025](#))
14. The ASIP Programmer Chess Compiler User Manual, found on the Versal AI Engines Secure Site, provides a list of all the pragmas and functions to help optimize your AI Engine kernel code.
15. AI Engine API User Guide ([UG1529](#))
16. Vitis Model Composer User Guide ([UG1483](#))
17. SmartLynq+ Module User Guide ([UG1514](#))
18. Versal adaptive SoC data sheets:
 - Versal Architecture and Product Data Sheet: Overview ([DS950](#))
 - Versal AI Core Series Data Sheet: DC and AC Switching Characteristics ([DS957](#))
19. Versal Adaptive SoC Design Guide ([UG1273](#))
20. Power Design Manager User Guide ([UG1556](#))

21. Versal Adaptive SoC Programmable Network on Chip and Integrated Memory Controller LogiCORE IP Product Guide ([PG313](#))

22. Answer Records:

- AI Engine Compiler - General Guidance and Known Issues for the Vitis 2020.2 tool and later versions ([Answer 75790](#))
- AIE Simulator - General Guidance and Known Issues for the Vitis 2020.2 tool and later versions ([Answer 75788](#))
- AI Engine Solution Center ([Answer 75837](#))

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
11/18/2024 Version 2024.2	
General Update	Removed references to classic AMD Vitis™ IDE, and AMD Vitis™ Analyzer because they were replaced by the AMD Vitis™ Unified IDE in 2024.2.
	Moved the following sections to <i>Vitis Reference Guide</i> (UG1702):
	• Using the Vitis Unified IDE • Viewing Compilation Results in the Analysis View of the Vitis Unified IDE
	Moved the following sections to <i>Embedded Design Development Using Vitis</i> (UG1701):
	• Hardware Emulation • AI Engine Hardware Profile and Debug Methodology
	Removed Software Emulation because the feature is deprecated.
Compiling using v++ (Unified Compiler)	Clarified the acceptable characters to use in file and directory names.
Event Tracing Options	Removed deprecated event trace configuration values: <code>functions_partial_stalls</code> , <code>functions_all_stalls</code> . Now, these values must be specified in the <code>xrt.ini</code> file for hardware emulation and run.
Compiling AI Engine Graph for Independent Partitions	Clarified that the Analysis view of the Vitis Unified IDE can open only one partition compilation result at a time.
Data Snapshots	• Added example of the configuration file contents <code>./Work/options/x86sim_dump.config</code> that the compiler generates to specify what data to dump when running <code>x86simulator --dump</code>. • Added X86simulator dump configurations categories.
Latency and Throughput Estimates	Updated right-click option labels available from the Latency Table in the Vitis IDE.

Section	Revision Summary
Profiling Memory Tiles	Updated with additional metric name changes: - input_throughputs is now s2mm_throughputs. - output_throughputs is now mm2s_throughputs.
Generating AI Engine Status using XSDB	- For the -aie-version option, updated the filename in path that contains the hw_gen value to aie_trace_config.json. - Updated the Tcl script to source the aie_status.
XSDB Flow	Updated JSON file generated (-config-file <json-file>) to reflected per 2024.2 change. Added options: -tile-based-aie-metrics <column all>:<metric_set>, -tile-based-aie-metrics <mincolumn>:<maxcolumn>:<metric_set>. Updated valid metrics: - Core and Memory modules: Added s2mm_throughputs and mm2s_throughputs; Removed read_throughputs and write_throughputs. - Interface tiles: Added input_stalls and output_stalls; Modified output_throughputs, and input_throughputs. - Memory tiles: Modified s2mm_channels_details and mm2s_channels_details; Added conflict_stats1, conflict_stats2, conflict_stats3, and conflict_stats4.
Profiling for AI Engine	Added the stream_put_get metric. Mentioned that s2mm_throughputs and mm2s_throughputs metrics are automatically selected to match the specified Memory Module metric sets.
Controlling Data Transfers between AI Engine and Global Memory	Added information about external buffer objects, which is an alternative to GMIO objects for memory-mapped connections between global memory and the AI Enginegraph. Updated code example, and added code example.
Controlling AI Engine Partitions	New topic that includes an example of code to control multiple partitions.
06/27/2024 Version 2024.1	
General Updates	Editorial updates only. No technical content updates.
05/30/2024 Version 2024.1	
General Updates	Deprecated the aiesimulator --end-wait-time option in 2024.1.
	Renamed Event Trace Option metrics: functions_all_stalls changed to all_stalls, and functions_partial_stalls changed to partial_stalls.
	Changed aiecompiler to v++ -c --mode aie per 2024.1 recommendation.
Setting Up the Vitis Tool Environment	Added that the AI Engine tools require a license which can be generated for free from any eligible AMD licensing account.
Aiecompiler to Vitis Command Line Interface Migration	Added that the --aiecompiler --export option is used to generate a aiecompiler.cfg configuration file. Updated sample aiecompiler.cfg file.

Section	Revision Summary
Module Specific Options	Specified that <code>fast</code> is the default value for <code>--float-accuracy</code> .
Miscellaneous Options	Added the <code>--swfifo-threshold</code> option.
Compiling AI Engine Graph for Independent Partitions	Provides details on AI Engine support of partitions.
Managing AI Engine Components with the Python Command Line Interface	Clarified that you can use Python Command Line Interface (CLI) commands to automate the AI Engine component creation and build process in the Vitis IDE, and with a Python script.
Compiling the Design	Removed support for the preprocessor directive in x86 simulator debug.
Enabling Third-Party Simulators in a Configuration File	Updated configuration file settings for the release.
AI Engine Simulator	Updated to support threads in the AI Engine multithreaded simulator.
	Updated to show PLIO Average Throughput report generated in AI Engine simulation.
Simulator Options	<code>--hang-detect-time=<time in ns></code> option now supported in AI Engine-ML devices.
Reusing AI Engine Simulator Options	- Updated to reflect that the <code>--profile</code> option is valid only for the AMD Vivado™ simulator. - Updated to reflect that to view the AMD Vivado™ simulator waveform GUI, use the <code>launch_hw_emu</code> script with <code>-g</code> option, and also use the <code>-g</code> option during the <code>v+ + --link</code> stage as well. - Removed the <code>AIE_Profile</code> option because it is no longer supported for hardware emulation. - Added the <code>AIE_PKG_DIR</code> option to set the Work directory used by the AI Engine compiler.
Evaluating FIFO Depth To Break Deadlocks	The <code>--evaluate-fifo-depth</code> option is available for AI Engine-ML designs.
Latency and Throughput Estimates	Added new topic.
Exporting Summary Tables from the Analysis View	New section added to explain how to export summary tables to a CSV file.
AI Engine Stall Analysis in the Vitis IDE	Added note that a run summary file is generated for third-party simulators.
DMA Stall Analysis	New section to explain how to configure and analyze DMA stalls.
Profiling for Memory Modules, Profiling Memory Tiles, Profiling for Interface Tiles, and Metric Sets	Updated metric sets.
DMA Event Trace in AI Engine-ML Tiles	New section to explain that DMA event tracing is available for AI Engine-ML tiles.
XRT Support for Event APIs	New section that introduces XRT support for event profiling.
XRT Flow	Updated <code>-graph-based-memory-tile-metrics</code> to <code>graph_based_memory_tile_metrics</code> . Updated <code>-tile-based-memory-tile-metrics</code> to <code>tile_based_memory_tile_metrics</code> .
Trace Offload Methodology, Event Tracing in Hardware, and Profiling the AI Engine in Hardware	Added note regarding support on various BDC platforms.
Buffer Data Types	Identified which data types are applicable to AI Engine and/or AI Engine-ML.

Section	Revision Summary
Handling Large Designs	New section provides suggestions for handling large designs to alleviate placement challenges.
Event API	Moved to UG1076 from UG1079 & UG1603.
12/04/2023 Version 2023.2	
General Updates	Continued to update to reflect the usage of the new unified AMD Vitis™ IDE.
Simulator Options	Added <code>--dump-size=SIZE</code> option.
AI Engine Compiler Options	Added more details about the <code>pl-register-threshold</code> option.
Inline Keywords	Added details about the <code>inline</code> keyword.
FIFO Depth Evaluation	Added content about FIFO depth evaluation.
Compiling and Linking Host Code with ADF API	Added new section.
AI Engine Simulation-Based Profiling	Further defined information generated in the Function Reports section of the Profile report.
10/18/2023 Version 2023.2	
Throughout the document	Updated to reflect the usage of the new unified AMD Vitis™ IDE.
	Updated to reflect AI Engine-ML support.
Migration to Vitis Unified IDE	Added information about migrating 2023.1 command-line projects to unified Vitis IDE, and migrating 2023.1 Vitis Classic IDE projects to Vitis Unified IDE.
Using the Vitis Unified IDE	Added information on how to create AI Engine components, and simulate and debug them in the new unified Vitis IDE.
AI Engine Compiler Options	Added <code>--part</code> , <code>-runtime-opt</code> , <code>--float-accuracy arg</code> , <code>--lock-fence-mode</code> , and <code>--evaluate-fifo-depth</code> options.
Mapper and Router Options	Added <code>disableMultiDMAFifo</code> and <code>skipUnroutableTraceNets</code> options.
Macro for Device Architectures	Added <code>_AIE_ARCH_</code> macro details.
Compiling using v++ (Unified Compiler)	Updated to reflect the usage of the new unified Vitis command line AI Engine compiler.
Simulation Input and Output Data Streams, and CSV File Format	Added support for CSV file format for PLIO inputs and outputs.
Evaluating FIFO Depth To Break Deadlocks	Added information on how to use the FIFO Depth Report from the <code>aiesimulator</code> to alleviate deadlocks.
Reusing AI Engine Simulator Options	Clarified that the <code>AIE_Profile</code> option is not supported for third-party simulators.
Inputs	Added note that the <code>main</code> function must have a return statement. Otherwise, <code>aiecompiler</code> will error out.
Simulator Options	Added <code>--dump-size x86simulator</code> option.
Memory Tile Event Trace for AI Engine-ML-based Devices	Added event trace details of memory tiles for AI Engine-ML based applications.
Profiling Memory Tiles	Added profiling details of memory tiles for AI Engine-ML based applications.
AI Engine Trace from VCD	Updated the directory path of the generated event trace data.

Section	Revision Summary
Generating VCD with Select Signals	Added row, column and time window options in the <code>options.txt</code> file which are used to generate the VCD.
Using Vitis IDE to Generate a VCD with Select Signals	Added information on how to use the Vitis Unified IDE to specify row, column and time window options which are used to generate the VCD.
Profiling the NoC	Added information on how to monitor network performance on the NoC.
XSDB Flow	Updated table to reflect profiling options for memory tiles. Added multi-level hierarchy graph support for interface tile metrics.
XRT Flow	Updated table to reflect profiling options for memory tiles. Added multi-level hierarchy graph support for interface tile metrics.
XSDB Flow	Updated multi-level hierarchy graph support example.
XRT Flow	Removed the <code>offload_interval_us</code> , and <code>file_dump_interval_s</code> options. Added multi-level hierarchy graph support example.
Viewing Profile Results in the Vitis IDE	Added the new Performance Annotation in Graph View section.
Event Trace Offload using High Speed Debug Port	Added information in support of using high speed debug port to offload trace data.
Event Trace Routing	Added information on how to handle event trace routing failures.
Integrating the Application Using the Vitis Tools Flow chapter	Moved content to the <i>Vitis Unified Software Platform Documentation: Application Acceleration Development</i> (UG1393).
Debugging the AI Engine Application chapter	Moved content to the <i>Vitis Unified Software Platform Documentation: Application Acceleration Development</i> (UG1393).
06/23/2023 Version 2023.1	
Chapter 4: Simulating an AI Engine Graph Application , and Chapter 6: Performance Analysis of AI Engine Graph Application on Hardware	Added tip regarding the precompiled AI Engine test harness, with link to GitHub for more information.
Compiling the Embedded Application for the x86 Processor	Added note to set the GCC path from the AMD Vitis™ Install.
05/16/2023 Version 2023.1	
General Updates	Update screen captures to reflect the new Vitis IDE to view compilation and simulation reports, and summary files.
	Added the <code>-a</code> option for launching Vitis Analyzer with a summary file, as in, <code>vitis_analyzer -a</code> throughout the document.
	Moved "Using the Restrict Keyword in AI Engine Kernels" and "Non-Templated Versions of Window and Stream APIs" appendixes to UG1079.
	Updated window to buffer, as in <code>input_window</code> to <code>input_buffer</code> and <code>output_window</code> to <code>output_buffer</code> .
Tools	Added new Vitis IDE to view compilation and simulation reports, and analyze output files generated by the AI Engine compiler. Added Vitis unified command line interface.

Section	Revision Summary
AI Engine Compiler Options	Added <code>--graph-iterator-event</code> event trace compiler option. Added <code>--evaluate-fifo-depth</code> miscellaneous option that helps to avoid the manual and iterative process of adding FIFO depth. When the design is compiled with this option and simulated using <code>aiesimulator</code> , FIFO depth recommendations are generated by the simulator.
Opening AI Engine Compilation Summary Reports	Updated examples, and screen captures in this section to reflect the new Vitis IDE.
Chapter 4: Simulating an AI Engine Graph Application	Updated Simulation Flows, Simulation Models, and Simulation Features tables.
Simulator Options	Updated description for <code>--display-run-interval=<time in ns></code> , <code>--dump-vcd=<file></code> , <code>--enable-handshake-ext-tb</code> , <code>--enable-memory-check</code> , <code>--hang-detect-time=<time in ns></code> , and <code>--online</code> .
Simulator Options	Added the <code>--options-file</code> option to specify the text file for selective VCD.
Generating VCD with Select Signals	New section on generating AI Engine selective VCD modules.
External Traffic Generator	Added new section.
Reusing AI Engine Simulator Options	Added information about viewing the run summary.
Trace Compare	Added recommendation to use <code>vitis_analyzer --classic</code> for the trace compare feature.
Stream Switch FIFO Visualization	New section on data visualization of stream switch FIFOs.
Profiling the AI Engine, Memory Modules and Interface Tiles and Profiling Flows	Updated bandwidth with throughput.
XSDB Flow	Added <code>-start-type</code> , <code>-start-time</code> , and <code>-start-iteration</code> options, and examples. Renamed <code>-tile-based-aie-tile-metrics</code> to <code>-tile-based-aie-tile-metrics</code> , <code>-graph-based-aie-tile-metrics</code> to <code>-graph-based-aie-tile-metrics</code> , and <code>-tile-based-interface-tile-metrics</code> to <code>-tile-based-interface-tile-metrics</code> .
XRT Flow	Added <code>start_type</code> , <code>start_time</code> , and <code>start_iteration</code> options, and examples.
Using the Delayed Event Trace	New section.
FIFO Depth Visualization in the Vitis IDE	Added section for data visualization of stream switch FIFOs.
Analyzing AI Engine Status	Added note that mentions that you can access error events in classic Vitis Analyzer with <code>vitis_analyzer --classic</code> , and not in the new Vitis IDE Analysis view.
Troubleshooting Event Trace in Hardware	Updated <code>-config-level</code> to use <code>-graph-based-aie-tile-metrics</code> in example, because <code>-config-level</code> is no longer supported.
Chapter 7: Programming the PS Host Application	Removed references to ADF APIs, and updated to use XRT API only.
Controlling the Application with the XRT C++ API	Updated the execution model for the XRT API controlling PL kernels and AI Engine graphs, and updated example code.
Controlling Data Transfers between AI Engine and Global Memory	Added note that only non-cacheable buffers are supported for AI Engine GMIO buffers.
Compiling and Linking Host Code for Linux	New section.

Section	Revision Summary
Compiling and Linking Host Code for Bare-Metal	New section.
Resetting and Reloading Flow of the AI Engine Array	New section.
Platforms	Updated Types of Platforms.
Compiling the Embedded Application for the Cortex-A72 Processor	Updated for XRT-only compile.
Resetting and Reloading Flow of the AI Engine Array	Added new section.
Introduction to the new Vitis Unified IDE	Added new appendix to introduce the preview mode tool.
10/05/2022 Version 2022.2	
Document title	Changed title to <i>AI Engine Tools and Flows User Guide</i> .
Throughout the document	Moved graph programming content to UG1079.
AI Engine Compiler Options	Added DRC Options. Updated Event Trace options.
Simulation Input and Output Data Streams	Added simulation option file details.
Enabling Third-Party Simulators in a Configuration File	Updated <code>v++ --link</code> Configuration to use generic <code><SIMULATOR DIRECTORY></code> variable.
Scalar RTP Data Analysis	New topic.
Profiling Graph Latency	Added example code for hardware and hardware emulation flows.
Profiling for AI Engine	Removed <code>stream_put_get</code> Metric.
XSDB Flow	Added XSDB Trace options.
Limitations	New topic.
Memory Model	Added explanation of the macro <code>X86SIM_THREAD_LOCAL</code> to make the global read/write thread safe.
Profiling for Interface Tiles	Updated <code>input_bandwidths</code> and <code>output_bandwidths</code> metrics. New <code>packets</code> metrics.
XSDB Flow	Updated and added new <code>aieprofile</code> options.
XRT Flow , and XRT Flow	Updated <code>xrt.ini</code> options, and examples throughout.
Troubleshooting Event Trace in Hardware	Updated troubleshooting options for when trace packets are being dropped on certain streams.
Controlling Data Transfers between AI Engine and Global Memory	New topic.
Iterative AI Engine Application Compilation	New topic.
Viewing Data from Buffer Port Interfaces	New topic.
Linking the System	Added <code>sp</code> Connectivity Section Options
Analyzing AI Engine Status in Hardware Emulation	New topic to describe Vitis Analyzer support for AI Engine profile and status in hardware emulation.
Compiling the Embedded Application for the x86 Processor	New topic to describe support for PS on x86 flow in software emulation.
Creating Traffic Generators using System Verilog/Verilog	New topic to describe how to use Verilog or System Verilog modules/testbenches to drive traffic in and out of an ADF graph running in the AI Engine Simulator.
Appendix B: Encrypting AI Engine Kernels	New appendix, which refers to Encryption web lounge for more details.
05/25/2022 Version 2022.1	
AI Engine Compiler Options	Added note that two reserved words, <code>aie</code> and <code>adf</code> , are not valid namespace identifiers in graph programming

Section	Revision Summary
Opening AI Engine Compilation Summary Reports	Updated graphs for the 2022.1 release.
I/O, Nets, and Tiles	Updated example details and Vitis Analyzer GUI screen captures for the 2022.1 release.
Interface Channels	Added new graph interface channel details.
Event Profile APIs for Graph Inputs and Outputs	Added details for events on nets.
Viewing Guidance in the Vitis IDE	Added this guidance.
Using Traffic Generators for AI Engine System Projects	Clarified that you can write the external traffic generator in HDL, in addition to C++, and Python.
04/26/2022 Version 2022.1	
Chapter 1: Overview	Updated Vitis core development kit details for the 2022.1 release.
Creating a Data Flow Graph (Including Kernels)	Updated to the 2022.1 programming model.
Synchronous Window Access	Added sections to explain window to window broadcasting, and multi-rate design support.
Stream-Based Access	Added section to explain stream-based access using cascade stream.
Using Streams in Parallel	Updated list of 32-bits and 64-bit macros.
Run-Time Parameter Support Summary	Removed AI engine-to-AI engine run time parameter constructs, which are no longer supported.
Multicast Support	Updated Multicast Support Scenarios table.
Design Flow Using RTL Programmable Logic	Updated examples to reflect 2022.1 programming model changes.
Chapter 12: Graph Programming Model	Updated programming model details and examples throughout the chapter.
AI Engine Compiler Options	Added new multi-rate options. Updated <code>--Xrouter=<string></code> example.
Mapper and Router Options	Removed <code>enableSplitAsBroadcast</code> option (always on). Added <code>disablePathBalancing</code> option.
x86 Functional Simulator	Added ability to visualize X86 simulation output in Vitis Analyzer.
Compiling the Design	Added X86 simulator options.
Data Snapshots	Added ability to visualize snapshots in Vitis Analyzer.
Limitations	Removed 'Simulation Output File Processing Considerations' and 'adf::headers Constraint and aie_api Include Files' sections because these x86 simulation limitations have been addressed.
Simulator Options	Added new options, and hang detection details.
Enabling Third-Party Simulators in a Configuration File	Updated VCS details. Added Riviera simulator information.
Profiling the AI Engine in Hardware Profiling for Interface Tiles Profiling the AI Engine, Memory Modules and Interface Tiles	Added the events used for DMA write/read_bandwidths. Added the ability to profile interface events.
FIFO Depth Visualization in the Vitis IDE	Add ability to visualize DMA FIFO Depth in Vitis Analyzer from simulation VCD data.
XSDB Flow XRT Flow	Added ability to specify event trace start time. Added ability to periodically offload trace data at regular intervals.

Section	Revision Summary
Viewing Profile Results in the Vitis IDE	Added interface metrics example. Added ability to consolidate multiple profile results in Vitis Analyzer.
Analyzing AI Engine Status in Hardware Generating AI Engine Status Analyzing AI Engine Status	Added ability to report out AI Engine status in hardware, and added ability to open and analyze the report in Vitis Analyzer.
Packaging for DFX Platforms	Added ability to use the DFX platform in addition to the base platform, and added information on using this platform in hardware.
Multi-Process and Multi-Thread Control of AI Engine Graphs	Add clarification about <code>xrtGraphClose</code> and <code>xrtDeviceClose</code> behavior.
Platforms	Updated Types of Platforms to include DFX platforms.
Performance Metrics	Added Show Percentage button description.
Lock Stall Analysis Stream Stall Analysis Cascade Stall Analysis Memory Stall Analysis	Added program counter (PC) option, which allows you to cross-probe to the source code from the Trace view in Vitis Analyzer.
Using Traffic Generators for AI Engine Systems Using Traffic Generators in AI Engine Graphs	Added support for using traffic generators in x86 functional simulator, AI Engine simulator, software emulation, and hardware emulation. These traffic generators can be written in Python, C++, or HDL.
Host Programming for Bare-Metal Building a Bare-Metal System Host Programming Support Comparison Between Linux and Bare-Metal	Added details about Bare-Metal software stack. Compared and contrasted the capabilities of running host application in Bare-Metal vs. Linux operating systems.
Integrating the Application Using the Vitis Tools Flow Linking the System Packaging the System for Hardware	Updated to reflect the fact that v++ link now produces an XSA file.
AI Engine Hardware Profile and Debug Methodology	New chapter about the AI engine hardware profile and debug methodology.
<code>input_gmio/output_gmio</code> <code>input_plio/output_plio</code>	Updated document to reflect changes to programming models, which includes <code>input_gmio/output_gmio</code> , and <code>input_plio/output_plio</code> .
Other Constraints	Added the <code>repetition_count</code> constraint for multi-rate designs.
12/17/2021 Version 2021.2	
Chapter 8: Window and Streaming Data API	Added more supported unsigned integer data types.
Specifying Run-Time Data Parameters	Clarified description.
Programming Model Features	Changed heading of section.
AI Engine Compiler Options	Added table.
Profiling the AI Engine	New section.
Profiling Graph Throughput	Added information.
Profiling the AI Engine in Hardware	New section.
Event Tracing in Hardware	New section.
Hardware Event Trace	New section.
Troubleshooting Event Trace in Hardware	New section.

Section	Revision Summary
10/22/2021 Version 2021.2	
AI Engine Components	Updated.
Prepare the Kernels	Updated for AI Engine API.
Creating a Data Flow Graph (Including Kernels)	Figure representing graph connectivity added.
Chapter 8: Window and Streaming Data API	Updated data types for AI Engine API and template support.
Packet Switching Graph Constructs	Added an example for floating-point data.
Area Location Constraints	New section.
Hierarchical Constraints	Added information.
Programming Model Features	New section.
AI Engine Compiler Options	New options added.
Chapter 4: Simulating an AI Engine Graph Application	Added simulation flow related information.
Data Snapshots	New section.
Deadlock Detection	New section.
Trace Report	New section.
Memory Access Violations and Valgrind	New section.
Memory Model	Updated information.
Simulation Output File Processing Considerations	New section.
adf::headers Constraint and aie_api Include Files	New section.
Software Emulation	New section.
Simulator Options	New option added.
Hardware Emulation	New section.
Reusing AI Engine Simulator Options	Added information about setting the AI Engine compiler <code>workdir</code> environment variable, as well as manual creation of the sim options file.
AI Engine Simulation-Based Profiling	New section.
Buffer Data Types	Updated data types.
Supported Stream Data Types	
AI Engine Stall Analysis in the Vitis IDE	New section.
Multi-Process and Multi-Thread Control of AI Engine Graphs	New section.
AI Engine Error Events	Updated errors, as well as debug tips.
Running Software Emulation	New section.
Area Group Constraint	Updated properties.
Creating the AI Engine Graph Project and Top-Level System Project	Updated screenshots.
Building and Running the System	Updated to add software emulation.
Debugging the AI Engine Application	Debug information added.
Software Emulation Debug from the Vitis IDE	New section.
Running Software Emulation from the Command Line	New section.
Using the Debug Environment	Updated screenshots.
Watchpoints	New section.
Vitis IDE Layout for Software Emulation Debug	New section.

Section	Revision Summary
Non-Templated Versions of Window and Stream APIs	Appendix describing non-templated version of window and stream data types and APIs.
07/19/2021 Version 2021.1	
FIFO Location Constraints	Updated FIFO constraints examples.
Buffer Data Types	New topic.
Supported Stream Data Types	New topic.
Building a Bare-metal AI Engine in the Vitis IDE	Updated Step 4.
06/16/2021 Version 2021.1	
Run-Time Ratio	New topic.
Stream Data Types Reading and Advancing an Input Stream Writing and Advancing an Output Stream	New Stream types added.
Run-Time Parameter Support Summary	AI Engine RTP Support table added.
Stream Switch FIFO DMA FIFO AI Engine Tile DMA Performance	New FIFO topics.
Packet Switching Graph Constructs	Allowed number of packet streams updated.
Multicast Support	New topic.
Chapter 11: AI Engine/Programmable Logic Integration	Updated content.
Hardware Emulation and Hardware Flows	ADF_FRONTEND removed.
Performance Comparison Between AI Engine/PL and AI Engine/NoC Interfaces	New topic.
AI Engine Compiler Options	• Heap and Stack size updated. • <code>--broadcast-enable-core</code> CDO option added. • Tracing options updated. • <code>xlopt</code> updated.
Graph and Array Details	New section added.
AI Engine Compiler Guidance	New topic.
Reusing AI Engine Simulator Options	<code>--profile/AIE_PROFILE</code> added to options.
Enabling Third-Party Simulators in a Configuration File	Simulators added and versions updated.
x86 Functional Simulator	Updated content and new sections added.
Viewing the Run Summary in the Vitis IDE	Content updated.
Trace View Data Visualization	New section.
Run-Time Event API Performance Counters Usage Summary	New topic.
Chapter 7: Programming the PS Host Application	ADF_FRONTEND removed.
Controlling the Application with the XRT C++ API	New topic.
AI Engine Error Reporting	<code>xbutil</code> scope updated.
Host Code Reference with ADF API and XRT API	Updated for <code>printf</code> .
Syncing Clocks with the AI Engine	Updated topic.
Compiling the Embedded Application for the Cortex-A72 Processor	Code updated: <code>aarch64-linux-gnu-g++</code> to <code>aarch64-xilinx-linux-g++</code>
Running Hardware Emulation	New section.

Section	Revision Summary
Using the Vitis IDE	Screenshot updates.
Chapter 8: Mapper/Router Methodology	New chapter.
Event API	Removed extra Enumeration section.
FIFO Constraint	New topic.
Using the Restrict Keyword in AI Engine Kernels	Updated to C++.

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2020-2024 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Versal, Vitis, Vivado, and combinations thereof are trademarks of Advanced Micro Devices, Inc. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. MATLAB and Simulink are registered trademarks of The MathWorks, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.